UCD-SeRG Lab Manual

Last updated: 2026-10-12

Contents

1 Welcome to UCD-SeRG! 1
- 1.1 About the lab . . . 1
- 1.2 About this lab manual . . . 1

2 Culture and conduct 2
- 2.1 Lab culture . . . 2
- 2.2 Diversity, equity, and inclusion . . . 2
- 2.3 Protecting human subjects . . . 2
- 2.4 Authorship . . . 3

3 Communication and coordination 4
- 3.1 Microsoft Teams . . . 4
- 3.2 Email . . . 4
- 3.3 Direct Hyperlinks in Internal Correspondence . . . 5
- 3.4 Managing Notifications and Work-Life Balance . . . 5
- 3.5 Setting Away Status and Autoresponse Checklist . . . 5
- 3.6 Task Management . . . 6
- 3.7 Shared Documents . . . 6
- 3.8 UC Davis Box and SharePoint . . . 6
- 3.9 Meetings . . . 7
- 3.10 Conference Room Booking . . . 8
- 3.11 Code Review . . . 8

4 Reproducibility 9
- 4.1 What is the reproducibility crisis? . . . 9
- 4.2 Study design . . . 9
- 4.3 Register study protocols . . . 10
- 4.4 Write and register pre-analysis plans . . . 10
- 4.5 Create reproducible workflows . . . 10
- 4.6 Process and analyze data with internal replication and masking . . . 11
- 4.7 Use reporting checklists with manuscripts . . . 11
- 4.8 Publish preprints . . . 11
- 4.9 Publish data (when possible) and replication scripts . . . 11

5 Code repositories 12
- 5.1 Template repositories . . . 12
- 5.2 Package Structure . . . 12
- 5.3 `.Rproj` files . . . 17
- 5.4 Organizing the `data-raw` folder . . . 18
- 5.5 Repository Status Badges . . . 19

6 R Coding Practices 22
- 6.1 Lab Protocols for Code and Data . . . 22
- 6.2 The Tool-Building Mindset . . . 22

6.3 Essential R Package Development Tools . . . 23
6.4 Complete Package Development Workflow . . . 28
6.5 Organizing scripts . . . 31
6.6 Testing Requirements . . . 31
6.7 Test-Driven Development . . . 34
6.8 Benchmarking . . . 35
6.9 Iterative Operations . . . 45
6.10 Reading and Saving Data . . . 46
6.11 Version Control and Collaboration . . . 47
6.12 Quality Assurance Checklist . . . 49
6.13 Automated Code Styling . . . 50
6.14 Documenting your code . . . 51
6.15 Design Documentation with ADRs . . . 56
6.16 Object naming . . . 58
6.17 Function calls . . . 58
6.18 Function decomposition . . . 59
6.19 When to decompose a function . . . 59
6.20 How to decompose in practice . . . 59
6.21 Decomposing validation blocks . . . 60
6.22 Examples borrowed from lab package PRs . . . 61
6.23 Related references . . . 61
6.24 One function per file . . . 61
6.25 Function length limits . . . 62
6.26 Keep It Short and Simple (KISS) . . . 63
6.27 Decomposing Code Chunks in Quarto Documents . . . 64
6.28 The here package . . . 64
6.29 Reading/Saving Data . . . 64
6.30 Integrating Box and Dropbox . . . 65
6.31 r-lib Packages . . . 66
6.32 Package Development . . . 66
6.33 Testing and Quality . . . 69
6.34 Core Language and Infrastructure . . . 69
6.35 System and Process . . . 70
6.36 HTTP and Web . . . 71
6.37 Utilities . . . 72
6.38 GitHub Actions . . . 74
6.39 Tidyverse . . . 74
6.40 Core Tidyverse Packages . . . 75
6.41 Base R to Tidyverse Translation . . . 77
6.42 Programming with Tidyverse . . . 78
6.43 Coding with R and Python . . . 78
6.44 Repeating analyses with different variations . . . 78
6.45 Reviewing Code . . . 82
6.46 Getting Help with Code . . . 83
6.47 Additional Resources . . . 84

7 R Code Style **86**
7.1 General Principles . . . 86
7.2 Clean Code Principles . . . 86
7.3 Function Structure and Documentation . . . 88
7.4 Assertions and Input Validation . . . 89

7.5 Avoiding Deep Nesting . . . 91
7.6 Prefer Existing Packaged Functions . . . 92
7.7 Prefer Per-Operation Grouping . . . 93
7.8 Avoid Hard-Coding Data with an External Source of Truth . . . 93
7.9 Construct Complex Inputs Before the Call . . . 96
7.10 Comments . . . 97
7.11 Line Breaks and Formatting . . . 98
7.12 Markdown and Quarto Formatting . . . 101
7.13 Messaging and User Communication . . . 101
7.14 Package Code Practices . . . 103
7.15 Tidyverse Replacements . . . 103
7.16 The here Package . . . 104
7.17 Object Naming . . . 105
7.18 Automated Tools for Style and Project Workflow . . . 106
7.19 Additional Resources . . . 112

8 Big data **113**
8.1 The data.table package . . . 113
8.2 Using downsampled data . . . 113
8.3 Optimal RStudio set up . . . 113

9 Data masking **115**
9.1 General Overview . . . 115
9.2 Data masking vs. tidy selection . . . 116
9.3 Technical Overview . . . 118

10 Quarto **120**
10.1 Introduction . . . 120
10.2 Quarto Basics . . . 121
10.3 Building Quarto Books . . . 124
10.4 Dashboards . . . 127
10.5 Quarto Profiles . . . 127
10.6 Advanced Features . . . 129
10.7 Mermaid Diagrams . . . 132
10.8 Troubleshooting . . . 135
10.9 Additional Resources . . . 137

11 Github **139**
11.1 Basics . . . 139
11.2 GitHub Education and Copilot Access . . . 139
11.3 Graphical Git Clients . . . 141
11.4 GitHub CLI and GitLab CLI . . . 142
11.5 Git Branching . . . 143
11.6 Personal Dev Branches . . . 143
11.7 Example Workflow . . . 144
11.8 Pull Request Roles and Protocol . . . 145
11.9 Commonly Used Git Commands . . . 145
11.10 How often should I commit? . . . 146
11.11 Repeated Amend Workflow . . . 146
11.12 What should be pushed to Github? . . . 147
11.13 GitLab . . . 147
11.14 Customizing How Files Appear on GitHub . . . 148

11.15 Exporting Issue and Pull Request Conversations . . . 150
11.16 Removing Sensitive Data from a Repository . . . 153

12 Collaborative Software Development **154**
12.1 Pull Requests . . . 154
12.2 Code Review . . . 159

13 Continuous Integration **164**
13.1 Understanding GitHub Actions . . . 164
13.2 Setting Up GitHub Actions . . . 166
13.3 Reusable Workflow Collections . . . 167
13.4 How GitHub Actions Workflows Work . . . 168
13.5 Workflow Files and Security . . . 168
13.6 Pipeline Design Patterns . . . 169
13.7 Troubleshooting Failed Workflows . . . 172
13.8 Pull Request Comment Automation . . . 172
13.9 Fast Package Installation with r2u . . . 177
13.10 Additional Resources . . . 178

14 Unix **180**
14.1 User interfaces: CLI, TUI, and GUI . . . 180
14.2 Basics . . . 182
14.3 Terminal Emulators . . . 183
14.4 Syntax for both Mac/Windows . . . 185
14.5 Zsh and Oh My Zsh . . . 187
14.6 Running Bash Scripts . . . 191
14.7 Shell Scripting Style . . . 192
14.8 Running Rscripts in Windows . . . 193
14.9 Checking tasks and killing jobs . . . 194
14.10 Running big jobs . . . 195
14.11 External Resources . . . 198

15 Reproducible Environments **199**
15.1 Package Version Control with renv . . . 199
15.2 Fast Package Installation on Ubuntu with r2u . . . 201
15.3 Managing R Versions with rig . . . 202
15.4 Clearing Docker Cache . . . 202

16 Code Publication **204**
16.1 Checklist overview . . . 204
16.2 Fill out file headers . . . 204
16.3 Clean up comments . . . 204
16.4 Document functions . . . 204
16.5 Remove deprecated filepaths . . . 204
16.6 Ensure project runs via bash . . . 205
16.7 Complete the README . . . 205
16.8 Clean up feature branches . . . 207
16.9 Create GitHub release . . . 207
16.10 Releasing to CRAN . . . 207

17 Data Publication **210**
17.1 Overview . . . 210

17.2 Removing PHI . . . 211
17.3 Create public IDs . . . 212
17.4 Create a data repository . . . 213
17.5 Edit and test analysis scripts . . . 214
17.6 Create a public GitHub page for public scripts . . . 214
17.7 Go live . . . 215

18 High-performance computing (HPC) 216
18.1 UC Davis Computing Resources . . . 216
18.2 Getting started with SLURM clusters . . . 216
18.3 Moving files to the cluster . . . 218
18.4 Installing packages on the cluster . . . 218
18.5 Testing your code . . . 219
18.6 Running graphical programs with X11 forwarding . . . 221
18.7 Recovering a disconnected interactive session . . . 224
18.8 Storage & group storage access . . . 225
18.9 Running big jobs . . . 226

19 Working with AI 228
19.1 Institutional AI account access . . . 228

20 Checklists 230
20.1 Onboarding checklist . . . 230
20.2 Pre-analysis plan checklist . . . 230
20.3 Code checklist . . . 231
20.4 Manuscript checklist . . . 231
20.5 Figure checklist . . . 232
20.6 Offboarding checklist . . . 232

21 Resources 233
21.1 Resources for R . . . 233
21.2 Resources for Git & Github . . . 235
21.3 Practical Computing and Command Line . . . 235
21.4 Resources for Python . . . 236
21.5 Resources for Julia . . . 236
21.6 Artificial Intelligence & Coding Agents . . . 236
21.7 Scientific figures . . . 236
21.8 Writing . . . 236
21.9 Presentations . . . 237
21.10 Professional advice . . . 237
21.11 Funding . . . 237
21.12 Ethics and global health research . . . 237
21.13 Open Science and Reproducibility Initiatives . . . 237
21.14 Comparable Lab Manuals . . . 238
21.15 Reading and Watching List . . . 238

22 Professional Development 239
22.1 Mentoring Philosophy . . . 239
22.2 Individual Development Plans . . . 239
22.3 Navigating the Hidden Curriculum . . . 239
22.4 Presentations and Conferences . . . 241
22.5 Scientific Figures . . . 242

22.6 Grant Writing . . . 242
22.7 PhD Dissertation Requirements . . . 242
22.8 Teaching and Outreach . . . 244
22.9 Networking . . . 245

23 Writing **246**
23.1 Writing to Clarify Your Thinking . . . 246
23.2 Plain, Direct Prose . . . 246
23.3 Scanning for AI Tells . . . 247
23.4 Show, Don't Tell . . . 247

24 Manuscript Preparation and Publication **249**
24.1 Publication Process . . . 249
24.2 Responding to Peer Review . . . 249
24.3 Preprints and Open Access . . . 250
24.4 Reporting Checklists . . . 250
24.5 Manuscript Checklist . . . 251
24.6 Formatting tables with `gtsummary` + `flextable` + `flexlsx` . . . 251
24.7 Scientific Writing: Claims and Evidence . . . 252

References **253**

Appendices **257**

Copilot Instructions File **257**

Copilot Setup Steps File **258**

Document Generation Metadata **259**

1 Welcome to UCD-SeRG!

1.1 About the lab

Welcome to the UC Davis Seroepidemiology Research Group (SeRG), led by Drs. Kristen Aiemjoy and Ezra Morrison. Accurate methods to measure infectious disease burden are essential for guiding public health decisions, yet many infectious diseases remain underrecognized due to limited diagnostics and costly, resource-intensive surveillance systems. Our work addresses this gap by developing seroepidemiologic methods to characterize infection burden in populations. Currently, we focus on enteric fever (Salmonella Typhi and Paratyphi), Scrub Typhus (Orientia tsutsugamushi), Melioidosis (Burkholderia pseudomallei), Shigella (Shigella spp.), and Cholera (Vibrio cholerae). We are supported by the US National Institutes of Health, the Bill and Melinda Gates Foundation, and the Department of Defense, and collaborate with partners around the world. To learn more about the lab, visit ucdserg.ucdavis.edu[1].

1.2 About this lab manual

This lab manual covers our communication strategy, code of conduct, and best practices for reproducibility of computational workflows. It is a living document that is updated regularly.

This manual is a fork of the Benjamin-Chung Lab Manual (Benjamin-Chung et al. 2024), adapted for UCD-SeRG. We are grateful to Dr. Jade Benjamin-Chung and her team for developing and openly sharing their excellent lab manual. You can view the original manual at jadebc.github.io/lab-manual[2]. Original contributors include Jade Benjamin-Chung, Kunal Mishra, Stephanie Djajadi, Nolan Pokpongkiat, Anna Nguyen, Iris Tong, and Gabby Barratt Heitmann. The Data Publication chapter also draws from Fanice Nyatigo and Ben Arnold's public-data chapter in the Proctor Lab at UCSF handbook[3].

Feel free to draw from this manual (and please cite it if you do!).

This work is licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("Creative Commons Attribution-NonCommercial 4.0 International License," n.d.).

[1] https://ucdserg.ucdavis.edu
[2] https://jadebc.github.io/lab-manual/index.html
[3] https://proctor-ucsf.github.io/dcc-handbook/publicdata.html

2 Culture and conduct

2.1 Lab culture

We are committed to a lab culture that is collaborative, supportive, inclusive, open, and free from discrimination and harassment.

We encourage students / staff of all experience levels to respectfully share their honest opinions and ideas on any topic. Our group has thrived upon such respectful honest input from team members over the years, and this document is a product of years of student and staff input (and even debate) that has gradually improved our productivity and overall quality of our work.

2.2 Diversity, equity, and inclusion

UCD-SeRG recognizes the importance of and is committed to cultivating a culture of diversity, equity, and inclusion. This means being a safe, supportive, and anti-racist environment in which students from diverse backgrounds are equally and inclusively supported in their education and training. Diversity takes many forms, and includes, but is not limited to, differences in race, ethnicity, gender, sexuality, socioeconomic status, religion, disability, and political affiliation.

2.3 Protecting human subjects

All lab members must complete CITI Human Subjects Biomedical Group 1[1] training and share their certificate with the lab leadership. Team members will be added to relevant Institutional Review Board protocols prior to their start date to ensure they have permission to work with identifiable datasets.

One of the most relevant aspects of protecting human subjects in our work is maintaining confidentiality. For students supporting our data science efforts, in practice this means:

- Be sure to understand and comply with project-specific policies about where data can be saved, particularly if the data include personal identifiers.
- Do not share data with anyone without permission, including to other members of the group, who might not be on the same IRB protocol as you (check with lab leadership first).

Remember, data that looks like it does not contain identifiers to you might still be classified as data that requires special protection by our IRB or under HIPAA, so always proceed with caution and ask for help if you have any concerns about how to maintain study participant confidentiality.

[1] https://research.ucdavis.edu/policiescompliance/irb-admin/education/

2.4 Authorship

We adhere to the ICMJE Definition of authorship[2] (International Committee of Medical Journal Editors, n.d.) and are happy for team members who meet the definition of authorship to be included as co-authors on scientific manuscripts. To qualify for authorship, individuals must meet all four criteria:

1. Substantial contributions to conception/design, or acquisition/analysis/interpretation of data
2. Drafting the work or revising it critically for important intellectual content
3. Final approval of the version to be published
4. Agreement to be accountable for all aspects of the work

Authorship practices:

- **First authorship**: Typically goes to the person who led the work
- **Corresponding author**: Usually the PI, unless otherwise agreed
- **Co-authorship**: Determined by substantial intellectual contributions
- **Author order**: Should be discussed and agreed upon by all authors
- **Acknowledgments**: For contributions that don't meet authorship criteria

Authorship should be discussed early in a project and revisited as the work evolves to ensure transparency and fairness. We encourage using the CRediT Taxonomy[3] to document specific author contributions.

[2] http://www.icmje.org/recommendations/browse/roles-and-responsibilities/defining-the-role-of-authors-and-contributors.html
[3] https://credit.niso.org/

3 Communication and coordination

One benefit of the academic environment is its schedule flexibility and autonomy. This means that lab members may choose to work in the early morning, afternoon, evening, or weekends. Lab members should manage their own notification schedules to ensure they can maintain healthy work-life boundaries. We do not expect lab members to respond outside of normal business hours (unless there are special circumstances). Because of the nature of our work across time zones, frequent travel, and international collaborations, there are no set working hours. Team members are not expected to reply to messages outside their own working hours, even if a message arrives outside of those hours.

3.1 Microsoft Teams

- Use Microsoft Teams for scheduling, coding related questions, quick check ins, etc. If your Teams message exceeds 200 words, it might be time to use email.
- Use channels instead of direct messages unless you need to discuss something private.
- Please make an effort to respond to messages that mention you (e.g., `@username`) as quickly as possible.
- If you are unusually busy (e.g., preparing for QE/grant deadline, taking two exams) or on vacation please alert the team in advance so we can expect you not to respond at all / as quickly as usual. Please also set your status in Teams (e.g., it could say "On vacation") so we know not to expect to see you online.
- Please thread messages in Teams as much as possible.
- When referencing a pull request, issue, commit, or file, always include a direct hyperlink to where you want your teammates to look (see Direct Hyperlinks in Internal Correspondence[1]).
- Don't wait for meetings to ask questions. As soon as a question comes up, write it out in Teams. This benefits both you (by clarifying your thinking, as discussed in Chapter 23) and the team (by getting the conversation started earlier).

3.2 Email

- Use email for longer messages (>200 words) or messages that merit preservation.
- When discussing specific code, pull requests, or documents in emails, include direct hyperlinks so recipients do not have to search for the relevant items (see Direct Hyperlinks in Internal Correspondence[2]).
- Generally, strive to respond within a work day when you are directly addressed in an email. Again because we work across many timezones, this means responding within a work day in your own timezone. If you're on vacation, you're not expected to respond

[1] @sec-direct-hyperlinks
[2] @sec-direct-hyperlinks

until the end of your first work day back (or possibly longer, if you've been out for a while and have a backlog).
- As noted above, if you are unusually busy or on vacation please alert the team in advance and via an away message, so we can expect you not to respond at all / as quickly as usual.

3.3 Direct Hyperlinks in Internal Correspondence

In all internal correspondence—including Microsoft Teams messages, emails, GitHub issues, and pull request comments—always provide direct hyperlinks to the exact place you want your audience to look.

- **Link directly to the target:** Provide a direct, clickable URL or markdown link to the specific pull request, issue, commit, branch, file, or line number.
- **Do not make your audience search:** Avoid vague references (such as "I opened a PR", "see the latest changes on my branch", or "look at the helper script") or bare unlinked identifiers (such as `#123` in emails or chat). Your audience should never have to search repositories, branches, or directory trees to find what you are referring to.
- **Use descriptive markdown links:** Format links with descriptive anchor text (for example, `[PR #123: Add data validation](https://github.com/UCD-SERG/lab-manual/pull/123)` or `[cleaning-script.R lines 45-60](https://github.com/...)`) so collaborators know where the link leads before clicking.
- **Reduce friction for collaborators:** Direct links save everyone time, prevent confusion about which branch or version is being discussed, and allow reviewers to immediately jump into context with a single click.

3.4 Managing Notifications and Work-Life Balance

Lab members should actively manage their notification schedules, especially if they receive work email or Teams notifications on their phones.

- **Block notifications outside of work hours**: We recommend configuring your devices and applications to prevent work notifications during non-work time. We all need uninterrupted rest and relaxation time in order to stay healthy and productive in the medium and long term.
- **Sending messages outside work hours**: There's no expectation to only send emails or messages during normal work hours. Feel free to send messages whenever you compose them—don't spend effort scheduling messages for later delivery. Recipients will manage their own notification schedules, and they may be working different hours or want the information sooner.

3.5 Setting Away Status and Autoresponse Checklist

When you'll be away from work (vacation, conference, sick leave, etc.), please help the team stay coordinated by setting appropriate away indicators:

- ☐ **Outlook calendar**: Mark yourself as away on your Outlook calendar for the dates you'll be unavailable.

- ☐ **Autoresponse**: Set an autoresponse message that indicates when you expect to be back at work.
- ☐ **Teams status**: Update your Teams status to indicate you're away (e.g., "On vacation until [date]").

3.6 Task Management

We use a combination of tools to track and manage project tasks:

- **GitHub Issues and Projects**: For code-related tasks, feature requests, and bug tracking. Lab leadership will assign issues and organize them in GitHub Projects. Issues are prioritized within projects, and you can track your assigned tasks there.
- **Microsoft Teams & Planner**: For general lab tasks and personal task management. Lab leadership may assign tasks through these tools, which integrate with Microsoft Teams. Lab members are expected to set up a Teams page for important degree requirements (QE) and projects (grant proposals, papers, etc.). Invite both Dr Morrison and Aiemjoy to the Teams page and assign them to review tasks with the deadline you need the review by. We appreciate 2 weeks' notice for letters and larger review projects.
- Generally, strive to complete assigned tasks by the date listed.
- Use checklists to break down tasks into smaller chunks. Sometimes leadership will create these for you, but you can also add them yourself.
- Update task status as you make progress so the team can stay coordinated.

3.7 Shared Documents

- We mostly use Microsoft 365 to collaborate and write internally and GoogleDocs when working with external collaborators. These documents may be linked to in GitHub Issues or other task tracking tools. Lab leadership often shares these with the whole team since tasks are overlapping, and even if a task is assigned to one person, others may have valuable insights.

3.8 UC Davis Box and SharePoint

- Human subjects data for research studies are generally stored in UC Davis Box or SharePoint. Please check with lab leadership about whether there are special storage and transfer requirements for the datasets you are working with for each study.
- You can access Box via your UC Davis credentials. For more information, visit UC Davis Box Support[3].
- SharePoint is also used for collaborative document storage and team file sharing. Access SharePoint through your UC Davis Microsoft 365 account.

[3] https://servicehub.ucdavis.edu/servicehub?id=ucd_kb_article&sysparm_article=KB0000184

3.9 Meetings

- Our meetings start on the hour.
- If you are going to be late, please send a message in our Teams channel.
- If you are regularly not able to come on the hour, notify the team and we might choose to modify the agenda order or the start time.

3.9.1 Developing Meeting Agendas

An effective meeting agenda ensures that collaborative time is productive, focused, and respectful of everyone's schedule. Whether organizing a weekly lab meeting, a project synchronization check-in, or a one-on-one advising session, follow these guidelines when preparing agendas.

3.9.1.1 Why agendas are important

- **Clear purpose and focus**: Meetings without an agenda easily drift into unstructured conversation or unfocused status updates. A written agenda clarifies the meeting's objective before bringing people together.
- **Preparation and respect for time**: Circulating an agenda in advance allows participants to review relevant materials, gather required data, and arrive ready to contribute substantive insights.
- **Accountability and action items**: A structured agenda provides a natural framework for tracking decisions, assigning clear ownership for action items, and reviewing progress at subsequent meetings.

3.9.1.2 Core content: Inform, Discuss, Ask

Organize your agenda items using three distinct functional categories:

- **Inform**: Brief announcements, status summaries, or project milestone updates. These items share essential context that requires little to no debate. Whenever possible, share background details asynchronously before the meeting so live time can focus on deeper discussion.
- **Discuss**: Topics requiring collaborative problem-solving, methodological debate, troubleshooting, or synthesis of differing viewpoints. These items benefit most from synchronous conversation.
- **Ask**: Specific requests for decisions, approvals, feedback, or resources from advisors, PIs, or team members. Clearly framing what you need (for example, "Decision on primary analysis model" or "Approval to submit abstract") helps attendees provide decisive guidance.

3.9.1.3 Formatting and style guidelines

- **Use clear category tags**: Prefix each bullet point with its category tag (such as `[Inform]`, `[Discuss]`, or `[Ask]`) so participants immediately understand their expected role for each topic.

- **Include direct links**: Always include direct hyperlinks to relevant draft manuscripts, code repositories, pull requests, or data summaries rather than requiring attendees to search for materials.
- **Allocate time estimates**: Assign estimated time allocations to each major agenda topic (for example, `[Discuss] Simulation study edge cases (15 min)`). This prevents early topics from crowding out crucial decisions scheduled at the end.
- **Distribute in advance**: Share the agenda in the calendar invitation, by email, or in the relevant Microsoft Teams channel at least 24 hours before the meeting.
- **Record action items**: Conclude every meeting by summarizing agreed-upon next steps, assigned owners, and target completion dates.

3.10 Conference Room Booking

The MS1C Conference Room can be booked through its Outlook calendar.

To book the room:

1. Create a new event in Outlook Calendar
2. Add the conference room as an attendee by typing "ms1c"
3. Select "MS1C Conference Room" from the results
4. Send the meeting invitation

The room will automatically accept or decline based on its availability.

To view the room's calendar:

1. In Outlook Calendar, click "Add Calendar"
2. Select "Add Shared Calendar…" or use "Open Calendar…"
3. Search for "ms1c"
4. Select "MS1C Conference Room" to add it to your calendar view

This allows you to check the room's availability before scheduling.

3.11 Code Review

Code review guidance has moved to its own chapter: see Collaborative Software Development[4].

[4] @sec-collab-dev

4 Reproducibility

Our lab adopts the following practices to maximize the reproducibility of our work.

1. Design studies with appropriate methodology and adherence to best practices in epidemiology and biostatistics
2. Register study protocols
3. Write and register pre-analysis plans
4. Create reproducible workflows
5. Process and analyze data with internal replication and masking
6. Use reporting checklists with manuscripts
7. Publish preprints
8. Publish data (when possible) and replication scripts

4.1 What is the reproducibility crisis?

In the past decade, an increasing number of studies have found that published study findings could not be reproduced. Researchers found that it was not possible to reproduce estimates from published studies: 1) with the same data and same or similar code and 2) with newly collected data using the same (or similar) study design. These "failures" of reproducibility were frequent enough and broad enough in scope, occurring across a range of disciplines (epidemiology, psychology, economics, and others) to be deeply troubling. Program and policy decisions based on erroneous research findings could lead to wasted resources, and at worst, could harm intended beneficiaries. This crisis has motivated new practices in reproducibility, transparency, and openness. Our lab is committed to adopting these best practices, and much of the remainder of the lab manual focuses on how to do so.

Recommended readings on the "reproducibility crisis":

- Nuzzo R. How scientists fool themselves – and how they can stop (Nuzzo 2015)
- Stoddart C. Is there a reproducibility crisis in science? (Stoddart 2019)
- Munafò MR, et al. A manifesto for reproducible science (Munafò et al. 2017)

4.2 Study design

Appropriate study design is beyond the scope of this lab manual and is something trainees develop through their coursework and mentoring.

4.3 Register study protocols

We register all randomized trials on clinicaltrials.gov, and in some cases register observational studies as well.

4.4 Write and register pre-analysis plans

We write pre-analysis plans for most original research projects that are not exploratory in nature, although in some cases, we write pre-analysis plans for exploratory studies as well. The format and content of pre-analysis plans can vary from project to project. Here is an example of one: https://osf.io/tgbxr/. Generally, these include:

1. Brief background on the study (a condensed version of the introduction section of the paper)
2. Hypotheses / objectives
3. Study design
4. Description of data
5. Definition of outcomes
6. Definition of interventions / exposures
7. Definition of covariates
8. Statistical power calculation
9. Statistical analysis:

- Type of model
- Covariate selection / screening
- Standard error estimation method
- Missing data analysis
- Assessment of effect modification / subgroup analyses
- Sensitivity analyses
- Negative control analyses

4.5 Create reproducible workflows

Reproducible workflows allow a user to reproduce study estimates and ideally figures and tables with a "single click". In practice, this typically means running a single bash script that sources all replication scripts in a repository. These replication scripts complete data processing, data analysis, and figure/table generation. The following chapters provide detailed guidance on this topic:

- Chapter 5: Code repositories
- Chapter 6: Coding practices
- Chapter 7: Coding style
- Chapter 8: Code publication
- Chapter 9: Working with big data
- Chapter 10: Github
- Chapter 11: Unix

For additional learning resources on reproducible research practices, see the UC Davis DataLab workshop on reproducible research[1].

4.6 Process and analyze data with internal replication and masking

See my video on this topic: https://www.youtube.com/watch?v=WoYkY9MkbRE

4.7 Use reporting checklists with manuscripts

Using reporting checklists helps ensure that peer-reviewed articles contain the information needed for readers to assess the validity of your work and/or attempt to reproduce it. A collection of reporting checklists is available from the EQUATOR Network[2] ("EQUATOR Network," n.d.). Commonly used checklists in our lab include:

- CONSORT[3] (Consolidated Standards of Reporting Trials) for randomized trials
- STROBE[4] (Strengthening the Reporting of Observational Studies in Epidemiology) for observational studies
- STARD[5] (Standards for Reporting Diagnostic Accuracy Studies) for diagnostic accuracy studies

Please ensure that your manuscript follows these reporting guidelines before sending it for review.

4.8 Publish preprints

A preprint is a scientific manuscript that has not been peer reviewed. Preprint servers create digital object identifiers (DOIs) and can be cited in other articles and in grant applications. Because the peer review process can take many months, publishing preprints prior to or during peer review enables other scientists to immediately learn from and build on your work. Importantly, NIH allows applicants to include preprint citations in their biosketches. In most cases, we publish preprints on medRxiv ("medRxiv," n.d.).

4.9 Publish data (when possible) and replication scripts

Publishing data and replication scripts allows other scientists to reproduce your work and to build upon it. We typically publish data on the Open Science Framework ("Open Science Framework," n.d.), share links to Github[6] repositories, and archive code on Zenodo[7].

[1] https://github.com/ucdavisdatalab/workshop_reproducible_research
[2] https://www.equator-network.org/
[3] https://www.equator-network.org/reporting-guidelines/consort/
[4] https://www.equator-network.org/reporting-guidelines/strobe/
[5] https://www.equator-network.org/reporting-guidelines/stard/
[6] github.com
[7] zenodo.org

5 Code repositories

Each study has at least one code repository that typically holds R code, shell scripts with Unix code, and research outputs (results .RDS files, tables, figures). Repositories may also include datasets. This chapter outlines how to organize these files. Adhering to a standard format makes it easier for us to efficiently collaborate across projects.

UCD-SeRG projects use R package structure for most R-based work. This provides benefits for reproducibility, collaboration, and code quality even for analysis-only projects.

5.1 Template repositories

When starting a new package, begin from the lab's template repository (`UCD-SERG/rpt`[1]) rather than an empty repository. On GitHub, click **Use this template** to create a new repository that already includes our standard structure, CI workflows, and documentation scaffolding. Then replace the placeholder package name, `Title`, and `Description` in `DESCRIPTION` with your project's details.

5.1.1 A note on GitLab

GitHub-style template repositories do **not** have a direct equivalent in GitLab Community Edition (CE):

- **Built-in project templates** (Rails, Node, and similar) ship with all editions, including CE, but you cannot add your own to that list.
- **Custom project templates** (instance- or group-level) — the feature most analogous to GitHub's template repositories — are Premium/Ultimate tier[2] only, so they are unavailable in CE.
- On CE, the practical workaround is to **fork** (or clone and re-push) a "seed" project to start new work from it.

In short: if the lab ever mirrors this workflow onto a self-managed GitLab CE instance, plan to either fork a seed repository or budget for a paid tier.

5.2 Package Structure

All R projects in our lab should be structured as R packages, even if they are primarily analysis projects and not intended for distribution on CRAN or Bioconductor. This standardized structure provides numerous benefits:

[1] https://github.com/UCD-SERG/rpt
[2] https://docs.gitlab.com/user/group/custom_project_templates/

5.2.1 Why Use R Package Structure?

1. **Organized code**: Clear separation of functions (`R/`), documentation (`man/`), tests (`tests/`), data (`data/`), and vignettes/analyses
2. **Dependency management**: `DESCRIPTION` file explicitly declares all package dependencies and version restrictions, which simplifies installing those dependencies.
3. **Automatic documentation**: `roxygen2` generates help files from inline comments
4. **Built-in testing**: `testthat` framework integrates seamlessly with package structure
5. **Code quality**: Tools like `devtools::check()` and `lintr` enforce best practices
6. **Reproducibility**: Package structure makes it easy to share and reproduce analyses
7. **Reusable functions**: Decompose complex analyses into well-documented, testable functions
8. **Version control**: Track changes to code, documentation, and data together

5.2.2 Basic Package Structure

```
myproject/
  DESCRIPTION          # Package metadata and dependencies
  NAMESPACE            # Auto-generated, don't edit manually
  R/                   # All R functions (reusable code)
     analysis_functions.R
     data_prep.R
     plotting.R
  man/                 # Auto-generated documentation
  tests/
     testthat/       # Unit tests
  data/                # Processed data objects (.rda files)
  data-raw/            # Raw data and data processing scripts
     0-prep-data.sh  # Shell scripts for data preparation
     process_survey_data.R
     clean_lab_results.R
  vignettes/          # Long-form documentation
     intro.qmd       # Main vignettes (shipped with package)
     tutorial.qmd
     articles/       # Website-only articles (not shipped)
         advanced-topics.qmd
         case-studies.qmd
  inst/               # Additional files to include in package
     extdata/        # External data files and .RDS results
        analysis_results.rds
        processed_data.rds
     output/         # Figure and table outputs
        figures/
           fig1.pdf
           fig2.png
        tables/
            table1.csv
            table2.xlsx
     analyses/       # Analyses using restricted data (see below)
  .Rproj              # RStudio project file
```

5.2.3 Where to Place Analysis Files

5.2.3.1 Vignettes vs Articles

Vignettes (`vignettes/*.qmd`): - **Shipped with the package** when installed - Accessible via `vignette()` and `browseVignettes()` in R - Displayed on CRAN - Built at package build time - Use for core package documentation and tutorials - Created with `usethis::use_vignette("name")`

Articles (`vignettes/articles/*.qmd`): - **Website-only** (not shipped with the package) - Only appear on the pkgdown website - Not accessible via `vignette()` in R - Not displayed on CRAN - Use for supplementary content, blog posts, extended tutorials, or frequently updated material - Created with `usethis::use_article("name")` - Automatically added to `.Rbuildignore`

When to use each: - **Vignette**: Essential tutorials users need offline, core package workflows - **Article**: Supplementary material, case studies, advanced topics, blog-style content

Additional considerations for choosing articles over vignettes:

Articles are particularly useful when (Wickham and Bryan 2023, sec. 17.4.1):

- **Code shouldn't execute on CRAN**: When your code requires authentication credentials, depends on external services, or takes too long to run
- **Demonstrating integration with other packages**: When you want to show your package working with packages you don't want to formally depend on
- **Graphics-heavy content**: When including many or large graphics would make the package too large for CRAN
- **Trade-off**: Articles are less accessible than vignettes for users with limited internet access, as they don't ship with the package and only appear on the pkgdown website

5.2.3.2 Public Analyses (vignettes/)

Use `vignettes/` for analysis workbooks that:

- Use publicly available data
- Should be accessible to all package users
- Are core to understanding the package

Use `vignettes/articles/` for:

- Extended case studies
- Blog-style posts
- Supplementary analyses
- Material that updates frequently

All vignettes and articles will be rendered by `pkgdown::build_site()` on your package website.

5.2.3.3 Analyses with Restricted Data (inst/analyses/)

For analyses that rely on **private, sensitive, or restricted data**, place `.qmd` or `.qmd` files in `inst/analyses/`:

```
myproject/
  inst/
      analyses/
          01-confidential-data-analysis.qmd
          02-unpublished-results.qmd
          README.md  # Document data access requirements
      extdata/
  vignettes/
      01-public-analysis.qmd
      02-demo-with-simulated-data.qmd
```

Benefits of this approach:

- Analyses with restricted data are included in version control alongside your code
- They're clearly separated from public documentation
- `inst/analyses/` is **excluded from `pkgdown` builds** and package documentation
- Collaborators with data access can still run these analyses
- You maintain a complete record of all project work

Note on privacy: Files in `inst/analyses/` are not inherently private—they will be visible if your repository is public. Use this folder for analyses that rely on restricted data that is stored separately, not for storing the restricted data itself. If you need to keep the analysis code private, use a private repository.

Best practices for analyses with restricted data:

1. **Document data requirements**: Include a README.md in `inst/analyses/` explaining:
    - What data is required
    - Where to obtain it (if permissible)
    - Data access restrictions
    - How to set up data paths
2. **Use relative paths carefully**: Structure your code so data paths can be configured:

```
# In inst/analyses/01-analysis.qmd
# Users should set this based on their local setup
data_dir <- Sys.getenv("MYPROJECT_DATA",
                       default = "~/restricted_data/myproject")
raw_data <- readr::read_csv(file.path(data_dir, "sensitive.csv"))
```

3. **Create public alternatives**: When possible, create companion vignettes in `vignettes/` using:
    - Simulated data that mimics the structure
    - Publicly available datasets
    - Aggregated/de-identified summaries

4. **Add to .Rbuildignore**: Ensure `inst/analyses/` doesn't cause package checks to fail:

```
# Use usethis to add to .Rbuildignore
usethis::use_build_ignore("inst/analyses")
```

5.2.4 Keep Analysis Workbooks Tidy

Decompose reusable functions from your analysis notebooks into the `R/` directory. Your vignettes should:

- Be clean, readable narratives of your analysis
- Call well-documented functions from your package
- Focus on the "what" and "why" rather than implementation details
- Be reproducible by others with a single click (or with documented data access for private analyses)

Example of what NOT to do (all code in vignette):

```
# Bad: 100 lines of data manipulation in vignette
raw_data <- read_csv("data.csv")
# ... 100 lines of cleaning, transforming, reshaping ...
cleaned_data <- final_result
```

Example of what TO do (functions in R/, simple calls in vignette):

```
# Good: Clean vignette calling documented functions
raw_data <- read_csv("data.csv")
cleaned_data <- prep_study_data(raw_data)  # Function in R/data_prep.R
```

5.2.5 Shell Scripts and Automation

Shell scripts are useful for automating workflows and ensuring reproducibility. Place shell scripts in `data-raw/` alongside the R scripts they coordinate:

```
data-raw/
   0-prep-data.sh          # Shell script to run all data prep
   01-load-survey.R
   02-clean-survey.R
   03-merge-datasets.R
   04-create-analysis-data.R
```

Using shell scripts:

```
# data-raw/0-prep-data.sh
#!/bin/bash
Rscript data-raw/01-load-survey.R
Rscript data-raw/02-clean-survey.R
Rscript data-raw/03-merge-datasets.R
Rscript data-raw/04-create-analysis-data.R
```

This is especially useful when data upstream changes — you can simply run the shell script to reproduce everything. After running shell scripts, `.Rout` log files will be generated for each script. It is important to check these files to ensure everything has run correctly.

5.2.6 Storing Analysis Outputs

Results files (.RDS): Save analysis results in `inst/extdata/`:

```
# Save results
readr::write_rds(analysis_results, here("inst", "extdata", "analysis_results.rds"))

# Load results later
results <- readr::read_rds(here("inst", "extdata", "analysis_results.rds"))
```

Figures and tables: Save publication outputs in `inst/output/`:

```
# Save figure
ggsave(here("inst", "output", "figures", "fig1_incidence_trends.pdf"),
       width = 8, height = 6)

# Save table
readr::write_csv(summary_table,
                 here("inst", "output", "tables", "table1_demographics.csv"))
```

Organization:

```
inst/
  extdata/
      analysis_results.rds
      model_fits.rds
      processed_data.rds
  output/
      figures/
          fig1_incidence_trends.pdf
          fig2_risk_factors.png
          figS1_sensitivity.pdf
      tables/
          table1_demographics.csv
          table2_main_results.xlsx
          tableS1_detailed_results.csv
```

5.3 `.Rproj` files

An "R Project" can be created within RStudio by going to `File >> New Project`. Depending on where you are with your research, choose the most appropriate option. This will save preferences, working directories, and even the results of running code/data (though I'd recommend starting from scratch each time you open your project, in general). Then, ensure that whenever you are working on that specific research project, you open your

created project to enable the full utility of `.Rproj` files. This also automatically sets the directory to the top level of the project.

5.4 Organizing the `data-raw` folder

The `data-raw` folder serves as a catch-all for scripts that do not (yet) fit into the package structure described above. The `data-raw` folder should still be organized. We recommend the following subdirectory structure for `data-raw`:

```
0-run-project.sh
0-config.R
1 - Data-Management/
    0-prep-data.sh
    1-prep-cdph-fluseas.R
    2a-prep-absentee.R
    2b-prep-absentee-weighted.R
    3a-prep-absentee-adj.R
    3b-prep-absentee-adj-weighted.R
2 - Analysis/
    0-run-analysis.sh
    1 - Absentee-Mean/
        1-absentee-mean-primary.R
        2-absentee-mean-negative-control.R
        3-absentee-mean-CDC.R
        4-absentee-mean-peakwk.R
        5-absentee-mean-cdph2.R
        6-absentee-mean-cdph3.R
    2 - Absentee-Positivity-Check/
    3 - Absentee-P1/
    4 - Absentee-P2/
3 - Figures/
    0-run-figures.sh
    ...
4 - Tables/
    0-run-tables.sh
    ...
5 - Results/
    1 - Absentee-Mean/
        1-absentee-mean-primary.RDS
        2-absentee-mean-negative-control.RDS
        3-absentee-mean-CDC.RDS
        4-absentee-mean-peakwk.RDS
        5-absentee-mean-cdph2.RDS
        6-absentee-mean-cdph3.RDS
    ...
.gitignore
```

For brevity, not every directory is "expanded", but we can glean some important takeaways from what we *do* see.

5.4.1 Configuration ('config') File

This is the single most important file for your project. It will be responsible for a variety of common tasks, declare global variables, load functions, declare paths, and more. *Every other file in the project* will begin with `source("0-config")`, and its role is to reduce redundancy and create an abstraction layer that allows you to make changes in one place (`0-config.R`) rather than 5 different files. To this end, paths which will be reference in multiple scripts (i.e. a `merged_data_path`) can be declared in `0-config.R` and simply referred to by its variable name in scripts. If you ever want to change things, rename them, or even switch from a downsample to the full data, all you would then to need to do is modify the path in one place and the change will automatically update throughout your project. See the example config file for more details. The paths defined in the `0-config.R` file assume that users have opened the `.Rproj` file, which sets the directory to the top level of the project.

5.4.2 Order Files and Directories

This makes the jumble of alphabetized filenames much more coherent and places similar code and files next to one another. This also helps us understand how data flows from start to finish and allows us to easily map a script to its output (i.e. `2 - Analysis/1 - Absentee-Mean/1-absentee-mean-primary.R` => `5 - Results/1 - Absentee-Mean/1-absentee-mean-primary.RDS`). If you take nothing else away from this guide, this is the single most helpful suggestion to make your workflow more coherent. Often the particular order of files will be in flux until an analysis is close to completion. At that time it is important to review file order and naming and reproduce everything prior to drafting a manuscript.

5.4.3 Using Bash scripts to ensure reproducibility

Bash scripts are useful components of a reproducible workflow. At many of the directory levels (i.e. in `3 - Analysis`), there is a bash script that runs each of the analysis scripts. This is exceptionally useful when data "upstream" changes – you simply run the bash script. See Chapter 14 for further details.

After running bash scripts, `.Rout` log files will be generated for each script that has been executed. It is important to check these files. Scripts may appear to have run correctly in the terminal, but checking the log files is the only way to ensure that everything has run completely.

5.5 Repository Status Badges

Communicating the development and support status of a repository helps both human collaborators and automated tools understand whether a project is actively maintained, a work in progress, or an archived analysis. The repostatus.org[3] project provides a standardized, machine-readable and human-readable convention for declaring project status through badges in the repository `README.md`.

[3] https://www.repostatus.org/

5.5.1 Standard repository statuses

repostatus.org defines eight standardized statuses spanning both implementation progress and ongoing maintenance commitment:

- **Concept:** Minimal or no implementation has been done yet, or the repository is only intended to be a limited example, demo, or proof-of-concept.
- **WIP (Work in Progress):** Initial development is in progress, but there has not yet been a stable, usable release suitable for the public.
- **Suspended:** Initial development has started, but there has not yet been a stable, usable release; the project has been put on hold or stopped.
- **Abandoned:** Initial development has started, but has been abandoned; the project is no longer being worked on.
- **Active:** The project has reached a stable, usable state and is being actively developed.
- **Inactive:** The project has reached a stable, usable state but is no longer being actively developed; support/maintenance will be provided as time allows.
- **Unsupported:** The project has reached a stable, usable release, but the person or organization responsible for it has decided to abandon it or drop support for it permanently.
- **Moved:** The project has been moved to a new location, and the version at that location should be considered current and authoritative.

5.5.2 Adding a status badge

Place the status badge near the top of your repository `README.md`, immediately beneath the main title. The badge links to the corresponding status definition on repostatus.org.

For an active project:

```
[![Project Status: Active - The project has reached a stable, usable state and is being act
```

For a project in initial development:

```
[![Project Status: WIP - Initial development is in progress, but there has not yet been a s
```

For a completed analysis repository that is stable but no longer actively adding features:

```
[![Project Status: Inactive - The project has reached a stable, usable state but is no long
```

(Note: An ASCII hyphen - is used in the badge alt text above as an ASCII-safe approximation of repostatus.org's Unicode en-dash to ensure compatibility across all plain-text and build tools.)

5.5.3 Lifecycle management

Update the status badge as your project evolves through its lifecycle:

1. **Start as Concept or WIP:** Begin new repositories with `Concept` (for early prototypes) or `WIP` (when building an analysis pipeline or package).
2. **Promote to Active:** Switch to `Active` once the codebase reaches a functional, reproducible state with working scripts (and, where applicable, tests).
3. **Transition upon project completion:** When an epidemiology study or manuscript is published and the analysis code is finalized, update the badge to `Inactive` or `Unsupported` so readers know whether future bug fixes or dependency updates are planned.

6 R Coding Practices

6.1 Lab Protocols for Code and Data

Just as wet labs have strict safety protocols to ensure reproducible results and prevent contamination, our computational lab has protocols for coding and data management. These protocols are not suggestions—they are essential practices that:

- **Ensure reproducibility**: Others (including your future self) can recreate your analysis
- **Prevent errors**: Systematic approaches reduce the risk of mistakes
- **Enable collaboration**: Consistent practices allow team members to work together efficiently
- **Maintain data integrity**: Proper handling prevents data corruption and loss
- **Support publication**: Well-documented, reproducible code is increasingly required for publication

Violating these protocols can have serious consequences, including invalid results, wasted time, inability to publish, and damage to scientific credibility. Treat coding and data management protocols with the same seriousness as you would safety protocols in a wet lab.

6.2 The Tool-Building Mindset

A tool is anything that makes a task easier when that task would otherwise be repetitive, hard, or impossible — a script, a function, a package, a spreadsheet, a checklist, or a CI workflow. Aaron Crosman's Tool Building Mindset[1] argues for taking that expansive view deliberately: everyone who works with technology should, can, and already does build tools for themselves, whether or not they think of themselves as a "developer."

The mindset in practice:

- **Notice the friction.** When you catch yourself repeating a task, following a fragile manual sequence, or avoiding something because it is tedious, treat that as a signal to build (or find) a tool for it.
- **Look before you build — then chip away.** Check whether an existing tool already solves the problem (see Section 7.6); when nothing quite fits, don't let "not worth the effort" stop you: a tool that removes part of an annoying task is already good enough to start with.
- **Start small and grow the tool as it earns it.** A copy-pasted snippet becomes a function; a function used across analyses becomes a package (see Section 6.4); a workflow copied between repositories becomes a shared, reusable one.

[1] https://spinningcode.org/2024/05/tool-building/

- **Build for your own repeated use first.** A tool that saves you an hour a week justifies itself quickly, and the discipline of packaging it (documentation, tests) makes it usable by the next lab member too.
- **Don't gold-plate.** The tool only needs to beat doing the task by hand; polish it further only when its use grows to warrant it.

Much of our lab's shared infrastructure came from exactly this pattern: the `{snapr}`[2] package packages the lab's snapshot-testing helpers, the `{lms}`[3] linter package packages our shared lint configuration, and the d-morrison/gha[4] repository collects reusable CI workflows and composite actions shared across the lab's repositories.

6.3 Essential R Package Development Tools

The following tools are essential for R package development in our lab:

6.3.1 usethis: Package Setup and Management

`usethis` automates common package development tasks:

```
# Install usethis
install.packages("usethis")

# Create a new package
usethis::create_package("~/myproject")

# Add common components
usethis::use_mit_license()          # Add a license
usethis::use_git()                  # Initialize git
usethis::use_github()               # Connect to GitHub
usethis::use_testthat()             # Set up testing infrastructure
usethis::use_vignette("intro")      # Create a vignette (shipped with package)
usethis::use_article("case-study")  # Create an article (website-only)
usethis::use_data_raw("dataset")    # Create data processing script
usethis::use_package("dplyr")       # Add a dependency
usethis::use_pipe()                 # Import magrittr pipe operator (no longer recommended)

# Increment version
usethis::use_version()              # Increment package version
```

6.3.2 devtools: Development Workflow

`devtools` provides the core development workflow:

[2] https://morrison-lab.github.io/snapr/
[3] https://github.com/UCD-SERG/lab-manual/tree/main/lms
[4] https://github.com/d-morrison/gha

```r
# Install devtools
install.packages("devtools")

# Load your package for interactive development
devtools::load_all()                # Like library(), but for development

# Documentation
devtools::document()                # Generate documentation from roxygen2

# Testing
devtools::test()                    # Run all tests
devtools::test_active_file()        # Run tests in current file

# Checking
devtools::check()                   # R CMD check (comprehensive validation)
devtools::check_man()               # Check documentation only

# Dependencies
devtools::install_dev_deps()        # Install all development dependencies

# Building
devtools::build()                   # Build package bundle
devtools::install()                 # Install package locally
```

Run `devtools::check()` locally before pushing

Running `R CMD check` workflows on GitHub Actions introduces runner startup overhead and delay, as virtual environments must be provisioned and system-level dependencies configured on each run. Always run `devtools::check()` locally before pushing to GitHub to catch errors, warnings, and NOTEs early while conserving CI resources.

6.3.3 pkgdown: Package Websites

`pkgdown` builds beautiful documentation websites from your package:

```r
# Install pkgdown
install.packages("pkgdown")

# Set up pkgdown
usethis::use_pkgdown()

# Build website locally
pkgdown::build_site()

# Preview in browser
pkgdown::build_site(preview = TRUE)

# Build components separately
```

```
pkgdown::build_reference()          # Function reference
pkgdown::build_articles()           # Vignettes
pkgdown::build_home()               # Home page from README
```

Configure your pkgdown site with `_pkgdown.yml`:

```
url: https://ucd-serg.github.io/YOURPROJECT

template:
  bootstrap: 5

reference:
  - title: "Data Preparation"
    desc: "Functions for preparing and cleaning data"
    contents:
    - prep_study_data
    - validate_data

  - title: "Analysis"
    desc: "Core analysis functions"
    contents:
    - run_primary_analysis
    - sensitivity_analysis

articles:
  - title: "Analysis Workflow"
    navbar: Analysis
    contents:
    - 01-data-preparation
    - 02-primary-analysis
    - 03-sensitivity-analysis
```

6.3.4 Alternatives to pkgdown

While `{pkgdown}`[5] is the standard tool for creating package documentation websites, several alternatives exist that may better suit specific needs, particularly for generating multiple output formats beyond HTML.

6.3.4.1 altdoc

`{altdoc}`[6] is a flexible alternative that supports multiple documentation frameworks, including Quarto, Docsify, MkDocs, and Docute. It is especially useful when you need to generate documentation in multiple formats.

Key features:

- Supports multiple documentation frameworks (Quarto, Docsify, MkDocs, Docute)

[5] https://pkgdown.r-lib.org/
[6] https://altdoc.etiennebacher.com/

- Renders Quarto and R Markdown vignettes to HTML websites
- When using Quarto as the framework, vignettes can be authored to support multiple output formats (HTML, PDF, DOCX, reveal.js presentations), and Quarto can include download links for alternative formats on the HTML site (see Quarto documentation on multiple formats[7])
- Handles function reference pages, README, NEWS, and other standard documentation
- Easy preview and deployment workflow

Basic usage:

```
# Install altdoc
install.packages("altdoc")

# Set up documentation with Quarto
altdoc::setup_docs(tool = "quarto_website")

# Or use other frameworks
# altdoc::setup_docs(tool = "docsify")
# altdoc::setup_docs(tool = "mkdocs")
# altdoc::setup_docs(tool = "docute")

# Render documentation
altdoc::render_docs()

# Preview in browser
altdoc::preview_docs()
```

When to choose altdoc:

- You want to use Quarto's modern publishing system for your documentation website
- You need vignettes that can be downloaded in multiple formats (when authored with Quarto's multi-format support)
- You prefer a different documentation framework than pkgdown's Bootstrap-based approach (Docsify, MkDocs, or Docute)
- You need more flexibility in site design and structure

6.3.4.2 pkgsite

`{pkgsite}`[8] provides a minimal, lightweight alternative to pkgdown, focusing on simplicity and ease of customization.

Key features:

- Minimal CSS framework (no Bootstrap)
- Simple, clean design that is easy to customize
- Similar `build_site()` function to pkgdown
- Lightweight and fast
- Can be published via GitHub Pages or other static site hosts

[7] https://quarto.org/docs/output-formats/html-multi-format.html
[8] https://github.com/pachadotdev/pkgsite

Basic usage:

```
# Install pkgsite (not on CRAN as of early 2026)
pak::pkg_install("pachadotdev/pkgsite")

# Build and preview site
pkgsite::build_site(preview = TRUE)

# Build with custom URL and lazy rebuilding
pkgsite::build_site(
  # Replace with your real docs domain.
  url = "https://example.com",
  lazy = TRUE,
  preview = TRUE
)
```

When to choose pkgsite:

- You want a minimal, lightweight documentation site
- You prefer to customize CSS styling from scratch
- You don't need the extensive features of pkgdown
- You want faster build times for simple packages

6.3.4.3 Quarto for Package Documentation

While not a dedicated package documentation tool, Quarto can be used to create sophisticated documentation websites for R packages, particularly when combined with custom scripts or tools like `{ecodown}`[9] (experimental and intended for internal use).

Discussion and resources:

- Quarto discussion on pkgdown alternative[10]
- pkgdown itself now supports Quarto vignettes (as of version 2.1.0)

When to consider Quarto:

- You want complete control over site structure and design
- You're already using Quarto for other documentation
- You need advanced publishing features beyond what pkgdown offers
- You're comfortable writing custom scripts for function reference extraction

6.3.4.4 Comparison Summary

[9] https://github.com/edgararuiz/ecodown
[10] https://github.com/orgs/quarto-dev/discussions/1591

Tool	Complexity	Website Output	Multi-format Support	Customization	Best For
pkgdown	Standard	HTML	Limited (via Quarto vignettes)	Template-based	Most R packages, standard documentation needs
altdoc	Moderate	HTML	Yes (via Quarto-authored vignettes with download links)	Framework-dependent	Quarto-based workflows, flexible frameworks
pkgsite	Minimal	HTML	No	High (simple CSS)	Lightweight sites, custom styling
Quarto	Advanced	HTML (for websites)	Yes (native - can output to PDF, DOCX, reveal.js, EPUB)	Complete	Full control, advanced features

Recommendation: Use pkgdown for most standard R package documentation needs. Consider altdoc when you prefer Quarto's publishing system or want flexibility to choose between documentation frameworks (Quarto, Docsify, MkDocs, Docute). If you choose altdoc with Quarto, you can author vignettes to provide downloadable PDF and DOCX versions alongside the HTML website. Choose pkgsite for minimalist websites with easy CSS customization. Use Quarto directly only if you need complete control and are comfortable with more complex setup.

6.4 Complete Package Development Workflow

Here's the typical workflow for developing an R package in our lab:

6.4.1 1. Initial Setup

Starting from a template (recommended):

Using our R package template is the fastest way to get started with a new R package, as it provides pre-configured settings, GitHub Actions workflows, and development tools:

- **UCD-SeRG R Package Template** - Our recommended template with pre-configured development tools and CI workflows:
  - Repository: https://github.com/UCD-SERG/rpt
  - Click "Use this template" → "Create a new repository" on GitHub
  - Clone your new repository and start developing

The template includes pre-configured:

- GitHub Actions workflows for R CMD check, test coverage, and pkgdown deployment
- Development tools setup (`{usethis}`[11], `{devtools}`[12], `{roxygen2}`[13])
- Testing infrastructure (`{testthat}`[14])
- Code styling and linting configurations
- Package documentation structure

While the template jumpstarts your project with up-to-date configuration and workflow files, you should still come up to speed on what all the config files do so you can modify and debug them as needed. The template serves as a central location for the most current versions of these files and best practices.

Starting from scratch:

If you prefer to start from scratch or need to understand each setup step, you can create a new package manually:

```
# Create package structure
usethis::create_package("~/myproject")

# Set up infrastructure
usethis::use_git()
usethis::use_github()
usethis::use_testthat()
usethis::use_pkgdown()
usethis::use_mit_license()
usethis::use_readme_rmd()
```

6.4.2 2. Add Dependencies

```
# Add packages your project depends on
usethis::use_package("dplyr")
usethis::use_package("ggplot2")
usethis::use_package("readr")

# Add packages only needed for development/testing
usethis::use_package("testthat", type = "Suggests")
```

6.4.3 3. Write Functions

Create functions in `R/` directory with roxygen2 documentation:

[11] https://usethis.r-lib.org/
[12] https://devtools.r-lib.org/
[13] https://roxygen2.r-lib.org/
[14] https://testthat.r-lib.org/

```r
#' Prepare Study Data
#'
#' Clean and prepare raw study data for analysis.
#'
#' @param raw_data A data frame containing raw study data
#' @param validate Logical; whether to run validation checks
#'
#' @returns A cleaned data frame ready for analysis
#'
#' @examples
#' raw_data <- read_csv("data.csv")
#' clean_data <- prep_study_data(raw_data)
#'
#' @export
prep_study_data <- function(raw_data, validate = TRUE) {
  # Function implementation
}
```

6.4.4 4. Document

```r
# Generate documentation from roxygen2 comments
devtools::document()
```

6.4.5 5. Test

Create tests in `tests/testthat/`:

```r
# tests/testthat/test-data_prep.R
test_that("prep_study_data handles missing values", {
  raw_data <- data.frame(x = c(1, NA, 3))
  result <- prep_study_data(raw_data)
  expect_false(anyNA(result$x))
})
```

Run tests:

```r
devtools::test()
```

6.4.6 6. Check

```r
# Comprehensive package check
devtools::check()
```

Fix any warnings or errors before proceeding.

6.4.7 7. Build Documentation Site

```
pkgdown::build_site()
```

6.4.8 8. Share and Publish

```
# Push to GitHub
# The pkgdown site can be automatically deployed to GitHub Pages
# using GitHub Actions
```

6.5 Organizing scripts

Just as your data "flows" through your project, data should flow naturally through a script. Very generally, you want to:

1. describe the work completed in the script in a comment header
2. source your configuration file (`0-config.R`)
3. load all your data
4. do all your analysis/computation
5. save your data.

Each of these sections should be "chunked together" using comments. See this file[15] for a good example of how to cleanly organize a file in a way that follows this "flow" and functionally separate pieces of code that are doing different things.

6.6 Testing Requirements

ALWAYS establish tests BEFORE modifying functions. This ensures changes preserve existing behavior and new behavior is correctly validated.

6.6.1 Add New Tests for New Features

When a PR adds a feature, give it a new test rather than editing an existing test's inputs or expectations to also cover it. Repurposing a test this way folds two changes into one diff — the reviewer can no longer tell, at a glance, whether the edited assertions still protect the old behavior or now only protect the new one.

For example, suppose a `run_model()` test originally covered the `"power"` method and gets edited in place to cover `"exponential"` instead, rather than gaining a new test alongside it:

[15] https://github.com/kmishra9/Flu-Absenteeism/blob/master/Master's%20Thesis%20-%20Spatial%20Epidemiology%20of%20Influenza/2a%20-%20Statistical-Inputs.R

```r
# Avoid --- repurposing an existing test to also cover a new method
test_that("run_model produces expected output", {
  result <- run_model(data, method = "exponential") # was "power"
  expect_snapshot_value(result)
})

# Preferred --- a new test for the new feature
test_that("run_model handles the exponential method", {
  result <- run_model(data, method = "exponential")
  expect_snapshot_value(result)
})
```

If the new feature makes an existing test redundant, leave that redundancy alone for now. Remove or consolidate redundant tests in a separate, focused PR, so a reviewer can evaluate "does the new feature work" and "which old tests are now redundant" as two independent questions.

6.6.2 When to Use Snapshot Tests

Use snapshot tests (`expect_snapshot()`, `expect_snapshot_value()`) when:

- Testing complex data structures (data frames, lists, model outputs)
- Validating statistical results where exact values may vary slightly
- Output format stability is important

```r
test_that("prep_study_data produces expected structure", {
  result <- prep_study_data(raw_data)
  expect_snapshot_value(result, style = "serialize")
})
```

Our lab's `{snapr}`[16] package (on CRAN[17]) adds convenience wrappers on top of `{testthat}`[18]'s snapshot functions: `expect_snapshot_data()` for data frames (adapted, with permission, from the `{ssdtools}`[19] package) and `expect_snapshot_object()`, which generalizes it to any R object.

```r
test_that("prep_study_data output is stable", {
  result <- prep_study_data(raw_data)
  snapr::expect_snapshot_data(result, name = "prepped_study_data")
})
```

[16] https://morrison-lab.github.io/snapr/
[17] https://cran.r-project.org/package=snapr
[18] https://testthat.r-lib.org/
[19] https://cran.r-project.org/package=ssdtools

6.6.3 When to Use Explicit Value Tests

Use explicit tests (`expect_equal()`, `expect_identical()`) when:

- Testing simple scalar outputs
- Validating specific numeric thresholds
- Testing Boolean returns or categorical outputs

```
test_that("calculate_mean returns correct value", {
  expect_equal(calculate_mean(c(1, 2, 3)), 2)
  expect_equal(calculate_ratio(3, 7), 0.4285714, tolerance = 1e-6)
})
```

6.6.4 Choosing Between Snapshot and Explicit Tests

The two styles answer different questions. A snapshot test asks "does this output still match what it looked like before?"; an explicit test asks "does this specific property equal this exact value?". That difference drives every practical trade-off, as Table 6.1 summarizes:

Table 6.1: Trade-offs between snapshot and explicit tests

	Snapshot test	Explicit test
Conciseness	High: one snapshot call pins the whole output, standing in for a long list of assertions, and the first run records the reference automatically	Lower: you write an assertion per property you care about, which adds up fast for complex outputs
Readability	Lower: the test shows *that* output is pinned, not *why*	Higher: each assertion documents an intended behavior
When output legitimately changes	One review-and-accept of the new snapshot	Rewrite the affected assertions by hand
Failure diagnosis	A diff of the whole output; the relevant change may be buried	Points at the exact property that broke

Each style has a characteristic failure mode to guard against:

- **Snapshot blindness.** Because a snapshot diff can be long, it is tempting to accept an updated snapshot without reading it — which silently converts a regression into the new expected output. Treat every `testthat::snapshot_accept()` like a code review: read the diff (`testthat::snapshot_review()`) before accepting.
- **Brittle snapshots.** Non-deterministic output (timestamps, random numbers, file paths) makes a snapshot fail on every run; seed or fix the inputs first (see the seeding advice under Section 6.6.5).
- **Assertion sprawl.** Explicitly asserting every field of a large object takes a page of fragile code — exactly the case where one snapshot is clearer.

- **Over-specified assertions.** Explicit tests pinned to incidental details (element order that doesn't matter, formatting) fail on harmless refactors and train you to distrust them.

In practice the strongest suites combine both: explicit assertions pin the core logic (the value a calculation must produce, the condition an input must trigger), and a snapshot guards the surrounding large structure (the full prepared data frame, a printed summary) against unnoticed drift. The `{testthat}` snapshotting vignette[20] covers the workflow — recording, reviewing, and accepting snapshots — in more depth.

6.6.5 Testing Best Practices

- **Seed randomness**: Use `withr::local_seed()` for reproducible tests
- **Use small test cases**: Keep tests fast
- **Test edge cases**: Missing values, empty inputs, boundary conditions
- **Test errors**: Verify functions fail appropriately with invalid input

When testing conditions with `{testthat}`[21], assert on the condition `class` whenever possible. Message text (`regexp`) can be checked as a secondary assertion, but should not be the only signal.

```
test_that("prep_study_data handles edge cases", {
  # Empty input
  expect_error(
    prep_study_data(data.frame()),
    class = "prep_study_data_empty_input_error"
  )

  # Missing required columns
  expect_error(
    prep_study_data(data.frame(x = 1)),
    class = "prep_study_data_missing_columns_error",
    regexp = "Missing required columns"
  )

  # Valid input with missing values
  result <- prep_study_data(data.frame(id = 1:3, value = c(1, NA, 3)))
  expect_true(all(!is.na(result$value)))
})
```

6.7 Test-Driven Development

Test-driven development (TDD)[22] inverts the usual order of coding and testing: you write the test for a behavior *before* writing the code that implements it. The practice was popularized by Kent Beck, who developed it as part of extreme programming and described it in *Test-Driven Development: By Example* (2002).

[20] https://testthat.r-lib.org/articles/snapshotting.html
[21] https://testthat.r-lib.org/
[22] https://en.wikipedia.org/wiki/Test-driven_development

Work proceeds in a short, repeated cycle (often called *red-green-refactor*):

1. **Red.** Write a small test for the next behavior you want, and run it to confirm it fails — this proves the test can actually detect the missing behavior.
2. **Green.** Write the simplest code that makes the test pass, resisting the urge to build ahead of the tests.
3. **Refactor.** Clean up the code (and the tests) while everything stays green, leaning on the tests to catch anything the cleanup breaks.

Why it helps:

- The test pins down what the function *should* do before you are invested in what it happens to do, turning requirements into executable checks.
- Every unit of behavior ends up covered by construction, so later changes get regression protection for free.
- Code written to be testable tends toward small, single-purpose functions with explicit inputs and outputs — the same design our function-decomposition

  (1) guidance pushes toward.

You do not have to practice strict TDD to benefit from it. Our testing guidance (Section 6.6) already applies its core move — establish tests *before* modifying existing functions — and writing the test first is just that same discipline applied to new code as well. In R packages, the mechanics are light: `usethis::use_test()` creates the matching `{testthat}`[23] test file, and `devtools::test()` runs the suite (see R Packages, ch. "Testing basics"[24]).

6.8 Benchmarking

> **Note**
>
> This section draws from the Measuring Performance[a] and Improving Performance[b] chapters in *Advanced R* by Hadley Wickham, along with documentation for `{bench}`[c] and `{profvis}`[d].
>
> [a] https://adv-r.hadley.nz/perf-measure.html
> [b] https://adv-r.hadley.nz/perf-improve.html
> [c] https://bench.r-lib.org/
> [d] https://profvis.r-lib.org/

Performance optimization starts with measurement. Benchmarking helps identify bottlenecks, compare alternative implementations, and ensure your code meets performance requirements. This section covers when and how to benchmark R code, with a focus on integration with package development workflows.

6.8.1 When to Benchmark

Benchmark when:

[23] https://testthat.r-lib.org/
[24] https://r-pkgs.org/testing-basics.html

- **Choosing between implementations**: Compare different approaches to the same problem
- **Optimizing critical paths**: Identify and improve performance bottlenecks
- **Preventing regressions**: Ensure new code doesn't slow down existing functionality
- **Processing large datasets**: Verify scalability for biostatistics/epidemiology workflows
- **Implementing computational methods**: Test algorithms involving bootstrapping, simulation, or resampling

Don't benchmark prematurely. Write correct, readable code first, then measure to find what actually needs optimization.

6.8.2 Setting Up Benchmarking Infrastructure

Organize benchmarking code separately from unit tests:

```
mypackage/
  tests/
    testthat/          # Unit tests (run by R CMD check)
    benchmarks/        # Benchmarking scripts (run manually or in CI)
      01-data-prep.R
      02-analysis.R
```

Add benchmarking dependencies to `DESCRIPTION`:

```
# Add bench and profvis as development dependencies
usethis::use_package("bench", type = "Suggests")
usethis::use_package("profvis", type = "Suggests")

# Add any packages used in benchmarks
# For example, if comparing with data.table or gbm:
# usethis::use_package("data.table", type = "Suggests")
# usethis::use_package("gbm", type = "Suggests")
```

Create a benchmark template:

```
# tests/benchmarks/01-data-prep.R
library(bench)
library(mypackage)

# Generate test data
n <- 10000
test_data <- data.frame(
  id = 1:n,
  exposure = rnorm(n),
  outcome = rbinom(n, 1, 0.3)
)

# Compare implementations
```

```r
results <- bench::mark(
  original = prep_data_v1(test_data),
  optimized = prep_data_v2(test_data),
  check = FALSE,  # Set TRUE to verify outputs are identical
  iterations = 100,
  time_unit = "ms"
)

print(results)
```

6.8.3 Using bench for Performance Comparisons

{bench}[25] provides accurate performance measurements and makes it easy to compare multiple implementations.

Basic usage:

```r
library(bench)

# Compare different approaches
results <- bench::mark(
  base_subset = data[data$group == "treatment", ],
  dplyr_filter = dplyr::filter(data, group == "treatment"),
  data.table = dt[group == "treatment"],
  iterations = 100
)

# View results
print(results)
#> # A tibble: 3 x 6
#>   expression        min   median `itr/sec` mem_alloc `gc/sec`
#>   <bch:expr>   <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
#> 1 base_subset    1.2ms    1.4ms       714.    1.5MB     2.14
#> 2 dplyr_filter   2.1ms    2.3ms       435.    2.1MB     4.35
#> 3 data.table   850.0us  950.0us      1053.  800.0KB     0.00

# Plot results
plot(results)
```

Key features:

- **Accurate timing**: Accounts for overhead and runs multiple iterations
- **Memory tracking**: Shows memory allocation for each approach
- **Garbage collection**: Reports GC frequency
- **Output verification**: Optional checking that all implementations produce identical results

Example for biostatistics workflow:

[25] https://bench.r-lib.org/

```r
# Compare propensity score estimation methods
library(bench)

# Simulate cohort data
n <- 5000
cohort <- data.frame(
  age = rnorm(n, 50, 15),
  bmi = rnorm(n, 27, 5),
  treatment = rbinom(n, 1, 0.5),
  outcome = rbinom(n, 1, 0.3)
)

# Note: Install gbm first if not already installed
# usethis::use_package("gbm", type = "Suggests")

# Compare GLM vs GBM for propensity scores
ps_benchmark <- bench::mark(
  glm_method = {
    model <- glm(treatment ~ age + bmi,
                 data = cohort,
                 family = binomial())
    predict(model, type = "response")
  },
  gbm_method = {
    model <- gbm::gbm(treatment ~ age + bmi,
                      data = cohort,
                      distribution = "bernoulli",
                      n.trees = 100,
                      verbose = FALSE)
    predict(model, n.trees = 100, type = "response")
  },
  check = FALSE,
  iterations = 50
)

print(ps_benchmark)
```

6.8.4 Using profvis for Memory and CPU Profiling

{profvis}[26] identifies where your code spends time and allocates memory. Use it to find bottlenecks in existing functions.

Basic usage:

```r
library(profvis)

# Profile a function
profvis({
```

[26] https://profvis.r-lib.org/

```
  data <- prep_study_data(raw_data)
  results <- run_primary_analysis(data)
})
```

The profvis output shows:

- **Flame graph**: Visual representation of time spent in each function
- **Data view**: Line-by-line time and memory usage
- **Memory profile**: Allocation patterns over time

Example for data preparation:

```
library(profvis)

profvis({
  # Profile data cleaning pipeline
  cleaned <- raw_data |>
    dplyr::filter(!is.na(id)) |>
    dplyr::mutate(
      age_group = cut(age, breaks = c(0, 18, 65, Inf)),
      bmi_category = cut(bmi, breaks = c(0, 18.5, 25, 30, Inf))
    ) |>
    dplyr::left_join(exposure_data, by = "id") |>
    dplyr::group_by(age_group) |>
    dplyr::summarize(
      n = n(),
      mean_bmi = mean(bmi, na.rm = TRUE)
    )
})
```

Interpreting profvis output:

- **Wide bars**: Functions taking substantial time
- **Tall stacks**: Deep call chains (potential for simplification)
- **Memory spikes**: Large allocations (consider chunking or data.table)

Common bottlenecks in epidemiology code:

- Repeated subsetting operations -> Use data.table or pre-filter
- Growing objects in loops -> Pre-allocate vectors
- Complex joins on large datasets -> Index properly or use data.table
- Unnecessary copies -> Use reference semantics where appropriate

6.8.5 Integration with Package Development Workflow

Integrate benchmarking into your development cycle:

1. Benchmark before optimization:

```
# tests/benchmarks/baseline.R
# Run this before making changes to establish baseline

library(bench)
library(mypackage)

baseline <- bench::mark(
  current_implementation = analyze_cohort(test_data),
  iterations = 100
)

saveRDS(baseline, "tests/benchmarks/baseline.rds")
```

2. Profile to identify bottlenecks:

```
library(profvis)

profvis({
  analyze_cohort(test_data)
})
# Identify which functions are slow
```

3. Optimize and re-benchmark:

```
# After optimization
library(bench)

baseline <- readRDS("tests/benchmarks/baseline.rds")

comparison <- bench::mark(
  baseline = analyze_cohort_v1(test_data),
  optimized = analyze_cohort_v2(test_data),
  check = FALSE,
  iterations = 100
)

print(comparison)
```

4. Document performance characteristics:

```
#' Analyze cohort data
#'
#' @param data A data frame with cohort information
#' @return Analysis results
#'
#' @details
#' Performance characteristics (as of 2025-01):
#' - Typical runtime: ~50ms for 10,000 observations
#' - Memory usage: ~5MB per 10,000 observations
```

```
#' - Scales linearly with sample size
#'
#' @examples
#' \dontrun{
#' results <- analyze_cohort(cohort_data)
#' }
#' @export
analyze_cohort <- function(data) {
  # Implementation
}
```

6.8.6 CI/CD Integration for Automated Benchmarking

Set up GitHub Actions to track performance over time.

Create `.github/workflows/benchmark.yaml`:

Key considerations for CI benchmarks:

- **Variability**: CI runners have variable performance; use thresholds (e.g., >10% regression)
- **Baseline storage**: Commit baseline results or use GitHub Actions artifacts
- **Selective running**: Only benchmark on specific branches or when performance-critical files change
- **Manual triggers**: Use `workflow_dispatch` for on-demand benchmarking
- **Security**: For production workflows, consider pinning action versions to commit SHAs instead of tags (see the wai site's "Best Practices for Safe and Successful Use" section[27])

Alternative: Comment benchmark results on PRs:

Use the `{touchstone}`[28] package for more sophisticated CI benchmarking with automated PR comments.

6.8.7 Performance Testing with testthat

For critical performance requirements, add performance tests to your test suite.

Example performance test:

```
# tests/testthat/test-performance.R

test_that("data preparation meets performance requirements", {
  skip_on_cran()  # Skip on CRAN (timing-based tests can be flaky)

  # Run on CI only if BENCHMARK_ON_CI is set
  if (identical(Sys.getenv("CI"), "true") &&
      !identical(Sys.getenv("BENCHMARK_ON_CI"), "true")) {
    skip("Skipping performance test on CI")
```

[27] https://morrison-lab.github.io/wai/chapters/coding-agents.html#sec-ai-best-practices
[28] https://github.com/lorenzwalthert/touchstone

```
  }

  n <- 10000
  test_data <- generate_test_cohort(n)

  # Measure execution time
  timing <- bench::mark(
    prep_study_data(test_data),
    iterations = 10,
    check = FALSE
  )

  # Require median time under threshold
  max_time_ms <- 100
  median_time_ms <- as.numeric(timing$median) * 1000

  expect_true(
    median_time_ms < max_time_ms,
    info = sprintf(
      "prep_study_data took %.1f ms (threshold: %.1f ms)",
      median_time_ms,
      max_time_ms
    )
  )
})

test_that("analysis scales linearly with sample size", {
  skip_on_cran()

  # Run on CI only if BENCHMARK_ON_CI is set
  if (identical(Sys.getenv("CI"), "true") &&
      !identical(Sys.getenv("BENCHMARK_ON_CI"), "true")) {
    skip("Skipping performance test on CI")
  }

  # Test at different sample sizes
  sizes <- c(1000, 5000, 10000)

  timings <- vapply(sizes, function(n) {
    data <- generate_test_cohort(n)
    result <- bench::mark(
      analyze_cohort(data),
      iterations = 10
    )
    as.numeric(result$median)
  }, numeric(1))

  # Check approximate linearity (R^2 > 0.95)
  model <- lm(timings ~ sizes)
  r_squared <- summary(model)$r.squared
```

```
  expect_true(
    r_squared > 0.95,
    info = sprintf("Scaling R^2 = %.3f (expected > 0.95)", r_squared)
  )
})
```

When to use performance tests:

- Functions with documented performance requirements
- Critical paths that must stay fast
- Code that processes large datasets

When not to use performance tests:

- Functions without specific performance needs
- Tests that would be flaky due to system variability
- Development environments with limited resources

See Section 6.6 for more on testing with `{testthat}`[29].

6.8.8 Best Practices

Focus benchmarks on realistic scenarios:

```
# Good: Realistic data size
n <- 50000  # Typical cohort size in our studies
test_data <- generate_realistic_cohort(n)

# Avoid: Unrealistically small data
n <- 10
```

Establish baselines:

Before optimizing, measure current performance to understand the improvement.

```
# Document baseline
baseline <- bench::mark(current_implementation(data), iterations = 100)
saveRDS(baseline, "benchmarks/baseline-2025-01.rds")
```

Compare on equal footing:

```
# Good: Same random seed, same data
set.seed(123)
data <- generate_test_data(10000)

bench::mark(
  method_a = analyze_a(data),
  method_b = analyze_b(data),
```

[29] https://testthat.r-lib.org/

```
  check = FALSE
)

# Avoid: Different random data
bench::mark(
  method_a = analyze_a(generate_test_data(10000)),
  method_b = analyze_b(generate_test_data(10000))
)
```

Benchmark critical paths only:

Don't optimize everything—focus on code that:

- Runs frequently
- Processes large datasets
- Is called in loops or simulations
- Has noticeable user-facing delays

Use appropriate sample sizes:

```
# For data prep: use typical dataset size
cohort_data <- generate_cohort(n = 50000)

# For simulation: use realistic iteration counts
n_iterations <- 1000
```

Document optimization decisions:

```
# In package documentation or vignette

# We use data.table for joins because:
# - Benchmarks show 5x speedup over dplyr for n > 100,000
# - Typical cohort sizes: 50,000 - 500,000 observations
# - See tests/benchmarks/join-comparison.R for details
```

6.8.9 Additional Resources

- `{bench}`[30] - Accurate benchmarking
- `{profvis}`[31] - Interactive profiling
- `{memoise}`[32] - Function-call caching to eliminate repeated bottlenecks (Section 6.37.2)
- Measuring Performance[33] chapter in Advanced R
- Improving Performance[34] chapter in Advanced R
- `{touchstone}`[35] - CI benchmarking with PR comments

[30] https://bench.r-lib.org/
[31] https://profvis.r-lib.org/
[32] https://memoise.r-lib.org/
[33] https://adv-r.hadley.nz/perf-measure.html
[34] https://adv-r.hadley.nz/perf-improve.html
[35] https://github.com/lorenzwalthert/touchstone

6.9 Iterative Operations

When applying analyses with different variations (outcomes, exposures, subgroups), use functional programming approaches:

6.9.1 `lapply()` and `sapply()`

```
# Apply function to each element
results <- lapply(outcomes, function(y) {
  run_analysis(data, outcome = y)
})

# Simplify to vector if possible
summary_stats <- sapply(data_list, mean)
```

6.9.2 `purrr::map()` Family

The `purrr` package provides type-stable alternatives:

```
library(purrr)

# Always returns a list
results <- map(outcomes, ~ run_analysis(data, outcome = .x))

# Type-specific variants
means <- map_dbl(data_list, mean)        # Returns numeric vector
models <- map(splits, ~ lm(y ~ x, data = .x))  # Returns list of models
```

6.9.3 `base::Vectorize()` vs `purrr::map_*()`

`base::Vectorize()` and {purrr}[36] solve different problems.

- Use `base::Vectorize()` when you already have a scalar function and want a quick base-R wrapper that accepts vector inputs.
- Use `purrr::map_*()` when you are iterating over lists or list-columns and want explicit, type-stable outputs (`map_dbl()`, `map_int()`, etc.) that compose naturally in tidyverse pipelines.

`base::Vectorize()` is often convenient for small, local wrappers. In contrast, {purrr}[37] is usually more idiomatic for analysis workflows because output types are explicit and iteration intent is clearer.

Also note that neither approach is parallel by default. For parallel execution, use `furrr::future_map()` from {furrr}[38].

[36] https://purrr.tidyverse.org/
[37] https://purrr.tidyverse.org/
[38] https://furrr.futureverse.org/

6.9.4 `purrr::pmap()` for Multiple Arguments

When iterating over multiple parameter lists:

```
params <- tibble(
  outcome = c("outcome1", "outcome2", "outcome3"),
  exposure = c("exp1", "exp2", "exp3"),
  covariate_set = list(c("age", "sex"), c("age"), c("age", "sex", "bmi"))
)

results <- pmap(params, function(outcome, exposure, covariate_set) {
  run_analysis(
    data = study_data,
    outcome = outcome,
    exposure = exposure,
    covariates = covariate_set
  )
})
```

6.9.5 Parallel Processing

For computationally intensive work, use `future` and `furrr`:

```
library(future)
library(furrr)

# Set up parallel processing
plan(multisession, workers = availableCores() - 1)

# Parallel version of map()
results <- future_map(large_list, time_consuming_function, .progress = TRUE)
```

6.10 Reading and Saving Data

6.10.1 RDS Files (Preferred)

Use RDS format for R objects:

```
# Save single object
readr::write_rds(analysis_results, here("results", "analysis.rds"))

# Read back
results <- readr::read_rds(here("results", "analysis.rds"))
```

Avoid `.RData` files because: - You can't control object names when loading - Can't load individual objects - Creates confusion in older code

6.10.2 CSV Files

For tabular data that may be shared with non-R users:

```
# Write
readr::write_csv(data, here("data-raw", "clean_data.csv"))

# Read
data <- readr::read_csv(here("data-raw", "clean_data.csv"))

# For very large files, use data.table
data.table::fwrite(large_data, "big_file.csv")
data <- data.table::fread("big_file.csv")
```

6.11 Version Control and Collaboration

6.11.1 Git Branches and Tags

To understand how Git tracks history, think of commits as Polaroid snapshots of your repository connected to their parent snapshots by strings. Branches and tags are both labels (like sticky notes) pointing to specific snapshots in this history graph:

- **Branches are movable labels**: A branch label points to the latest commit in a line of development. When you make a new commit while working on a branch, Git automatically advances the branch label forward to point to the new commit. Think of a branch as a sticky note that moves automatically as history grows.
- **Tags are permanent labels**: A tag marks a specific point in time (such as a published release like `v1.0.0` or a manuscript submission). Tags do not move when new commits are added; they remain anchored to their specific commit. Git provides two types of tags: *lightweight tags* (simple references directly pointing to a commit) and *annotated tags* (full Git objects storing a tagger identity, timestamp, and release message; recommended for official versions and public releases).

For deeper background on Git's object model and branching mechanisms, consult the official Git Branching in a Nutshell[39] and Git Tagging[40] guides in *Pro Git* by Scott Chacon and Ben Straub.

6.11.1.1 Practical branch and tag commands

```
# Create and switch to a new feature branch
git switch -c feat/new-analysis

# Publish the feature branch to GitHub and set upstream tracking
git push -u origin feat/new-analysis

# View all local and remote-tracking branches
```

[39] https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell
[40] https://git-scm.com/book/en/v2/Git-Basics-Tagging

```
git branch -a

# Create a lightweight tag at the current commit
git tag v0.1.0

# Create an annotated tag with a release message (recommended for releases)
git tag -a v1.0.0 -m "Release version 1.0.0"

# Push a specific tag to GitHub
git push origin v1.0.0

# Push reachable annotated tags alongside branch commits (safer than pushing all tags)
git push --follow-tags
```

6.11.2 Version Numbers

Follow semantic versioning (MAJOR.MINOR.PATCH):

- Development versions: `0.0.0.9000`, `0.0.0.9001`, etc.
- First release: `0.1.0`
- Bug fixes: increment PATCH (e.g., `0.1.0` → `0.1.1`)
- New features: increment MINOR (e.g., `0.1.1` → `0.2.0`)
- Breaking changes: increment MAJOR (e.g., `0.2.0` → `1.0.0`)

> **!** Keep the development version ahead
>
> Always keep the development version ahead of the main branch version. When working on a development branch, ensure the version in `DESCRIPTION` is higher than the version on `main`.

```
# Increment version
usethis::use_version()
```

See R Packages - Version numbers[41] for full guidelines.

6.11.3 NEWS File

Document all user-facing changes in `NEWS.md`.

Formatting conventions:

- Use `(#123)` notation to link to related issues and pull requests (GitHub uses a shared number space for both)
- Use `@username` to credit **external** contributors only (not internal team members)

[41] https://r-pkgs.org/lifecycle.html#sec-lifecycle-version-number

```
# myproject 0.2.0

## New features

- Added function for data validation (#45).
- Improved error messages (#52, @external-contributor).

## Bug fixes

- Fixed issue with missing values (#38).
- Corrected calculation error in summary stats (#41).
```

See R Packages - NEWS.md[42] for full conventions.

6.11.4 README.Rmd and README.md

If your project uses `README.Rmd` (a README with executable R code), **always edit `README.Rmd`, never `README.md` directly.** `README.md` is auto-generated from `README.Rmd`.

To regenerate `README.md` after editing `README.Rmd`, use `devtools::build_readme()` (preferred for R packages):

```
devtools::build_readme()
```

Alternatively, you can use `{rmarkdown}`[43]'s `rmarkdown::render("README.Rmd")` if `{devtools}`[44] is not available.

6.12 Quality Assurance Checklist

Before requesting human review on a pull request or finalizing analysis, verify:

- ☐ All functions have complete roxygen2 documentation
- ☐ All functions have corresponding tests
- ☐ `devtools::document()` has been run
- ☐ `devtools::test()` passes with no failures
- ☐ `devtools::check()` passes with no errors, warnings, or notes
- ☐ `lintr::lint_package()` shows no issues (or only acceptable ones); run `pkgload::load_all()` first so `object_usage_linter()` checks current source
- ☐ `spelling::spell_check_package()` passes
- ☐ Version number has been incremented
- ☐ `NEWS.md` has been updated with changes
- ☐ `README.Rmd` has been updated (if needed) and `README.md` regenerated via `devtools::build_readme()`
- ☐ `pkgdown::build_site()` builds successfully

[42] https://r-pkgs.org/other-markdown.html#sec-news
[43] https://rmarkdown.rstudio.com/
[44] https://devtools.r-lib.org/

- ☐ All changes committed and pushed to GitHub
- ☐ Copilot review completed iteratively until no valuable suggestions remain (typically 1-3 iterations, with all comments addressed or dismissed)

6.13 Automated Code Styling

6.13.1 RStudio Built-in Formatting

Use RStudio's built-in autoformatter (keyboard shortcut: `CMD-Shift-A` or `Ctrl-Shift-A`) to quickly format highlighted code.

6.13.2 styler Package

For automated styling of entire projects:

```
# Install styler
install.packages("styler")

# Style all files in R/ directory
styler::style_dir("R/")

# Style entire package
styler::style_pkg()

# Note: styler modifies files in-place
# Always use with version control so you can review changes
```

6.13.3 lintr Package

For checking code style without modifying files:

```
# Install lintr (and pkgload, used below)
install.packages(c("lintr", "pkgload"))

# For package code, load the package first so object_usage_linter()
# checks your current source (see the style guide for details)
pkgload::load_all()

# Lint the entire package
lintr::lint_package()

# Lint a specific file
lintr::lint("R/my_function.R")
```

The linter checks for:

- Unused variables
- Improper whitespace

- Line length issues
- Style guide violations

You can customize linting rules by creating a `.lintr` file in your project root.

6.13.4 jarl (Fast Rust-Powered Linter)

`{jarl}`[45] is a high-performance R linter written in Rust by Etienne Bacher. It checks many common style and syntax rules significantly faster than `{lintr}`, making it well-suited for fast pre-commit hooks and large codebases.

See also Automated Tools[46].

6.14 Documenting your code

6.14.1 Function headers

Every function you write must include documentation to describe its purpose, inputs, and outputs. For any reproducible workflows, this is essential, because R is dynamically typed. This means you can pass a `string` into an argument that is meant to be a `data.table`, or a `list` into an argument meant for a `tibble`. It is the responsibility of a function's author to document what each argument is meant to do and its basic type.

We use `{roxygen2}`[47] (Wickham et al. 2024) for function documentation. Roxygen2 allows you to describe your functions in special comments next to their definitions, and automatically generates R documentation files (`.Rd` files) and helps manage your package `NAMESPACE`. The roxygen2 format uses `#'` comments placed immediately before the function definition.

Here is an example of documenting a function using roxygen2:

```
#' Calculate flu season means by site
#'
#' Make a dataframe with rows for flu season and site
#' containing the number of patients with an outcome, the total patients,
#' and the percent of patients with the outcome.
#'
#' @param data A data frame with variables flu_season, site, studyID, and yname
#' @param yname A string for the outcome name
#' @param silent A boolean specifying whether to suppress console output
#'   (default: TRUE)
#'
#' @returns A dataframe as described above
#'
#' @examples
#' calc_fluseas_mean(my_data, "hospitalized", silent = FALSE)
#'
```

[45] https://jarl.etiennebacher.com/
[46] @sec-style-auto-style
[47] https://roxygen2.r-lib.org/

```r
calc_fluseas_mean <- function(data, yname, silent = TRUE) {
  ### function code here

}
```

The roxygen2 header tells you what the function does, its various inputs, and how you might use it. Also notice that all optional arguments (i.e. ones with pre-specified defaults) follow arguments that require user input.

For more information on roxygen2 syntax and features, see https://roxygen2.r-lib.org/.

6.14.2 Using ... (dots) and `@inheritDotParams`

The ... argument (pronounced "dots" or "ellipsis") is a special R construct that allows functions to accept additional arguments that are passed to other functions. This is particularly useful when creating wrapper functions that call other functions internally.

When to use ...:

- You're creating a wrapper function that calls another function
- You want to allow users to pass additional arguments to an internal function
- You want to provide flexibility without explicitly listing all possible arguments

Basic example with ...:

```r
#' Plot data with custom ggplot2 styling
#'
#' A wrapper function that creates a scatter plot with custom theme settings.
#' Additional arguments are passed to ggplot2::geom_point().
#'
#' @param data A data frame containing the variables to plot
#' @param x A string specifying the x-axis variable name
#' @param y A string specifying the y-axis variable name
#' @param ... Additional arguments passed to ggplot2::geom_point()
#'
#' @returns A ggplot2 object
#'
#' @examples
#' # Pass color and size arguments to geom_point
#' plot_with_style(my_data, "age", "height", color = "blue", size = 3)
#'
plot_with_style <- function(data, x, y, ...) {
  ggplot2::ggplot(data, ggplot2::aes(.data[[x]], .data[[y]])) +
    ggplot2::geom_point(...) +
    ggplot2::theme_minimal()  # Apply a minimal theme
}
```

While the example above documents ... with a simple description, roxygen2 provides `@inheritDotParams` to automatically inherit parameter documentation from the function you're calling. This is more robust and maintainable because it automatically stays synchronized with the target function's documentation.

Using `@inheritDotParams`:

```
#' Plot data with custom ggplot2 styling
#'
#' A wrapper function that creates a scatter plot with custom theme settings.
#'
#' @param data A data frame containing the variables to plot
#' @param x A string specifying the x-axis variable name
#' @param y A string specifying the y-axis variable name
#' @inheritDotParams ggplot2::geom_point -mapping -data -stat -position
#'
#' @returns A ggplot2 object
#'
#' @examples
#' # Pass color and size arguments to geom_point
#' plot_with_style(my_data, "age", "height", color = "blue", size = 3)
#'
plot_with_style <- function(data, x, y, ...) {
  ggplot2::ggplot(data, ggplot2::aes(.data[[x]], .data[[y]])) +
    ggplot2::geom_point(...) +
    ggplot2::theme_minimal()  # Apply a minimal theme
}
```

The `@inheritDotParams` tag:

- Automatically imports parameter documentation from `ggplot2::geom_point()`
- Uses `-mapping -data -stat -position` to exclude parameters that don't make sense in this context
- Keeps documentation synchronized if the underlying function changes
- Makes it clear which function receives the `...` arguments

Best practices for `...`:

1. **Always document what receives the dots**: Use `@inheritDotParams` when passing to a specific function, or clearly describe where the arguments go
2. **Exclude irrelevant parameters**: Use the `-param_name` syntax to exclude parameters that don't apply
3. **Validate unexpected arguments**: Consider using the `{ellipsis}`[48] package to catch misspelled argument names:

   ```
   my_function <- function(x, y, ...) {
     ellipsis::check_dots_used()
     # function code
   }
   ```

4. **Consider alternatives**: If you're only passing a few specific arguments, it may be clearer to list them explicitly rather than using `...`

For more details on `@inheritDotParams`, see the roxygen2 documentation on inheriting parameters[49].

[48] https://ellipsis.r-lib.org/
[49] https://roxygen2.r-lib.org/articles/rd.html#inheriting-documentation

6.14.3 Documenting internal helpers: `@keywords internal` over `@noRd`

Document every function you write, including internal helpers that only your other package functions call. The question is *how* the documentation is exposed, not whether it exists.

A helper is kept internal by omitting `@export`, which leaves it out of the `export()` directives in `NAMESPACE` (the function is still in the package namespace, which is why `pkg:::fn` works). Once the function is non-exported, two roxygen tags control how its documentation is generated and indexed, and they behave differently:

Table 6.2: Comparison of `@noRd` and `@keywords internal` roxygen tags

Tag	Generates an `.Rd` file?	Appears in pkgdown reference index?	Accessible via `?pkg:::fn`?
`@noRd`	No	No	No
`@keywords internal`	Yes	No	Yes

Prefer `@keywords internal` for any helper that isn't part of the package's public API.

For more on these tags, see the roxygen2 documentation[50] and R Packages (2e): Function documentation[51]. You still get a real `.Rd` file — so future maintainers (or future-you) can run `?pkg:::internal_helper` and see the documentation — but the function stays out of the pkgdown reference and the package's main help page.

`@noRd` is for cases where you genuinely don't want documentation to exist at all (e.g., a one-line internal shim where the docstring would be longer than the function). Reach for it sparingly.

Example:

```
#' Compute the per-row score
#'
#' Internal worker for [public_summary()].
#'
#' @param row A one-row data frame.
#' @returns A numeric scalar.
#'
#' @keywords internal
score_one_row <- function(row) {
  ### function code here
}
```

> **Note**
>
> As someone trying to call a function, it is possible to access a function's documentation (and internal code) by `CMD-Left-Click`ing the function's name in RStudio

[50] https://roxygen2.r-lib.org/articles/rd.html
[51] https://r-pkgs.org/man.html

> **Note**
>
> Depending on how important your function is, the complexity of your function code, and the complexity of different types of data in your project, you can also add "type-checking" to your function with the `assertthat::assert_that()` function. You can, for example, `assert_that(is.data.frame(statistical_input))`, which will ensure that collaborators or reviewers of your project attempting to use your function are using it in the way that it is intended by calling it with (at the minimum) the correct type of arguments. You can extend this to ensure that certain assumptions regarding the inputs are fulfilled as well (i.e. that `time_column`, `location_column`, `value_column`, and `population_column` all exist within the `statistical_input` tibble).

6.14.4 Script headers

Every file in a project that doesn't have roxygen function documentation should at least have a header that allows it to be interpreted on its own. It should include the name of the project and a short description for what this file (among the many in your project) does specifically. You may optionally wish to include the inputs and outputs of the script as well, though the next section makes this significantly less necessary.

```
##############################################################################
# @Organization - Example Organization
# @Project - Example Project
# @Description - This file is responsible for [...]
##############################################################################
```

6.14.5 Sections and subsections

Rstudio (v1.4 or more recent[52]) supports the use of Sections and Subsections. You can easily navigate through longer scripts using the navigation pane in RStudio, as shown on the right below.

```
# Section -------

## Subsection -------

### Sub-subsection -------
```

6.14.6 Code folding

Consider using RStudio's code folding[53] feature to collapse and expand different sections of your code. Any comment line with at least four trailing dashes (-), equal signs (=), or pound signs (#) automatically creates a code section. For example:

[52] https://blog.rstudio.com/2020/12/02/rstudio-v1-4-preview-little-things/
[53] https://support.posit.co/hc/en-us/articles/200484568-Code-Folding-and-Sections

6.14.7 Comments in the body of your code

Commenting your code is an important part of reproducibility and helps document your code for the future. When things change or break, you'll be thankful for comments. There's no need to comment excessively or unnecessarily, but a comment describing what a large or complex chunk of code does is always helpful. See this file[54] for an example of how to comment your code and notice that comments are always in the form of:

```
# This is a comment -- first letter is capitalized and spaced away from the pound sign
```

6.14.8 Variable labels

Add a descriptive label as soon as you create each key variable (any variable that might end up in a figure, table, or output model). The label should explicitly include measurement units where applicable.

In R, prefer the `{labelled}`[55] package over raw base R attributes. `{labelled}` integrates seamlessly with tidyverse pipelines:

```r
library(labelled)

# Set variable labels directly in a pipeline
data <- data |>
  set_variable_labels(
    blood_lead_level = "Blood lead concentration (ug/dL)",
    pm25_exposure = "Annual average PM2.5 exposure (ug/m^3)"
  )

# Set a single variable label using var_label()
var_label(data$blood_lead_level) <- "Blood lead concentration (ug/dL)"
```

Labeling key variables at the point of creation ensures that downstream table generators (such as `{gtsummary}`[56]) can automatically display human-readable descriptions and correct units without error-prone manual relabeling.

See also Section 7.3 for function documentation style guidelines.

6.15 Design Documentation with ADRs

An Architecture Decision Record (ADR) is a short, versioned document that captures a significant design decision: the context that forced the choice, the alternatives considered, the decision itself, and its consequences. Function-level documentation and inline comments explain *code*; an ADR explains a *decision* that shapes many files at once and would otherwise live only in someone's memory or a long-merged pull request thread.

[54] https://github.com/kmishra9/Flu-Absenteeism/blob/master/Master's%20Thesis%20-%20Spatial%20Epidemiology%20of%20Influenza/1b%20-%20Map-Management.R
[55] https://larmarange.github.io/labelled/
[56] https://www.danieldsjoberg.com/gtsummary/

6.15.1 When to write one

Write an ADR for a non-obvious design choice that affects multiple files or systems: a concurrency or security trade-off, a data layout or dependency choice with long-term consequences, or a naming convention that needs justification. Routine choices that any reader would make the same way do not need one.

6.15.2 Recommended format

Use the four-part template from Michael Nygard's "Documenting Architecture Decisions"[57]:

- **Status**: proposed, accepted, deprecated, or superseded (with a pointer to the successor).
- **Context**: the forces at play — what problem existed and what constraints applied.
- **Decision**: what was chosen, stated in full sentences with an active voice ("We will …").
- **Consequences**: what becomes easier, what becomes harder, and what future work the decision commits you to.

6.15.3 Where to store them

Keep ADRs in the repository they describe, as `docs/adr-<topic>.md` files (or a `docs/decisions/` directory once there are more than a few), so they are versioned, reviewable, and greppable alongside the code.

6.15.4 How to reference them from code

Point to the ADR with a terse inline comment — `# see docs/adr-<topic>.md, "<section>" section` — instead of duplicating the rationale in comments. This keeps scripts readable while making the full reasoning one hop away, and it means the rationale has one home to update when the decision changes.

6.15.5 Relationship to pull request descriptions

A PR description explains *why this change, now*, and effectively disappears from view once the PR merges; an ADR persists in the repository and keeps answering "why is it built this way?" long after the merge. When a PR implements a significant decision, write the ADR in that PR and let the PR description summarize and link to it.

[57] https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

6.16 Object naming

Generally we recommend using nouns for objects and verbs for functions. This is because functions are performing actions, while objects are not.

Try to make your variable names both more expressive and more explicit. Being a bit more verbose is useful and easy in the age of autocompletion! For example, instead of naming a variable `vaxcov_1718`, try naming it `vaccination_coverage_2017_18`. Similarly, `flu_res` could be named `absentee_flu_residuals`, making your code more readable and explicit.

- For more help, check out Be Expressive: How to Give Your Variables Better Names[58]

We recommend you use **snake_case**.

- Base R allows `.` in variable names and functions (such as `read.csv()`), but this goes against best practices for variable naming in many other coding languages. For consistency's sake, `snake_case` has been adopted across languages, and modern packages and functions typically use it (i.e. `readr::read_csv()`). As a very general rule of thumb, if a package you're using doesn't use `snake_case`, there may be an updated version or more modern package that *does*, bringing with it the variety of performance improvements and bug fixes inherent in more mature and modern software.

> **Note**
>
> You may also see `camelCase` throughout the R code you come across. This is *okay* but not ideal – try to stay consistent across all your code with `snake_case`.

> **Note**
>
> Again, it's also worth noting there's nothing inherently wrong with using `.` in variable names, just that it goes against style best practices that are cropping up in data science, so it's worth getting rid of these bad habits now.

See also Section 7.17.

6.17 Function calls

In a function call, use "named arguments" and put each argument on a separate line to make your code more readable.

Here's an example of what not to do when calling the function a function `calc_fluseas_mean` (defined above):

```
mean_Y = calc_fluseas_mean(flu_data, "maari_yn", FALSE)
```

And here it is again using the best practices we've outlined:

[58] https://spin.atomicobject.com/2017/11/01/good-variable-names/

```
mean_Y <- calc_fluseas_mean(
  data = flu_data,
  yname = "maari_yn",
  silent = FALSE
)
```

6.18 Function decomposition

Function decomposition improves code readability and maintainability by breaking complex logic into focused, reusable units. When a function becomes too long or handles too many concerns, split it into smaller helper functions with clear responsibilities.

Key goals of decomposition include:

- Breaking a big problem into smaller subproblems that are easier to reason about
- Making code easier to read, review, and test
- Pushing low-level complexity "downward" into helper functions so the main workflow stays clear

6.19 When to decompose a function

Consider decomposition when one or more of these are true:

- A function is doing multiple jobs (validation, transformation, modeling, and formatting)
- Most of the function body is a single `if-else`; split the two cases into separate functions so each path has a clear name and contract
- You need comments to explain every few lines just to follow the logic
- You are repeating the same block in multiple places
- You want to test one part of a workflow independently of the rest
- The current function is hard to scan because implementation details hide the main idea

6.20 How to decompose in practice

A practical sequence is:

1. Write down the top-level steps in plain language.
2. Extract one step at a time into a helper function.
3. Give each helper a single, specific contract (inputs, outputs, and assumptions).
4. Keep the orchestration function short, so it reads like a workflow outline.
5. Add or update tests around each helper and the top-level function.

6.21 Decomposing validation blocks

A common case for decomposition is a function that opens with a long sequence of input-validation checks. When the same validation pattern repeats for multiple inputs, extract it into a helper:

```
# Before: repeated validation blocks inline
build_analysis_cache <- function(
    rebuild_signal, signal_files, signal_dir,
    rebuild_finance, finance_files, finance_dir,
    ...) {
  if (rebuild_signal && length(signal_files) == 0L) {
    cli::cli_abort(c(
      "Cannot rebuild Signal source data.",
      "x" = "No raw files found in {.path {signal_dir}}.",
      "i" = "Use {.val auto} to reuse an existing RDS."
    ))
  }
  if (rebuild_finance && length(finance_files) == 0L) {
    cli::cli_abort(c(
      "Cannot rebuild finance source data.",
      "x" = "No raw files found in {.path {finance_dir}}.",
      "i" = "Use {.val auto} to reuse an existing RDS."
    ))
  }
  # ... same pattern for mapping, then more for cached-RDS checks ...
}

# After: extract the repeated pattern into a helper
validate_source_rebuild <- function(
    rebuild,
    source_files,
    label,
    search_path,
    call = rlang::caller_env()) {
  if (!rebuild || length(source_files) > 0L) return(invisible(NULL))
  cli::cli_abort(
    c(
      "Cannot rebuild {label} source data.",
      "x" = "No raw {label} files were found in {.path {search_path}}.",
      "i" = "Use {.val auto} to reuse an existing source RDS."
    ),
    call = call
  )
}

build_analysis_cache <- function(
    rebuild_signal, signal_files, signal_dir,
    rebuild_finance, finance_files, finance_dir,
    rebuild_mapping, mapping_files, s_drive_path,
    ...) {
```

```
  validate_source_rebuild(rebuild_signal, signal_files, "Signal", signal_dir)
  validate_source_rebuild(rebuild_finance, finance_files, "finance", finance_dir)
  validate_source_rebuild(rebuild_mapping, mapping_files, "mapping", s_drive_path)
  # ... main logic is now immediately visible
}
```

Benefits of this pattern:

- The main function reads as a workflow outline rather than a wall of `if`/`cli_abort` blocks.
- Each validation rule is defined once and tested once.
- Adding a new data source requires one new call, not a copy-pasted block.
- Error message formatting stays consistent across all sources.

Place each extracted helper in its own file (Section 6.24), not above or below the caller in the same file.

6.22 Examples borrowed from lab package PRs

In `UCD-SERG/rpt` PR #100[59], a top-level function was updated to call dedicated helpers, with helper logic split into separate files. The workflow moved from "do everything in one function" toward a clearer sequence of validation, cleaning, calculation, and formatting steps. This made the top-level function easier to read, while isolating validation and formatting details in reusable units.

In `UCD-SERG/shigella` PR #11[60], focused analysis functions were added in separate files, moving away from larger script-style helpers toward single-purpose, testable package functions.

6.23 Related references

For background and related ideas, see Decomposition (computer science)[61], Code refactoring[62], and Functional decomposition[63].

6.24 One function per file

Give each function its own file rather than bundling multiple functions into one file. This applies to exported functions, internal helpers, and test helpers alike — the only exception is a trivial two-line wrapper that is inseparable from its caller.

[59] https://github.com/UCD-SERG/rpt/pull/100
[60] https://github.com/UCD-SERG/shigella/pull/11
[61] https://en.wikipedia.org/wiki/Decomposition_(computer_science)
[62] https://en.wikipedia.org/wiki/Code_refactoring
[63] https://en.wikipedia.org/wiki/Functional_decomposition

6.24.1 R packages

Under `R/`, name the file after the function it contains: `simulate_data()` lives in `R/simulate_data.R`. A private helper that is more than a two-liner still gets its own file, named after the helper, even if only one caller uses it.

6.24.2 Why

A file with one function is easier to find, easier to diff (a change to one function does not sit inside an unrelated grep hit for a different one), and matches how `git blame` and code-review tools attribute changes. Bundling unrelated functions into one file — because they were added in the same PR, or because an existing file happened to be open — accumulates unrelated concerns over time.

6.24.3 Adding to an existing multi-function file

When adding a function to a file that already holds others (common in older code that predates this convention), put the **new** function in its own file so at least new code follows the convention going forward. Splitting every pre-existing function out is a separate, larger refactor.

6.24.4 Relationship to function decomposition

When decomposing a large function (1), place each extracted helper in its own file rather than above or below the caller in the same file. The caller file then contains only the orchestration function, and each helper is independently findable and testable.

6.25 Function length limits

Function length matters because very long functions can hide workflow intent, increase maintenance burden, and make targeted testing harder. The sources below combine community discussions (opinion-based) with a formal complexity reference. Use the community links for examples of practitioner viewpoints, and the complexity reference for a formal metric.

Sources:

- Reddit (r/learnprogramming)[64]: February 2016 community thread asking whether one long function or many smaller functions is preferable.
- SoloLearn discussion[65]: commenters describe different line-count conventions (including 20-line and 100-line limits) and discuss readability and testability tradeoffs.

[64] https://www.reddit.com/r/learnprogramming/comments/47avb5/one_long_function_2001000_lines_of_codes_or_one/
[65] https://www.sololearn.com/en/Discuss/1278705/optimal-number-of-lines-of-code-of-functions

- Stack Overflow: "When is a function too long?"[66] and Software Engineering Stack Exchange: "What is the ideal length of a method for you?"[67]: older Q&A threads (September 2008 and February 2012, respectively) focused on practical criteria for deciding when to split methods.
- Cyclomatic complexity[68]: defines complexity by counting independent execution paths/decision points; this reinforces that branching complexity is often a better risk indicator than raw line count alone.

Synthesizing these viewpoints for lab code yields the following practical rules:

1. Use `<150` lines as a provisional heuristic trigger to reassess decomposition, not a hard constraint.
2. Prefer single-purpose functions with clear names and contracts.
3. Watch decision complexity (branching and nesting), not only length.
4. Decompose when the top-level story is hard to follow or hard to test.

6.26 Keep It Short and Simple (KISS)

The KISS principle[69] — "Keep It Short and Simple" — says that most systems work best when they are kept simple, so unnecessary complexity should be avoided rather than managed. A blunter phrasing — the one aircraft engineer Kelly Johnson actually coined at the Lockheed Skunk Works — is "keep it simple, stupid": his standing challenge was that his team's aircraft must be repairable by an average mechanic in the field with only a handful of common tools.

The principle carries directly into software development: code is read, debugged, and modified far more often than it is written, and it is usually maintained by someone with less context than the original author had — often your own future self.

Rationales for keeping code short and simple:

- **Fewer bugs.** Less code and simpler control flow leave fewer places for defects to hide and make the remaining logic easier to reason about.
- **Easier review.** A reviewer can hold a short, single-purpose function in their head; they cannot do that with a sprawling one.
- **Cheaper maintenance.** Simple code can be modified confidently; complex code makes every change risky and slow.
- **Faster onboarding.** New lab members become productive sooner when the codebase does things the plain, expected way.

Simplicity is not an excuse to drop requirements: the goal is the simplest solution that still does the whole job, not a simpler job. The preceding sections on function decomposition and function length limits, and the guidance on avoiding deep nesting[70], are concrete applications of this principle.

[66] https://stackoverflow.com/questions/475675/when-is-a-function-too-long
[67] https://softwareengineering.stackexchange.com/questions/133404/what-is-the-ideal-length-of-a-method-for-you
[68] https://en.wikipedia.org/wiki/Cyclomatic_complexity
[69] https://en.wikipedia.org/wiki/KISS_principle
[70] @sec-avoid-nesting

6.27 Decomposing Code Chunks in Quarto Documents

A Quarto document should read as narrative plus orchestration: each code chunk calls documented functions and displays their results, and the surrounding prose carries the analysis story. Complex logic belongs in named functions, not inline in chunks.

When a chunk grows into substantial logic — a multi-step data manipulation, a function defined inline, a non-trivial transformation, or the same block copied across chunks — extract that logic into a function and have the chunk call it. Following the function-decomposition (1), one-function-per-file (Section 6.24), and length-limit (Section 6.25) principles, give the function its own file under `R/`, with full `{roxygen2}`[71] documentation, the same way any other package function is written. The chunk then shrinks to a call plus whatever narrative or display wraps it.

Why this matters: a function can be unit-tested, reused across chunks and analyses, and documented; inline chunk code can be none of these. Keeping logic inline also buries the analysis narrative under implementation detail, and duplicates code that should have one home.

Our shared linter set enforces the function-definition half of this rule: `lms::function_location_linter()` (part of `lms::default_linters()`) flags a named function defined at the top level of any file outside the designated directories (`R/` and `tests/` by default), including inside a `.qmd` chunk. Loose chunk logic that was never wrapped in a function at all is not something a linter can catch, so watch for it when writing and reviewing analyses: a chunk carrying substantial inline logic is a decomposition finding, and the fix is to move the logic into `R/` and leave the chunk calling it.

6.28 The here package

The `here` package is one great R package that helps multiple collaborators deal with the mess that is working directories within an R project structure. Let's say we have an R project at the path `/home/oski/Some-R-Project`. My collaborator might clone the repository and work with it at some other path, such as `/home/bear/R-Code/Some-R-Project`. Dealing with working directories and paths explicitly can be a very large pain, and as you might imagine, setting up a Config with paths requires those paths to flexibly work for all contributors to a project. This is where the `here` package comes in and this a great vignette describing it[72].

See also Section 7.16 for code style guidelines on using the here package.

6.29 Reading/Saving Data

6.29.1 `.RDS` vs `.RData` Files

One of the most common ways to load and save data in Base R is with the `load()` and `save()` functions to serialize multiple objects in a single `.RData` file. The biggest problems with this practice include an inability to control the names of things getting loaded in,

[71] https://roxygen2.r-lib.org/
[72] https://github.com/jennybc/here_here

the inherent confusion this creates in understanding older code, and the inability to load individual elements of a saved file. For this, we recommend using the RDS format to save R objects.

> **Note**
>
> If you have many related R objects you would have otherwise saved all together using the `save` function, the functional equivalent with `RDS` would be to create a (named) list containing each of these objects, and saving it.

6.29.2 CSVs

Once again, the `readr` package as part of the Tidvyerse is great, with a much faster `read_csv()` than Base R's `read.csv()`. For massive CSVs (> 5 GB), you'll find `data.table::fread()` to be the fastest CSV reader in any data science language out there. For writing CSVs, `readr::write_csv()` and `data.table::fwrite()` outclass Base R's `write.csv()` by a significant margin as well.

6.30 Integrating Box and Dropbox

Box and Dropbox are cloud-based file sharing systems that are useful when dealing with large files. When our scripts generate large output files, the files can slow down the workflow if they are pushed to GitHub. This makes collaboration difficult when not everyone has a copy of the file, unless we decide to duplicate files and share them manually. The files might also take up a lot of local storage. Box and Dropbox help us avoid these issues by automatically storing the files, reading data, and writing data back to the cloud.

Box and Dropbox are separate platforms, but we can use either one to store and share files. To use them, we can install the packages that have been created to integrate Box and Dropbox into R. The set-up instructions are detailed below.

Make sure to authenticate before reading and writing from either Box or Dropbox. The authentication commands should go in the configuration file; it only needs to be done once. This will prompt you to give your login credentials for Box and Dropbox and will allow your application to access your shared folders.

6.30.1 Box

Follow the instructions in this section to use the `boxr` package. Note that there are a few setup steps that need to be done on the box website before you can use the `boxr` package, explained here[73] in the section "Creating an Interactive App." This gets the authentication keys that must be put in box. Once that is done, add the authentication keys to your code in the configuration file, with `box_auth(client_id = "<your_client_id>", client_secret = "<your_client_secret_id>")`. It is also important to set the default working directory so that the code can reference the correct folder in box: `box_setwd(<folder_id>)`. The folder ID is the sequence of digits at the end of the URL.

[73] https://r-box.github.io/boxr/articles/boxr-app-interactive.html#create

Further details can be found here[74].

6.30.2 Dropbox

Follow the instructions at this link[75] to use the `rdrop2` package. Similar to the `boxr` package, you must authenticate before reading and writing from Dropbox, which can be done by adding `drop_auth()` to the configuration file.

Saving the authentication token is not required, although it may be useful if you plan on using Dropbox frequently. To do so, save the token with the following commands. Tokens are valid until they are manually revoked.

```
# first time only
# save the output of drop_auth to an RDS file
token <- drop_auth()
# this token only has to be generated once, it is valid until revoked
saveRDS(token, "/path/to/tokenfile.RDS")

# all future usages
# to use a stored token, provide the rdstoken argument
drop_auth(rdstoken = "/path/to/tokenfile.RDS")
```

6.31 r-lib Packages

The r-lib[76] GitHub organization, maintained primarily by the Posit team, hosts a collection of foundational R packages for package development, infrastructure, and general programming. These packages underpin much of the modern R ecosystem and are widely used in our lab's workflows.

6.32 Package Development

6.32.1 `{usethis}`[77]

`{usethis}`[78] automates repetitive tasks that arise during project setup and development, both for R packages and non-package projects. It provides functions for setting up testing, CI, licenses, Git, and GitHub integrations, making it the go-to companion for `devtools`.

[74] https://github.com/r-box/boxr
[75] https://github.com/karthik/rdrop2
[76] https://github.com/r-lib
[77] https://usethis.r-lib.org/
[78] https://usethis.r-lib.org/

6.32.2 {devtools}[79]

{devtools}[80] provides a suite of tools for the R package development workflow. It wraps many lower-level tools such as pkgbuild, pkgload, and rcmdcheck behind a convenient interface. Key functions include load_all(), document(), test(), check(), and install().

6.32.3 {roxygen2}[81]

{roxygen2}[82] lets you write documentation in-source, alongside the code it documents, using #' comment blocks. It generates .Rd files for the man/ directory and updates NAMESPACE. It is the standard approach to R package documentation.

6.32.4 {pkgdown}[83]

{pkgdown}[84] builds a documentation website for an R package from its source. It converts README, vignettes, and function documentation into a navigable website, and integrates with GitHub Actions for automatic deployment.

6.32.5 {pkgload}[85]

{pkgload}[86] simulates the process of installing and loading a package, enabling rapid interactive development. It is the backend for devtools::load_all().

6.32.6 {pkgbuild}[87]

{pkgbuild}[88] provides tools for building R packages, finding compilers, and checking whether a build environment (such as Rtools on Windows) is available.

6.32.7 {rcmdcheck}[89]

{rcmdcheck}[90] runs R CMD check from within R and captures the results in a structured format. It is used by CI systems and devtools::check().

[79] https://devtools.r-lib.org/
[80] https://devtools.r-lib.org/
[81] https://roxygen2.r-lib.org/
[82] https://roxygen2.r-lib.org/
[83] https://pkgdown.r-lib.org/
[84] https://pkgdown.r-lib.org/
[85] https://pkgload.r-lib.org/
[86] https://pkgload.r-lib.org/
[87] https://pkgbuild.r-lib.org/
[88] https://pkgbuild.r-lib.org/
[89] https://rcmdcheck.r-lib.org/
[90] https://rcmdcheck.r-lib.org/

6.32.8 {remotes}[91]

{remotes}[92] installs R packages from remote sources including GitHub, GitLab, Bitbucket, URLs, and local files. It is a lightweight alternative to `devtools::install_github()`.

6.32.9 {pak}[93]

{pak}[94] installs R packages from CRAN, GitHub, and other sources, with fast parallel downloads and automatic system-dependency handling. It is r-lib's modern alternative to `install.packages()` and `remotes` for interactive use.

6.32.10 {revdepcheck}[95]

{revdepcheck}[96] automates reverse dependency checking — checking that changes to a package do not break packages that depend on it. It is used by package maintainers before CRAN releases.

6.32.11 {desc}[97]

{desc}[98] provides tools for reading, writing, and manipulating `DESCRIPTION` files. It is used internally by `usethis` and other developer-facing tools.

6.32.12 {pkgcache}[99]

{pkgcache}[100] provides a local cache of CRAN-like metadata and package files, speeding up package installation. It is used internally by `pak`.

6.32.13 {lifecycle}[101]

{lifecycle}[102] manages the life cycle of functions and arguments, providing the experimental/superseded/deprecated badges and deprecation warnings used across the tidyverse and r-lib ecosystems.

[91] https://remotes.r-lib.org/
[92] https://remotes.r-lib.org/
[93] https://pak.r-lib.org/
[94] https://pak.r-lib.org/
[95] https://revdepcheck.r-lib.org/
[96] https://revdepcheck.r-lib.org/
[97] https://desc.r-lib.org/
[98] https://desc.r-lib.org/
[99] https://r-lib.github.io/pkgcache/
[100] https://r-lib.github.io/pkgcache/
[101] https://lifecycle.r-lib.org/
[102] https://lifecycle.r-lib.org/

6.33 Testing and Quality

6.33.1 {testthat}[103]

{testthat}[104] is the most widely used R testing framework. It provides a readable, expressive way to write unit tests for R packages, with functions like `expect_equal()`, `expect_error()`, and `expect_snapshot()`.

6.33.2 {covr}[105]

{covr}[106] measures and reports test coverage for R packages. It can generate HTML reports, submit results to Codecov, and is commonly integrated into CI pipelines.

6.33.3 {waldo}[107]

{waldo}[108] finds and describes differences between R objects. It provides clearer, more informative output than `identical()` or `all.equal()`, and is used internally by `testthat` to display test failures.

6.33.4 {lintr}[109]

{lintr}[110] performs static code analysis (linting) for R. It checks for style issues, potential errors, and common anti-patterns, and integrates with RStudio, VS Code, and CI pipelines.

6.34 Core Language and Infrastructure

6.34.1 {rlang}[111]

{rlang}[112] provides a low-level API for working with R's core language features, including environments, quosures, symbols, and non-standard evaluation (tidy eval). It is a foundational dependency for tidyverse packages and tidy eval programming.

[103] https://testthat.r-lib.org/
[104] https://testthat.r-lib.org/
[105] https://covr.r-lib.org/
[106] https://covr.r-lib.org/
[107] https://waldo.r-lib.org/
[108] https://waldo.r-lib.org/
[109] https://lintr.r-lib.org/
[110] https://lintr.r-lib.org/
[111] https://rlang.r-lib.org/
[112] https://rlang.r-lib.org/

6.34.2 {vctrs}[113]

{vctrs}[114] defines a principled type system for R vectors, providing tools for coercion, casting, and size-stability. It underpins the consistent behavior of `dplyr` and `tidyr` across vector types.

6.34.3 {withr}[115]

{withr}[116] allows code to be run with temporarily modified global state — changing options, environment variables, working directories, or random seeds — and reliably restoring them afterward. Essential for writing side-effect-free tests.

6.34.4 {cli}[117]

{cli}[118] provides a rich toolkit for building command-line interfaces in R. It supports color, ANSI styling, progress bars, spinners, and formatted messages. It is the modern replacement for `crayon` and is used throughout the tidyverse.

6.34.5 {R6}[119]

{R6}[120] implements encapsulated, reference-based object-oriented programming in R. Unlike S3 or S4, R6 classes support mutable state and inheritance in a style familiar from Python or Java.

6.34.6 {cpp11}[121]

{cpp11}[122] provides a modern C++11 interface for interacting with R's C API. It is a lightweight, header-only alternative to `Rcpp`, designed to simplify writing and debugging C++ extensions for R.

6.35 System and Process

6.35.1 {fs}[123]

{fs}[124] provides a cross-platform, uniform interface to file system operations. It replaces inconsistent base R file functions with a tidy API using `path_*()`, `file_*()`, `dir_*()`, and `link_*()` functions that always return character vectors.

[113] https://vctrs.r-lib.org/
[114] https://vctrs.r-lib.org/
[115] https://withr.r-lib.org/
[116] https://withr.r-lib.org/
[117] https://cli.r-lib.org/
[118] https://cli.r-lib.org/
[119] https://r6.r-lib.org/
[120] https://r6.r-lib.org/
[121] https://cpp11.r-lib.org/
[122] https://cpp11.r-lib.org/
[123] https://fs.r-lib.org/
[124] https://fs.r-lib.org/

6.35.2 {processx}[125]

{processx}[126] runs external system processes asynchronously or synchronously from R, with full control over stdin, stdout, and stderr. It is the backend for `callr`, and is used internally by `rcmdcheck`.

6.35.3 {callr}[127]

{callr}[128] calls R functions in separate R processes. This is useful for running code in a clean environment, isolating side effects, or parallelizing work. It builds on top of `processx`.

6.35.4 {ps}[129]

{ps}[130] provides a cross-platform API for listing, querying, and manipulating system processes from R. It is used internally by `processx` and `callr`.

6.36 HTTP and Web

6.36.1 {httr2}[131]

{httr2}[132] is a modern, pipe-friendly HTTP client for R. It replaces `httr` with a cleaner API, better support for OAuth, rate limiting, retries, and request/response bodies. Preferred for new code.

6.36.2 {httr}[133]

{httr}[134] is a widely-used HTTP client for R, providing wrappers for `GET()`, `POST()`, `PUT()`, `DELETE()`, and other HTTP verbs, as well as OAuth support. It has been superseded by `httr2` for new development.

6.36.3 {gh}[135]

{gh}[136] provides a minimal client for the GitHub REST API. It handles authentication and pagination, and is used internally by `usethis` for GitHub operations.

[125] https://processx.r-lib.org/
[126] https://processx.r-lib.org/
[127] https://callr.r-lib.org/
[128] https://callr.r-lib.org/
[129] https://ps.r-lib.org/
[130] https://ps.r-lib.org/
[131] https://httr2.r-lib.org/
[132] https://httr2.r-lib.org/
[133] https://httr.r-lib.org/
[134] https://httr.r-lib.org/
[135] https://gh.r-lib.org/
[136] https://gh.r-lib.org/

6.36.4 {gargle}[137]

{gargle}[138] provides authentication utilities for Google APIs, including OAuth 2.0, service accounts, and Application Default Credentials. It is used by googledrive, googlesheets4, and other Google-backed R packages.

6.37 Utilities

6.37.1 {here}[139]

{here}[140] builds file paths relative to the project root, so scripts work regardless of the current working directory. This manual requires {here} for file paths (see Section 6.28).

6.37.2 {memoise}[141]

{memoise}[142] adds memoisation (function-call caching) to R functions. Wrapping an expensive, pure function with memoise::memoise() causes it to cache computed return values in memory or on disk, so repeated calls with identical arguments return instantly from cache instead of recomputing. It supports multiple storage backends via the cache argument (such as cachem::cache_mem() for in-memory caching and cachem::cache_disk() for persistent disk caching across sessions), and provides memoise::forget() to clear caches, memoise::is.memoised() to test whether a function is memoised, and memoise::has_cache() to check whether specific arguments have cached results. Use {memoise} only for deterministic functions without side effects (such as pure mathematical evaluations, stable API queries, or idempotent data processing pipelines). Never memoise stochastic simulation routines or functions that generate pseudorandom numbers; memoisation suppresses stochastic variation across repeated calls, silently collapsing random variability by returning stale cached draws.

6.37.3 {scales}[143]

{scales}[144] converts data values into perceptual properties and formats labels for dates, currencies, and percentages; it powers the scale system of {ggplot2}[145].

[137] https://gargle.r-lib.org/
[138] https://gargle.r-lib.org/
[139] https://here.r-lib.org/
[140] https://here.r-lib.org/
[141] https://memoise.r-lib.org/
[142] https://memoise.r-lib.org/
[143] https://scales.r-lib.org/
[144] https://scales.r-lib.org/
[145] https://ggplot2.tidyverse.org/

6.37.4 {sessioninfo}[146]

{sessioninfo}[147] collects and prints detailed information about the current R session, including package versions and sources. It is a more informative replacement for base R's `sessionInfo()`.

6.37.5 {downlit}[148]

{downlit}[149] provides syntax highlighting for R code and automatically links function names to their documentation. It is used by `pkgdown` and Quarto for rendering R code in documentation websites.

6.37.6 {prettyunits}[150]

{prettyunits}[151] formats quantities such as file sizes, time durations, and byte counts in human-readable form. It is used internally by `devtools`, `remotes`, and progress-reporting packages.

6.37.7 {lobstr}[152]

{lobstr}[153] provides tools for visualizing R data structures, including shared-reference trees (`ref()`), abstract syntax trees (`ast()`), and object sizes (`obj_size()`). It is a useful learning and debugging tool.

6.37.8 {crayon}[154]

{crayon}[155] adds color and formatting to terminal output in R using ANSI codes. It has been largely superseded by `cli` but remains widely used in older code and packages.

6.37.9 {styler}[156]

{styler}[157] formats R code according to the tidyverse style guide. It provides an RStudio add-in and functions for styling individual files, packages, or projects. See Section 6.13 for more details.

[146] https://sessioninfo.r-lib.org/
[147] https://sessioninfo.r-lib.org/
[148] https://downlit.r-lib.org/
[149] https://downlit.r-lib.org/
[150] https://r-lib.github.io/prettyunits/
[151] https://r-lib.github.io/prettyunits/
[152] https://lobstr.r-lib.org/
[153] https://lobstr.r-lib.org/
[154] https://r-lib.github.io/crayon/
[155] https://r-lib.github.io/crayon/
[156] https://styler.r-lib.org/
[157] https://styler.r-lib.org/

6.37.10 {rex}[158]

{rex}[159] provides a human-readable interface for constructing regular expressions in R without having to write complex, unreadable regex strings manually. It allows regular expressions to be composed modularly using functions such as `rex()`, `character_class()`, `capture()`, and `some_of()`.

When using {rex} inside another R package:

- Add `rex` to `Imports` in the package `DESCRIPTION`.
- Call `rex::register_shortcuts(pkgname)` in `.onLoad()` to prevent `R CMD check` NOTEs regarding undefined global variables when using `rex` shortcuts in package code.
- Call `rex::rex(...)` to compile expressions into regular expression strings for use with functions like `grepl()`, `gsub()`, or {stringr}[160] functions.

6.38 GitHub Actions

6.38.1 actions[161]

The `r-lib/actions`[162] repository provides reusable GitHub Actions workflows for R package development. Commonly used actions include:

- `r-lib/actions/setup-r` — installs R
- `r-lib/actions/setup-renv` — restores an `renv` environment
- `r-lib/actions/check-r-package` — runs `R CMD check`
- `r-lib/actions/setup-r-dependencies` — installs package dependencies

These actions are widely used across R packages hosted on GitHub, including this lab's repositories.

6.39 Tidyverse

Throughout this document there have been references to the Tidyverse, but this section is to explicitly show you how to transform your Base R tendencies to Tidyverse (or Data.Table, Tidyverse's performance-optimized competitor). For most of our work that does not utilize very large datasets, we recommend that you code in Tidyverse rather than Base R. Tidyverse is quickly becoming the gold standard[163] in R data analysis and modern data science packages and code should use Tidyverse style and packages unless there's a significant reason not to (i.e. big data pipelines that would benefit from Data.Table's performance optimizations). Note that {dtplyr}[164] provides a data.table backend for dplyr, enabling you to use most of dplyr's tidy syntax with data.table's performance optimizations.

[158] https://rex.r-lib.org/
[159] https://rex.r-lib.org/
[160] https://stringr.tidyverse.org/
[161] https://github.com/r-lib/actions
[162] https://github.com/r-lib/actions
[163] https://rviews.rstudio.com/2017/06/08/what-is-the-tidyverse/
[164] https://dtplyr.tidyverse.org/

The package author has published R for Data Science (Wickham et al. 2023), which leans heavily on many Tidyverse packages and may be worth checking out.

6.40 Core Tidyverse Packages

The tidyverse[165] is a collection of R packages designed for data science that share an underlying design philosophy, grammar, and data structures. As of tidyverse 1.3.0, the following nine packages are included in the core tidyverse and are loaded automatically when you run `library(tidyverse)`:

6.40.1 ggplot2[166]

`{ggplot2}`[167] is a system for declaratively creating graphics, based on The Grammar of Graphics. You provide the data, tell ggplot2 how to map variables to aesthetics and what graphical primitives to use, and it takes care of the details.

6.40.2 dplyr[168]

`{dplyr}`[169] provides a grammar of data manipulation, providing a consistent set of verbs that solve the most common data manipulation challenges. Key functions include `filter()`, `select()`, `mutate()`, `summarize()`, and `arrange()`.

6.40.3 tidyr[170]

`{tidyr}`[171] provides a set of functions that help you get to tidy data. Tidy data is data with a consistent form: in brief, every variable goes in a column, and every column is a variable. Key functions include `pivot_longer()`, `pivot_wider()`, `separate()`, and `unite()`.

6.40.3.1 When to use dplyr vs tidyr

While both `{dplyr}`[172] and `{tidyr}`[173] work with data frames, they serve different purposes:

- **Use dplyr** for data manipulation within the current structure: filtering rows, selecting columns, creating new variables, summarizing data, or joining datasets. These operations work with your data as-is.
- **Use tidyr** for reshaping your data structure itself: converting between wide and long formats (`pivot_longer()`, `pivot_wider()`), splitting or combining columns (`separate()`, `unite()`), or handling missing values explicitly (`complete()`, `fill()`).

[165] https://www.tidyverse.org/
[166] https://ggplot2.tidyverse.org/
[167] https://ggplot2.tidyverse.org/
[168] https://dplyr.tidyverse.org/
[169] https://dplyr.tidyverse.org/
[170] https://tidyr.tidyverse.org/
[171] https://tidyr.tidyverse.org/
[172] https://dplyr.tidyverse.org/
[173] https://tidyr.tidyverse.org/

These packages work together seamlessly in data analysis workflows. A typical pattern is to use tidyr to reshape your data into the right structure, then use dplyr to manipulate and analyze it.

6.40.4 readr[174]

`{readr}`[175] provides a fast and friendly way to read rectangular data (like csv, tsv, and fwf). It is designed to flexibly parse many types of data found in the wild, while still cleanly failing when data unexpectedly changes.

6.40.5 purrr[176]

`{purrr}`[177] enhances R's functional programming[178] (FP) toolkit by providing a complete and consistent set of tools for working with functions and vectors. Once you master the basic concepts, purrr allows you to replace many for loops with code that is easier to write and more expressive. See Section 6.9 for more details on using purrr.

6.40.6 tibble[179]

`{tibble}`[180] is a modern re-imagining of the data frame, keeping what time has proven to be effective, and throwing out what it has not. Tibbles are data.frames that are lazy and surly: they do less and complain more forcing you to confront problems earlier, typically leading to cleaner, more expressive code.

6.40.7 stringr[181]

`{stringr}`[182] provides a cohesive set of functions designed to make working with strings as easy as possible. It is built on top of stringi, which uses the ICU C library to provide fast, correct implementations of common string manipulations.

6.40.8 forcats[183]

`{forcats}`[184] provides a suite of useful tools that solve common problems with factors. R uses factors to handle categorical variables, variables that have a fixed and known set of possible values.

[174] https://readr.tidyverse.org/
[175] https://readr.tidyverse.org/
[176] https://purrr.tidyverse.org/
[177] https://purrr.tidyverse.org/
[178] https://en.wikipedia.org/wiki/Functional_programming
[179] https://tibble.tidyverse.org/
[180] https://tibble.tidyverse.org/
[181] https://stringr.tidyverse.org/
[182] https://stringr.tidyverse.org/
[183] https://forcats.tidyverse.org/
[184] https://forcats.tidyverse.org/

6.40.9 lubridate[185]

{lubridate}[186] provides a set of functions for working with date-times, extending and improving on R's existing support for them. Key functions include `ymd()`, `mdy()`, `dmy()` for parsing dates, and `year()`, `month()`, `day()` for extracting components.

6.41 Base R to Tidyverse Translation

The following list is not exhaustive, but is a compact overview to begin to translate Base R into something better:

Base R	Better Style, Performance, and Utility
—	—
`read.csv()`	`readr::read_csv()` or `data.table::fread()`
`write.csv()`	`readr::write_csv()` or `data.table::fwrite()`
`readRDS` `saveRDS()`	`readr::read_rds()` `readr::write_rds()`
—	—
`data.frame()`	`tibble::tibble()` or `tibble::tribble()`
`rbind()`	`dplyr::bind_rows()`
`cbind()`	`dplyr::bind_cols()`
`df$some_column`	`df |> dplyr::pull(some_column)`
`df$some_column = ...`	`df |> dplyr::mutate(some_column = ...)`
`df[get_rows_condition,]`	`df |> dplyr::filter(get_rows_condition)`
`df[,c(col1, col2)]`	`df |> dplyr::select(col1, col2)`
`merge(df1, df2, by = ..., all.x = ..., all.y = ...)`	`df1 |> dplyr::left_join(df2, by = ...)` or `dplyr::full_join` or `dplyr::inner_join` or `dplyr::right_join`
—	—
`str()`	`dplyr::glimpse()`
`grep(pattern, x)`	`stringr::str_which(string, pattern)`
`gsub(pattern, replacement, x)`	`stringr::str_replace(string, pattern, replacement)`
`ifelse(test_expression, yes, no)`	`if_else(condition, true, false)`
Nested: `ifelse(test_expression1, yes1, ifelse(test_expression2, yes2, ifelse(test_expression3, yes3, no)))`	`case_when(test_expression1 ~ yes1, test_expression2 ~ yes2, test_expression3 ~ yes3, TRUE ~ no)`
`proc.time()`	`tictoc::tic()` and `tictoc::toc()`
`stopifnot()`	`assertthat::assert_that()` or `assertthat::see_if()` or `assertthat::validate_that()`
—	—

[185] https://lubridate.tidyverse.org/
[186] https://lubridate.tidyverse.org/

Base R	Better Style, Performance, and Utility
`sessionInfo()`	`sessioninfo::session_info()`

For a more extensive set of syntactical translations to Tidyverse, you can check out this document[187].

6.42 Programming with Tidyverse

Working with Tidyverse within functions can be somewhat of a pain due to non-standard evaluation (NSE) semantics. If you're an avid function writer, we'd recommend checking out the following resources:

- Programming with dplyr[188] (package vignette)
- Using dplyr in packages[189] (package vignette)
- Tidy Eval in 5 Minutes[190] (video)
- Tidy Evaluation[191] (e-book)
- Evaluation[192] (advanced details)
- Data Frame Columns as Arguments to Dplyr Functions[193] (blog)
- Standard Evaluation for *_join[194] (stackoverflow)

See also Section 7.15

6.43 Coding with R and Python

If you're using both R and Python, you may wish to check out the Feather package[195] for exchanging data between the two languages extremely quickly[196].

6.44 Repeating analyses with different variations

In many cases, we will need to apply our modeling on different combinations of interests (outcomes, exposures, etc.). We can certainly use a `for` loop to repeat the execution of a wrapper function, but generally, `for` loops request high memory usage and produce the results in long computation time.

Fortunately, R has some functions which implement looping in a compact form to help repeating your analyses with different variations (subgroups, outcomes, covariate sets, etc.) with better performances.

[187] https://tavareshugo.github.io/data_carpentry_extras/base-r_tidyverse_equivalents/base-r_tidyverse_equivalents.html
[188] https://dplyr.tidyverse.org/articles/programming.html
[189] https://dplyr.tidyverse.org/articles/in-packages.html
[190] https://www.youtube.com/watch?v=nERXS3ssntw
[191] https://dplyr.tidyverse.org/articles/programming.html
[192] https://adv-r.hadley.nz/evaluation.html
[193] https://www.brodrigues.co/blog/2016-07-18-data-frame-columns-as-arguments-to-dplyr-functions/
[194] https://stackoverflow.com/questions/28125816/r-standard-evaluation-for-join-dplyr
[195] https://www.rdocumentation.org/packages/feather/versions/0.3.3
[196] https://blog.rstudio.com/2016/03/29/feather/

6.44.1 `lapply()` and `sapply()`

`lapply()` is a function in the base R package that applies a function to each element of a list and returns a list. It's typically faster than `for`. Here is a simple generic example:

```
result <- lapply(X = mylist, FUN = func)
```

There is another very similar function called `sapply()`. It also takes a list as its input, but if the output of the `func` is of the same length for each element in the input list, then `sapply()` will simplify the output to the simplest data structure possible, which will usually be a vector.

6.44.2 `mapply()` and `pmap()`

Sometimes, we'd like to employ a wrapper function that takes arguments from multiple different lists/vectors. Then, we can consider using `mapply()` from the base R package or `pmap()` from the `purrr` package.

Please see the simple specific example below where the two input lists are of the same length and we are doing a pairwise calculation:

```
mylist1 = list(0:3)
mylist2 = list(6:9)
mylists = list(mylist1, mylist2)

square_sum <- function(x, y) {
  x^2 + y^2
}

#Use `mapply()`
result1 <- mapply(FUN = square_sum, mylist1, mylist2)

#Use `pmap()`
library(purrr)
result2 <- pmap(.l = mylists, .f = square_sum)

#unlist(as.list(result1)) = result2 = [36 50 68 90]
```

There are two major differences between `mapply()` and `pmap()`. The first difference is that `mapply()` takes seperate lists as its input arguments, while `pmap()` takes a list of list. Secondly, the output of `mapply()` will be in the form of a matrix or an array, but `pmap()` produces a list directly.

However, when **the input lists are of different lengths AND/OR the wrapper function doesn't take arguments in pairs, `mapply()` and `pmap()` may not give the preferable results.**

Both `mapply()` and `pmap()` will recycle shorter input lists to match the length of the longest input list. Assume that now `mylist2 = list(6:12)`. Then, `pmap(mylists, square_sum)` will generate `[36  50  68  90 100 122 148]` where elements 0, 1, and 2 are recycled to

match 10, 11, and 12. And it will return an error message that "longer object length is not a multiple of shorter object length."

Thus, unless the recycling pattern described above is desirable feature for a certain experiment design, **when the input lists are of different lengths, the best practice is probably to use `lapply()` and then combine the results.**

Here is an example where we'd like to find the `square_sum` for every element combination of `mylist1` and `mylist2`.

```
mylist1 <- list(0:3)
mylist2 <- list(6:12)

square_sum <- function(x, y) {
  x^2 + y^2
}

results <- list()

for (i in seq_along(mylist1[[1]])) {
  result <- lapply(X = mylist2, FUN = function(y) square_sum(mylist1[[1]][i], y))
  results[[i]] <- result
}
```

This example doesn't work in the way that 0 is paired to 6, 1 is paired to 7, and so on. Instead, every element in `mylist1` will be paired with every element in `mylist2`. Thus, the "unlisted" results from the example will have $4 * 7 = 28$ elements.

We can use `flatten()` or `unlist()` functions to decrease the depths of our results. If the results are data frames, then we will need to use `bind_rows()` to combine them.

6.44.3 Parallel processing with `parallel` and `future` packages

One big drawback of `lapply()` is its long computation time, especially when the list length is long. Fortunately, computers nowadays must have multiple cores which makes parallel processing possible to help make computation much faster.

Assume you have a list called `mylist` of length 1000, and `lapply(X = mylist, FUN = func)` applies the function to each of the 1000 elements one by one in T seconds. If we could execute the `func` in n processors simultaneously, then ideally, we would shrink the computation time to T/n seconds.

In practice, using functions under the `parallel` and the `future` packages, we can split `mylist` into smaller chunks and apply the function to each element of the several chunks in parallel in different cores to significantly reduce the run time.

6.44.3.1 `parLapply()`

Below is a generic example of `parLapply()`:

```
library(parallel)

# Set how many processors will be used to process the list and make cluster
n_cores <- 4
cl <- makeCluster(n_cores)

#Use parLapply() to apply func to each element in mylist
result <- parLapply(cl = cl, x = mylist, FUN = func)

#Stop the parallel processing
stopCluster(cl)
```

Let's still assume `mylist` is of length 1000. The `parLapply` above splits `mylist` into 4 sub-lists each of length 250 and applies the function to the elements of each sub-list in parallel. To be more specific, first apply the function to element 1, 251, 501, 751; second apply to element 2, 252, 502, 752; so on and so forth. As such, the computation time will be greatly reduced.

You can use `parallel::detectCores()` to test how many cores your machine has and to help decide what to put for `n_cores`. It would be a good idea to leave at least one core free for the operating system to use.

We will always start `parLapply()` with `makeCluster()`. **`stopCluster()` is not fully necessary but follows the best practices.** If not stopped, the processing will continue in the back end and consuming the computation capacity for other software in your machine. But keep in mind that stopping the cluster is similar quitting R, meaning that you will need to re-load the packages needed when you need to do parallel processing use `parLapply()` again.

6.44.3.2 `future.lapply()`

Below is a generic example of `future.lapply()`:

```
library(future)
library(future.apply)

# First, plan how the future_lapply() will be resolved
future::plan(
  multisession, workers = future::availableCores() - 1
)

# Use future_lapply() to apply func to each element in mylist
future_lapply(x = mylist, FUN = func)
```

Here, `future::availableCores()` checks how many cores your machine has. Similar to `parLapply()` showed above, `future_lapply()` parallelizes the computation of `lapply()` by executing the function `func` simultaneously on different sub-lists of `mylist`.

6.45 Reviewing Code

Peer code review is a collaborative practice that improves scientific rigor, fosters shared understanding, and catches bugs before analysis results are shared or published. The lab's pull request workflow, author and reviewer expectations, and response protocol are consolidated in Chapter 12.

6.45.1 Why We Review Code

Code review serves several vital purposes:

- **Error prevention**: Catching data processing and analytical bugs early, before findings enter manuscripts or reports.
- **Reproducibility**: Verifying that scripts execute cleanly from scratch without undocumented manual interventions or environment-specific dependencies.
- **Knowledge transfer**: Sharing analytical methods, R idioms, and package features across the lab team.
- **Maintainability**: Ensuring that analysis pipelines can be understood, modified, and extended by other lab members in the future.

6.45.2 What Reviewers Look For

When reviewing a colleague's code, evaluate these key dimensions:

1. **Reproducibility**:
    - Does the script run cleanly from start to finish in a fresh R session?
    - Are package dependencies managed reproducibly via `{renv}`[197] (see Chapter 15)?
    - Are file paths relative and constructed with `{here}`[198] (see Section 6.28), avoiding machine-specific absolute paths?
2. **Code correctness and data integrity**:
    - Are data transformations and filtering steps logical and documented?
    - Are missing values (`NA`), edge cases, and empty groupings handled appropriately?
    - Are key variables assigned descriptive labels and measurement units where applicable (see Section 6.14.8)?
3. **Style and structure**:
    - Does the code follow the lab's R code style (see Chapter 7)?
    - Are long chunks broken into focused functions (see 1)?
    - Are variable and function names self-describing (see Section 7.17)?
4. **Documentation**:
    - Do comments explain non-obvious design choices or statistical assumptions (see Section 6.14)?
    - Are architectural or analytical decisions documented with an ADR where appropriate (see Section 6.15)?

[197] https://rstudio.github.io/renv/
[198] https://here.r-lib.org/

6.45.3 How to Give Constructive Feedback

Effective code review builds a supportive, high-trust research culture:

- **Be kind and respectful**: Critique the code, not the author. Assume positive intent and acknowledge elegant solutions or clear documentation.
- **Be specific and actionable**: Explain why a change is recommended and provide concrete code suggestions or examples where possible.
- **Distinguish blocking vs. non-blocking comments**: Clearly separate essential correctness or reproducibility issues (blocking) from optional stylistic preferences or ideas for future exploration (non-blocking nits).
- **Ask clarifying questions**: When logic is unclear, ask the author to explain their intent before proposing a rewrite.

6.45.4 When to Approve

Approve a pull request when:

- All automated CI checks and workflows pass cleanly.
- The code runs end-to-end and produces expected outputs.
- All blocking review findings have been resolved or explicitly deferred to a tracked follow-up issue.
- The author has completed the pre-review quality checks in Section 6.12.

6.46 Getting Help with Code

When you encounter a coding problem, creating a **reprex** (minimal **repr**oducible **ex**ample) is one of the most effective ways to get help—and often helps you solve the problem yourself.

A good reprex (Bryan et al. 2024):

- Is **reproducible**: Contains all necessary code, including `library()` calls and data
- Is **minimal**: Strips away everything not directly related to your problem
- Uses small, simple example data (often built-in datasets)

Why create a reprex:

- 80% of the time, creating a reprex helps you discover the solution yourself
- 20% of the time, you'll have a clear example that makes it easy for others to help you
- It respects others' time by making your problem easy to understand and reproduce

Resources:

- `{reprex}`[199] package: Automates creation of reproducible examples
- R for Data Science: Making a reprex[200] (Wickham et al. 2023): Step-by-step guide to creating effective reproducible examples

[199] https://reprex.tidyverse.org/
[200] https://r4ds.hadley.nz/workflow-help.html#making-a-reprex

6.47 Additional Resources

6.47.1 R Package Development

- R Packages (Wickham and Bryan 2023) - comprehensive guide to R package development
- Tidyverse design guide (Wickham 2023b) - principles for designing R packages and APIs that are intuitive, composable, and consistent with tidyverse philosophy
- usethis documentation[201] - workflow automation for R projects
- devtools documentation[202] - essential development tools
- pkgdown documentation[203] - create package websites
- testthat documentation[204] - unit testing framework

6.47.2 General R Programming

- R for Data Science (Wickham et al. 2023) - learn data science with the tidyverse
- Advanced R (Wickham 2019) - deep dive into R programming and internals

6.47.3 Shiny Development

- Mastering Shiny (Wickham 2021) - comprehensive guide to building web applications with Shiny
- Engineering Production-Grade Shiny Apps (Fay et al. 2021) - best practices for production Shiny applications

6.47.4 Git and Version Control

- Happy Git and GitHub for the useR (Bryan 2023) - essential guide to using Git and GitHub with R
- `{gitdown}`[205] - R package for documenting git commit history as a bookdown report, organized by patterns like tags or issues

[201] https://usethis.r-lib.org/
[202] https://devtools.r-lib.org/
[203] https://pkgdown.r-lib.org/
[204] https://testthat.r-lib.org/
[205] https://thinkr-open.github.io/gitdown/

Listing 6.1 `.github/workflows/benchmark.yaml`

```yaml
name: Benchmark

on:
  pull_request:
    branches: [main, master]
  workflow_dispatch:  # Manual trigger

jobs:
  benchmark:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: r-lib/actions/setup-r@v2
        with:
          use-public-rspm: true

      - uses: r-lib/actions/setup-r-dependencies@v2
        with:
          extra-packages: any::bench, any::profvis

      - name: Run benchmarks
        run: |
          library(bench)
          library(mypackage)

          # Create benchmarks directory if it doesn't exist
          dir.create("tests/benchmarks", recursive = TRUE, showWarnings = FALSE)

          # Generate test data (or load existing test data)
          test_data <- generate_test_cohort(n = 10000)

          # Run benchmark
          results <- bench::mark(
            current_implementation = analyze_cohort(test_data),
            iterations = 100
          )

          # Save results
          saveRDS(results, "tests/benchmarks/current.rds")
        shell: Rscript {0}

      - name: Compare with baseline
        run: |
          # Load baseline and current results
          baseline <- readRDS("tests/benchmarks/baseline.rds")
          current <- readRDS("tests/benchmarks/current.rds")

          # Calculate percentage change in median time
          baseline_time <- as.numeric(baseline$median[1])
          current_time <- as.numeric(current$median[1])
          pct_change <- (current_time - baseline_time) / baseline_time * 100
```

7 R Code Style

Follow these code style guidelines for all R code:

7.1 General Principles

Our lab follows the Google R Style Guide[1], which in turn mostly follows the tidyverse style guide[2] (Wickham 2023a). The following principles apply to all R code:

- **Follow tidyverse conventions**: Our style is built on the tidyverse style guide, which provides comprehensive guidance on naming, syntax, pipes, functions, and more
- **Naming**: Use `snake_case` for functions and variables; acronyms may be uppercase (e.g., `prep_IDs_data`)
- **Write tidy code**: Keep code clean, readable, and well-organized
- **Avoid redundant logical comparisons**: Use logical variables directly in conditional statements (e.g., `if (x)` instead of `if (x == TRUE)` or `if (x == 1)`)
- **Use pipes to emphasize primary inputs**: When writing functions and code, use the pipe operator to clearly show transformations on a primary object. The primary input should flow as the first argument to each function in the chain. Design functions so the most important argument (usually data) comes first, enabling natural pipeline composition. See the tidyverse design principles[3] for more details.
- **Use native pipe**: `|>` not `%>%` (available in R >= 4.1.0). This is enforced by our `.lintr.R` configuration via `pipe_consistency_linter(pipe = "|>")`

Many of these rules are automatically enforced through our `.lintr.R`[4] configuration file. See Section 7.18 for details on automated style checking.

7.2 Clean Code Principles

Robert C. Martin's *Clean Code* (Martin 2008) provides timeless principles for writing readable, maintainable, and robust software. While originally framed in object-oriented languages, these core tenets translate directly to scientific computing and R programming:

[1] https://google.github.io/styleguide/Rguide.html
[2] https://style.tidyverse.org/
[3] https://design.tidyverse.org/important-args-first.html
[4] https://github.com/UCD-SERG/lab-manual/blob/main/.lintr.R

7.2.1 Meaningful Names

- **Use intention-revealing names**: A variable or function name should tell you why it exists, what it does, and how it is used (e.g., `days_since_exposure` rather than `d`).
- **Make meaningful distinctions**: Avoid arbitrary suffixes like `data1` and `data2` or `info` and `data`.
- **Use pronounceable and searchable names**: Names should be easily communicated in conversations and found via search tools.
- **Functions should use verbs**: Function names should describe actions (e.g., `calculate_incidence()`, `fit_model()`, `plot_titers()`).

7.2.2 Functions

- **Small and focused**: Functions should do one thing, do it well, and do it only.
- **Single level of abstraction**: Keep operations in a function at the same conceptual level rather than mixing high-level orchestration with low-level data munging.
- **Few arguments**: Aim for 0 to 2 arguments where practical; if a function requires many parameters, group them into a configuration object or list.
- **Avoid side effects**: Functions should return values predictably without unexpectedly mutating global state.

7.2.3 Comments

- **Explain why, not what**: As detailed in Section 7.10, code should explain what it does through expressive naming and clear structure. Use comments to document the reasoning, statistical rationale, edge cases, or scientific domain decisions behind an implementation.
- **Do not leave commented-out code**: Rely on version control (Git) to preserve historical code rather than leaving dead blocks in source files.

7.2.4 The Boy Scout Rule

- **Leave the codebase cleaner than you found it**: When editing a script or chapter, make small improvements in stride—fix typos, improve a variable name, or break up a long expression. Over time, continuous incremental cleanup prevents code degradation.

7.2.5 Don't Repeat Yourself (DRY)

- **Eliminate duplication**: Originating in *The Pragmatic Programmer* (Hunt and Thomas 1999) and emphasized in *Clean Code*, the DRY principle states that every piece of knowledge must have a single, unambiguous representation. Duplicate logic is a primary source of bugs and maintenance overhead. When the same transformation or calculation appears in multiple places, extract it into a reusable helper function (see also Section 7.14).

7.3 Function Structure and Documentation

Every function should follow this pattern:

```
#' Short Title (One Line)
#'
#' Longer description providing details about what the function does,
#' when to use it, and important considerations.
#'
#' @param param1 Description of first parameter, including type and constraints
#' @param param2 Description of second parameter
#'
#' @returns Description of return value, including type and structure
#'
#' @examples
#' # Example usage
#' result <- my_function(param1 = "value", param2 = 10)
#'
#' @export
my_function <- function(param1, param2) {
  # Implementation
  return(result)
}
```

See also Section 6.14 for general code documentation practices.

7.3.1 Explicit Return Statements

Following the Google R Style Guide's recommendation[5], always use explicit `return()` statements in functions, even when R's implicit return would work. This makes the function's intent clear and improves readability.

Example:

```
# Good: Explicit return
calculate_mean <- function(x) {
  result <- mean(x, na.rm = TRUE)
  return(result)
}

# Less clear: Implicit return (avoid)
calculate_mean <- function(x) {
  mean(x, na.rm = TRUE)
}
```

Note: Our `.lintr.R` configuration disables the `return_linter` to allow flexibility in code, but we still require explicit returns as a lab standard.

[5] https://google.github.io/styleguide/Rguide.html#use-explicit-returns

7.4 Assertions and Input Validation

Validating function arguments and preconditions early (often called "failing fast") prevents bugs from propagating deep into analysis pipelines or statistical models. R offers several strategies and packages for writing assertions, each suited to different stages of development:

7.4.1 Base R: `stopifnot()`

Base R provides `stopifnot()`, which halts execution if any of the supplied expressions are not all `TRUE`:

```
calculate_rate <- function(count, person_years) {
  stopifnot(
    is.numeric(count),
    length(count) == 1,
    !is.na(count),
    count >= 0,
    is.numeric(person_years),
    length(person_years) == 1,
    !is.na(person_years),
    person_years > 0
  )
  return(count / person_years)
}
```

- **Pros**: Zero dependencies; built into every R installation.
- **Cons**: Error messages can be cryptic in older R versions (though R 4.0+ improved messages to display the failing expression); does not provide prepackaged assertion helpers for complex data types.

7.4.2 `{checkmate}`: Fast, comprehensive argument checks

The `{checkmate}`[6] package (on CRAN[7]) is widely recommended for package development and production pipelines:

```
calculate_rate <- function(count, person_years) {
  checkmate::assert_number(count, lower = 0)
  # lower bound is inclusive; use smallest positive double to require person_years > 0
  checkmate::assert_number(person_years, lower = .Machine$double.xmin, finite = TRUE)
  return(count / person_years)
}
```

- **Pros**: Extremely fast (written in C); comprehensive family of assertion functions (`assert_*`, `check_*`, `test_*`); generates clear, informative error messages automatically.
- **Cons**: Adds a package dependency.

[6] https://mllg.github.io/checkmate/
[7] https://cran.r-project.org/package=checkmate

7.4.3 {cli} and {rlang}: Custom, friendly error conditions

Within the Tidyverse, {cli}[8] and {rlang}[9] are the modern standard for emitting structured, formatted error messages:

```
calculate_rate <- function(count, person_years, call = rlang::caller_env()) {
  if (!is.numeric(count) || length(count) != 1 || is.na(count)) {
    cli::cli_abort(
      "{.arg count} must be a single non-missing number, not {.obj_type_friendly {count}}."
      call = call
    )
  }
  if (count < 0) {
    cli::cli_abort(
      "{.arg count} must be non-negative ({.val {count}} was provided).",
      call = call
    )
  }
  if (!is.numeric(person_years) || length(person_years) != 1 || is.na(person_years)) {
    cli::cli_abort(
      "{.arg person_years} must be a single non-missing number, not {.obj_type_friendly {pe
      call = call
    )
  }
  if (person_years <= 0) {
    cli::cli_abort(
      "{.arg person_years} must be positive ({.val {person_years}} was provided).",
      call = call
    )
  }
  return(count / person_years)
}
```

- **Pros**: Outstanding user feedback with inline formatting (`{.arg}`, `{.val}`); properly attributes the error to the caller environment via `call = rlang::caller_env()`; supports signaling custom condition classes (via `class = ...`) that callers can catch programmatically.
- **Cons**: Requires writing manual validation branches rather than one-line assertion calls.

7.4.4 Decomposing validation into helpers

When a function validates multiple inputs with the same pattern, extract the repeated checks into a helper function rather than repeating the `if`/`cli_abort` block for each input. See Section 6.21 for a worked example.

[8] https://cli.r-lib.org/
[9] https://rlang.r-lib.org/

7.4.5 Note on `{assertthat}`

The `{assertthat}`[10] package, created by Hadley Wickham, popularized user-friendly assertions in R. While still widely used across existing codebases, it has not seen active development since 2019. For new code, prefer `{checkmate}`[11] for comprehensive argument checking or `{cli}`[12] paired with `cli::cli_abort()` for custom error messages.

7.5 Avoiding Deep Nesting

When writing code, avoid nested function calls and nested function definitions where feasible:

- Prefer named intermediate variables (or a pipe, e.g. `|>` in R) over deeply nested calls like `f(g(h(x)))`. Naming each step makes the data flow read top-to-bottom and leaves intermediate values inspectable in a debugger.
- Prefer standalone, top-level function definitions over functions defined inside other functions. Nested definitions hide reusable logic, complicate unit testing, and obscure scope.

This is a readability and maintainability default, not an absolute rule — keep the nesting when flattening it would be more convoluted (a trivial one-argument wrapper, or a closure that genuinely needs the enclosing scope).

7.5.1 Prefer more, simpler steps over fewer, denser ones

The named-intermediate rule above operates within one expression. The same trade runs one level up, across a pipeline: given a choice, do less per step and take more steps.

Advanced R makes the observation while comparing a `purrr` pipeline against the base-R and `for`-loop versions of the same task, in Purrr style[13]:

> It's interesting to note that as you move from purrr to base apply functions to for loops you tend to do more and more in each iteration. In purrr we iterate 3 times (`map()`, `map()`, `map_dbl()`), with apply functions we iterate twice (`lapply()`, `vapply()`), and with a for loop we iterate once. I prefer more, but simpler, steps because I think it makes the code easier to understand and later modify.

The gain is the same one named intermediates buy: each step is separately readable, separately testable, and separately replaceable, and a change lands in one step rather than in the middle of a compound one. Cost only shows up when a step is traversed enough times for the extra passes to matter — which is a claim to settle with performance benchmarking, not by assumption.

[10] https://cran.r-project.org/package=assertthat
[11] https://mllg.github.io/checkmate/
[12] https://cli.r-lib.org/
[13] https://adv-r.hadley.nz/functionals.html#purrr-style

7.5.2 Lambdas in map() and apply-family calls

The nested-definition rule applies to `purrr::map*()` / `pmap*()` / `lapply()`-family call sites too: do not wrap a named function in an anonymous function (lambda) just to fix constant arguments. Pass the mapped elements positionally and the constants through the mapping function's `...`:

```
# Preferred --- hr/power match schoenfeld_events()'s leading parameters
# positionally; the constant fractions ride along via map2's `...`
purrr::map2_dbl(
  df$hr, df$power, schoenfeld_events,
  p1 = frac_op, p2 = frac_nonop
)

# Avoid --- a lambda that only fixes constant arguments
purrr::map2_dbl(df$hr, df$power, function(h, p) {
  schoenfeld_events(hr = h, power = p, p1 = frac_op, p2 = frac_nonop)
})
```

`purrr`'s own documentation mildly recommends shorthand lambdas over `...`-passing. This preference deliberately overrides that: when the mapped elements line up with the callee's leading parameters, use `...` and skip the wrapper.

When a wrapper genuinely is necessary — the mapped element is not the callee's leading argument, is used more than once in the body, or the body is a real expression rather than a single call — define a named wrapper function in its own file rather than an inline lambda. The one exception is a demonstrated performance reason to define the wrapper nested inside the calling function (e.g. it must close over a large enclosing-scope object that would otherwise be passed repeatedly); that is the same closure escape hatch as the nested-definition rule above.

7.6 Prefer Existing Packaged Functions

Before writing a function, look for an existing packaged one that already does the job — and prefer it over rolling your own:

- Check, roughly in this order: base R and the tidyverse / r-lib packages, then a focused, well-maintained CRAN package, then our own lab packages (e.g. `{bcs}`, `{ettbc}`, `{gha}`, and the shared workflows there). Packages can depend on each other, so reuse across our repos is fine.
- Reach for the packaged version unless it is genuinely unfit — the wrong API, a heavy dependency for a one-liner, or it does not quite do what you need.

Packaged functions are tested, documented, and maintained by other people; hand-rolling an equivalent duplicates that work, adds surface area to maintain, and risks subtle bugs the package already fixed. For example, use `withr::with_seed()` to set a seed and restore the RNG stream, rather than hand-rolling a `.Random.seed` save/restore.

This is a default, not an absolute rule. A tiny, dependency-free helper can beat pulling in a package, and sometimes nothing fits — but look first, and prefer the standard, well-known way over a bespoke one.

One reason to skip a packaged function does not count: an environment your own change chose. "The script runs on a bare R, so `rlang` is unreachable" is not evidence that `rlang` does not fit when this change is what decided the script would run that way. Install the package, fix the CI job, and re-run the comparison.

- **Do:** say whether a constraint ruling out a package is external or one of ours, before letting it decide.
- **Don't:** verify a self-imposed constraint and report that as having justified the hand-rolled version.

7.7 Prefer Per-Operation Grouping

When reviewing or writing `dplyr` code, prefer **per-operation grouping** (the `.by` argument) over persistent `group_by()` / `ungroup()` pairs. Apply it when the grouping is only needed for one operation.

```
# Preferred --- grouping is scoped to this summarise(), no ungroup() needed
df |> summarise(mean_x = mean(x), .by = group_col)

# Avoid --- group_by() persists and must be manually ungroup()'d
df |> group_by(group_col) |> summarise(mean_x = mean(x)) |> ungroup()
```

Reference: https://dplyr.tidyverse.org/reference/dplyr_by.html

Flag persistent `group_by()` calls during code review when `.by` would work — that is, when the grouping feeds exactly one downstream verb and no subsequent operation needs it to persist.

7.8 Avoid Hard-Coding Data with an External Source of Truth

Avoid hard-coding data that already has a reliable external source of truth — a version number, a package list, a dependency's release date, a set of downstream consumers, a schema, or an enum's valid values. Read or generate it from that source instead of copying a snapshot into the codebase:

- **Versions and pins.** Do not retype a dependency's version in prose or a second config file when a lockfile, `DESCRIPTION`, or manifest already states it — reference that file, or generate the mention from it.
- **Generated lists.** A list of consumers, plugins, or registered items that the source system can enumerate (an API, a directory scan, a registry) should be produced by querying that system, not maintained by hand alongside it.
- **Cross-file duplication.** When the same fact must appear in two places (a usage example and a reference doc, a schema and its example), generate the second from the first, or have CI check they agree, rather than trusting two hand-edited copies to stay in sync.

7.8.1 Prose enumerations count, and they are the ones that rot unnoticed

The "generated lists" bullet reads as being about code, and the rule is easiest to break in a sentence. An enumeration written into documentation — "the rules below (A, B, C, …)" — is a hand-maintained copy of a directory listing, and it has no generator, no test, and no linter behind it. Nothing fails when the directory grows; the sentence simply becomes wrong and stays wrong, while still reading as authoritative.

When a prose list mirrors something the filesystem or an API already enumerates, prefer a pointer to the source over the list: "every fragment under `coding-style/`" cannot drift, while a parenthetical naming seven of them silently can. Keep an explicit list only where the *selection* is the point — a curated subset, an ordering that matters — and then say that it is a selection, so a reader knows not to trust it as complete.

Fixing a drifted list by refreshing it only resets the clock. The list will drift again on the next addition, by exactly the same mechanism. Replace it with the pointer instead, and treat "this needs updating again" as the signal that it should not have been a list.

A softened count is still a pinned count. The rule that refreshing a drifted list only resets the clock has a near-miss. The move it does not rule out is replacing the figure with a vaguer version of itself — "on the order of 90K+ tokens across the ~60 files in it" in place of "~74-92K tokens across 59 files". Hedging acknowledges the imprecision and changes nothing about the mechanism, so the substitution reads as careful while drifting on the same schedule: a tree can shrink past "90K+" as easily as it grew past an exact figure.

The tell is that such a sentence has to describe its own restraint. A form needing a clause like "a count this sentence deliberately does not pin" pinned one, or it would have nothing to disclaim. The pointer form needs no such clause, because it names the query rather than an answer.

Hedging is the wrong answer whether or not the figure has a source. What the source decides is which right answer applies, and the discriminator is whether a *source can be named* rather than whether a command exists. A figure whose source is nameable — a directory, an API, a lockfile — gets the pointer. A count of the items in the block directly beneath it names nothing, so it gets dropped or re-derived instead.

- **Do:** replace a figure the filesystem or an API enumerates with the command that derives it.
- **Don't:** hedge such a figure and keep it — an approximation drifts on the same schedule the exact value did.

This is conditioned on the external source being **reliably available** — do not add a network fetch or a fragile dependency where a static value would do. A constant that has no external owner (a magic number intrinsic to the algorithm, a default chosen by this project) is not "hard-coded data" in this sense — it is just a value. The target is duplicated *ownership* of a fact: if updating the external source should have updated this value too, and did not, that is the bug this guidance prevents.

7.8.1.1 A count in the prose above a block is the same duplicate, one line away

The section above describes a list mirroring something *elsewhere* — a directory the sentence cannot see, drifting over weeks as files land. The tighter case is a count of the items in the block directly beneath it: "Three reads settle it", above three commands. Same defect, since

the count is a hand-maintained copy of something the block already enumerates. What differs is who invalidates it and when. Nobody adding a file to some other directory breaks this one. **You** break it, in the same review round, by fixing the block the count describes.

Two things keep it out of view at exactly that moment:

1. The count was **correct when written**, so it was never a mistake to notice and carry forward — it became false only when the block gained a command. And a review finding points at the block, so correcting the block feels like the whole action. The sentence introducing it is not part of what the reviewer flagged, so nothing prompts a re-read.
2. Adjacency reads as safe. A count of items in a distant file is obviously fragile, and that visible fragility is what makes anyone check it. A count one line above the thing it counts feels like it cannot drift, since both are on the screen at once — which is precisely why nobody looks at the prose while editing the block.

Two remedies that work:

- **Drop the count.** The block is immediately below, so "These reads settle it" loses nothing a reader could not get by looking down. This is the better answer whenever the number carries no argument.
- **Re-derive it mechanically before pushing**, when the number is doing real work in the sentence. One command decides it exactly, which makes this a check to automate rather than something to settle by recollection.

Scope that counting command to the block, not to the whole file. A file-wide `grep -c` is right only while the pattern happens to match nothing outside the block, which is a property of the file today rather than of the command. An unrelated edit that adds one matching line anywhere else silently inflates the count, so the instrument acquires exactly the failure mode it was reached for to prevent.

- **Do:** re-read the sentence introducing a block whenever a review finding changes what is in that block.
- **Do:** delete a count the neighbouring block already states, and re-derive by command any count you keep.
- **Don't:** treat a fix to the block as complete because the finding named only the block.
- **Don't:** read adjacency as protection — the nearest duplicate is the one your own edit falsifies first.

7.8.1.2 A qualitative generalization above a block goes unchecked where a count would be re-derived

The section above governs a **count** stated above the block it counts, and its remedy is to drop the count or re-derive it by command. A qualitative claim over the same block is the same shape, and it slips past that remedy entirely, because it carries no number to re-derive.

A count invites verification because it is obviously a number someone must have measured, and a stale one reads as a typo waiting to be caught. A qualitative claim — "always", "independently of", "in every case", "the same in both" — carries no such tell. It reads as characterization rather than as a checkable fact, so it survives review exactly where a count would not.

The check is not "re-derive the number" — there is none — but "read the introducing sentence against the block", applied here to the sentence a table or block sits directly beneath.

- **Do:** re-read a lead-in sentence against its block for what it generalizes or quantifies, not only for a stated count.
- **Do:** treat "always", "independently of", "in every case", "the same in both" as tells that a claim needs checking against the data beneath it.
- **Don't:** assume a qualitative lead-in is safe because it carries no number that could go stale.

7.8.2 Where the rule stops: text that records what was observed

Everything above pushes toward replacing a literal with whatever owns it. There is one boundary it must not cross, and a consistency sweep is precisely the operation that crosses it without noticing.

Text that **asserts what was observed** is not configuration. A command someone actually ran, the output it actually produced, and the conditions a measurement was actually taken under are claims about the past, and their literals are the evidence for those claims. Parameterizing them does not generalize the record. It falsifies it, in the name of consistency, and leaves no trace that anything was changed.

Three forms, each of which looks like the hard-coding this section bans:

- **A command that was executed.** `git worktree add /tmp/wt-ums main` reports a run. Rewriting it to `<default-branch>` asserts a run that never happened.
- **A verbatim error string.** `fatal: invalid reference: origin/main` is what the tool printed. A reader matches it against their own terminal, so a parameterized version matches nothing and stops being findable.
- **The conditions of a measurement.** A sentence saying the runs used a repo whose default branch is literally `main` states the scope of the result. It is usually the sentence that explains why the measurement did not surface the bug.

The tell is tense and mood rather than syntax. Prescriptive text tells a reader what to do next, and should name the parameter. Evidentiary text says what happened, and should keep the literal. One file routinely carries both, so decide occurrence by occurrence.

- **Do:** parameterize the occurrences that instruct, and leave the ones that record.
- **Do:** decide per occurrence in a file that carries both, reading each one's surrounding sentence.
- **Don't:** run a whole-file replace over a literal that also appears inside quoted commands, quoted output, or a statement of measurement conditions.
- **Don't:** treat an unparameterized literal inside a case record as a defect left behind — there it is the evidence.

7.9 Construct Complex Inputs Before the Call

Build a complex argument as a named intermediate first, then pass that name to the function. Naming the intermediate keeps the call short, makes the data flow read top to bottom, and lets you inspect the value in a debugger.

```
# Good: name the intermediate, then pass it
model_vars <- c("age", "sex", "titer")
model_formula <- reformulate(model_vars, response = "outcome")
fit <- lm(model_formula, data = study_data)

# Avoid: complex input constructed inline in the call
fit <- lm(reformulate(c("age", "sex", "titer"), response = "outcome"), data = study_data)
```

A pipe is the other idiomatic way to avoid an inline-constructed argument, when the input is the result of a short sequence of transformations:

```
# Good: build the input with a pipe, then pass it
recent_cases <-
  case_data |>
  filter(year >= 2017) |>
  arrange(onset_date)

ggplot(recent_cases, aes(x = onset_date)) +
  geom_histogram()
```

This is the same readability goal as Section 7.5: prefer named steps you can read and check over one dense expression.

7.10 Comments

Use comments to explain *why*, not *what*:

```
# Good: Explains reasoning
# Use log scale because distribution is highly skewed
ggplot(data, aes(x = log10(income))) + geom_histogram()

# Bad: States the obvious
# Create a histogram
ggplot(data, aes(x = income)) + geom_histogram()
```

File headers (for scripts in `data-raw/` or `inst/analyses/`):

```
##############################################################################
# @Organization - Example Organization
# @Project - Example Project
# @Description - This file is responsible for [...]
##############################################################################
```

File Structure - Just as your data "flows" through your project, data should flow naturally through a script. Very generally, you want to

1) source your config =>
2) load all your data =>

3) do all your analysis/computation => save your data.

Each of these sections should be "chunked together" using comments. See this file[14] for a good example of how to cleanly organize a file in a way that follows this "flow" and functionally separate pieces of code that are doing different things.

> **Note**
>
> If your computer isn't able to handle this workflow due to RAM or requirements, modifying the ordering of your code to accommodate it won't be ultimately helpful and your code will be fragile, not to mention less readable and messy. You need to look into high-performance computing (HPC) resources in this case.

Single-Line Comments - Commenting your code is an important part of reproducibility and helps document your code for the future. When things change or break, you'll be thankful for comments. There's no need to comment excessively or unnecessarily, but a comment describing what a large or complex chunk of code does is always helpful. See this file[15] for an example of how to comment your code and notice that comments are always in the form of:

```
# This is a comment -- first letter is capitalized and spaced away from the pound sign
```

Multi-Line Comments - Occasionally, multi-line comments are necessary. You should manually insert line breaks to "hard-wrap" code and comments, whenever lines become longer than 80 characters. `lintr` should object otherwise, even for comments. Try to break lines at semantic boundaries: ends of sentences or phrases. Long lines in source code files make it more difficult to see and comment on diffs in pull requests.

In prose text chunks, Quarto ignores single line breaks, so you should also line-break your prose text in .qmd files to keep them under 80 characters.

You can configure RStudio's settings to display the 80-character margin.

7.11 Line Breaks and Formatting

7.11.1 Blank Lines Before Lists

Always include a blank line before starting a bullet list or numbered list in markdown/Quarto documents. This ensures proper rendering and readability.

Correct:

```
Here are the requirements:

- First item
- Second item
```

[14] https://github.com/kmishra9/Flu-Absenteeism/blob/master/Master's%20Thesis%20-%20Spatial%20Epidemiology%20of%20Influenza/2a%20-%20Statistical-Inputs.R

[15] https://github.com/kmishra9/Flu-Absenteeism/blob/master/Master's%20Thesis%20-%20Spatial%20Epidemiology%20of%20Influenza/1b%20-%20Map-Management.R

Incorrect:

```
Here are the requirements:
- First item
- Second item
```

Here's what happens if you don't add the blank line:

Here are the requirements: - First item - Second item

7.11.2 Semantic Line Breaks in Plain Text

Add a newline at the end of every phrase or logical unit of text in plain-text source files. A phrase is typically a complete thought, clause, or sentence. This applies to:

- Plain-text paragraphs in `.qmd` files
- Source code text: comments, documentation strings, and error messages

Correct (prose in .qmd):

```
When talking about code in prose sections,
use backticks to apply code formatting.
This helps maintain readability in source files
and makes diffs easier to review.
```

Incorrect (prose in .qmd):

```
When talking about code in prose sections, use backticks to apply code formatting. This hel
```

Correct (R code comment):

```
# First, check if the input is valid.
# Then, process the data.
# Finally, return the result.
```

Incorrect (R code comment):

```
# First, check if the input is valid. Then, process the data. Finally, return the result.
```

This practice is also known as semantic line breaks[16].

Guidelines:

- Break after complete sentences (at periods)
- Break after long phrases or clauses (at commas or conjunctions)
- Aim for lines under 80 characters
- Keep related short phrases together on one line
- Do not break in the middle of inline code, links, or formatting

[16] https://sembr.org/

7.11.3 Line Breaks in Code

- For `ggplot` calls and `dplyr` pipelines, do not crowd single lines. Here are some nontrivial examples of "beautiful" pipelines, where beauty is defined by coherence:

```
# Example 1
school_names = list(
  OUSD_school_names = absentee_all |>
    filter(dist.n == 1) |>
    pull(school) |>
    unique |>
    sort,

  WCCSD_school_names = absentee_all |>
    filter(dist.n == 0) |>
    pull(school) |>
    unique |>
    sort
)
```

```
# Example 2
absentee_all = fread(file = raw_data_path) |>
  mutate(program = case_when(schoolyr %in% pre_program_schoolyrs ~ 0,
                             schoolyr %in% program_schoolyrs ~ 1)) |>
  mutate(period = case_when(schoolyr %in% pre_program_schoolyrs ~ 0,
                            schoolyr %in% LAIV_schoolyrs ~ 1,
                            schoolyr %in% IIV_schoolyrs ~ 2)) |>
  filter(schoolyr != "2017-18")
```

And of a complex `ggplot` call:

```
# Example 3
ggplot(data=data) +

  aes(x=.data[["year"]], y=.data[["rd"]], group=.data[[group]]) +

  geom_point(mapping = aes(col = .data[[group]], shape = .data[[group]]),
             position=position_dodge(width=0.2),
             size=2.5) +

  geom_errorbar(mapping = aes(ymin=.data[["lb"]], ymax= .data[["ub"]], col= .data[[group]]),
                position=position_dodge(width=0.2),
                width=0.2) +

  geom_point(position=position_dodge(width=0.2),
             size=2.5) +

  geom_errorbar(mapping=aes(ymin=lb, ymax=ub),
                position=position_dodge(width=0.2),
                width=0.1) +
```

```
scale_y_continuous(limits=limits,
                   breaks=breaks,
                   labels=breaks) +

scale_color_manual(std_legend_title,values=cols,labels=legend_label) +
scale_shape_manual(std_legend_title,values=shapes, labels=legend_label) +
geom_hline(yintercept=0, linetype="dashed") +
xlab("Program year") +
ylab(yaxis_lab) +
theme_complete_bw() +
theme(strip.text.x = element_text(size = 14),
      axis.text.x = element_text(size = 12)) +
ggtitle(title)
```

Imagine (or perhaps mournfully recall) the mess that can occur when you don't strictly style a complicated `ggplot` call. Trying to fix bugs and ensure your code is working can be a nightmare. Now imagine trying to do it with the same code 6 months after you've written it. Invest the time now and reap the rewards as the code practically explains itself, line by line.

7.12 Markdown and Quarto Formatting

7.12.1 Writing about code in Quarto documents

When writing about code in prose sections of quarto documents, use backticks to apply a code style: for example, `dplyr::mutate()`. When talking about packages, use backticks and curly-braces with a hyperlink to the package website. For example: `{dplyr}`[17].

Important: Do not use raw HTML (`<a href="...">`) in `.qmd` files. Always use Quarto/markdown link syntax instead.

7.13 Messaging and User Communication

Use `{cli}`[18] package functions for all user-facing messages in package functions. This is enforced by our `.lintr.R` configuration via `undesirable_function_linter()`.

Required messaging functions:

- Use `cli::cli_inform()` instead of `message()`, `inform()`, or older `cli_alert_*()` functions
- Use `cli::cli_warn()` instead of `warning()` or `warn()`
- Use `cli::cli_abort()` instead of `stop()` or `abort()`

This provides better formatting, color support, and consistent messaging across our packages.

Examples:

[17] https://dplyr.tidyverse.org/
[18] https://cli.r-lib.org/

```r
# Good
cli::cli_inform("Analysis complete")
cli::cli_warn("Missing data detected")
cli::cli_abort("Invalid input: {.arg x} must be numeric")

# Bad - don't use these in package code
message("Analysis complete")
warning("Missing data detected")
stop("Invalid input")
```

7.13.1 CLI Styling: Progress Notes vs. Outcomes

When writing command-line tools, pipeline scripts, or package messaging, maintain a clear visual distinction between intermediate progress updates and terminal outcomes:

- **Progress updates should use neutral, non-emphatic styling**: Routine status notes describing in-flight actions (such as "Starting to compute…", "Loading input data…", or "Querying remote endpoint…") should remain quiet and plain. Avoid bold fonts, bright highlight colors, or heavy icons for intermediate steps. Applying emphatic styling to routine progress causes visual fatigue and desensitizes users to important messages.
- **Outcomes should receive emphatic styling**: Definitive conclusions and status transitions (such as "Succeeded", "Failed", or "Warning") signal results that require user awareness or intervention. Emphasize these outcomes with bold text, standard semantic colors (such as green for success, yellow for warning, and red for failure), or recognizable status indicators (such as check marks or crosses).

This separation creates an effective visual hierarchy: users can scan terminal output quickly without getting bogged down by intermediate noise, while immediately noticing whether a process succeeded or encountered issues.

Examples:

```r
# Good: quiet progress note, emphatic outcome
cli::cli_inform("Computing bootstrap estimates...")
cli::cli_inform(c("v" = "{.strong Succeeded:} computed {.val {n_reps}} replicates."))

# Good: emphatic warning status in pipeline reporting (does not signal an R warning conditi
# (use cli_warn() in package code to signal an R warning condition)
cli::cli_inform(c("!" = "{.strong Warning:} {.val {n_dropped}} missing observations dropped

# Good: emphatic failure status in batch scripts (non-aborting -- processing continues)
cli::cli_inform(c("x" = "{.strong Failed:} input file {.file data.csv} not found."))

# Good: emphatic failure condition that halts execution in package functions
cli::cli_abort("Failed to converge after {.val {max_iter}} iterations.")

# Bad: over-emphasized progress note (distracting and noisy)
cli::cli_inform(c("*" = "{.strong STARTING TO COMPUTE ESTIMATES...}"))
```

```
# Bad: plain unstyled failure (and wrong function if execution should halt; use cli_abort()
cli::cli_inform("failed: check inputs")
```

7.14 Package Code Practices

- **No `library()` in package code**: Use `::` notation or declare in DESCRIPTION Imports. This is enforced by our `.lintr.R` configuration via `undesirable_function_linter()`. Instead, use:
  - `::` for explicit namespace references (e.g., `dplyr::mutate()`)
  - `usethis::use_import_from()` to declare imports in `NAMESPACE`
  - `withr::local_package()` for temporary package loading in tests
- This keeps the global search path clean and makes dependencies explicit. See R Packages - The R landscape[19] for more details.
- **Document all exports**: Use roxygen2 (`@title`, `@description`, `@param`, `@returns`, `@examples`)
- **Avoid code duplication**: Extract repeated logic into helper functions

7.15 Tidyverse Replacements

Use modern tidyverse/alternatives for base R functions:

```
# Data structures
tibble::tibble()           # instead of data.frame()
tibble::tribble()          # instead of manual data.frame creation

# I/O
readr::read_csv()          # instead of read.csv()
readr::write_csv()         # instead of write.csv()
readr::read_rds()          # instead of readRDS()
readr::write_rds()         # instead of saveRDS()

# Data manipulation
dplyr::bind_rows()         # instead of rbind()
dplyr::bind_cols()         # instead of cbind()

# String operations
stringr::str_which()       # instead of grep()
stringr::str_replace()     # instead of gsub()

# Date/time operations
lubridate::NA_Date_        # instead of as.Date(NA)

# Variable labels
labelled::set_variable_labels() # instead of attr(x, "label") <- ...
```

[19] https://r-pkgs.org/code.html#sec-code-r-landscape

```
# Session info
sessioninfo::session_info() # instead of sessionInfo()
```

See also Section 6.39.

7.16 The here Package

The `here` package helps manage file paths in projects by automatically finding the project root and building paths relative to it:

```
library(here)

# Automatically finds project root and builds paths
data <- readr::read_csv(here("data-raw", "survey.csv"))
saveRDS(results, here("inst", "analyses", "results.rds"))
```

This solves the problem of different working directory paths across collaborators. For example, one person might have the project at `/home/oski/Some-R-Project` while another has it at `/home/bear/R-Code/Some-R-Project`. The `here` package handles this automatically.

This works regardless of where collaborators clone the repository. For more details, see the here package vignette[20].

7.16.1 Pass subpath components directly to `here()`

`here::here()` already accepts path components as arguments and joins them with the project root. Do not wrap `here::here()` in `file.path()` or `fs::path()`:

```
# Good: pass subpath components directly
rds_path <- here::here("inst", "extdata", "mapping.rds")

# Bad: redundant wrapping adds noise and defeats the purpose
rds_path <- file.path(here::here(), "inst", "extdata", "mapping.rds")
rds_path <- fs::path(here::here(), "inst", "extdata", "mapping.rds")
```

The wrapped form is functionally equivalent but harder to read and obscures the intent. `here()` was designed to replace `file.path()` for project-relative paths, so let it do its job.

See also Section 6.28 for detailed explanation of the here package.

[20] https://github.com/jennybc/here_here

7.17 Object Naming

Use descriptive names that are both expressive and explicit. Being verbose is useful and easy in the age of autocompletion:

```
# Good
vaccination_coverage_2017_18
absentee_flu_residuals

# Less good
vaxcov_1718
flu_res
```

Prefer nouns for objects and verbs for functions:

```
# Good
clean_data <- prep_study_data(raw_data)  # verb for function, noun for object

# Less clear
data <- process(input)
```

Generally we recommend using nouns for objects and verbs for functions. This is because functions are performing actions, while objects are not.

Use consistent prefixes to signal what a function returns:

- `add_...()` for functions that add one or more columns to a `data.frame` or `tibble` and return the modified table.
- `compute_...()` for functions that compute and return a single vector.

```
# Good
add_age_group <- function(data) {
  data |> dplyr::mutate(age_group = cut(age, breaks = c(0, 18, 65, Inf)))
}

compute_age_group <- function(age) {
  cut(age, breaks = c(0, 18, 65, Inf))
}

# Less clear
make_age_group <- function(data) { ... }
get_age_group  <- function(age)  { ... }
```

This distinction makes the return type visible from the call site without reading the function body.

Use `snake_case` for all variable and function names. Avoid using `.` in names (as in base R's `read.csv()`), as this goes against best practices in modern R and other languages. Modern packages like `readr::read_csv()` follow this convention.

Uppercase acronyms are allowed in `snake_case` names (e.g., `prep_IDs_data`, `calculate_BMI_score`). This is enforced via a custom `object_name_linter` regex pattern in our `.lintr.R` configuration.

Try to make your variable names both more expressive and more explicit. Being a bit more verbose is useful and easy in the age of autocompletion! For example, instead of naming a variable `vaxcov_1718`, try naming it `vaccination_coverage_2017_18`. Similarly, `flu_res` could be named `absentee_flu_residuals`, making your code more readable and explicit.

Base R allows `.` in variable names and functions (such as `read.csv()`), but this goes against best practices for variable naming in many other coding languages. For consistency's sake, `snake_case` has been adopted across languages, and modern packages and functions typically use it (i.e. `readr::read_csv()`). As a very general rule of thumb, if a package you're using doesn't use `snake_case`, there may be an updated version or more modern package that *does*, bringing with it the variety of performance improvements and bug fixes inherent in more mature and modern software.

> **Note**
>
> You may also see `camelCase` throughout the R code you come across. This is *okay* but not ideal – try to stay consistent across all your code with `snake_case`.

> **Note**
>
> Again, it's also worth noting there's nothing inherently wrong with using `.` in variable names, just that it goes against style best practices that are cropping up in data science, so it's worth getting rid of these bad habits now.

For more help, check out Be Expressive: How to Give Your Variables Better Names[21]

7.18 Automated Tools for Style and Project Workflow

7.18.1 Styling

7.18.1.1 RStudio shortcuts

1. **Code Autoformatting** - RStudio includes a fantastic built-in utility (keyboard shortcut: `CMD-Shift-A` (Mac) or `Ctrl-Shift-A` (Windows/Linux)) for autoformatting

[21] https://spin.atomicobject.com/2017/11/01/good-variable-names/

highlighted chunks of code to fit many of the best practices listed here. It generally makes code more readable and fixes a lot of the small things you may not feel like fixing yourself. Try it out as a "first pass" on some code of yours that *doesn't* follow many of these best practices!

2. **Assignment Aligner** - A cool R package[22] allows you to very powerfully format large chunks of assignment code to be much cleaner and much more readable. Follow the linked instructions and create a keyboard shortcut of your choosing (recommendation: `CMD-Shift-Z`). Here is an example of how assignment aligning can dramatically improve code readability:

```
# Before
OUSD_not_found_aliases = list(
  "Brookfield Village Elementary" = str_subset(string = OUSD_school_shapes$schnam, pattern
  "Carl Munck Elementary" = str_subset(string = OUSD_school_shapes$schnam, pattern = "Munck
  "Community United Elementary School" = str_subset(string = OUSD_school_shapes$schnam, pat
  "East Oakland PRIDE Elementary" = str_subset(string = OUSD_school_shapes$schnam, pattern
  "EnCompass Academy" = str_subset(string = OUSD_school_shapes$schnam, pattern = "EnCompass
  "Global Family School" = str_subset(string = OUSD_school_shapes$schnam, pattern = "Global
  "International Community School" = str_subset(string = OUSD_school_shapes$schnam, pattern
  "Madison Park Lower Campus" = "Madison Park Academy TK-5",
  "Manzanita Community School" = str_subset(string = OUSD_school_shapes$schnam, pattern = "
  "Martin Luther King Jr Elementary" = str_subset(string = OUSD_school_shapes$schnam, patte
  "PLACE @ Prescott" = "Preparatory Literary Academy of Cultural Excellence",
  "RISE Community School" = str_subset(string = OUSD_school_shapes$schnam, pattern = "Rise
)
```

```
# After
OUSD_not_found_aliases = list(
  "Brookfield Village Elementary"      = str_subset(string = OUSD_school_shapes$schnam, pat
  "Carl Munck Elementary"              = str_subset(string = OUSD_school_shapes$schnam, pat
  "Community United Elementary School" = str_subset(string = OUSD_school_shapes$schnam, pat
  "East Oakland PRIDE Elementary"      = str_subset(string = OUSD_school_shapes$schnam, pat
  "EnCompass Academy"                  = str_subset(string = OUSD_school_shapes$schnam, pat
  "Global Family School"               = str_subset(string = OUSD_school_shapes$schnam, pat
  "International Community School"     = str_subset(string = OUSD_school_shapes$schnam, pat
  "Madison Park Lower Campus"          = "Madison Park Academy TK-5",
  "Manzanita Community School"         = str_subset(string = OUSD_school_shapes$schnam, pat
  "Martin Luther King Jr Elementary"   = str_subset(string = OUSD_school_shapes$schnam, pat
  "PLACE @ Prescott"                   = "Preparatory Literary Academy of Cultural Excellen
  "RISE Community School"              = str_subset(string = OUSD_school_shapes$schnam, pat
)
```

7.18.1.2 `{styler}`[23]

`{styler}`[24] is another cool R package from the Tidyverse[25] that can be powerful and used as a first pass on entire projects that need refactoring. The most useful function of the

[22] https://www.r-bloggers.com/align-assign-rstudio-addin-to-align-assignment-operators/
[23] https://styler.r-lib.org/
[24] https://styler.r-lib.org/
[25] https://tidyverse.org/blog/2017/12/styler-1.0.0/

package is the `style_dir` function, which will style all files within a given directory. See the function's documentation[26] and the vignette linked above for more details.

> **Note**
>
> The default Tidyverse styler is subtly different from some of the things we've advocated for in this document. Most notably we differ with regards to the assignment operator (`<-` vs `=`) and number of spaces before/after "tokens" (i.e. Assignment Aligner add spaces before `=` signs to align them properly). For this reason, we'd recommend the following: `style_dir(path = ..., scope = "line_breaks", strict = FALSE)`. You can also customize `{styler}`[a] even more[b] if you're really hardcore.
>
> [a] https://styler.r-lib.org/
> [b] http://styler.r-lib.org/articles/customizing_styler.html

> **Note**
>
> As is mentioned in the package vignette linked above, `{styler}`[a] modifies things *in-place*, meaning it overwrites your existing code and replaces it with the updated, properly styled code. This makes it a good fit on projects *with version control*, but if you don't have backups or a good way to revert back to the initial code, I wouldn't recommend going this route.
>
> [a] https://styler.r-lib.org/

> **styler Package**
>
> For automated styling of entire projects:

```
# Install styler
install.packages("styler")

# Style all files in R/ directory
styler::style_dir("R/")

# Style entire package
styler::style_pkg()

# Note: styler modifies files in-place
# Always use with version control so you can review changes
```

7.18.1.3 `{lintr}`[27]

Linters are programming tools that check adherence to a given style, syntax errors, and possible semantic issues. The R linter, called `{lintr}`[28], helps keep files consistent across different authors and even different organizations. For example, it notifies you if you have unused variables, global variables with no visible binding, not enough or superfluous

[26] https://www.rdocumentation.org/packages/styler/versions/1.1.0/topics/style_dir
[27] https://lintr.r-lib.org/
[28] https://lintr.r-lib.org/

whitespace, and improper use of parentheses or brackets. A list of its other purposes can be found in this link[29], and most guidelines are based on the Tidyverse R Style Guide[30].

> **Note**
>
> You can customize your settings to set defaults or to exclude files. More details can be found here[a].
>
> [a] https://cran.r-project.org/web/packages/lintr/readme/README.html#project-configuration

> **Note**
>
> The `lintr` package goes hand in hand with the `styler` package. The styler can be used to automatically fix the problems that the lintr catches.

7.18.2 Using Lintr

> **`lintr` package**
>
> For checking code style without modifying files:

```
# Install lintr (and pkgload, used below)
install.packages(c("lintr", "pkgload"))

# For package code, load the package first (see note below)
pkgload::load_all()

# Lint the entire package
lintr::lint_package()

# Lint a specific file
lintr::lint("R/my_function.R")
```

> The linter checks for:
>
> - Unused variables
> - Improper whitespace
> - Line length issues
> - Style guide violations
>
> For package code, run `pkgload::load_all()` (or `devtools::load_all()`) **before** `lintr::lint_package()`. Our configuration includes `object_usage_linter()`, which resolves symbols using the loaded package; without loading first, it checks against a stale installed copy (or none), producing spurious `no visible binding for global variable` warnings. Loading also runs any `.onLoad()` side effects: for example, our `lms`[a] linter package registers its `rex` shortcuts in `.onLoad()`, and the linter only sees them once the package is loaded.
> Our lab uses `.lintr.R` files for configuration (the `.lintr` format is also supported by lintr, but we prefer `.lintr.R` for better R syntax support).

[29] https://cran.r-project.org/web/packages/lintr/readme/README.html#available-linters
[30] https://style.tidyverse.org/

[a] https://github.com/UCD-SERG/lab-manual/tree/main/lms

7.18.3 Our Lab's Lintr Configuration

Our lab uses a custom `.lintr.R` configuration file in each repository to enforce our style standards. You can view the lab-manual's configuration at https://github.com/UCD-SERG/lab-manual/blob/main/.lintr.R.

Key linters we enable:

- `pipe_consistency_linter(pipe = "|>")`: Enforces use of native pipe `|>` instead of `%>%`
- `object_name_linter()`: Enforces `snake_case` with custom regex allowing uppercase acronyms
- `undesirable_function_linter()`: Prohibits base messaging functions and `library()` in package code
- `redundant_equals_linter()`: Catches redundant `= TRUE` when `TRUE` is the default

Linters we disable:

- `return_linter(return_style = "explicit")`: Every function should end with `return(return_value)` rather than just `return_value`. We may sometimes disable this linter in older projects when we aren't ready to clean this issue up, but for all new code, we require explicit returns as a lab standard, following the Google R Style Guide[31].
- `trailing_whitespace_linter`: Disabled (handled by `styler` instead)

Exceptions:

Our configuration allows relaxed rules for certain directories:

- `data-raw/`: Pipe consistency and undesirable function rules relaxed (exploratory scripts)
- `vignettes/`: Undesirable function and object naming rules relaxed (tutorial code may need `library()`)
- `inst/examples/`: Undesirable function rules relaxed
- `tests/testthat.R`: Undesirable function rules relaxed

7.18.3.1 `{jarl}`[32]

`{jarl}`[33] (Just Another R Linter) is a fast, standalone R linter written in Rust by Etienne Bacher. It is designed to provide high-speed static analysis for R code with minimal overhead.

Because `{jarl}` is implemented in Rust, it performs static analysis significantly faster than standard R-based linters.

[31] https://google.github.io/styleguide/Rguide.html#use-explicit-returns
[32] https://jarl.etiennebacher.com/
[33] https://jarl.etiennebacher.com/

Key Features

- **Fast execution:** Written in Rust to deliver near-instant linting feedback.
- **`{lintr}` compatibility:** Implements many common `{lintr}` rules (such as unused variables, improper whitespace, indentation, object naming, and syntax checks).
- **Flexible invocation:** Can be run as a standalone CLI tool, via pre-commit hooks, or integrated into editors via its editor integrations[34].
- **Auto-fixing:** Can automatically fix supported rule violations via `--fix`.
- **Project configuration:** Configured using a `jarl.toml`[35] file in the project root.

Basic Usage

Run the standalone CLI (see the getting started guide[36]):

```
# Lint all R files in the current project
jarl check .

# Automatically fix supported lint violations
jarl check . --fix
```

Comparison with `{lintr}`

Feature	`{lintr}`	`{jarl}`
Implementation	R	Rust
Execution speed	Standard	Extremely fast (Rust binary)
Ecosystem maturity	Industry standard, wide rule coverage	Emerging, expanding rule set
Custom R linters	Yes (e.g., `{lms}`[37])	No (Rust-only rule definitions)
Primary use case	CI verification, custom lab rules	Fast pre-commit checks, local interactive editing

Lab Guidance

Our lab uses `{lintr}`[a] with our custom `.lintr.R`[b] configuration and `{lms}`[c] package as the primary standard for CI pipelines and package validation. However, `{jarl}` is an excellent choice for local pre-commit hooks and interactive developer workflows where instant lint feedback saves time on large repositories.

[a] https://lintr.r-lib.org/
[b] https://github.com/UCD-SERG/lab-manual/blob/main/.lintr.R
[c] https://github.com/UCD-SERG/lab-manual/tree/main/lms

[34] https://jarl.etiennebacher.com/howto/editors
[35] https://jarl.etiennebacher.com/reference/config-file
[36] https://jarl.etiennebacher.com/getting-started
[37] https://github.com/UCD-SERG/lab-manual/tree/main/lms

7.18.4 Linting Changed Files vs. the Whole Project in CI

A project can fall out of lint compliance without any change to its own source code or `.lintr` configuration: `{lintr}`[38] releases sometimes add default linters or tighten existing ones, so code that passed yesterday can fail after a routine package update.

When a continuous-integration job lints the whole project (for example, `lintr::lint_dir()` on every pull request), those newly introduced findings surface on whatever pull request happens to run CI next. That pull request then has to expand into unrelated files just to turn the lint check green, which conflicts with our expectation that a pull request stays scoped to one concern.

In most cases, a better default is to lint **changed files only**, so pre-existing findings elsewhere in the project don't block unrelated work. r-lib/actions[39] provides a ready-made `lint-changed-files` example workflow; install it with `usethis::use_github_action("lint-changed-files")`.

Whole-project linting still has a place: run it on a schedule or as a manually triggered workflow, so that project-wide drift is still detected and cleaned up deliberately in its own dedicated pull request, rather than as a side effect of someone else's change.

7.19 Additional Resources

- Tidyverse style guide (Wickham 2023a): Detailed coding style conventions for writing clear, consistent R code. Covers naming, syntax, pipes, functions, and more.

[38] https://lintr.r-lib.org/
[39] https://github.com/r-lib/actions/blob/v2-branch/examples/lint-changed-files.yaml

8 Big data

8.1 The data.table package

It may also be the case that you're working with very large datasets. Generally I would define this as 10+ million rows. As is outlined in this document, the 3 main players in the data analysis space are Base R, `Tidvyerse` (more specificially, `dplyr`), and `data.table`. For a majority of things, Base R is inferior to both `dplyr` and `data.table`, with concise but less clear syntax and less speed. `Dplyr` is architected for medium and smaller data, and while its very fast for everyday usage, it trades off maximum performance for ease of use and syntax compared to `data.table`. An overview of the `dplyr` vs `data.table` debate can be found in this stackoverflow post[1] and all 3 answers are worth a read.

You can also achieve a performance boost by running `dplyr` commands on `data.table`s, which I find to be the best of both worlds, given that a `data.table` is a special type of `data.frame` and fairly easy to convert with the `as.data.table()` function. The speedup is due to `dplyr`'s use of the `data.table` backend and in the future this coupling should become even more natural.

If you want to test whether using a certain coding approach increases speed, consider the `tictoc` package. Run `tic()` before a code chunk and `toc()` after to measure the amount of system time it takes to run the chunk. For example, you might use this to decide if you *really* need to switch a code chunk from `dplyr` to `data.table`.

8.2 Using downsampled data

In our studies with very large datasets, we save "downsampled" data that usually includes a 1% random sample stratified by any important variables, such as year or household id. This allows us to efficiently write and test our code without having to load in large, slow datasets that can cause RStudio to freeze. Be very careful to be sure which dataset you are working with and to label results output accordingly.

8.3 Optimal RStudio set up

Using the following settings will help ensure a smooth experience when working with big data. In RStudio, go to the "Tools" menu, then select "Global Options". Under "General":

Workspace

- **Uncheck** Restore RData into workspace at startup
- Save workspace to RData on exit – choose **never**

[1] https://stackoverflow.com/questions/21435339/data-table-vs-dplyr-can-one-do-something-well-the-other-cant-or-does-poorly/27840349#27840349

History

- **Uncheck** Always save history

Unfortunately RStudio often gets slow and/or freezes after hours working with big datasets. Sometimes it is much more efficient to just use Terminal / gitbash to run code and make updates in git.

9 Data masking

For information about UC Davis computing resources for data-intensive work, see Chapter 18.

9.1 General Overview

This chapter covers data masking, a unique process in R in which columns are treated as distinct objects within their dataframe's environment. In our lab, data masking most frequently comes up when writing wrapper functions where arguments to indicate column names are supplied as strings. We often do this when we repeat the same code on multiple columns, and want to apply a function to a vector of strings that correspond to column names in a dataframe. For example, we might want to clean multiple columns using the same function or estimate the same model under different feature sets. Here, we try to break down what data masking is, why this error comes up, and common approaches to solve this problem.

9.1.1 What is Data Masking?

Within certain tidyverse operations, columns are called as if they were variables. For example, while running `df |> mutate(X = …)` R recognizes that `X` specifically references a column in `df` without explicitly stating its membership `df |> mutate(df$X = …)` or calling the column name as a string `df |> mutate("X" = …)`.

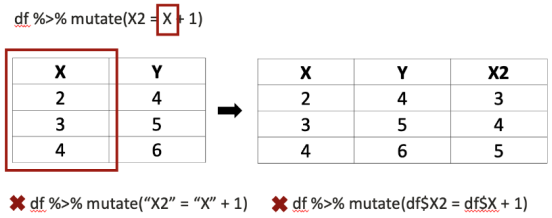


Figure 9.1: Data masking in tidyverse operations

However, this behavior may introduce errors when we attempt to incorporate variables from the global environment within these tidyverse pipelines. In the example shown in Figure 9.1, `column_name = "X"` followed by `df |> mutate(X2 = column_name + 1)` would yield an error, since `column_name` is not a column in `df` and the variable `column_name` is not defined within the environment of `df`

9.1.2 Using tidy evaluation for data masking

In dplyr-based R programming, we make use of tidy evaluation. This allows us to avoid using base R syntax to reference specific columns in a data frame. By leveraging Tidy evaluation-based data masking, we can employ long pipes with several dplyr verbs to manipulate our data using stand-alone variables that store column names as strings.

For example, consider a data frame "df" that contains a column called "heavyrain" that we want to manipulate. Suppose we wanted to convert the values of "heavyrain" into a factor.

Using base R, which does not mask data, heavyrain must have quotes to be treated as a data-variable:

```
df[["outcome"]] = as.factor(df[["heavyrain"]])
```

In a dplyr pipe, heavyrain is being masked using tidy evaluation and will be correctly interpreted as a column because it is recognized as a data-variable:

```
df |> mutate(outcome = as.factor(heavyrain))
```

With modified data masking, heavyrain is a string that is coerced into being recognized as a data-variable:

```
var_name = "heavyrain"
df |> mutate(outcome = as.factor(!!sym(var_name)))
```

While cleaner and often more convenient, the data frame that var_name is in is now "masked" and we refer to the vectors in the dataframe (data-variables) as though it is an object of its own (an environmental-variable). This is why we can just say the variable's name in the context of a pipe – we treat it as though it's an object defined in our environment. Within normal scripts, this is usually fine, because the data frame is "held on to" in the pipe. However, it can cause some programming hurdles when writing functions that take strings of variable/column names as arguments. In the next section, we briefly describe how to troubleshoot common errors in data masking, as relevant to our lab's work.

9.2 Data masking vs. tidy selection

Many `{tidyverse}`[1] verbs use *two* different flavors of tidy evaluation, and the way you refer to a column stored in a string variable depends on which one you are in.

- **Data-masking** contexts treat column names as values you compute *with*: the `{dplyr}`[2] verbs `mutate()`, `filter()`, `summarize()`/`summarise()` (the summary expressions), `group_by()`, and `arrange()`, plus `aes()` in `{ggplot2}`[3].
- **Tidy-selection** contexts treat column names as columns you *choose*: `select()`, `rename()`, `relocate()`, `pull()`, the `.cols` argument of `across()`, the `.by` argument of `mutate()`/`summarize()`, and the `cols`, `names_from`, `values_from`, and `id_cols` arguments of `pivot_*()` in `{tidyr}`[4].

[1] https://www.tidyverse.org/
[2] https://dplyr.tidyverse.org/
[3] https://ggplot2.tidyverse.org/
[4] https://tidyr.tidyverse.org/

To refer to a column named by a string variable, use the tool that matches the context:

Table 9.1: Referring to a column named by a string variable

Context	Use	Avoid
Data-masking	`!!sym(var)` or `.data[[var]]`	bare `var` (uses the string value itself, not the column it names)
Tidy-selection	`all_of(var)` / `any_of(var)`	the `.data` pronoun (`.data[[var]]`, `.data$col`)

Use `all_of()` when every selected name must exist (it errors if one is missing) and `any_of()` when names that are absent should simply be dropped.

9.2.1 The `.data` pronoun does not belong in tidy selection

A common (and **silent**) mistake is reaching for the `.data` pronoun inside a tidy-selection context:

```
# Soft-deprecated: .data used in selection contexts
df |> select(.data$Subject, .data[[var]])
df |> summarize(.by = c(.data$group), med = median(value))
df |> tidyr::pivot_wider(names_from = .data$Parameter)
```

The `.data` pronoun is a *data-masking* tool, not a selection tool. Using it in a selection context was deprecated in `{tidyselect}`[5] 1.2.0 (release notes[6]), but it is a **soft** deprecation, signaled through `{lifecycle}`[7]. Per the `{lifecycle}` documentation[8], a soft-deprecated feature warns only users who trigger it from the global environment and developers who use it directly (for example when running `{testthat}`[9] tests); there is *no warning when the deprecated feature is called indirectly by another package.* So in installed-package code it usually emits *no warning at all*, and it slips past interactive use, CI, and code review unnoticed. Rewrite these with `all_of()`/`any_of()` and plain strings:

```
# Idiomatic tidy selection
df |> select(all_of(c("Subject", var)))
df |> summarize(.by = all_of("group"), med = median(value))
df |> tidyr::pivot_wider(names_from = "Parameter")
```

To make this class of soft deprecation fail loudly, set `options(lifecycle_verbosity = "error")` in your tests (for example in `tests/testthat/setup.R`). This option promotes deprecation signals to errors regardless of the soft-versus-warning distinction (`{lifecycle}` reference[10]). Plain `options(warn = 2)` will *not* catch it, because `warn = 2` only converts warnings into errors once a warning is actually signaled, and for a soft deprecation in installed-package code no warning is signaled in the first place.

[5] https://tidyselect.r-lib.org/
[6] https://www.tidyverse.org/blog/2022/10/tidyselect-1-2-0/
[7] https://lifecycle.r-lib.org/
[8] https://lifecycle.r-lib.org/articles/communicate.html
[9] https://testthat.r-lib.org/
[10] https://lifecycle.r-lib.org/reference/deprecate_soft.html

9.3 Technical Overview

This section covers the R functions and tools that we often use in the context of data masking, focusing on the bang bang operator (`!!`) with symbol coercion (`sym()`) and the Walrus operator (`:=`).

The combined use of `!!` and `sym()` allows us to use strings, rather than data-variables, to reference column names within dplyr. Together, `!!sym("column_name")` forces dplyr to recognize "column_name" as a data-variable prior to evaluating the rest of the expression, enabling the ability to perform calculations on the column while referring to it as a string. `sym()` is a function that turns strings into symbols. In the context of a dplyr pipe, these symbols are interpreted as data-variables. The `!!` (bang bang) operator tells dplyr to evaluate the sym() expression first, e.g. to unquote its expression (e.g. "column_name") and evaluate it as a pre-existing object, first. This is helpful because often we use `sym("column_name")` within a larger expression, and dplyr might evaluate other elements of the expression first without !!, causing errors.

When we want to create a new column (via mutate or summarize), the Walrus operator (`:=`) allows us to specify the new column's name using a string. For example, while `df |> mutate("new_column" = values)` would yield an error, `df |> mutate("new_column" := values)` will correctly create a new column called "new_column". If we want to use a variable representing a string, we can use !! to force the variable to be evaluated before using `:=` to assign the value of the new column.

```
col_name = "new_column"
df |> mutate(!!col_name := values)
```

9.3.1 Example

Suppose we want to write a function "generate_descriptive_table" to summarize how the prevalence of "outcome" varies under different levels of a "risk_factor" in a data frame "df"

We can start by writing the function shell:

```
generate_descriptive_table <- function (df, outcome, rf) {
outcome_dist_by_rf <- ….
return(outcome_dist_by_rf)
}
```

Next, we can filter the data frame for only rows in which "rf" and "outcome" are not missing. We can use !! and sym() within filter to evaluate the strings stored in "rf" and "outcome". Note that defining `!!sym(outcome)` or `!!sym(rf)` in variables *outside of the dplyr pipeline* will *not* work.

```
generate_descriptive_table <- function(df, outcome, rf) {
  outcome_dist_by_rf <- df |>
    filter(!is.na(!!sym(outcome)), !is.na(!!sym(rf))) |>
    ….
  return(outcome_dist_by_rf)
}
```

Similarly, we use !! and sym() in group_by to evaluate column name, stored as a string in the argument "rf"

```
generate_descriptive_table <- function(df, outcome, rf) {
  outcome_dist_by_rf <- df |>
    filter(!is.na(!!sym(outcome)), !is.na(!!sym(rf))) |>
    group_by(!!sym(rf)) |>
    ….
  return(outcome_dist_by_rf)
}
```

Finally, we can use the walrus operator, !! and sym() with "summarize" to create a new column that takes the mean of the column referenced in "rf". We also use "glue" or "paste" to give the new column an informative name that includes the "outcome" it describes.

```
generate_descriptive_table <- function(df, outcome, rf) {
  outcome_dist_by_rf <- df |>
    filter(!is.na(!!sym(outcome)), !is.na(!!sym(rf))) |>
    group_by(!!sym(rf)) |>
    summarize(!!(glue::glue("{outcome}_prev")) := mean(!!sym(outcome)))
  return(outcome_dist_by_rf)
}
```

OR

```
generate_descriptive_table <- function(df, outcome, rf) {
  outcome_dist_by_rf <- df |>
    filter(!is.na(!!sym(outcome)), !is.na(!!sym(rf))) |>
    group_by(!!sym(rf)) |>
    summarize(!!(paste0(outcome, "_prev")) := mean(!!sym(outcome)))
  return(outcome_dist_by_rf)
}
```

OR

```
generate_descriptive_table <- function(df, outcome, rf) {
  new_column_name = paste0(outcome, "_prev")
  outcome_dist_by_rf <- df |>
    filter(!is.na(!!sym(outcome)), !is.na(!!sym(rf))) |>
    group_by(!!sym(rf)) |>
    summarize(!!(new_column_name) := mean(!!sym(outcome)))
  return(outcome_dist_by_rf)
}
```

10 Quarto

10.1 Introduction

Quarto[1] is an open-source scientific and technical publishing system that allows you to create documents, books, websites, presentations, and more. Quarto provides a unified authoring framework for data science, combining your code, its results, and your prose. Quarto documents are fully reproducible and support dozens of output formats, like PDFs, Word files, presentations, and more.

Quarto files are designed to be used in three ways:

1. **For communicating to decision-makers,** who want to focus on the conclusions, not the code behind the analysis.
2. **For collaborating with other data scientists** (including future you!), who are interested in both your conclusions, and how you reached them (i.e., the code).
3. **As an environment in which to do data science,** as a modern-day lab notebook where you can capture not only what you did, but also what you were thinking.

10.1.1 Key Features

Multi-format Output: Quarto documents can be rendered into HTML, PDF, MS Word, ePub, PowerPoint, Revealjs presentations, dashboards, websites, and books from a single source file. This allows authors to maintain one document but publish it in multiple formats without rewriting content.

Rich Markdown Authoring: Content is created in markdown, with support for figures, tables, equations (LaTeX), citations, cross-references, and advanced layout features like tabs, callouts, and panels.

Embedded Executable Code: Integrate code chunks (R, Python, Julia, Observable JS) that can be executed and the results rendered directly in the document. This allows for dynamic results, data analysis, plots, and reproducible research workflows.

Interactivity: Add interactive components such as widgets, tab sets, and collapsible sections for richer communication with readers.

Customization: Extensive theming and styling options, including custom CSS and advanced layout controls for polished, publication-quality output.

Project Management: Organize large projects and integrate with version control tools like Git. Use Quarto projects to group related documents, manage dependencies, and orchestrate rendering.

[1] https://quarto.org/

10.1.2 Why Quarto?

If you're an R Markdown user, you might be thinking "Quarto sounds a lot like R Markdown." You're not wrong! Quarto unifies the functionality of many packages from the R Markdown ecosystem (rmarkdown, bookdown, distill, xaringan, etc.) into a single consistent system as well as extends it with native support for multiple programming languages like Python and Julia in addition to R. In a way, Quarto reflects everything that was learned from expanding and supporting the R Markdown ecosystem over a decade.

10.1.3 Getting Started

To get started with Quarto:

- **Installation:** Quarto CLI is included with RStudio, so if you have a recent version of RStudio, you already have Quarto. Otherwise, visit https://quarto.org/docs/get-started/
- **Documentation:** The official Quarto documentation is available at https://quarto.org/docs/guide/
- **R4DS Chapter:** For an excellent introduction to using Quarto with R, see the Quarto chapter in R for Data Science[2] (Wickham et al. 2023)

10.2 Quarto Basics

A Quarto document is a plain text file with the extension `.qmd`. It contains three important types of content:

1. An (optional) **YAML header** surrounded by `---`s
2. **Chunks** of code surrounded by ```` ``` ````
3. Text mixed with simple text formatting like `# heading` and `_italics_`

10.2.1 Creating a New Quarto Document

In RStudio, create a new Quarto document using **File > New File > Quarto Document...** in the menu bar. RStudio will launch a wizard that you can use to pre-populate your file with useful content that reminds you how the key features of Quarto work.

10.2.2 Visual vs. Source Editor

RStudio provides two ways to edit Quarto documents:

Visual Editor: The Visual editor provides a WYSIWYM[3] interface for authoring Quarto documents. If you're new to computational documents but have experience using tools like Google Docs or MS Word, the visual editor is the easiest way to get started. In the visual editor you can use the buttons on the menu bar to insert images, tables, cross-references, etc., or you can use the catch-all + / or Ctrl + / shortcut to insert just about anything.

[2] https://r4ds.hadley.nz/quarto.html
[3] https://en.wikipedia.org/wiki/WYSIWYM

Source Editor: The source editor allows you to edit the raw markdown and code. While the visual editor will feel familiar to those with experience in word processors, the source editor will feel familiar to those with experience writing R scripts or R Markdown documents. The source editor can also be useful for debugging any Quarto syntax errors since it's often easier to catch these in plain text.

You can switch between the visual and source editors at any time using the toggle in the top-left of the editor pane.

10.2.3 Rendering Documents

To produce a complete report containing all text, code, and results:

- Click **"Render"** in RStudio, or
- Press **Cmd/Ctrl + Shift + K**, or
- Use `quarto::quarto_render("document.qmd")` in R, or
- Use `quarto render document.qmd` in the terminal

When you render the document, Quarto sends the `.qmd` file to **knitr**, which executes all of the code chunks and creates a new markdown (`.md`) document which includes the code and its output. The markdown file generated by knitr is then processed by **pandoc**, which is responsible for creating the finished file in your chosen format (HTML, PDF, Word, etc.).

10.2.4 Code Chunks

To run code inside a Quarto document, you need to insert a chunk. There are three ways to do so:

1. The keyboard shortcut **Cmd + Option + I** / **Ctrl + Alt + I**
2. The **"Insert"** button icon in the editor toolbar
3. By manually typing the chunk delimiters ```` ```{r} ```` and ```` ``` ````

Chunks can be given an optional label and various chunk options:

````
```{r}
#| label: simple-addition
#| echo: false
1 + 1
```
````

Common chunk options include:

- `#| label:` - give the chunk a name
- `#| echo: false` - hide the code but show the output
- `#| code-fold: true` - allow readers to toggle code visibility (useful when output is more important to the narrative than the code)
- `#| eval: false` - show the code but don't run it
- `#| include: false` - run the code but hide both code and output
- `#| warning: false` - hide warnings
- `#| message: false` - hide messages

Use `code-fold: true` for chunks where the output is important to the narrative and not the code used to produce it. This allows interested readers to expand and view the code while keeping the document focused on results.

10.2.5 Format-Specific Settings

When rendering to multiple output formats (HTML, PDF, DOCX, EPUB), you may want different chunk options or behavior for different formats. Use `knitr::pandoc_to()` with `if ()` statements to detect the output format and set format-specific settings.

Example: Different figure sizes for different formats

````
```{r}
#| label: example-plot
#| fig-width: !expr if (knitr::pandoc_to("html")) 8 else 6
#| fig-height: !expr if (knitr::pandoc_to("html")) 6 else 4

plot(1:10)
```
````

Example: Conditional code execution based on format

````
```{r}
if (knitr::pandoc_to("docx")) {
  # DOCX-specific code
  knitr::kable(data, format = "simple")
} else if (knitr::pandoc_to("html")) {
  # HTML-specific code
  knitr::kable(data, format = "html")
} else {
  # PDF or other formats
  knitr::kable(data, format = "latex")
}
```
````

Common format detection patterns:

- `knitr::pandoc_to("html")` - returns `TRUE` for HTML output
- `knitr::pandoc_to("latex")` - returns `TRUE` for PDF output
- `knitr::pandoc_to("docx")` - returns `TRUE` for Word output
- `knitr::pandoc_to("epub")` - returns `TRUE` for EPUB output

This technique is particularly useful when you need to:

- Adjust figure dimensions for different page sizes
- Use different table formatting for different outputs
- Include or exclude content based on output format
- Set format-specific styling or options

10.2.6 Text Formatting

Quarto uses Pandoc's markdown for text formatting:

- `*italic*` or `_italic_` produces *italic* text
- `**bold**` or `__bold__` produces **bold** text
- `` `code` `` produces `code` formatting
- `# Heading 1`, `## Heading 2`, `### Heading 3` for headings
- Bullet lists start with `-` or `*`
- Numbered lists start with `1.`, `2.`, etc.
- `[link text](url)` creates hyperlinks
- `![alt text](image.png)` inserts images

Important: Always include a blank line before bullet lists and numbered lists in markdown and Quarto documents.

For more details on using Quarto for writing and analysis, see the Quarto chapter in R for Data Science[4] (Wickham et al. 2023).

10.3 Building Quarto Books

Quarto books let you author entire books (or course notes, manuals, dissertations, etc.) in markdown. Quarto books are ideal for documentation, tutorials, lab manuals, and other long-form content.

10.3.1 Creating a Quarto Book

Starting from a template (recommended):

Using a template is the fastest way to get started with a Quarto book, as it provides pre-configured settings, example content, and GitHub Actions workflows for automated deployment:

1. **UCD-SeRG Quarto Book Template** - Our recommended template with pre-configured settings for lab publications:
   - Repository: https://github.com/UCD-SERG/qbt
   - Click "Use this template" → "Create a new repository" on GitHub
   - Clone your new repository and start editing
   - Includes GitHub Actions for automatic deployment to GitHub Pages
2. **Coatless Tutorials Quarto Book Template** - Another template with helpful examples:
   - Repository: https://github.com/coatless-tutorials/quarto-book-template
   - Includes examples of common Quarto book features
3. **DataLab Quarto Template** - Template from the UC Davis DataLab and Davis R Users Group:
   - Repository: https://github.com/d-rug/datalab_template_quarto
   - Provides a starting point for DataLab workshop materials and tutorials

[4] https://r4ds.hadley.nz/quarto.html

While these templates jumpstart your project with up-to-date configuration and workflow files, you should still come up to speed on what all the config files do (particularly `_quarto.yml` and any GitHub Actions workflows) so you can modify and debug them as needed. The templates serve as central locations for the most current versions of these files and best practices.

Starting from scratch:

If you prefer to start from scratch, you can create a new Quarto book project using the Quarto CLI:

```
# Create a new Quarto book project
quarto create project book mybook
cd mybook
```

This will create a basic book structure with:

- `_quarto.yml` - configuration file for your book
- `index.qmd` - the home page / preface
- Sample chapter files
- `references.qmd` - bibliography/references page

10.3.2 Building and Previewing

Once you have a Quarto book project, you can build and preview it:

```
# Render the entire book
quarto render

# Preview with live reload (recommended during development)
quarto preview
```

The `quarto preview` command starts a local web server and automatically refreshes the preview whenever you save changes to your files.

10.3.3 Book Structure

A typical Quarto book is organized as follows:

`_quarto.yml`: The main configuration file that defines:

- Book metadata (title, author, date)
- Chapter order
- Output formats (HTML, PDF, ePub, etc.)
- Styling and theme
- Navigation options

Chapter files: Individual `.qmd` files for each chapter. These are listed in the `chapters` section of `_quarto.yml`.

Parts: You can organize chapters into parts for better structure:

```yaml
book:
  chapters:
    - index.qmd
    - part: "Getting Started"
      chapters:
        - intro.qmd
        - basics.qmd
    - part: "Advanced Topics"
      chapters:
        - advanced.qmd
```

10.3.4 Book Features

Quarto books support many advanced features:

Cross-references: Reference figures, tables, equations, and sections throughout your book:

```
See @fig-plot for details.
As shown in @tbl-results.
Refer to @sec-introduction.
```

Citations: Include a bibliography and cite sources:

```
According to @smith2020, the method works well.
```

Search: Automatic full-text search in HTML output.

Downloads: Offer PDF, ePub, and Word versions alongside HTML.

Navigation: Automatic table of contents, previous/next chapter buttons, and breadcrumbs.

Customization: Custom themes, CSS, and templates for professional appearance.

10.3.5 Example: This Lab Manual

This lab manual itself is a Quarto book! You can view its source code at https://github.com/UCD-SERG/lab-manual to see how we've structured chapters, used includes for modular content, and configured various output formats.

10.3.6 Resources

- Quarto Books Guide[5] - comprehensive documentation for Quarto books
- Quarto Publishing Guide[6] - how to publish your book online
- R4DS Quarto chapter[7] (Wickham et al. 2023) - excellent introduction to Quarto

[5] https://quarto.org/docs/books/
[6] https://quarto.org/docs/publishing/
[7] https://r4ds.hadley.nz/quarto.html

10.4 Dashboards

Quarto dashboards[8] turn a `.qmd` (or Jupyter notebook) into a data dashboard by setting `format: dashboard` in the document YAML.

Layout is driven by ordinary markdown headings, which define pages, rows, columns, and tabsets; each computational cell becomes a "card" holding a plot, table, or text. Value boxes highlight key metrics, and sidebars and toolbars hold interactive inputs.

Dashboards work with R, Python, Julia, and Observable JavaScript, and display both interactive widgets (`{plotly}`[9], `{leaflet}`[10], `{htmlwidgets}`[11]) and static graphics (`{ggplot2}`[12], `Matplotlib`).

By default a dashboard renders to static HTML, so it can be published anywhere a web page can (including GitHub Pages, like the rest of our Quarto output) with no server behind it. When server-side interactivity is genuinely needed, a Shiny[13] backend can drive the dashboard, at the cost of needing a Shiny server to host it. For most lab reporting, start with the static form and add Shiny only if filtering or recomputation on the reader's side becomes necessary.

See the dashboard examples gallery[14] for layouts to copy from.

10.5 Quarto Profiles

Quarto profiles allow you to customize rendering behavior for different purposes. A profile is a named set of configuration options that can be activated when rendering. This is particularly useful when you want to render the same source files in different formats or for different audiences.

10.5.1 What are Profiles?

Profiles let you maintain multiple `_quarto.yml` configuration files in the same project. For example, you might have:

- `_quarto.yml` - default book configuration
- `_quarto-revealjs.yml` - configuration for rendering chapters as slide decks
- `_quarto-print.yml` - configuration optimized for PDF printing

[8] https://quarto.org/docs/dashboards/
[9] https://plotly.com/r/
[10] https://rstudio.github.io/leaflet/
[11] https://www.htmlwidgets.org/
[12] https://ggplot2.tidyverse.org/
[13] https://shiny.posit.co/
[14] https://quarto.org/docs/gallery/#dashboards

10.5.2 Example: Rendering Chapters as Slides

A excellent example of using Quarto profiles comes from the Regression Models for Epidemiology[15] course materials by D. Morrison.

The project includes a `_quarto-revealjs.yml` profile that allows each chapter to be compiled as a RevealJS slide deck, in addition to being part of the book.

To render a single chapter as slides:

```
quarto render chapter-name.qmd --profile=revealjs
```

To render all chapters listed in the profile as slides:

```
quarto render --profile=revealjs
```

10.5.3 Creating a Profile

To create a profile:

1. Create a new YAML file named `_quarto-{profile-name}.yml`
2. Include only the configuration options that differ from your default `_quarto.yml`
3. Activate the profile when rendering using `--profile={profile-name}`

Example profile structure:

`_quarto-slides.yml`:

```yaml
project:
  type: default
  output-dir: slides

format:
  revealjs:
    theme: serif
    slide-number: true
    preview-links: auto
```

10.5.4 Common Use Cases

Multiple output formats: Maintain separate configurations for web, print, and presentation versions of your content.

Different audiences: Create versions with or without solutions, technical details, or instructor notes.

Development vs. production: Use a development profile with faster rendering options during writing, and a production profile with full features for final output.

Course materials: Render the same content as both a reference book and lecture slides, as demonstrated in the RME course[16].

[15] https://d-morrison.github.io/rme/
[16] https://d-morrison.github.io/rme/

10.5.5 Resources

- Quarto Profiles Documentation[17]
- RME Example[18] - see the source at https://github.com/d-morrison/rme for a working example

10.6 Advanced Features

10.6.1 Cross-References

Quarto provides a powerful cross-reference system for figures, tables, equations, and sections. Cross-references automatically number your content and create clickable links in HTML and PDF output.

Required label prefixes:

- Figures: `#fig-` (e.g., `#fig-workflow-diagram`)
- Tables: `#tbl-` (e.g., `#tbl-summary-stats`)
- Equations: `#eq-` (e.g., `#eq-regression-model`)
- Sections: `#sec-` (e.g., `#sec-introduction`)
- Theorems: `#thm-`, Lemmas: `#lem-`, Corollaries: `#cor-`
- Propositions: `#prp-`, Examples: `#exm-`, Exercises: `#exr-`

For figures (static images):

```
![Caption text](path/to/image.png){#fig-label}
```

For code-generated figures:

````
```{r}
#| label: fig-plot-name
#| fig-cap: "Caption text describing the plot"

# R code to generate plot
ggplot(data, aes(x, y)) + geom_point()
```
````

For tables (markdown tables):

```
| Column 1 | Column 2 |
|----------|----------|
| Data     | Data     |

: Caption text {#tbl-label}
```

For code-generated tables:

[17] https://quarto.org/docs/projects/profiles.html
[18] https://d-morrison.github.io/rme/

```
```{r}
#| label: tbl-table-name
#| tbl-cap: "Caption text"

# R code to generate table
knitr::kable(data)
```
```

Referencing in text:

- Figures: `@fig-label` produces "Figure X"
- Tables: `@tbl-label` produces "Table X"
- Equations: `@eq-label` produces "Equation X"
- Sections: `@sec-label` produces "Section X"

Benefits:

- Automatic numbering of figures, tables, and equations
- Automatic updates when content is reordered
- Clickable cross-references in HTML and PDF output
- Consistent formatting across all output formats
- Better accessibility for screen readers

For complete details, see the Quarto Cross-References documentation[19].

10.6.2 Using Includes for Modular Content

Quarto's include feature allows you to decompose large documents into smaller, more manageable files. This is particularly useful for books and long documents.

Basic syntax:

```
{{< include path/to/file.qmd >}}
```

Benefits:

1. **Better Git History:** When sections are reordered, only the main chapter file changes (moving include statements), making it immediately clear that content was reorganized rather than edited.
2. **Easier Code Review:** Reviewers can see exactly what changed—either the organization (main file) or the content (include file).
3. **Modular Maintenance:** Each section lives in its own file, making it easier to find and edit specific content, reuse sections across chapters, and work on different sections simultaneously without merge conflicts.
4. **Clear Structure:** The main chapter file becomes a table of contents showing the organization at a glance.

[19] https://quarto.org/docs/authoring/cross-references.html

Recommended pattern:

Main chapter file (e.g., `05-coding-practices.qmd`):

```
# Coding Practices

## Section Heading

{{< include coding-practices/section-name.qmd >}}

## Another Section

{{< include coding-practices/another-section.qmd >}}
```

Include files (e.g., `coding-practices/section-name.qmd`):

- Stored in a subdirectory matching the chapter name
- Contains only the content for that section (no heading)
- The heading stays in the main chapter file
- Named descriptively using kebab-case

Note: The heading must be in the main file, followed by a blank line, then the include statement. This keeps the document structure clear in the main file.

For more details, see the Quarto Includes documentation[20].

10.6.3 Including External Code Files

When embedding raw source code from external files without executing them, the `include-code-files`[21] extension provides a filter to include code directly into fenced code blocks with syntax highlighting and line or snippet selection.

Installation:

```
quarto add quarto-ext/include-code-files
```

Add the filter to your document or `_quarto.yml`:

```
filters:
  - include-code-files
```

Usage:

````
```{.r include="path/to/script.R"}
```
````

Key capabilities:

- Embed external scripts directly without copy-pasting
- Slice specific line ranges (e.g., `start-line=10 end-line=25`)
- Include labeled snippets (e.g., `snippet="main"`)
- Keep documentation in sync with working standalone script files

[20] https://quarto.org/docs/authoring/includes.html
[21] https://github.com/quarto-ext/include-code-files

10.7 Mermaid Diagrams

Mermaid diagrams are a powerful tool for creating flowcharts, sequence diagrams, state diagrams, and other visualizations directly in your Quarto documents using text-based syntax. While mermaid diagrams work well in HTML output, they have significant limitations when rendering to DOCX (Word) and PDF formats as of early 2026.

10.7.1 Creating Mermaid Diagrams

Mermaid diagrams are created using fenced code blocks with the `mermaid` language tag:

````
```{mermaid}
flowchart LR
  A[Start] --> B{Decision}
  B -->|Yes| C[Result 1]
  B -->|No| D[Result 2]
```
````

This creates a simple flowchart that renders beautifully in HTML output.

10.7.2 Known Issues with DOCX and PDF Output

As of early 2026, rendering mermaid diagrams to DOCX or PDF formats in Quarto is unreliable and problematic. These issues are well-documented in the Quarto community and stem from the underlying tooling used to convert diagrams to images.

10.7.2.1 Rendering Failures and Hangs

When rendering documents with mermaid diagrams to DOCX format, you may experience:

- **Indefinite hangs:** Quarto may hang indefinitely during rendering, leaving orphaned Chrome/Edge browser processes running (Gewerd-Strauss 2023)
- **Complete rendering failures:** The rendering process may fail entirely with errors
- **Missing diagrams:** The document may render successfully but with missing diagrams ("Quarto Cannot Render Mermaid or Dot Images to Docx and Pdf Formats" 2023)

These issues occur because Quarto relies on headless browser engines (Chrome or Edge via Puppeteer) to convert mermaid diagrams to images for inclusion in DOCX and PDF outputs. The browser integration can fail or hang, particularly when running in automated environments or when multiple diagrams are present.

10.7.2.2 Image Quality Issues

Even when rendering succeeds, mermaid diagrams in DOCX and PDF output often suffer from:

- **Small or blurry images:** Diagrams appear rasterized at low resolution
- **Incorrect sizing:** Diagrams may not respect sizing options specified in the document

10.7.3 Recommended Workarounds

Given these limitations, we recommend the following approaches when you need DOCX or PDF output:

10.7.3.1 1. Manual Export and Embedding (Most Reliable)

The most robust workflow is to:

1. First render your mermaid diagrams to HTML
2. Export them as PNG or SVG images manually using:
    - Browser developer tools (save rendered diagram)
    - Online mermaid editors (e.g., https://mermaid.live/)
    - Mermaid CLI tools
3. Include the exported images in your Quarto document using standard markdown syntax

Example:

```
![Workflow diagram description](assets/images/workflow-diagram.png){#fig-workflow}
```

This approach guarantees compatibility with all output formats and gives you full control over image quality.

10.7.3.2 2. Conditional Content for Different Formats

Use Quarto's conditional content features to show different content based on output format:

````
::: {.content-visible when-format="html"}
```{mermaid}
flowchart LR
  A[Start] --> B[End]
```
:::

::: {.content-visible unless-format="html"}
![Workflow diagram](assets/images/workflow-diagram.png)
:::
````

This shows the live mermaid diagram in HTML output and a static image in DOCX/PDF outputs.

10.7.3.3 3. Ensure Chromium is Installed (Partial Solution)

If you want to attempt rendering mermaid diagrams to DOCX, ensure you have Chrome or Edge installed. You can install Chromium via Quarto:

```
quarto tools install chromium
```

However, this does not guarantee successful rendering and may still result in hangs or failures.

10.7.3.4 4. Prefer HTML Output When Possible

When creating documentation that includes mermaid diagrams, prefer HTML as your primary output format. HTML rendering of mermaid diagrams is fast, reliable, and produces high-quality interactive diagrams.

If DOCX output is required for collaboration or submission, use one of the workarounds above.

10.7.4 Real-World Example

In UCD-SERG/rpt PR #70[22], our team removed mermaid diagrams from package vignettes specifically because they were incompatible with DOCX output. This demonstrates the practical impact of these limitations in production workflows.

10.7.5 Future Outlook

The Quarto development team is aware of these issues and working to improve diagram rendering support. Monitor the following resources for updates:

- Quarto CLI Issue #3809[23] - Tracking mermaid/dot rendering to DOCX/PDF
- Quarto Discussion #6085[24] - Mermaid rendering hangs with DOCX
- Quarto Diagrams Documentation[25] - Official documentation on diagram rendering

Check for improvements in newer Quarto versions, but as of early 2026, manual export and embedding remains the most reliable approach for DOCX and PDF outputs.

10.7.6 Summary

Key Takeaways:

- Mermaid diagrams work well in HTML output but are problematic for DOCX and PDF
- Rendering failures, hangs, and quality issues are common
- For reliable DOCX/PDF output, export diagrams manually as images
- Use conditional content to show different formats based on output type
- Prefer HTML output when mermaid diagrams are important to your document

[22] https://github.com/UCD-SERG/rpt/pull/70
[23] https://github.com/quarto-dev/quarto-cli/issues/3809
[24] https://github.com/orgs/quarto-dev/discussions/6085
[25] https://quarto.org/docs/authoring/diagrams.html

10.8 Troubleshooting

When encountering issues with Quarto, the official Quarto Troubleshooting Guide[26] provides comprehensive diagnostic tools and strategies. This section summarizes key troubleshooting approaches.

10.8.1 Checking Your Setup

Use `quarto check` to verify your installation and dependencies:

```
quarto check
```

This command checks for:

- Quarto version and binary dependencies (Pandoc, Dart Sass, Deno, Typst)
- LaTeX installation (TinyTeX or system TeX)
- Python, Jupyter, and available kernels
- R, knitr, and rmarkdown versions
- Library paths and installed packages

See the `quarto check` output and the official guide[27] for the complete, authoritative list of checks.

10.8.2 Diagnostic Tools

Enable verbose logging to see detailed rendering information.

On Unix-like systems (macOS, Linux):

```
export QUARTO_LOG_LEVEL=DEBUG
quarto render document.qmd
```

On Windows PowerShell:

```
$env:QUARTO_LOG_LEVEL="DEBUG"
quarto render document.qmd
```

Or pass at command line (all platforms):

```
quarto render document.qmd --log-level debug
```

Get stack traces for better error messages.

On Unix-like systems (macOS, Linux):

[26] https://quarto.org/docs/troubleshooting/
[27] https://quarto.org/docs/troubleshooting/

```
export QUARTO_PRINT_STACK="true"
quarto render document.qmd
```

On Windows PowerShell:

```
$env:QUARTO_PRINT_STACK="true"
quarto render document.qmd
```

Inspect log files for diagnostic information. Log file locations:

- **Windows:** `%LOCALAPPDATA%\quarto\logs`
- **macOS:** `~/Library/Application Support/quarto/logs`
- **Linux:** `~/.local/share/quarto/logs` (or `${XDG_DATA_HOME}/quarto/logs` if `${XDG_DATA_HOME}` is set)

10.8.3 Common Issues

Out-of-memory errors: When building large projects, increase memory allocation (example sets 8GB limit).

On Unix-like systems (macOS, Linux):

```
export QUARTO_DENO_V8_OPTIONS="--max-old-space-size=8192"
quarto render
```

On Windows PowerShell:

```
$env:QUARTO_DENO_V8_OPTIONS="--max-old-space-size=8192"
quarto render
```

PDF rendering issues: If you encounter PDF rendering problems, ensure you have an up-to-date LaTeX installation, or install Quarto's TinyTeX:

```
quarto install tinytex
```

Environment conflicts: If Quarto finds the wrong R or Python installation, use `quarto check` output to verify which installation Quarto is using. For R, you can render a diagnostic document with `sessionInfo()` to compare Quarto's environment with your interactive R session.

10.8.4 Advanced Troubleshooting

For more specialized issues, consult the official Quarto Troubleshooting Guide[28], which covers:

- Jupyter engine debugging and kernel issues
- Lua filter debugging and tracing
- Julia-specific troubleshooting
- Detailed environment and dependency diagnostics
- Installation problems (including macOS installer logs)

When seeking help on the Quarto issue tracker[29], include diagnostic information from `quarto check` and relevant log files.

10.9 Additional Resources

10.9.1 Official Documentation

- Quarto Official Guide[30] - comprehensive official documentation
- Quarto Books Guide[31] - documentation specific to creating books
- Quarto Publishing Guide[32] - how to publish your Quarto content online
- Quarto Getting Started[33] - installation and basic usage

10.9.2 Learning Resources

- R for Data Science - Quarto Chapter[34] (Wickham et al. 2023) - excellent introduction to using Quarto with R
- litedown: R Markdown Reimagined[35] - a book by Yihui Xie on `{litedown}`[36], a lightweight R Markdown implementation aimed at simplicity and speed
- Regression Models for Epidemiology[37] - example of a Quarto book with profiles for rendering chapters as slides
- UCD-SeRG Lab Manual Source[38] - this manual's source code provides examples of:
  - Book structure and organization
  - Using includes for modular content
  - Configuring multiple output formats (HTML, PDF, ePub, Word)
  - Cross-references for figures, tables, and sections

[28] https://quarto.org/docs/troubleshooting/
[29] https://github.com/quarto-dev/quarto-cli/issues
[30] https://quarto.org/docs/guide/
[31] https://quarto.org/docs/books/
[32] https://quarto.org/docs/publishing/
[33] https://quarto.org/docs/get-started/
[34] https://r4ds.hadley.nz/quarto.html
[35] https://pkg.yihui.org/litedown/book/
[36] https://pkg.yihui.org/litedown/
[37] https://d-morrison.github.io/rme/
[38] https://github.com/UCD-SERG/lab-manual

10.9.3 Templates

- UCD-SeRG Quarto Book Template[39] - our recommended template
- Coatless Tutorials Quarto Book Template[40] - another frequently-used template with helpful examples
- DataLab Quarto Template[41] - template from the UC Davis DataLab and Davis R Users Group

10.9.4 Extensions and Tools

- `include-code-files`[42] - Quarto filter extension to include external code files directly inside fenced code blocks with syntax highlighting and line slicing

10.9.5 Development Roadmap and Known Issues

The Quarto development team uses GitHub "epics" to track collections of related bugs and planned improvements. These are useful for staying informed about known limitations and upcoming features:

- All open Quarto epics[43] - index of all open epic issues in the Quarto CLI repository
- reveal.js bugs & improvements epic[44] - example epic tracking reveal.js-related issues

10.9.6 Related Lab Manual Chapters

For additional context about using Quarto in our lab:

- Chapter 6 - R coding practices that apply to code in Quarto documents
- Chapter 7 - Code style guidelines including formatting for Quarto documents
- Chapter 11 - Version control for Quarto projects

[39] https://github.com/UCD-SERG/qbt
[40] https://github.com/coatless-tutorials/quarto-book-template
[41] https://github.com/d-rug/datalab_template_quarto
[42] https://github.com/quarto-ext/include-code-files
[43] https://github.com/quarto-dev/quarto-cli/issues?q=is%3Aopen+is%3Aissue+label%3Aepic
[44] https://github.com/quarto-dev/quarto-cli/issues/4894

11 Github

11.1 Basics

- GitHub Learn[1] hosts official interactive courses, Skills exercises, learning pathways, and credentials.
- A detailed tutorial of Git can be found here on the CS61B website[2].
- If you are already familiar with Git, you can reference the summary at the end of Section B[3].
- If you have made a mistake in Git, you can refer to On undoing, fixing, or removing commits in git (Robertson, n.d.) to undo, fix, or remove commits in git.
- For hands-on Git practice, see the UC Davis DataLab Git Sandbox[4] - a collaborative repository for learning Git workflows.

11.2 GitHub Education and Copilot Access

11.2.1 GitHub Education Benefits

Students, faculty, and researchers at UC Davis can access GitHub Education benefits, which include free access to GitHub Pro features and various developer tools.

To sign up for GitHub Education:

1. Visit the GitHub Education website[5]
2. Click "Get benefits" or "Join GitHub Education"
3. Sign in with your GitHub account (or create one if you don't have one)
4. Complete the application form using your UC Davis email address (ending in `ucdavis.edu`)
5. Provide proof of your academic affiliation (e.g., upload a photo of your student ID or university letter)
6. Wait for approval (typically takes a few days)

Once approved, you'll have access to the GitHub Student Developer Pack[6] (for students) or GitHub Teacher Toolbox[7] (for faculty), which includes numerous free tools and services for learning and development.

[1] https://learn.github.com/
[2] https://sp19.datastructur.es/materials/guides/using-git#b-local-repositories-narrative-introduction
[3] https://sp19.datastructur.es/materials/guides/using-git#b-local-repositories-narrative-introduction
[4] https://github.com/ucdavisdatalab/sandbox_git
[5] https://education.github.com/
[6] https://education.github.com/pack
[7] https://education.github.com/teachers

11.2.2 GitHub Copilot Access for UC Davis Members

GitHub Copilot is an AI-powered coding assistant that can significantly accelerate your work. As a member of the UC Davis GitHub organization, you may be eligible for a free Copilot seat.

To request a Copilot seat:

If you are not yet a member of the UC Davis GitHub organization:

1. Ensure you have a GitHub account and have signed up for GitHub Education (see above)
2. Go to the UC Davis IT ServiceHub GitHub page[8]
3. Click the blue "Get GitHub!" button near the top right
4. When you receive the invitation email, make sure to check the "Ask for a GitHub Copilot seat (optional)" checkbox before joining (Figure 11.1)

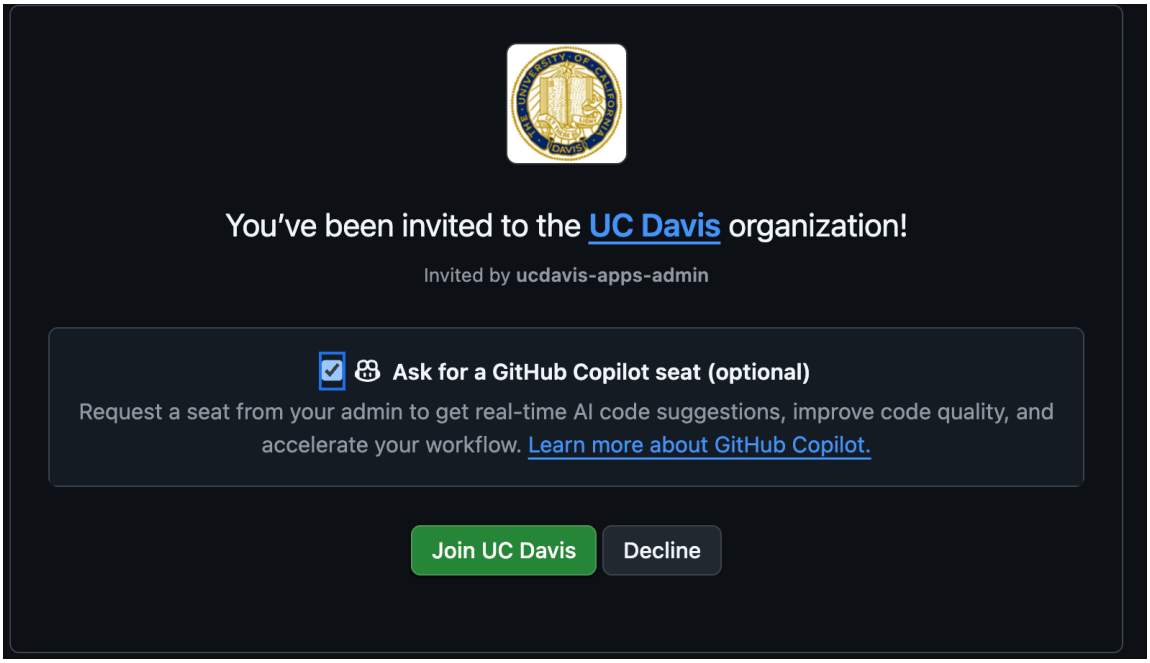


Figure 11.1: UC Davis GitHub invitation with Copilot seat option

5. Click "Join UC Davis" to accept the invitation
6. Follow the setup instructions to install and configure Copilot in your development environment

If you are already a member of the UC Davis GitHub organization:

1. Go to your GitHub Copilot settings[9]
2. Scroll to the "Get Copilot from an organization" section at the bottom
3. Find the "ucdavis" entry and click the request button

[8] https://servicehub.ucdavis.edu/servicehub?id=it_catalog_content&sys_id=951add951b5798103f4286ae6e4bcb12

[9] https://github.com/settings/copilot/features

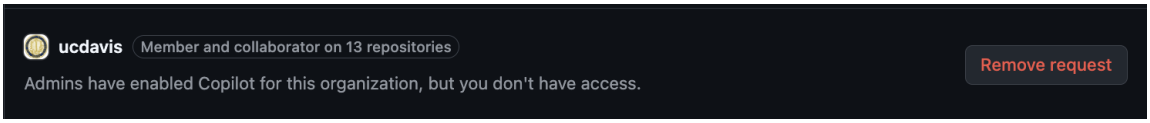


Figure 11.2: GitHub Copilot settings page showing organization options

4. Wait for approval from the UC Davis GitHub organization administrators
5. Once approved, follow the setup instructions to install and configure Copilot in your development environment

For guidance on using GitHub Copilot effectively, see the wai site's "Best Practices for Safe and Successful Use" section[10].

11.3 Graphical Git Clients

While knowing how to use Git on the command line remains essential because it provides the full power, flexibility, and scriptability of Git, graphical Git clients offer visual representations of repository history, interactive branch visualization, and simplified merge conflict resolution.

11.3.1 GitHub Desktop

GitHub Desktop[11] ("GitHub Desktop," n.d.) is a free, open-source graphical interface developed by GitHub. It simplifies basic Git workflows such as staging changes, creating commits, branching, and opening pull requests. It is an accessible starting point for beginners or for developers who prefer a visual interface for routine Git operations.

11.3.2 GitKraken

GitKraken[12] ("GitKraken," n.d.) is a cross-platform (macOS, Windows, Linux) Git client and developer experience platform. Key features include:

- An interactive, visual commit graph that simplifies navigating complex branch histories
- Drag-and-drop interactive rebasing and visual branch merging
- An in-app merge conflict editor with side-by-side diffing
- Multi-repository workspaces for tracking and managing multiple repositories together
- Integrations with major Git hosting providers (GitHub, GitLab, Bitbucket, and Azure DevOps) and issue trackers such as Jira

Beyond the standalone GitKraken Desktop client, the GitKraken ecosystem includes the GitLens extension for Visual Studio Code and terminal CLI integrations.

[10] https://morrison-lab.github.io/wai/chapters/coding-agents.html#sec-ai-best-practices
[11] https://desktop.github.com/
[12] https://gitkraken.com/

11.3.3 GitKraken Kepler

GitKraken Kepler[13] ("GitKraken Kepler," n.d.) is GitKraken's agentic development environment and workspace intelligence platform. It is designed to coordinate and manage agent-driven development workflows, providing a unified interface to:

- Plan implementation work and orchestrate parallel AI coding agents at scale
- Generate context-aware commits, pull request summaries, and code reviews
- Inspect and review diffs and generated changes before shipping code

11.4 GitHub CLI and GitLab CLI

`gh`[14] is the official GitHub command-line interface. It lets you manage pull requests, issues, repositories, and workflows without leaving the terminal. `glab`[15] is the GitLab equivalent.

11.4.1 Installing `gh`

Follow the official installation guide[16] for your platform. The most common routes are:

- **macOS:** `brew install gh`
- **Linux (Debian/Ubuntu):** `sudo apt install gh` after adding the GitHub CLI apt repository[17]
- **Linux (Fedora/RHEL):** `sudo dnf install gh`
- **Windows:** `winget install GitHub.cli` or `scoop install gh`

After installing, authenticate:

```
gh auth login
```

On Windows with Git Bash or MSYS2, see the wai site's "Installing Claude Code on Windows" section[18] for the `winpty` workaround if the interactive prompt fails.

11.4.2 Installing `glab`

Follow the official installation guide[19] for your platform. The most common routes are:

- **macOS:** `brew install glab`
- **Linux:** download the latest package from the glab releases page[20], or use `sudo snap install glab`
- **Windows:** `scoop install glab`, or download the installer from the glab releases page[21]

[13] https://gitkraken.com/kepler
[14] https://cli.github.com/
[15] https://gitlab.com/gitlab-org/cli
[16] https://github.com/cli/cli#installation
[17] https://github.com/cli/cli/blob/trunk/docs/install_linux.md
[18] https://morrison-lab.github.io/wai/chapters/coding-agents.html#sec-ai-claude-code-windows
[19] https://gitlab.com/gitlab-org/cli#installation
[20] https://gitlab.com/gitlab-org/cli/-/releases
[21] https://gitlab.com/gitlab-org/cli/-/releases

After installing, authenticate:

```
glab auth login
```

On Windows with Git Bash or MSYS2, see the wai site's "Installing Claude Code on Windows" section[22] for the `winpty` workaround if the interactive prompt fails.

11.5 Git Branching

Branches allow you to keep track of multiple versions of your work simultaneously, and you can easily switch between versions and merge branches together once you've finished working on a section and want it to join the rest of your code. Here are some cases when it may be a good idea to branch:

- You may want to make a dramatic change to your existing code (called refactoring) but it will break other parts of your project. But you want to be able to simultaneously work on other parts or you are collaborating with others, and you don't want to break the code for them.
- You want to start working on a new part of the project, but you aren't sure yet if your changes will work and make it to the final product.
- You are working with others and don't want to mix up your current work with theirs, even if you want to bring your work together later in the future.

A detailed tutorial on Git Branching can be found here[23]. You can also find instructions on how to handle merge conflicts when joining branches together.

11.6 Personal Dev Branches

Feature branches work best when each one stays small and scoped, but day-to-day exploratory work does not arrive in tidy, scoped units. Personal dev branches let you have both.

Each developer can keep a long-lived personal branch — named after them, for example `dev-ezra` — that they push to freely: experiments, half-finished ideas, notes-to-self, and fixes for whatever they bump into along the way. Nobody reviews a dev branch and nothing merges from it directly, so there is no pressure to keep it clean.

When some of that work is ready to contribute back to `main`:

1. Start a *new* feature branch from the **current** `main` (not from the dev branch).
2. Cherry-pick the commits you want to contribute from the dev branch onto the feature branch.
3. Open a pull request from the feature branch onto `main` as usual.

[22] https://morrison-lab.github.io/wai/chapters/coding-agents.html#sec-ai-claude-code-windows
[23] https://sp19.datastructur.es/materials/guides/using-git#e-git-branching-advanced-git-optional

```
# one-time setup: create your personal dev branch from current main
git switch -c dev-ezra origin/main
git push -u origin dev-ezra

# daily work: push freely
git switch dev-ezra

# ready to contribute: branch from current main ...
git fetch origin
git switch -c feat-cleaning-helpers origin/main

# ... pull over just the commits that belong in this contribution
git log --oneline dev-ezra          # find the commit hashes
git cherry-pick <hash1> <hash2>

git push -u origin feat-cleaning-helpers
# then open the PR from feat-cleaning-helpers onto main
```

(`git switch` is the modern, branch-only successor to `git checkout`; `git checkout dev-ezra` and `git checkout -b feat-cleaning-helpers origin/main`, matching the commands used elsewhere in this chapter, do the same thing.)

This way you can flow naturally while working — no stopping to plan a minimal, modular pull request in advance — and still deliver PRs that are minimal and modular, because the scoping happens at cherry-pick time instead.

Two habits make the cherry-picking step painless:

- **Commit in small, single-topic units on your dev branch.** A commit that mixes two unrelated changes cannot be cherry-picked into two different PRs.
- **Sync the dev branch with `main` periodically** (`git merge origin/main`), so your experiments build on current code and your cherry-picks transplant cleanly.

11.7 Example Workflow

A standard workflow when starting on a new project and contributing code looks like this:

Table 11.1: Standard Git workflow for new projects

Command	Description
SETUP: FIRST TIME ONLY: `git clone <url> <directory_name>`	Clone the repo. This copies of all the project files in its current state on Github to your local computer.
1. `git pull origin master`	update the state of your files to match the most current version on GitHub
2. `git checkout -b <new_branch_name>`	create new branch that you'll be working on and go to it
3. Make some file changes	work on your feature/implementation
4. `git add -p`	add changes to stage for commit, going through changes line by line

Command	Description
5. `git commit -m <commit message>`	commit files with a message
6. `git push -u origin <branch_name>`	push branch to remote and set to track (-u only needed if this is first push)
7. Repeat step 4-5.	work and commit often
8. `git push`	push work to remote branch for others to view
9. Follow the link given from the `git push` command to submit a pull request (PR) on GitHub online[24]	PR merges in work from your branch into master
(10.) Your changes and PR get approved, your reviewer deletes your remote branch upon merging	
11. `git fetch --all --prune`	clean up your local git by untracking deleted remote branches

11.8 Pull Request Roles and Protocol

The lab's pull request roles, author and reviewer expectations, stacked pull requests (Section 12.1.3), and review-response protocol are consolidated in Chapter 12. This GitHub chapter focuses on Git mechanics and GitHub-specific platform features.

11.9 Commonly Used Git Commands

Table 11.2: Commonly used Git commands

Command	Description
`git clone <url> <directory_name>`	clone a repository, only needs to be done the first time
`git pull origin master`	pull from `master` before making any changes
`git branch`	check what branch you are on
`git branch -a`	check what branch you are on + all remote branches
`git checkout -b <new_branch_name>`	create new branch and go to it (only necessary when you create a new branch)
`git checkout <branch name>`	switch to branch
`git add <file name>`	add file to stage for commit
`git add -p`	adds changes to commit, showing you changes one by one
`git commit -m <commit message>`	commit file with a message

[24] https://help.github.com/en/github/collaborating-with-issues-and-pull-requests/creating-a-pull-request#creating-the-pull-request

Command	Description
`git push -u origin <branch_name>`	push branch to remote and set to track (-u only works if this is first push)
`git branch --set-upstream-to origin <branch_name>`	set upstream to `origin/<branch_name>` (use if you forgot -u on first push)
`git push origin <branch_name>`	push work to branch
`git checkout <branch_name>` `git merge master`	switch to branch and merge changes from `master` into `<branch_name>` (two commands)
`git merge <branch_name> master`	switch to branch and merge changes from `master` into `<branch_name>` (one command)
`git checkout --track origin/<branch_name>`	pulls a remote branch and creates a local branch to track it (use when trying to pull someone else's branch onto your local computer)
`git push --delete <remote_name> <branch_name>`	delete remote branch
`git branch -d <branch_name>`	deletes local branch, -D to force
`git fetch --all --prune`	untrack deleted remote branches

11.10 How often should I commit?

It is good practice to commit every 15 minutes, or every time you make a significant change. It is better to commit more rather than less.

11.11 Repeated Amend Workflow

When working on a complex task, you may want to make frequent incremental commits to protect your progress, but avoid cluttering your Git history with many tiny "work in progress" commits. The **Repeated Amend** pattern lets you build up a polished commit gradually.

11.11.1 Basic Workflow

Start with a clean working tree in a functional state. Then:

1. Make a small change and verify your project still works
2. Stage and commit with a temporary message like "WIP" (work in progress)
3. **Do not push yet**
4. Make another small change and verify it works
5. Stage and amend the previous commit: `git commit --amend --no-edit`
6. Repeat steps 4-5 as needed
7. When finished, amend one final time with a proper commit message
8. Push your completed work

In RStudio, you can use the "Amend previous commit" checkbox when committing.

11.11.2 Key Points

- Each amend replaces the previous commit rather than creating a new one
- This keeps your history clean while letting you work incrementally
- Only use this pattern before pushing - never amend commits that others may have pulled
- If you need to undo changes, use `git reset --hard` to return to your last commit state
- Think of commits as climbing protection: use them when in uncertain territory

For more details and troubleshooting scenarios, see the Repeated Amend chapter[25] in Happy Git with R.

11.12 What should be pushed to Github?

Never push .Rout files! If someone else runs an R script and creates an .Rout file at the same time and both of you try to push to github, it is incredibly difficult to reconcile these two logs. If you run logs, keep them on your own system or (preferably) set up a shared directory where all logs are name and date timestamped.

There is a standardized `.gitignore` for `R` which you can download[26] and add to your project. This ensures you're not committing log files or things that would otherwise best be left ignored to GitHub. This is a great discussion of project-oriented workflows[27], extolling the virtues of a self-contained, portable projects, for your reference.

11.13 GitLab

Our lab primarily uses GitHub, but you may encounter GitLab[28] — most often when collaborating with a group that self-hosts its own GitLab instance.

The two platforms support the same core workflow (clone, branch, push, review, merge), with a few differences worth knowing:

- **Terminology.** A GitHub *pull request* is a GitLab *merge request* (MR); the concept is identical.
- **Hosting.** GitLab is open-core and designed for self-hosting, so organizations often run their own instance behind their firewall; GitHub repositories almost always live on github.com.
- **Continuous integration.** GitLab ships a built-in CI/CD system configured by a single `.gitlab-ci.yml` file at the repository root, where GitHub uses GitHub Actions workflows under `.github/workflows/` (see Chapter 13).
- **Command-line tool.** `glab` is the GitLab counterpart to `gh`; installation and authentication are covered in Section 11.4.

[25] https://happygitwithr.com/repeated-amend.html
[26] https://github.com/github/gitignore/blob/master/R.gitignore
[27] https://tidyverse.org/blog/2017/12/workflow-vs-script/
[28] https://about.gitlab.com/

Two R packages help when working with GitLab from R (the second works across both forges):

- `{gitlabr}`[29] provides R access to the GitLab API, with convenience wrappers for common tasks (repository file access, issue assignment and status, commenting).
- `{pr.helpers}`[30], our lab's own (experimental) package, provides helpers for common pull/merge request workflows from a local Git repository — creating and switching branches, fetching requests by number, generating request URLs, and syncing with the default branch — and works across both GitHub and GitLab remotes.

11.14 Customizing How Files Appear on GitHub

GitHub uses a tool called Linguist[31] to detect languages in your repository and generate language statistics. You can customize how certain files are treated by GitHub using a `.gitattributes` file in the root of your repository. This is particularly useful for marking generated files, documentation, or vendored code that shouldn't count toward your repository's language statistics.

11.14.1 The `linguist-generated` Attribute

One of the most useful attributes is `linguist-generated`, which marks files as generated code. Files marked this way are:

- **Excluded from language statistics** - they won't affect your repository's language breakdown
- **Hidden by default in diffs** - making pull request reviews cleaner and more focused on actual code changes

Common use cases for `linguist-generated` include:

- Compiled or minified files (e.g., `*.min.js`, `*.min.css`)
- Auto-generated documentation files
- Files generated by build tools or code generators
- Lock files that are updated automatically

For more details, see the Generated code documentation[32].

11.14.2 Using `.gitattributes`

Create a `.gitattributes` file in the root of your repository and add patterns for files you want to mark as generated:

[29] https://thinkr-open.github.io/gitlabr/
[30] https://github.com/d-morrison/pr.helpers
[31] https://github.com/github-linguist/linguist/
[32] https://github.com/github-linguist/linguist/blob/main/docs/overrides.md#generated-code

```
# Mark minified JavaScript as generated
*.min.js linguist-generated

# Mark search index as generated
search/index.json linguist-generated

# Mark compiled CSS as generated
dist/styles.css linguist-generated
```

To unmark a file that would normally be considered generated:

```
# Don't treat bootstrap as generated
bootstrap.min.css -linguist-generated
```

The `.gitattributes` file uses the same pattern matching rules as `.gitignore`. For more details, see the pattern format documentation[33] and the Using gitattributes guide[34].

11.14.3 Other Useful Linguist Attributes

Beyond `linguist-generated`, you can use several other attributes:

- `linguist-vendored` - Marks vendored code (libraries you didn't write) to exclude from stats (documentation[35])
- `linguist-documentation` - Marks documentation files to exclude from stats (documentation[36])
- `linguist-detectable` - Forces a file type to be included in language stats (useful for data or prose files) (documentation[37])
- `linguist-language=<name>` - Overrides the detected language for syntax highlighting (documentation[38])

Example `.gitattributes` file:

```
# Exclude vendored dependencies
vendor/* linguist-vendored

# Exclude generated files
*.generated.ts linguist-generated
dist/* linguist-generated

# Mark documentation
docs/* linguist-documentation

# Force R Markdown files to be detected
*.Rmd linguist-detectable
```

[33] https://www.git-scm.com/docs/gitignore#_pattern_format
[34] https://github.com/github-linguist/linguist/blob/main/docs/overrides.md#using-gitattributes
[35] https://github.com/github-linguist/linguist/blob/main/docs/overrides.md#vendored-code
[36] https://github.com/github-linguist/linguist/blob/main/docs/overrides.md#documentation
[37] https://github.com/github-linguist/linguist/blob/main/docs/overrides.md#detectable
[38] https://github.com/github-linguist/linguist/blob/main/docs/overrides.md#using-gitattributes

For complete documentation, see Customizing how changed files appear on GitHub[39] and the Linguist overrides documentation[40].

11.15 Exporting Issue and Pull Request Conversations

Issue threads and pull request reviews hold a lot of the reasoning behind a project: why an approach was rejected, what a reviewer worried about, which alternative was tried first. That record lives on GitHub's servers, not in your Git history, so `git clone` does not capture any of it.

Reasons to pull it out:

- **Archiving** a project before a repository is transferred, made private, or deleted.
- **Offline or full-text search** across years of discussion, which is faster and more thorough than GitHub's search box.
- **Supplementary material** documenting how an analysis evolved, for a manuscript or a reproducibility appendix.
- **Feeding context to an AI coding assistant** that cannot browse the web but can read files in the working directory.

11.15.1 A pull request holds three separate conversations

This is the detail that most home-grown export scripts miss. What the GitHub web interface presents as one continuous thread is actually three different resources in the API:

Stream	What it is	Where it lives
Issue comments	Top-level comments in the "Conversation" tab	`/issues/{n}/comments`
Reviews	The summary body attached to an Approve, Request changes, or Comment review	`/pulls/{n}/reviews`
Review comments	Inline comments anchored to a line of the diff	`/pulls/{n}/comments`

Fetch only the first and you will silently drop every line-level review comment and every reviewer's summary — usually the most substantive part of the discussion.

A fourth resource, the timeline (`/issues/{n}/timeline`), carries the non-prose events: labels applied, commits referenced, issues linked and closed, review requests. Export it too if you care about *what happened* and not only *what was said*.

Note also that GitHub models pull requests as a kind of issue. The `/issues` endpoint returns both, with pull requests distinguished by a `pull_request` key, so filter deliberately rather than assuming you got only issues.

[39] https://docs.github.com/en/repositories/working-with-files/managing-files/customizing-how-changed-files-appear-on-github
[40] https://github.com/github-linguist/linguist/blob/main/docs/overrides.md

11.15.2 Exporting with `gh`

The GitHub CLI (Section 11.4) is the shortest route for a handful of repositories. Its `--json` flag will assemble the separate streams for you.

Issues, with their comment threads inline:

```
gh issue list --repo UCD-SERG/serocalculator --state all --limit 1000 \
  --json number,title,state,author,createdAt,closedAt,labels,body,comments \
  > issues.json
```

Pull requests need all three streams named explicitly:

```
gh pr list --repo UCD-SERG/serocalculator --state all --limit 1000 \
  --json number --jq '.[].number' |
while read -r n; do
  gh pr view "$n" --repo UCD-SERG/serocalculator \
    --json number,title,state,author,createdAt,mergedAt,body,comments,reviews,reviewThreads
    > "pr-${n}.json"
done
```

`reviewThreads` is the one to watch: it nests the inline comments under the thread they belong to and reports whether the thread was resolved, which a flat list of review comments cannot tell you.

Note that `--limit` defaults to 30. Set it above your repository's issue count, or you will get a partial export with no warning.

11.15.3 Exporting with the REST API

When you need a field `gh`'s `--json` flag does not expose, call the API directly. `gh api` handles authentication and pagination for you:

```
gh api --paginate /repos/OWNER/REPO/issues/123/comments
gh api --paginate /repos/OWNER/REPO/pulls/123/comments
gh api --paginate /repos/OWNER/REPO/pulls/123/reviews
gh api --paginate /repos/OWNER/REPO/issues/123/timeline
```

Without `--paginate` you get the first 30 records and nothing to indicate more exist.

The same endpoints work from R via the `gh` package[41], which is convenient if the export feeds directly into an analysis:

```
comments <- gh::gh(
  "/repos/{owner}/{repo}/issues/{number}/comments",
  owner = "UCD-SERG",
  repo = "serocalculator",
  number = 123,
  .limit = Inf
)
```

[41] https://gh.r-lib.org/

`.limit = Inf` is that package's equivalent of `--paginate`.

11.15.4 Exporting with GraphQL

REST costs three or four requests per pull request. Across several hundred pull requests that is thousands of requests against a limit of 5000 per hour. GitHub's GraphQL API can return the comments, reviews, and review threads for a pull request in a single request, which is worth the extra complexity once a repository is large enough that the REST approach runs into the rate limit. Use `gh api graphql` to issue the query.

11.15.5 Whole-repository archives

GitHub's migration API builds a downloadable archive of your data as JSON. It is the most complete of the routes here.

The user migrations reference lists what the archive can hold ("REST API Endpoints for User Migrations," n.d.). Every stream named above is in it — `issues`, `issue_comments`, `pull_requests`, `pull_request_reviews`, and `review_comments` — along with `issue_events`, `commit_comments`, `milestones`, `releases`, `repositories`, and `users`.

Start a migration with `POST /user/migrations` for repositories you own, or `POST /orgs/{org}/migrations` for an organization's repositories if you are an organization owner ("REST API Endpoints for Organization Migrations," n.d.). Both are asynchronous: you poll the migration's status until it reports `exported`, then download the archive, which stays available for seven days.

Settings > Account > Export account data is the web-interface route to this same export rather than a separate feature. GitHub's documentation states that you can export your personal account's data "through your account settings on GitHub or with the User Migration API", and refers readers to the migrations API reference for what that data covers ("Requesting an Archive of Your Personal Account's Data," n.d.). Clicking through delivers the `.tar.gz` by emailed download link. That makes this the one route needing no code at all: a lab member who wants the whole record can click through Settings and skip everything above.

This route is the right one for archiving a repository before it is deleted or transferred. The per-repository scripts above are better for ongoing exports or for a subset of the discussion.

11.15.6 Practical cautions

- **Whether you get attachment files depends on the route.** Through `gh` or the REST API you get comment text, in which a pasted image or file is a link to GitHub's asset servers — the URL, not the file. Download those separately if they matter, and note that assets on a private repository require authentication to fetch. The migration archive is the exception: it carries an `attachments` directory of the files themselves, unless you ask for it to be left out ("REST API Endpoints for User Migrations," n.d.).

- **The authenticated rate limit is 5000 requests per hour.** A few hundred pull requests exported through REST will get close. Check your remaining budget with `gh api /rate_limit`.
- **Reactions and thread resolution need to be requested.** Neither comes along by default in a plain comment listing.
- **Deleted and edited content is gone.** An export captures the current state, not the edit history of a comment.

Whatever route you take, commit the export to a repository or deposit it somewhere durable. An archive that lives only on the laptop of whoever ran the script has not solved the problem it was made to solve.

11.16 Removing Sensitive Data from a Repository

Accidentally committing credentials, API tokens, private keys, or sensitive patient data to a Git repository is a security incident that requires prompt remediation.

Simply deleting the file in a subsequent commit does not remove it from Git history, as the data remains accessible in earlier commit snapshots.

11.16.1 Immediate steps

1. **Rotate exposed secrets:** Immediately revoke and rotate any leaked passwords, API keys, or tokens. Assume that any credential pushed to a remote repository has been compromised.
2. **Purge the file from history:** Use tools like `git-filter-repo`[42] (officially recommended by Git and GitHub) or BFG Repo-Cleaner[43] to scrub the sensitive file from all commits, branches, and tags in your local repository.
3. **Force-push rewritten branches:** Coordinate with your team before force-pushing rewritten branches with lease (`--force-with-lease --force-if-includes`) to GitHub.
4. **Notify all collaborators:** All collaborators must discard old clones or rebase (never merge) their local branches onto the rewritten history; merging an old branch will re-introduce the sensitive data into the repository.
5. **Contact GitHub Support to clear cached views (optional):** Cached commits and pull request views on GitHub may retain sensitive objects until they are garbage-collected; contact GitHub Support if you need cached views purged immediately.

For detailed step-by-step instructions, see GitHub's guide on Removing sensitive data from a repository[44].

[42] https://github.com/newren/git-filter-repo
[43] https://rtyley.github.io/bfg-repo-cleaner/
[44] https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository

12 Collaborative Software Development

This chapter covers the social side of writing code together: opening pull requests, reviewing each other's work, and responding to review feedback.

12.1 Pull Requests

12.1.1 Pull request roles

Every pull request involves several people, each with a distinct role.

12.1.1.1 Issue Creator

The issue creator reports a bug, suggests a feature, or requests other improvements. In projects with external contributors, the issue creator often cannot assign issues to developers and is typically distinct from the PR author.

12.1.1.2 Author

The author opens the PR, writes a clear description, and links it to the relevant issue with `Closes #N` in the PR body. The author implements review feedback and resolves merge conflicts before re-requesting review.

12.1.1.3 Reviewer

The reviewer reads the diff, checks the code for correctness and style, and either approves or requests changes.

12.1.1.4 Merger

The merger executes the final merge once the PR is approved and all CI checks pass. This is often the same person as the reviewer.

12.1.1.5 Assignee

The assignee is listed in the "Assignees" field on GitHub to indicate who is currently responsible for the PR. This field clarifies ownership and tracks who should be addressing open feedback.

12.1.1.6 PR Steward

A PR steward keeps the review queue healthy. The steward monitors open PRs, assigns reviewers when none are assigned, follows up on stalled reviews, and helps resolve disagreements between authors and reviewers. This is typically a rotating duty for a senior contributor or release manager. The p5.js project documents how it structures this role in its steward guidelines[1].

12.1.2 Opening pull requests

12.1.2.1 Write focused PRs

A focused pull request is **one self-contained change** that addresses just one thing. Writing focused PRs has several benefits:

- **Faster reviews**: It is easier for a reviewer to find 5-10 minutes for a single bug fix than to set aside an hour for one large PR.
- **More thorough reviews**: Large PRs can overwhelm reviewers, which makes it easier to miss important problems.
- **Fewer bugs**: Smaller changes are easier to reason about.
- **Easier merges**: Large PRs take longer and are more likely to develop merge conflicts.
- **Less wasted work**: If the overall direction is wrong, you lose less time on a small PR than on a large one.

As a guideline, 100 lines is usually a reasonable size for a PR, and 1000 lines is usually too large. The number of files matters too: a 200-line change in one file might be fine, but the same change spread across 50 files is usually too large.

12.1.2.2 Write clear PR titles and descriptions

When you submit a pull request, include a detailed title and description. A comprehensive description helps the reviewer and provides valuable historical context.

PR title: The title should be a short summary, ideally under 72 characters, that is specific enough for future developers to understand the change without opening the full PR.

Poor titles that lack context:

- “Fix bug”
- “Add patch”
- “Moving code from A to B”

Better titles that summarize the actual change:

- “Fix missing value handling in data processing function”
- “Add support for custom date formats in import functions”

PR description body: The description should give the reviewer the context they need to understand the change. Consider including:

- A brief description of the problem being solved

[1] https://p5js.org/contribute/steward_guidelines/

- Links to related issues, such as `Closes #123` or `Related to #456`
- A before-and-after example that shows the changed behavior
- Possible shortcomings or trade-offs in the current approach
- For complex PRs, a suggested reading order for the reviewer

The **Files changed** tab on GitHub also lets you add inline comments that explain non-obvious changes. Those notes live in the PR metadata, not in the source files, so they are a good place to explain *this change* without cluttering the code with temporary rationale.

12.1.2.3 Explain technical reasoning up front

For non-obvious technical decisions, proactively explain your reasoning in commit messages and PR descriptions. Do not make reviewers ask why you chose a particular approach.

Common situations that require explanation:

- **Version pinning**: Explain whether you are using floating tags (for example `@v2`), version tags (for example `@v2.9.4`), or commit SHA pins (for example `@abc1234...`) and why. For example: "Using `@v2` (moving tag) to receive bug fixes within the major version while maintaining stability."
- **Configuration changes**: Explain the rationale behind non-obvious choices, such as why a boolean is set a certain way.
- **Dependency updates**: Explain why you are updating and what benefit or fix it brings.
- **Workflow modifications**: Describe the problem you are solving and why this approach is preferred.

This proactive communication reduces review back-and-forth and helps future maintainers understand the decision later.

12.1.2.4 Request reviews in the right order

Before requesting a pull request review from the PIs, obtain fresh approvals from:

- at least one other student
- at least one AI reviewer

This requirement applies both to the initial PI review request and to later re-review requests after revisions.

When requesting a review (whether in Microsoft Teams, email, or GitHub), always provide a direct hyperlink to the pull request (see Direct Hyperlinks in Internal Correspondence[2]). Do not make reviewers search for the PR they are asked to review.

12.1.2.5 Include tests with behavior changes

Focused PRs should include related test code. A PR that adds or changes logic should add or update tests for the new behavior. Pure refactoring PRs should also be protected by tests. If tests do not already exist for code you are refactoring, add them in a separate PR first to confirm that behavior stays unchanged.

[2] @sec-direct-hyperlinks

12.1.2.6 Separate refactorings from feature work

It is usually best to do refactorings in a separate PR from feature changes or bug fixes. For example, moving and renaming a function should generally be separate from fixing a bug in that function. That separation makes it much easier for reviewers to understand what each PR actually changes.

Small cleanups, such as a local variable rename, can travel with a feature or bug fix. Large refactorings should not.

12.1.2.7 Standardize PR descriptions with a template

GitHub lets you create a pull request template in a repository so everyone starts with the same PR body structure.

To add one:

1. On GitHub, navigate to the main page of the repository.
2. Above the file list, click `Create new file`.
3. Use one of GitHub's recognized pull request template locations: `pull_request_template.md` or `PULL_REQUEST_TEMPLATE.md` in the repository root, `.github/`, or `docs/`. For multiple templates, use `.github/PULL_REQUEST_TEMPLATE/`. See GitHub's pull request template documentation[3].
4. Add the template content to the file on the default branch.

Here is an example template:

```
# Description

## Summary of change

Please include a summary of the change, including any new functions added and example usage

## Related Issues

Closes #(issue number)
Related to #(issue number)

## Testing

Describe how this change has been tested.

## Checklist

- [ ] Tests added/updated
- [ ] Documentation updated
- [ ] Code follows project style guidelines

## Who should review the pull request?
```

[3] https://docs.github.com/en/communities/using-templates-to-encourage-useful-issues-and-pull-requests/creating-a-pull-request-template-for-your-repository

```
@username
```

12.1.2.8 Stacked pull requests

When implementing a sequence of interdependent features or multi-stage refactorings, stack pull requests to keep each change small and focused (see Section 12.1.3 for detailed stack creation, CI, and merging guidelines).

12.1.3 Stacked pull requests

12.1.3.1 What are stacked pull requests?

A stacked pull request workflow breaks large or multi-step code changes into a linear chain (or stack) of smaller, dependent pull requests ("About Stacked Pull Requests," n.d.).

In a pull request stack:

- The bottom pull request targets the repository default branch (typically `main`).
- Each subsequent pull request targets the branch of the pull request immediately below it.

```
 +-- feat/frontend-ui    -> PR #3 (base: feat/api-endpoints) [top]
 +-- feat/api-endpoints  -> PR #2 (base: feat/auth-layer)
 +-- feat/auth-layer     -> PR #1 (base: main)               [bottom]
main (default base branch)
```

Each pull request in the stack isolates the diff for its specific layer, displaying only the changes between its branch and the layer beneath it.

12.1.3.2 Benefits of stacked pull requests

Stacked pull requests provide several practical advantages:

- **Unblocked continuous development**: You can begin and publish changes that build upon unmerged work without waiting for prior pull requests to complete review.
- **Smaller, faster code reviews**: Reviewers inspect focused, self-contained layers instead of reviewing an overwhelming monolithic diff.
- **Structured dependency tracking**: Foundational changes (such as database schemas, data structures, or shared utilities) live in lower layers, while dependent features (such as API endpoints, UI components, or specialized tests) live in higher layers.
- **Agent and high-volume compatibility**: When coding agents or developers generate modular increments in sequence, stacks give each logical task a dedicated, isolated pull request.

12.1.3.3 Core principles for stacking

When using stacked pull requests, follow these guidelines:

- **Enforce dependency order**: If code in one layer relies on code from another, that dependency must reside in the same branch or a lower branch in the stack.
- **Split at distinct concerns**: Start a new layer whenever shifting to a different concern (for example, moving from core data models to analysis scripts, or from backend logic to documentation and tests) or when the diff exceeds reviewable size guidelines.
- **Keep branches in the same repository**: Stacked pull requests require all branches to live in the same repository; cross-fork stacks are not supported.

12.1.3.4 CI, rebasing, and merging

GitHub provides native platform support for managing and merging pull request stacks:

- **Rules and CI enforcement**: Branch protection rules (such as required approvals) and GitHub Actions CI checks configured for the base branch (`main`) automatically apply to every pull request in the stack, ensuring consistent quality across every layer.
- **Bottom-up merging**: Pull requests merge in sequential order from the bottom up. When the bottom-most pull request merges into the default branch, the pull request immediately above it automatically retargets onto the default branch so each successive layer can land cleanly in turn.
- **Automated cascading rebases**: When lower branches in a stack are updated with review revisions or upstream changes, rebasing or merging those updates propagates them upward through dependent branches to keep the entire stack in sync and resolve conflicts early.

12.2 Code Review

12.2.1 Reviewing pull requests

When submitting code to colleagues or reviewing theirs, use practices that keep feedback constructive and specific. The Tidyverse code review principles[4] (Tidyverse Team 2023) are a strong baseline for R projects. As noted in the Tidyverse guide's acknowledgments[5], the guide is a modified fork of Google's Code Review Developer Guide[6].

12.2.1.1 Purpose of code review

The primary purpose of code review is to improve the overall code health of the project over time. Reviewers should balance the need to make forward progress with the need to maintain code quality.

The key principle is to approve a PR once it clearly improves the codebase, even if it is not perfect. There is no such thing as perfect code. There is only better code.

[4] https://code-review.tidyverse.org/
[5] https://code-review.tidyverse.org/#acknowledgements
[6] https://google.github.io/eng-practices/review/

12.2.1.2 Monitor PRs awaiting your review

To keep reviews moving, bookmark GitHub's review-requested page and check it regularly, ideally at least daily:

- **General bookmark**: https://github.com/pulls/review-requested shows all PRs across GitHub where you have been requested as a reviewer.
- **Project-specific bookmark**: for frequently reviewed repositories, bookmark the project-specific version. For this repository: https://github.com/UCD-SERG/lab-manual/pulls/review-requested/YOUR-USERNAME replacing `YOUR-USERNAME` with your GitHub username.

Checking these pages regularly helps keep PRs from languishing in the queue.

12.2.1.3 Write review comments that teach

When reviewing code, be courteous, clear, and helpful:

- Comment on the **code**, not the author.
- Explain **why** you are making a suggestion, referencing best practices, design patterns, or code health where appropriate.
- Balance problem finding with guidance, so the author can learn from the review.
- Highlight positive choices too, so good practices get reinforced.

Poor comment: "Why did you use this approach when there is obviously a better way?"

Better comment: "This approach adds complexity without clear benefits. Consider using [alternative approach] instead, which would simplify the logic and improve readability."

12.2.1.4 Use review as mentoring

Code review is also a mentoring tool. As a reviewer:

- Leave comments that help authors learn something new.
- Link to relevant style guides or best-practice documentation.
- For complex reviews, consider a live review or pair-programming session.

12.2.1.5 Balance direct fixes with guidance

In general, the author is responsible for fixing the PR, not the reviewer. Point out problems clearly, but do not feel obligated to design every solution. Sometimes it is better to name the issue and let the author work out the implementation details.

For very small tweaks, such as typo fixes or comment additions, use GitHub's suggestion feature so the author can accept the change directly in the UI.

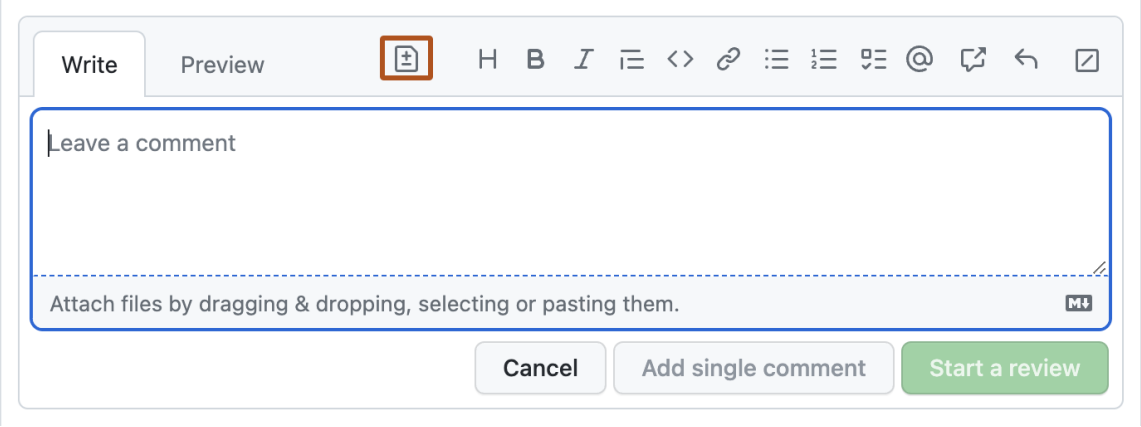


Figure 12.1: GitHub's suggestion feature in a PR review comment

12.2.1.6 Ignore auto-generated files when appropriate

When reviewing pull requests in R package repositories, you can usually ignore `.Rd` files in `man/`. These files are generated automatically by `{roxygen2}`[7] from roxygen comments in the source.

Why ignore `.Rd` files?

- They are generated files and should never be edited by hand.
- Their changes are already visible in the roxygen comments in the `.R` source files.
- Reviewing the source comments is more informative than reviewing the generated output.
- The files will be regenerated during the package build process.

What to review instead:

Review the roxygen comments in the `.R` source files. These begin with `#'` and appear immediately before function definitions.

If the repository has a preview workflow such as `pkgdown` or Quarto, review the rendered preview as well. The workflow should post a PR comment with a link to the preview build.

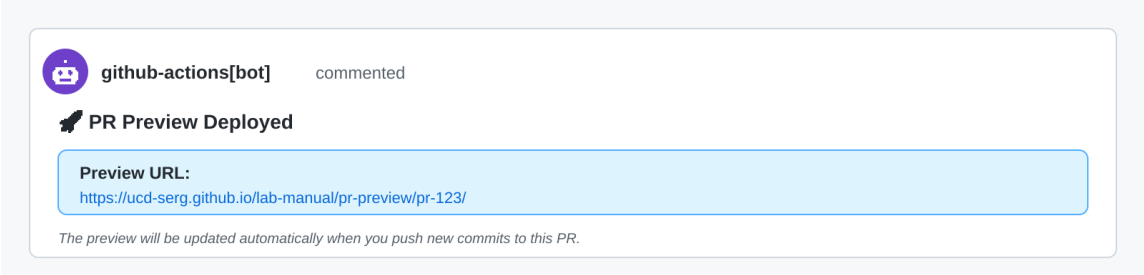


Figure 12.2: Example of an automated PR preview comment posted by GitHub Actions

In GitHub's **Files changed** view, you can click the three dots (`...`) next to a file and choose **View file** to hide it from the diff. That helps you focus on the meaningful changes.

[7] https://roxygen2.r-lib.org/

12.2.1.7 Reviewing Copilot-generated pull requests

Review pull requests created by GitHub Copilot coding agents using the same standards as any other PR, but watch for a few agent-specific issues.

Workflow approval requirements:

- You must manually approve GitHub Actions workflows for Copilot PRs.
- This is a security measure because Copilot can modify any file, including workflow files.
- Approve the workflow from the Actions tab or from the PR interface to start the checks.
- As of 2026-07-21, this manual approval step cannot be bypassed, including by repository owners.

Review focus areas:

- Verify that the solution actually addresses the issue.
- Check for over-engineering or extra features that were not requested.
- Review the test coverage for relevance and completeness.
- Check whether documentation is clear and follows repository conventions.
- Look for missed edge cases or incomplete error handling.

Iterating on Copilot PRs:

When you find issues in a Copilot PR, you can either:

1. Leave review comments and ask Copilot to address them.
2. Push commits directly to the Copilot PR branch yourself.

Direct edits are often faster for small fixes such as typos, formatting, or tiny adjustments.

Best practices:

- Do not push while Copilot is actively working on the branch.
- Review incrementally if the PR is large and the agent is updating it in stages.
- Trust the tool, but always verify the result with human review.

12.2.2 Responding to review comments

Once you receive a review from a human reviewer, you are obligated to respond promptly: typically within a week, unless there are extenuating circumstances. Address each of the reviewer's comments individually. You aren't obligated to agree with every comment; you are entitled to ask clarifying questions, raise objections, or ask to defer it to a later pull request.

When a reviewer leaves a comment on your code, respond to each one with exactly one of these:

- **Address it.** Make the change the reviewer asked for.
- **Rebut it.** If you think the code is correct as written, explain your reasoning and push back. You can disagree—just say why.

- **Defer it.** If the comment is valid but out of scope for this change, file a tracking issue, assign it to yourself (or delegate it, if applicable), and link it in your reply so the reviewer can see you are taking their concern seriously. This keeps the work visible and ensures it is not lost.
- **Acknowledge it.** For a comment that needs no action, a short acknowledgment is enough.

Never ignore a review comment. Always reply—even if only to say you filed a follow-up issue.

12.2.2.1 Resolving review comment threads

Before requesting a new review round, resolve every comment thread whose underlying item is settled:

- **When addressing an item:** Resolve the thread after pushing the commit that implements the fix. Include the commit hash in your thread reply so the reviewer can trace the change.
- **When deferring an item:** Resolve the thread after creating the tracking issue and linking it in your reply.
- **When acknowledging an item:** Resolve the thread after posting a brief acknowledgment.
- **When rebutting an item:** Leave the thread open initially with your explanation and evidence. If the reviewer agrees or does not re-raise the item in the subsequent round, resolve the thread.

If you need the reviewer to weigh in or clarify a specific point, leave that thread open and state explicitly in your reply that you are awaiting their guidance.

12.2.3 Avoid closing and restarting pull requests

Avoid closing a pull request and "starting over" on the same issue. Doing so disrupts the conversation between authors and reviewers, creates confusion, and leads to repeated work. Instead, keep iterating on the open pull request, where the review history stays attached to the code it discusses.

13 Continuous Integration

13.1 Understanding GitHub Actions

GitHub Actions[1] is GitHub's built-in automation platform that makes it easy to automate software workflows, including continuous integration and deployment (CI/CD). For R packages, this means you can automatically test your code, check for errors, and deploy documentation every time you push changes to GitHub.

Key benefits of GitHub Actions:

- **Automated testing**: Run `R CMD check` across multiple operating systems (Linux, macOS, Windows) and R versions
- **Immediate feedback**: Get notified of problems quickly, when they're easier to fix
- **Better collaboration**: External contributors can see if their changes pass all checks before you review
- **Quality assurance**: Catch platform-specific issues before they reach users
- **Documentation deployment**: Automatically build and deploy your pkgdown website

Even for solo developers, having automated checks run on different platforms helps avoid the "works on my machine" problem.

13.1.1 Action Versioning

When a workflow references an action, it must specify which version to use:

```
uses: r-lib/actions/setup-r@v2
```

The string after `@` is a Git ref—it can be a branch name, a commit SHA, or a tag. For most SERG workflows you will see a short tag like `v2`, and it is worth understanding what that actually means.

13.1.1.1 Floating (mutable) tags

`v2` is a **floating tag** (sometimes called a "sliding tag" or "mutable tag"): a Git tag that the action's maintainers deliberately move forward each time they publish a new backward-compatible release. When your workflow runs, GitHub resolves `v2` to whatever commit that tag currently points to, so you automatically pick up bug fixes, new platform support, and tool upgrades without editing your workflow file.

[1] https://github.com/features/actions

This is the officially recommended pattern for GitHub Actions[2]: maintainers commit to keeping all changes under a major-version floating tag backward-compatible, and bump to `v3` only for breaking changes.

> **Note**
>
> "Floating tag" is the standard term used across the ecosystem.

13.1.1.2 How a floating tag differs from a branch

Floating tags and branches are superficially similar—both are mutable named pointers to a commit. The key differences are:

- **Branches** advance automatically as commits are pushed. A floating tag only moves when the maintainer explicitly force-pushes it, so they retain deliberate control over when `v2` advances.
- **Semantics**: branches signal active development; a floating major-version tag signals a stable, maintained release line.
- In the GitHub Actions context the two are functionally equivalent as refs, but `@v2` communicates intent in a way that `@main` does not.

> **Note**
>
> Moving tags are generally considered bad practice in version control, because a tag that can change breaks the expectation that tagged commits are stable reference points—other collaborators who have already fetched the old target will silently diverge from the new one. See this GitHub Actions Toolkit discussion[a] for a detailed discussion of the pitfalls. GitHub Actions major-version tags are a deliberate, ecosystem-wide exception: maintainers accept the complexity in exchange for giving users automatic backward-compatible updates, and the pattern is explicitly endorsed in the Actions versioning guide[b].
>
> [a] https://github.com/actions/toolkit/issues/214
> [b] https://github.com/actions/toolkit/blob/main/docs/action-versioning.md

13.1.1.3 The reproducibility tradeoff

Using `@v2` is convenient but not fully reproducible: the same workflow file can silently resolve to a different commit on different days. For most R package development this is the right tradeoff—you benefit from fixes automatically. If you need exact reproducibility of your CI environment (e.g., for a frozen simulation study), you can pin to a specific commit SHA instead:

```
# Pinned to a specific commit; immune to tag movement
uses: r-lib/actions/setup-r@a51a8012b0aab7c32ef9d19bf54da93f3254335e # v2
```

The inline comment preserving the human-readable tag name is a community convention supported by dependency-update tools like Dependabot and Renovate.

[2] https://github.com/actions/toolkit/blob/main/docs/action-versioning.md

For SERG workflows, **`@v2` is the default recommendation**. Reach for SHA pinning only when exact CI reproducibility is a stated project requirement.

13.2 Setting Up GitHub Actions

The easiest way to add GitHub Actions to your R package is using `{usethis}`. The tidyverse team maintains a collection of ready-to-use workflows at r-lib/actions[3] that handle common R package tasks.

13.2.1 Essential Workflows

1. R CMD check (most important):

```
usethis::use_github_action("check-standard")
```

This runs `R CMD check` on Linux, macOS, and Windows to ensure your package works across platforms. If you only set up one workflow, make it this one.

2. Test coverage:

```
usethis::use_github_action("test-coverage")
```

Calculates what percentage of your code is covered by tests and reports to codecov.io[4].

3. Package website:

```
usethis::use_github_action("pkgdown")
```

Automatically builds and deploys your pkgdown documentation site to GitHub Pages.

13.2.2 Interactive Setup

Running `usethis::use_github_action()` without arguments shows a menu of recommended workflows:

```
usethis::use_github_action()
#> Which action do you want to add? (0 to exit)
#> (See <https://github.com/r-lib/actions/tree/v2/examples> for other options)
#>
#> 1: check-standard: Run `R CMD check` on Linux, macOS, and Windows
#> 2: test-coverage: Compute test coverage and report to https://about.codecov.io
#> 3: pr-commands: Add /document and /style commands for pull requests
```

[3] https://github.com/r-lib/actions
[4] https://about.codecov.io

13.2.3 Allowing Actions to Create Pull Requests

Some workflows open pull requests themselves — for example, a scheduled job that bumps a submodule pointer or updates generated files. By default, GitHub blocks workflows that authenticate with the built-in `GITHUB_TOKEN` from creating or approving pull requests, and such a workflow fails with a "GitHub Actions is not permitted to create or approve pull requests" error.

To allow it, go to **Settings > Actions > General > Workflow permissions** in the repository and check **Allow GitHub Actions to create and approve pull requests**.

A few related points:

- The workflow also needs write access: either select **Read and write permissions** on the same settings page, or grant it per workflow with a `permissions:` block (`contents: write` and `pull-requests: write`).
- For repositories in an organization, the same setting exists at the organization level, and the organization setting must allow it before the repository-level setting can take effect.
- This governs only the built-in `GITHUB_TOKEN`; workflows that authenticate with a personal access token are governed by that token's own scopes instead.

A workflow that creates or comments on pull requests depends on this setting and grants itself the needed scopes with a job-level `permissions:` block (the block can sit either at the top of a workflow file or under an individual job, as here):

```yaml
jobs:
  deploy:
    permissions:
      contents: write
      pull-requests: write
```

13.3 Reusable Workflow Collections

Several repositories collect reusable GitHub Actions and workflows for R projects, so you can call shared, community-maintained CI steps instead of writing your own:

- d-morrison/gha[5]: our lab's own collection of composite actions and reusable workflows for R-package and Quarto repositories, covering previews, publishing, test coverage, spell/link/character checks, bibliography DOI checks, and the Claude review bots used across our repos.
- r-lib/actions[6]: the tidyverse team's actions and example workflows (`R CMD check`, test coverage, `{pkgdown}`[7], `{lintr}`[8]), installable with `usethis::use_github_action()` as described in Section 13.2.
- eddelbuettel/r-ci[9]: a portable, shell-script-based CI setup for R that works across GitHub Actions, Azure DevOps, Docker, and other providers from one configuration; the successor to the r-travis project.

[5] https://github.com/d-morrison/gha
[6] https://github.com/r-lib/actions
[7] https://pkgdown.r-lib.org/
[8] https://lintr.r-lib.org/
[9] https://github.com/eddelbuettel/r-ci

- easystats/workflows[10]: reusable workflows the easystats collective runs across its R packages, useful as worked examples of centralizing many repos' automation in one place.
- RMI-PACTA/actions[11]: R and Docker actions plus example caller workflows; now archived ("no new work is expected"), but still readable as a reference for structuring an actions repo.

When several of your repositories need the same workflow, prefer calling a shared collection (or adding to ours) over copying workflow files between repos, for the same reason we avoid copy-pasted functions: fixes and improvements then land everywhere at once.

13.4 How GitHub Actions Workflows Work

When you set up a workflow, `usethis` creates a YAML configuration file in `.github/workflows/`. For example, `check-standard` creates `.github/workflows/R-CMD-check.yaml`.

This workflow automatically runs when you:

- Push commits to `main` or `master`
- Open or update a pull request

You can view workflow results in the "Actions" tab of your GitHub repository. A status badge is added to your README showing whether checks are passing.

13.5 Workflow Files and Security

> **Warning**
>
> **Important Security Consideration**
> Workflow files (`.github/workflows/*.yaml`) have access to repository secrets and can execute code. Always review workflow files carefully before committing them, especially if copied from external sources.
> See the wai site's "Best Practices for Safe and Successful Use" section[a] for guidance on working with workflow files using AI tools.
>
> [a] https://morrison-lab.github.io/wai/chapters/coding-agents.html#sec-ai-best-practices

The workflow YAML files in `.github/workflows/` are configuration files that tell GitHub Actions:

- When to run (on push, pull request, schedule, etc.)
- What operating systems and R versions to use
- What steps to execute (install dependencies, run checks, etc.)

[10] https://github.com/easystats/workflows
[11] https://github.com/RMI-PACTA/actions

13.6 Pipeline Design Patterns

CI/CD pipelines are infrastructure code, and they face engineering challenges that application code rarely does: several pipeline runs can execute at once, any step can fail partway through, and a re-run must not repeat side effects the first run already performed. The patterns below apply to any CI system; the examples name the GitHub Actions and GitLab CI features that implement them.

13.6.1 Concurrency Control

Two pipeline runs can execute at the same time whenever two triggering events land close together — two pushes, a push plus a scheduled run, or two merged pull requests. Runs that only *read* the repository (tests, lint, spell check) are safe to run in parallel. Runs that *write* to a shared resource — a deployment environment, a `gh-pages` branch, a package registry, a bot comment — must be serialized, or the runs will interleave and corrupt the resource.

Prefer the CI system's built-in serialization over hand-rolled lock files:

- GitHub Actions: a `concurrency:`[12] block assigns runs to a named group; at most one run per group executes at a time, and `cancel-in-progress: true` additionally cancels a superseded run instead of queueing behind it. Use cancellation for runs whose results only matter for the latest commit (reviews, previews), and plain queueing for runs that must all complete (deployments).
- GitLab CI: a `resource_group`[13] serializes jobs that name the same group, across pipelines.

When the platform primitive doesn't fit — for example, a lock that must span two different workflows, or one that a human must be able to hold — an *advisory lock* (a label, a file committed to a branch, an issue assignment) can work, but it inherits the race condition described in the next section (Section 13.6.3): acquiring it is a check followed by an act, so two runs can both "acquire" it unless the acquisition step is atomic.

13.6.2 Fail-Open vs. Fail-Closed

When a guard mechanism itself fails — the lock service is down, the label query errors, a duplicate check times out — the pipeline must choose between two failure modes:

- **Fail-open**: proceed as if the guard had allowed it. Risks duplicate work (two deploys, two bot comments), but never stalls the pipeline.
- **Fail-closed**: abort the run. Never duplicates work, but a broken guard now blocks all work behind it until someone intervenes.

Choose based on the cost of each failure: fail-open when duplicates are cheap and annoying (a second copy of a bot comment), fail-closed when duplicates are expensive or irreversible (a production deploy, a package release, an email to a mailing list). Either way, make the choice explicit in the pipeline code and its comments, and log loudly when the guard misbehaves so the degraded mode is visible rather than silent.

[12] https://docs.github.com/en/actions/using-jobs/using-concurrency
[13] https://docs.gitlab.com/ee/ci/resource_groups/

13.6.3 Avoiding Time-of-Check to Time-of-Use Races

A *TOCTOU* (time-of-check to time-of-use) race is the gap between checking a condition and acting on it:

```
check: is there already a "claimed" label?  # no
                                            # <- another run claims here
act:   add the label, start working         # both runs now think they own it
```

Under concurrency, `check -> act` sequences are unsafe no matter how small the gap, because another run can change the condition between the two steps. Closing the gap requires an *atomic* operation — one the platform guarantees to check and act in a single step, such as a git push that fails if the remote ref moved (the push atomically verifies "my branch is current" and updates it), or creating a file with an exclusive-create flag. When no atomic primitive is available, fall back to the platform's serialization (Section 13.6.1) so that only one run executes the `check -> act` sequence at a time, and design the action to be idempotent (Section 13.6.4) so that losing the race is harmless rather than corrupting.

13.6.4 Idempotency

A pipeline step is *idempotent* when running it twice has the same effect as running it once. Idempotent steps make re-runs safe: after a transient failure, a canceled run, or a manual retry, nobody has to reason about what the first attempt already did.

Common CI applications:

- **Bot comments**: update one "sticky" comment in place instead of appending a new comment per run (see 2).
- **Deploys**: overwrite the target with the built artifact (`rsync --delete`, a force-push to `gh-pages`) rather than applying increments to it.
- **Releases and tags**: check whether the tag already exists and skip cleanly, rather than failing on the collision.

The test for idempotency is concrete: re-run the job from the CI interface and inspect the result — one comment or two, one release or an error?

13.6.5 Retries and Timeouts

Pipelines call networks constantly (package installs, API queries, link checks), so transient failures are routine, not exceptional. Retry transient operations with **exponential backoff** (wait 2s, 4s, 8s, … between attempts) and a **bounded number of attempts**, and do not retry *permanent* failures — a `404 Not Found` will not succeed on attempt three, and retrying it just slows the pipeline and hides the real error. This repository's own DOI checker is an example: `.github/scripts/check-bibliography-dois.R` uses `{httr}`[14]'s `RETRY()` with three attempts, exponential pauses, and `terminate_on = c(404, 410)` so that genuinely-missing DOIs fail immediately while rate limits and timeouts are retried.

Give every job an explicit timeout (`timeout-minutes:` in GitHub Actions, `timeout:` in GitLab CI) sized to a small multiple of its normal runtime. The platform defaults are

[14] https://httr.r-lib.org/

generous (GitHub Actions allows a job six hours), so a hung job otherwise burns runner minutes for hours before anyone notices.

13.6.6 Pipeline Decomposition

The function-decomposition principles from Chapter 6 apply to pipelines: split a pipeline into jobs along the lines of **independent failure and re-run** — if one part can fail while another succeeds, and re-running only the failed part would save meaningful time, they belong in separate jobs. Separate jobs also parallelize across runners and show up as separate checks on a pull request, which makes failures legible at a glance.

Keep a pipeline monolithic when its steps share expensive setup (one job that restores an `renv` library and then lints, tests, and builds beats three jobs that each restore it) or when the steps are meaningless in isolation.

Pass data between jobs through the platform's declared channels — job `outputs` and `needs:` in GitHub Actions, artifacts in both systems — not through side effects like pushing intermediate state to the repository.

13.6.7 Secret Management

- Store credentials in the CI system's secret store (repository or organization secrets in GitHub Actions, masked CI/CD variables in GitLab), never in the repository — including its history.
- Grant the least privilege that works: prefer the ephemeral, automatically-scoped token (`GITHUB_TOKEN` with an explicit `permissions:` block) over a personal access token, and scope any long-lived token to the one repository and permission it needs.
- Never print secrets. Masking hides known secret *values* from logs, but not derived values (a URL that embeds the token, a `base64` encoding), so don't echo variables that might contain them.
- Rotate long-lived tokens on a schedule, and immediately when a person with access leaves the project.

13.6.8 Observability

Debugging a failed pipeline run is archaeology: all evidence must have been captured at run time.

- **Log decisions, not just actions.** When a guard skips a step, a retry fires, or a fail-open path activates, print *why* — the condition checked and the value found.
- **Structure long logs.** Group related output (`::group::` / `::endgroup::` in GitHub Actions, collapsible sections in GitLab) so a reader can scan to the failing step.
- **Upload artifacts for anything a re-run can't reproduce** — rendered output, test snapshots, the exact `renv.lock` used — with a retention period matched to how long debugging normally lags (the platform defaults, 90 days on GitHub, are usually more than enough).
- **Summarize outcomes** where reviewers look: a step summary (`$GITHUB_STEP_SUMMARY`) or a sticky PR comment

(2) beats a verdict buried in a thousand-line log.

13.7 Troubleshooting Failed Workflows

Before spending time debugging, check GitHub Status[15] to confirm that GitHub's services are fully operational. Outages can affect any GitHub service, not just CI— for example, as of May 2026, recent incidents have included:

- **Actions**: Hosted runners experiencing high queue times or failures, causing workflows to stall or fail with no change to your code
- **Copilot**: Inability to start Copilot Cloud Agent sessions or view running sessions, making the coding agent appear unresponsive
- **Pull Requests**: New PR comment threads and line comments failing to be created, making it seem as though review comments have disappeared or are not saving

If a GitHub-wide issue is reported, wait for the outage to resolve before investigating further—the problem is not in your code.

If GitHub Status[16] shows all services operational, check the "Actions" tab in your GitHub repository for detailed logs. Common causes of CI failures include:

- **Test failures**: Your tests found a bug (this is good! fix the bug)
- **Platform-specific issues**: Code works on your machine but not on other platforms
- **Missing dependencies**: System libraries needed for packages aren't installed
- **Linting errors**: Code style issues detected by automated checks

For help addressing workflow failures, see the wai site's "Addressing Failing GitHub Actions Workflows" section[17].

13.8 Pull Request Comment Automation

GitHub Actions can automatically comment on pull requests to provide feedback, status updates, or deployment previews. This section compares commonly used actions for managing PR comments, helping you choose the right tool for your workflow.

13.8.0.1 Common Use Cases

PR comment automation is particularly useful for:

- **CI/CD status updates**: Report test results, build status, or deployment progress
- **Code quality reports**: Post coverage reports, linting results, or security scan findings

- **Deployment previews**: Share links to preview deployments (e.g., documentation sites, app previews)
- **Bot feedback**: Provide automated feedback without cluttering the PR conversation

13.8.0.2 Comparison of Popular Actions

marocchino/sticky-pull-request-comment[18]

[15] https://www.githubstatus.com/
[16] https://www.githubstatus.com/
[17] https://morrison-lab.github.io/wai/chapters/coding-agents.html#sec-ai-addressing-failing-workflows
[18] https://github.com/marocchino/sticky-pull-request-comment

A widely-used action for creating or updating a single comment per workflow (as of early 2026, ~580 GitHub stars). Prevents comment spam by updating the same comment each time the workflow runs.

Key features:

- **Sticky comments**: Creates or updates a comment identified by a unique header
- **Multiple independent comments**: Different workflows can maintain separate sticky comments using different headers
- **Flexible update modes**: Replace, append, recreate, delete, or hide comments
- **File-based messages**: Load comment content from files for complex templates
- **Works with push events**: Can find and comment on PRs from push triggers (useful for monorepos)

Typical usage:

```yaml
- uses: marocchino/sticky-pull-request-comment@v2
  with:
    header: test-results
    message: |
      ## Test Results
      ```
      ${{ steps.test.outputs.summary }}
      ```
```

Best for: Projects needing clean, updatable status comments without duplicates. Ideal when you want the same type of information always visible in one place.

hasura/comment-progress[19]

Designed for tracking workflow progress with multiple updates as jobs complete. Similar to how Netlify or SonarCloud bots provide progressive feedback.

Key features:

- **Progress tracking**: Update comments as workflow steps complete or fail
- **Identifier-based updates**: Uses a hidden identifier to find and update the correct comment
- **Multiple update modes**: Append to existing comments, recreate, or delete
- **Flexible targets**: Comment on PRs, issues, or specific commits
- **Failure handling**: Optionally fail the workflow and append failure messages

Typical usage:

```yaml
- uses: hasura/comment-progress@v2.3.0
  with:
    github-token: ${{ secrets.GITHUB_TOKEN }}
    repository: ${{ github.repository }}
    number: ${{ github.event.number }}
    id: deploy-progress
    message: "Deploy in progress..."
    append: true
```

[19] https://github.com/hasura/comment-progress

Best for: Long-running workflows where you want to provide incremental status updates as different stages complete. Good for deployment pipelines or multi-stage builds.

thollander/actions-comment-pull-request[20]

A simpler, more straightforward action for posting or updating PR comments. Good balance of features and ease of use.

Key features:

- **Simple comment creation**: Easy to post one-time or updated comments
- **Comment updates**: Find and update existing comments by ID or content
- **Reactions**: Add emoji reactions to comments
- **Comment deletion**: Remove comments when no longer needed
- **Dynamic content**: Supports multi-line messages and environment variables

Typical usage:

```
- uses: thollander/actions-comment-pull-request@v2
  with:
    message: |
      ## Deployment Status
        Successfully deployed to preview environment
      Preview URL: https://preview-${{ github.event.number }}.example.com
```

Best for: Straightforward commenting needs without complex update logic. Good for simple status messages or one-time notifications.

rossjrw/pr-preview-action[21]

Specialized action for deploying pull request previews to GitHub Pages. Unlike the general-purpose comment actions above, this action handles the complete preview deployment lifecycle: building previews, deploying them to a GitHub Pages branch, and posting a comment with the preview link.

Key features:

- **Automated preview deployment**: Creates and deploys PR previews to a GitHub Pages branch (e.g., `gh-pages`)
- **Preview URLs**: Generates predictable preview URLs like `https://[owner].github.io/[repo]/pr-preview/pr-[number]/`
- **Sticky comments with links**: Posts and updates a comment with the preview URL and optional QR code for mobile access
- **Automatic cleanup**: Removes preview deployments when PRs are closed
- **Selective cleanup**: Can be configured to only remove previews for unmerged PRs, preserving merged PR previews for historical reference
- **Compatibility safeguards**: Designed to coexist with main branch deployments without conflicts

Typical usage:

[20] https://github.com/thollander/actions-comment-pull-request
[21] https://github.com/rossjrw/pr-preview-action

```yaml
- uses: rossjrw/pr-preview-action@v1
  with:
    source-dir: ./docs/
    preview-branch: gh-pages
    umbrella-dir: pr-preview
```

Advanced usage - only remove unmerged PR previews:

```yaml
# Deploy preview when PR is opened/updated
- uses: rossjrw/pr-preview-action@v1
  if: contains(['opened', 'reopened', 'synchronize'], github.event.action)
  with:
    source-dir: ./docs/
    action: deploy

# Remove preview only for unmerged PRs
- uses: rossjrw/pr-preview-action@v1
  if: github.event.action == "closed" && !github.event.pull_request.merged
  with:
    source-dir: ./docs/
    action: remove
```

This selective cleanup pattern (from the pr-preview-action documentation[22]) is particularly useful when you want to:

- Keep a historical record of what each merged PR changed
- Allow stakeholders to review merged changes after the fact
- Maintain preview URLs referenced in issue discussions or documentation

Best for: Projects using GitHub Pages for documentation, web apps, or other static sites where stakeholders benefit from previewing changes before merging. Essential when you want reviewers to see the rendered output (e.g., documentation sites, UI changes) rather than just the source code.

13.8.0.3 Feature Comparison

Table 13.1: Comparison of PR comment action features

Feature	marocchino/sticky-progress	hasura/comment-progress	thollander/actions-comment	rossjrw/pr-preview
Update existing comment	(by header)	(by identifier)	(by ID or content)	(automatic)

[22] https://github.com/rossjrw/pr-preview-action?tab=readme-ov-file#only-remove-previews-for-unmerged-prs

Feature	marocchino/sticky	hasura/comment-progress	thollander/actions-comment	rossjrw/pr-preview
Multiple independent comments	(via headers)	(via identifiers)	(limited)	(single preview link)
Append mode				
Delete comments				(on PR close)
Hide comments				
File-based messages				
Emoji reactions				
Works with push events		(requires PR number)	(requires PR number)	(requires PR number)
Progress tracking focus	(flexible)			
Deploy to GitHub Pages				
QR code generation				(optional)
Selective cleanup				(merged vs unmerged)

13.8.0.4 Choosing the Right Action

Use marocchino/sticky-pull-request-comment when:

- You need to maintain multiple independent status comments (test results, coverage, deployment, etc.)
- You want to prevent comment spam by updating the same comment
- You need advanced features like hiding outdated comments or file-based templates
- Your workflow triggers on push events and needs to find the associated PR

Use hasura/comment-progress when:

- You have long-running workflows with multiple stages
- You want to provide progressive feedback as each stage completes
- You need the workflow to fail and report the failure in the comment
- You want a pattern similar to third-party CI/CD service bots

Use thollander/actions-comment-pull-request when:

- You need simple comment posting without complex update logic
- You want to add emoji reactions to comments
- You're comfortable with the action's simpler update mechanism
- Your use case doesn't require the advanced features of the other options

Use rossjrw/pr-preview-action when:

- You need to deploy preview versions of your site/app to GitHub Pages for each PR
- You want stakeholders to review rendered output (documentation, UI) rather than just code
- You need automatic cleanup of preview deployments when PRs close
- You want to preserve previews of merged PRs for historical reference
- Your project uses GitHub Pages and benefits from preview environments

13.8.0.5 Security Considerations

> ⚠ Warning
>
> **Important: PR Comment Permissions**
> PR comment actions require write access to pull requests, which means they need the `pull-requests: write` permission.
> For workflows triggered by pull requests from forks (common in open-source projects), be careful about what information you expose in comments, as fork contributors can trigger these workflows. Never expose secrets or sensitive information in PR comments. See the wai site's "Best Practices for Safe and Successful Use" section[a] for more guidance on workflow security.
>
> [a] https://morrison-lab.github.io/wai/chapters/coding-agents.html#sec-ai-best-practices

13.9 Fast Package Installation with r2u

For GitHub Actions workflows running on Ubuntu, r2u[23] provides a faster alternative to installing R packages from source. r2u offers pre-compiled binary packages for all CRAN packages on Ubuntu, which can dramatically reduce CI build times.

[23] https://eddelbuettel.github.io/r2u/

Key benefits for CI workflows:

- **Faster installation**: Binary packages install in seconds rather than minutes
- **Automatic dependency resolution**: All system dependencies are handled by `apt`
- **Complete CRAN coverage**: Over 30,000 CRAN packages available as binaries (as of early 2025)
- **GitHub Actions support**: Easy integration with Ubuntu runners

Quick example:

Instead of waiting several minutes for `install.packages("tidyverse")` to compile from source, r2u can install it as binaries in under 20 seconds.

How to use r2u in GitHub Actions:

The r2u project provides Docker containers and setup actions. For most workflows, you can use the r2u setup action[24]:

```
- name: Setup r2u
  uses: eddelbuettel/github-actions/r2u-setup@master
```

Alternatively, use the rocker/r2u Docker containers directly in your workflow.

When to consider r2u:

- Your workflow installs many R packages
- Build times are a bottleneck in your CI pipeline
- You're running on Ubuntu (focal, jammy, or noble)
- You want to reduce GitHub Actions minutes usage

For complete documentation and setup instructions, see the r2u website[25].

13.10 Additional Resources

- GitHub Actions features overview[26]
- r-lib/actions repository[27] - R-specific actions and example workflows
- R Packages book: Continuous Integration[28]
- GitHub Actions documentation[29]
- Where to find help with r-lib/actions[30]
- r-ci: Easy and portable CI for R packages[31] - a portable shell script by Dirk Eddelbuettel for running R package CI across GitHub Actions, Azure Pipelines, CircleCI, and Docker
- r-codeberg migration guide[32] - step-by-step tutorial on porting R package development, documentation, and CI workflows from GitHub to Codeberg
- Woodpecker CI[33] - community-driven, container-based continuous integration system

[24] https://github.com/eddelbuettel/github-actions/tree/master/r2u-setup
[25] https://eddelbuettel.github.io/r2u/
[26] https://github.com/features/actions
[27] https://github.com/r-lib/actions
[28] https://r-pkgs.org/software-development-practices.html#sec-sw-dev-practices-ci
[29] https://docs.github.com/en/actions
[30] https://github.com/r-lib/actions#where-to-find-help
[31] https://eddelbuettel.github.io/r-ci/
[32] https://codeberg.org/r-codeberg/migration-guide
[33] https://woodpecker-ci.org/

that integrates with Codeberg, Forgejo, and Gitea for R package checks
- shinycoreci: CI Testing for shiny[34] - R package developed by the Posit Shiny team for testing the Shiny ecosystem across GitHub Actions runner environments
- GitHub Status[35] - real-time status of GitHub services

[34] https://rstudio.github.io/shinycoreci/
[35] https://www.githubstatus.com/

14 Unix

We typically use Unix commands in Terminal (for Mac users) or Git Bash (for Windows users) to

1. Run a series of scripts in parallel or in a specific order to reproduce our work
2. To check on the progress of a batch of jobs
3. To use git and push to github

14.1 User interfaces: CLI, TUI, and GUI

Software tools expose different interaction models depending on whether they run in a terminal or a desktop window. Understanding the trade-offs between Command-Line Interfaces (CLIs), Terminal User Interfaces (TUIs), and Graphical User Interfaces (GUIs) helps developers choose the right tool for local development, remote server administration, and automated analysis pipelines.

14.1.1 Command-line interfaces (CLI)

A Command-Line Interface (CLI) processes discrete text commands typed into a shell prompt. Commands execute sequentially, read arguments and configuration flags, and stream output to standard output (`stdout`) or standard error (`stderr`).

Key characteristics:

- **Scriptable and composable:** Commands can be chained with Unix pipes (`|`), redirected to files (`>`), and automated in shell scripts or CI/CD pipelines without human interaction.
- **Stateless execution:** Each command runs to completion and exits, returning control to the prompt.
- **Low overhead:** Requires minimal system memory and CPU, making it suitable for headless servers and high-performance computing (HPC) nodes.
- **Steep initial learning curve:** Users must memorize command syntax, flags, and arguments or consult manual pages (`man`) and `--help` output.

Common data science examples: `git`, `quarto`, `Rscript`, `curl`, and `gh`.

14.1.2 Terminal user interfaces (TUI)

A Terminal User Interface (TUI)—also called a text-based user interface—provides an interactive visual layout rendered entirely inside a text terminal. TUIs draw panels, borders, menus, and color highlights using ASCII or ANSI text grids rather than native windowing frameworks.

Key characteristics:

- **Interactive visual dashboards:** Displays real-time lists, split views, and scrollable panels while remaining inside the terminal.
- **Keyboard-driven efficiency:** Operates entirely via rapid keyboard shortcuts, avoiding mouse context-switching.
- **Full SSH and multiplexer compatibility:** Runs seamlessly over lightweight SSH connections and inside terminal multiplexers like `tmux` without X11 forwarding or browser overhead.
- **Terminal constraints:** Cannot display arbitrary high-resolution images or interactive plots without specialized terminal graphics extensions.

Common developer examples: `lazygit` (interactive Git client), `htop` / `btop` (interactive system process monitors), `ranger` (terminal file manager), and `tig` (text-mode Git repository browser).

14.1.3 Graphical user interfaces (GUI)

A Graphical User Interface (GUI) presents visual controls such as windows, buttons, menus, and dialog boxes, typically navigated using a mouse pointer and keyboard shortcuts.

Key characteristics:

- **High discoverability:** Menus, toolbars, and visual forms expose available actions without requiring users to memorize commands in advance.
- **Rich multi-panel layouts:** Enables simultaneous side-by-side inspection of plots, data tables, code editors, and environment variables.
- **Difficult to automate:** Actions taken by clicking buttons cannot be trivially replayed in batch pipelines unless the application provides an underlying scripting API.
- **Remote latency and display requirements:** Running a remote GUI requires an active graphical display server, such as X11 forwarding (see Section 18.6) or a web-based portal like RStudio Server on Mercury.

Common data science examples: RStudio, VS Code, GitHub Desktop, and GitKraken.

14.1.4 Summary and trade-offs

Table 14.1: Comparison of CLI, TUI, and GUI interface models

Interface	Primary interaction	Visual feedback	Automation / Scripting	Remote SSH performance	Primary use case
CLI	Text commands and flags	Sequential text streams	High (shell scripts, CI pipelines)	Excellent (native text)	Automated batch jobs, pipelines, scripting
TUI	Keyboard navigation and hotkeys	Interactive terminal dashboard	Low to moderate	Excellent (runs directly in shell or `tmux`)	Remote server inspection, fast staging, system monitoring

Interface	Primary interaction	Visual feedback	Automation / Scripting	Remote SSH performance	Primary use case
GUI	Mouse pointer and keyboard shortcuts	Windows, menus, graphic plots	Low	Requires X11 or web server	Exploratory analysis, visual editing, multi-panel IDEs

In research computing, the three paradigms complement one another. A typical workflow uses a **GUI** (such as RStudio) for exploratory analysis and visualization, a **TUI** (such as `lazygit` or `htop`) for managing version control and monitoring remote cluster jobs, and a **CLI** (such as `quarto render` or `Rscript`) for reproducible scripts and automated pipelines.

14.2 Basics

On the computer, there is a desktop with two folders, `folder1` and `folder2`, and a file called `file1`. Inside `folder1`, we have a file called `file2`. Mac users can run these commands on their terminal; it is recommended that Windows users use Git Bash, not Windows PowerShell.

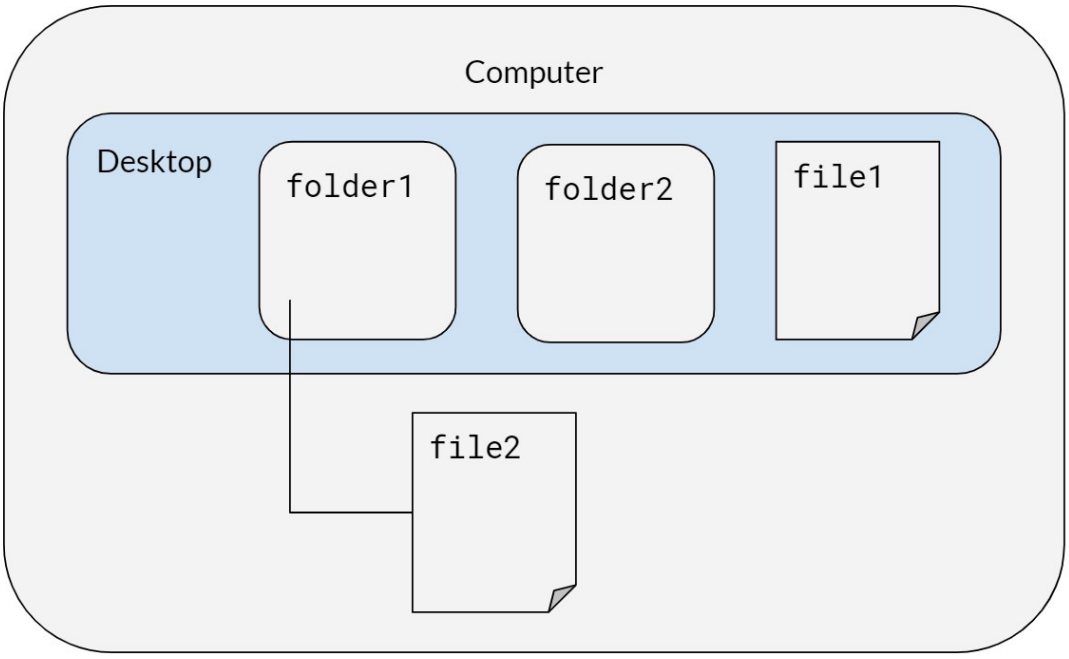


Figure 14.1: Example desktop with folders and files

14.3 Terminal Emulators

A terminal emulator is the application window that runs your shell (such as `zsh`, `bash`, or `fish`). While macOS ships with Terminal.app and Windows includes Command Prompt, modern terminal emulators provide significant advantages:

- **GPU acceleration**: Rendering text on the GPU keeps scrolling and large output streams smooth and responsive.
- **Modern font rendering**: Full support for true color, ligatures, and powerline or nerd font symbols.
- **Window management**: Native support for tabs, split panes, and multiple windows without requiring a separate multiplexer like `tmux`.
- **Cross-platform consistency**: Several modern emulators use identical configuration files across macOS, Linux, and Windows.

14.3.1 Recommended Options

14.3.1.1 Ghostty[1]

Ghostty is a fast, GPU-accelerated terminal emulator developed by Mitchell Hashimoto. It focuses on native platform integration, performance, and rich terminal features:

- Native user interface on macOS and Linux
- Cross-platform configuration using a simple plain-text configuration file
- Built-in tabs and split panes
- True color and modern terminal graphics protocol support
- Available on macOS and Linux (Windows support in active development as of September 2026)

14.3.1.2 WezTerm[2]

WezTerm is a GPU-accelerated terminal emulator and multiplexer written in Rust:

- Configured using Lua, enabling powerful custom keybindings and automation scripts
- Built-in multiplexing across tabs, splits, and workspaces
- Support for SSH domains to seamlessly reconnect to remote servers
- Rich graphics support including iTerm2 and Sixel image protocols
- Cross-platform support on macOS, Linux, and Windows

14.3.1.3 Alacritty[3]

Alacritty is a minimalist, GPU-accelerated terminal emulator written in Rust:

- Focused strictly on raw rendering speed and simplicity
- Configured using TOML
- Omits built-in tabs and split panes by design; often paired with `tmux` for multiplexing
- Cross-platform support on macOS, Linux, and Windows

[1] https://ghostty.org/
[2] https://wezfurlong.org/wezterm/
[3] https://alacritty.org/

14.3.1.4 Kitty[4]

Kitty is a GPU-accelerated terminal emulator designed for keyboard-driven power users:

- Rich graphics protocol for rendering images directly in the terminal
- Built-in support for window layouts, tabs, and startup sessions
- Configured via plain text configuration files
- Highly scriptable from the command line
- Available on Linux and macOS

14.3.1.5 iTerm2[5]

iTerm2 is a mature, feature-packed terminal emulator built specifically for macOS:

- Extensive GUI settings alongside scriptable configuration
- Native split panes, tab management, and search features
- Rich shell integration, inline image support, and paste history
- Available exclusively on macOS

14.3.1.6 Windows Terminal[6]

Windows Terminal is a modern, high-performance terminal emulator developed by Microsoft:

- GPU-accelerated text rendering engine (Direct3D-based AtlasEngine)
- Tabbed interface supporting multiple shells simultaneously (PowerShell, Command Prompt, Git Bash, and WSL)
- Customizable themes, color schemes, and keybindings via GUI or JSON
- Available on Windows 10 and Windows 11

14.3.2 Comparison Summary

Table 14.2: Modern terminal emulator comparison

Terminal Emulator	Platforms	GPU Accelerated	Built-in Tabs/Splits	Configuration
Ghostty[7]	macOS, Linux	Yes	Yes	Plain text
WezTerm[8]	macOS, Linux, Windows	Yes	Yes	Lua
Alacritty[9]	macOS, Linux, Windows	Yes	No (use `tmux`)	TOML
Kitty[10]	macOS, Linux	Yes	Yes	Plain text

[4] https://sw.kovidgoyal.net/kitty/
[5] https://iterm2.com/
[6] https://github.com/microsoft/terminal
[7] https://ghostty.org/
[8] https://wezfurlong.org/wezterm/
[9] https://alacritty.org/
[10] https://sw.kovidgoyal.net/kitty/

Terminal Emulator	Platforms	GPU Accelerated	Built-in Tabs/Splits	Configuration
iTerm2[11]	macOS	Yes	Yes	GUI / Property list
Windows Terminal[12]	Windows	Yes	Yes	JSON / GUI

14.4 Syntax for both Mac/Windows

When typing in directories or file names, quotes are necessary if the name includes spaces.

Table 14.3: Basic Unix commands for Mac and Windows

Command	Description
`cd desktop/folder1`	Change directory to `folder1`
`pwd`	Print working directory
`ls`	List files in the directory
`cp "file2" "newfile2"`	Copy file (remember to include file extensions when typing in file names like `.pdf` or `.R`)
`mv "newfile2" "file3"`	Rename `newfile2` to `file3`
`cd ..`	Go to parent of the working directory (in this case, `desktop`)
`mv "file1" folder2`	Move `file1` to `folder2`
`mkdir folder3`	Make a new folder in `folder2`
`rm <filename>`	Remove files
`rm -rf folder3`	Remove directories (`-r` will attempt to remove the directory recursively, `-rf` will force removal of the directory)
`clear`	Clear terminal screen of all previous commands

[11] https://iterm2.com/
[12] https://github.com/microsoft/terminal

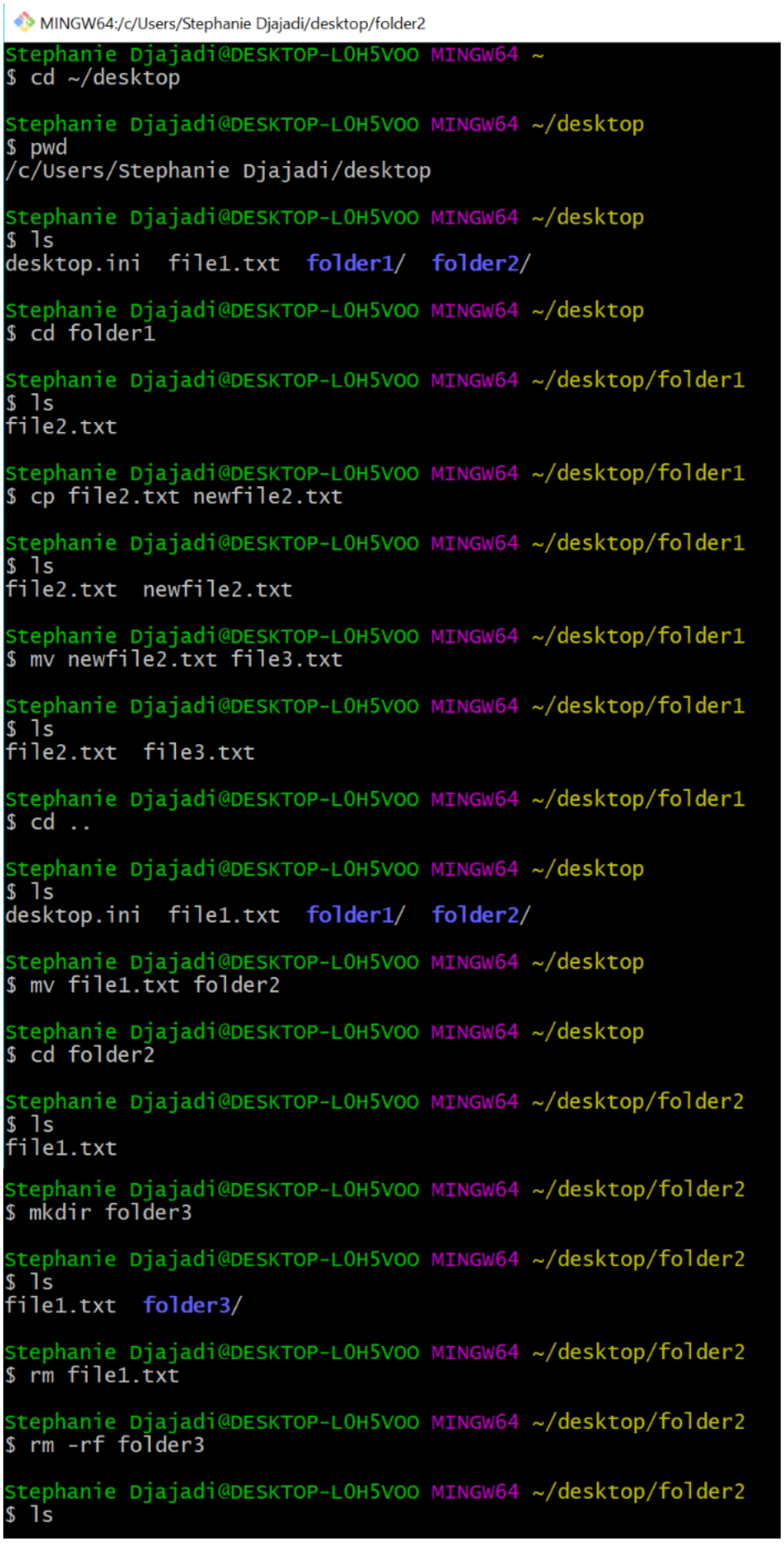


Figure 14.2: Terminal output after executing basic Unix commands

14.5 Zsh and Oh My Zsh

The commands in the tables above work the same in any shell. This section is about which shell to use *interactively* — the one that gives you a prompt, completes filenames, and remembers your history. It does not change how scripts run: a script beginning `#!/usr/bin/env bash` still runs under bash no matter which shell you launched it from.

We recommend Z shell (`zsh`) with Oh My Zsh[13], a configuration framework that supplies themes, completions, and a plugin system. Two reasons:

- `zsh` has been the default login shell on macOS since 10.15 (Catalina), so most of the lab is already running it.
- The plugin pair below — suggestions from your own history, and syntax coloring that shows a typo before you press enter — removes a real class of command-line mistake.

This applies to macOS, Linux, and Windows Subsystem for Linux (WSL). It does not apply to Git Bash on Windows, which is bash and has no `zsh` to switch to; Windows users who want this should work inside WSL.

14.5.1 Installing zsh

Check whether you already have it:

```
zsh --version
```

If that errors, install it (`brew install zsh` on macOS, `sudo apt install zsh` on Ubuntu and WSL), then make it your login shell:

```
chsh -s "$(command -v zsh)"
```

`chsh` prompts for your own password, and refuses any shell not listed in `/etc/shells`. An `apt` install registers itself there; a Homebrew build does not, so append its path first:

```
command -v zsh | sudo tee -a /etc/shells
```

The change takes effect at your next login, so open a new terminal and confirm with `echo $SHELL`. On WSL, close every window for that distribution first — or run `wsl --terminate <distro>` from PowerShell — since the shell is chosen when the distribution starts.

14.5.2 Installing Oh My Zsh

```
sh -c "$(curl -fsSL https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.
```

The installer clones the framework to `~/.oh-my-zsh`, replaces `~/.zshrc` with its own template, and backs up whatever was there to `~/.zshrc.pre-oh-my-zsh`. Check that backup for anything worth carrying forward before you delete it. If you would rather it not touch your login shell or drop you into `zsh` immediately, run it unattended instead:

[13] https://ohmyz.sh/

```
sh -c "$(curl -fsSL https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.
```

The empty `""` is not a typo. `sh -c` treats the argument after the script as `$0`, so without a placeholder there `--unattended` would be consumed as the script's own name instead of reaching it as a flag. The installer's header documents that flag as setting both `CHSH` and `RUNZSH` to `no`, which leaves the `chsh` step above to you.

14.5.3 The two plugins we use

`zsh-autosuggestions`[14] proposes the rest of a command in gray as you type it, drawn from your history; press the right arrow to accept it. `zsh-syntax-highlighting`[15] colors the command line as you type — a command that does not exist stays red, so a misspelled `git ceckout` is visible before you run it.

Neither ships with Oh My Zsh, so clone both into its custom-plugin directory:

```
git clone https://github.com/zsh-users/zsh-autosuggestions \
  "${ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/plugins/zsh-autosuggestions"
git clone https://github.com/zsh-users/zsh-syntax-highlighting \
  "${ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/plugins/zsh-syntax-highlighting"
```

Then enable them in `~/.zshrc`. Oh My Zsh's template ships `plugins=(git)`, so the line below is that default plus the two — if you have already added others, append to your list rather than pasting over it:

```
plugins=(git zsh-autosuggestions zsh-syntax-highlighting)
```

Keep `zsh-syntax-highlighting` last. Its own installation guide states that it "must be the last plugin sourced". The reason, from its FAQ, is that it hooks into the Zsh Line Editor, and those hooks run in the order they were registered — so it has to register after anything else that modifies the command-line buffer. The failure is quiet: a widget added by a later plugin still works, it just stops updating the highlighting, with no error to tell you why.

Open a new terminal, or run `exec zsh`, to load the changes.

14.5.4 More plugins, at no install cost

The two above had to be cloned because they live outside Oh My Zsh. Oh My Zsh itself ships several hundred plugins that are already on disk — adding a name to `plugins=()` is the whole installation. Run `ls ~/.oh-my-zsh/plugins` to see the full list. These are the ones that earn their place for the work this manual describes:

[14] https://github.com/zsh-users/zsh-autosuggestions
[15] https://github.com/zsh-users/zsh-syntax-highlighting

Table 14.4: Bundled Oh My Zsh plugins worth enabling

Plugin	What it gives you
`gh`	Completions for the GitHub CLI, which our workflow leans on heavily
`tmux`	Aliases for the `tmux` session commands this chapter uses for long-running jobs
`extract`	`x <file>` unpacks any archive, so you stop looking up `tar` flags
`command-not-found`	On Ubuntu and WSL, names the package that would supply a missing command
`history-substring-search`	Type a prefix, then press up to walk only the history entries that match
`colored-man-pages`	Syntax coloring in `man`, which matters most in the long pages
`safe-paste`	Holds a pasted command with a trailing newline instead of running it unread
`sudo`	Press escape twice to prefix the current or previous command with `sudo`

`history-substring-search` is the documented exception to the ordering rule above. Its own README says to load `zsh-syntax-highlighting` *before* it, which is the opposite of the "keep syntax-highlighting last" instruction, and both are correct. The plugin handles the overlap itself rather than leaving you to. Its source, at `~/.oh-my-zsh/plugins/history-substring-search/history-substring-search.zsh`, tests `ZSH_HIGHLIGHT_VERSION` inside its own redraw hook and, under the comment "If the zsh-syntax-highlighting plugin has been loaded … remove our hooks", unregisters both of its hooks with `add-zle-hook-widget -d`. So put it last of all:

```
plugins=(git zsh-autosuggestions zsh-syntax-highlighting history-substring-search)
```

Every plugin costs startup time, and a shell that takes a second to appear is one you will resent. Measure rather than guess, before and after a batch of additions:

```
time zsh -i -c exit
```

14.5.5 Prompt themes

Oh My Zsh sets `ZSH_THEME="robbyrussell"` by default, which shows the working directory and the git branch. `ls ~/.oh-my-zsh/themes` lists the bundled alternatives. Two non-bundled options come up often enough to be worth describing honestly.

Powerlevel10k[16] is the one people are usually told to install: a fast, information-dense prompt with an interactive setup wizard.

```
git clone --depth=1 https://github.com/romkatv/powerlevel10k \
  "${ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/themes/powerlevel10k"
```

[16] https://github.com/romkatv/powerlevel10k

Set `ZSH_THEME="powerlevel10k/powerlevel10k"` and run `p10k configure`.

Install it knowing its status. As of 2026-08-17 its README opens with a notice in capitals: the project has very limited support, no new features are in the works, most bugs will go unfixed, and help requests will be ignored. The author's position is that it does not need active maintenance to keep working — "if it works now for you, it'll keep working" — which is a reasonable thing to rely on for a prompt, and a bad thing to rely on if you expect a bug you hit to be fixed.

Starship[17] is the actively developed alternative, written in Rust and configured through one `~/.config/starship.toml`:

```
curl -sS https://starship.rs/install.sh | sh
```

Its distinguishing feature is that it is not a zsh theme at all — the same binary and the same config drive the prompt in bash, fish, PowerShell, and others. That matters if you move between a Mac laptop, a Linux server, and Git Bash on Windows, since one file follows you instead of three. Oh My Zsh bundles a `starship` plugin, so adding `starship` to `plugins=()` is the whole setup — the plugin unsets `ZSH_THEME` itself, and says so in a comment, so there is no leftover theme to remove by hand.

Both want a Nerd Font, and this is the step people miss. The prompts draw git status and other state with glyphs from the private-use area, which an unpatched font renders as boxes. Powerlevel10k's wizard offers to install MesloLGS NF for you. The font is a setting of the terminal *emulator*, so on a remote machine it is configured on your own laptop and not on the server — installing fonts on Shiva will not fix boxes in your local terminal.

14.5.6 Command-line tools that pair with the shell

These are not zsh plugins, and they are most of what people mean when they say someone else's terminal is nicer than theirs. Each stands alone; install the ones that answer a problem you actually have.

Table 14.5: Command-line tools worth knowing about

Tool	What it replaces or adds
`fzf`[18]	Fuzzy interactive filtering, most usefully over history — enable the bundled `fzf` plugin to wire up its key bindings
`ripgrep`[19] (`rg`)	Recursive search that is fast on a large repo and skips what `.gitignore` names
`fd`[20]	`find` with defaults you would have wanted anyway
`bat`[21]	`cat` with syntax highlighting and paging
`delta`[22]	Readable `git diff` and `git show` output, including word-level highlighting

[17] https://starship.rs/
[18] https://github.com/junegunn/fzf
[19] https://github.com/BurntSushi/ripgrep
[20] https://github.com/sharkdp/fd
[21] https://github.com/sharkdp/bat
[22] https://github.com/dandavison/delta

Tool	What it replaces or adds
`direnv`[23]	Per-directory environment variables, loaded on `cd` and unloaded on the way out
`zoxide`[24]	`z <fragment>` jumps to a directory you visit often, instead of retyping the path

`direnv` is the one with the most direct bearing on our own workflows: it is how a project can set `R_LIBS`, an API key, or a `RETICULATE_PYTHON` that applies inside that project and nowhere else. Oh My Zsh bundles `direnv` and `zoxide` plugins, so those two need no hook line in `~/.zshrc`.

14.5.7 Startup files, and what breaks when you switch

`zsh` does not read `~/.bashrc` or `~/.profile`. It reads `~/.zshenv` always, `~/.zprofile` at login, and `~/.zshrc` for every interactive shell — so anything you had accumulated in your bash startup files is simply gone the first time you log in under `zsh`.

The usual casualty is `PATH`. A line like

```
export PATH="$HOME/.local/bin:$PATH"
```

sitting at the bottom of `~/.bashrc` is what puts locally installed tools on your path, and nothing warns you when it stops being read — the tool just reports as not found. Copy any such lines into `~/.zshrc` when you switch, and check that the commands you rely on still resolve:

```
command -v quarto Rscript
```

14.6 Running Bash Scripts

Table 14.6: Commands for running Bash scripts

Windows	Mac / Linux	Description
`chmod +750 <filename.sh>`	`chmod +x <filename.sh>`	Change access permissions for a file (only needs to be done once)
`./<filename.sh>`	`./<filename.sh>`	Run file (`./` to run any executable file)
`bash bash_script_name.sh &`	`bash bash_script_name.sh &`	Run shell script in the background

[23] https://direnv.net/
[24] https://github.com/ajeetdsouza/zoxide

14.7 Shell Scripting Style

For shell scripts, we follow the Google Shell Style Guide[25] as the base authority, the same way our R chapters defer to the tidyverse style guide[26]. The points below are the lab's priorities and additions.

14.7.1 Error handling

Start every bash script with strict mode:

```
#!/usr/bin/env bash
set -euo pipefail
```

- `set -e` exits when a command fails unhandled (it deliberately skips commands tested in an `if` or joined with `&&`/`||`), `set -u` errors on undefined variables, and `set -o pipefail` makes a pipeline fail when any stage fails, not just the last.
- Use a `trap ... EXIT` handler for cleanup (temporary files, lock release) so it runs on failure as well as success.
- Decide explicitly whether a step should fail the script (fail-closed) or log and continue (fail-open), and comment the choice when it isn't obvious.

14.7.2 Functions and naming

Decompose scripts into functions the same way we decompose R code: small, single-purpose, and named in `snake_case`. Declare function-local variables with `local` so they don't leak into the global namespace, and return status with `return` codes (write data results to stdout).

14.7.3 Comments

The same principle as our R guidance applies: explain *why*, not *what*. Keep inline comments brief, and point to an Architecture Decision Record for rationale too long for a comment (see Section 6.15).

14.7.4 Logging

Prefer a small log helper over scattered `echo` calls, so messages get a consistent format and severity level, and send diagnostics to stderr so stdout stays parseable:

```
log() {
  local level=$1
  shift
  printf '%s [%s] %s\n' "$(date -u +%FT%TZ)" "$level" "$*" >&2
}
log INFO "starting import"
log ERROR "input file missing"
```

[25] https://google.github.io/styleguide/shellguide.html
[26] https://style.tidyverse.org/

The `shift` / `$*` pair joins every argument after the severity into the message, so a message that reaches the function as several words is still logged whole. Callers should still quote the message (`log ERROR "..."`) — the shell expands globs and splits words *before* `log()` runs, so an unquoted message can be mangled on the way in.

14.7.5 Security

- Never echo secrets; redact tokens before logging command lines that carry them.
- Quote every variable expansion (`"$var"`, `"$@"`) to prevent word-splitting and glob surprises.
- Pass untrusted values to `jq` as arguments (`jq --arg name "$value"`), not by interpolating them into the filter string.
- Keep credentials out of `curl` command lines, where any local process can read them from the process list: prefer `--netrc` or headers read from a file (`--header @headers.txt`) over a token expanded into an argument.

14.7.6 Linting

ShellCheck[27] is the shell equivalent of `{lintr}`[28]: run `shellcheck script.sh` locally and in CI, and fix or explicitly disable (with a justifying comment) every finding. `bash -n script.sh` is a quick syntax-only check.

14.7.7 Portability

Decide whether a script targets bash or POSIX `sh`, and match the shebang to the decision: bash-specific features (arrays, `[[ ]]`, `local -n`) need `#!/usr/bin/env bash`, while scripts that must run on minimal CI images (BusyBox, Alpine `ash`) should stick to POSIX constructs and use `#!/bin/sh`. ShellCheck understands both and checks against the declared shell.

14.7.8 Formatting

Follow the same readability conventions as our R style: lines under about 80 characters, two-space indentation, and one logical step per line, breaking long pipelines after the `|` with the continuation indented.

14.8 Running Rscripts in Windows

Note: This code seems to work only with Windows Command Prompt, not with Git Bash.

When R is installed, it comes with a utility called Rscript. This allows you to run R commands from the command line. If Rscript is in your `PATH,` then typing Rscript into the command line, and pressing enter, will not error. Otherwise, to use Rscript, you will either need to add it to your PATH (as an environment variable), or append the full directory of the location of Rscript on your machine. To find the full directory, search for where R

[27] https://www.shellcheck.net/
[28] https://lintr.r-lib.org/

is installed your computer. For instance, it may be something like below (this will vary depending on what version of R you have installed):

```
C:\Program Files\R\R-3.6.0\bin
```

For appending the `PATH` variable, please view this link[29]. I strongly recommend completing this option.

If you add the PATH as an environment variable, then you can run this line of code to test: `Rscript -e "cat('this is a test')"`, where the `-e` flag refers to the expression that will be executed.

If you do not add the PATH as an environment variable, then you can run this line of code to replicate the results from above: `"C:\Program Files\R\R-3.6.0\bin\Rscript.exe" -e "cat('this is a test')"`

To run an R script from the command line, we can say: `Rscript -e "source('C:/path/to/script/some_code`

14.8.1 Common Mistakes

- Remember to include all of the quotation marks around file paths that have a spaces.
- If you attempt to run an R script but run into `Error: '\U' used without hex digits in character string starting "'C:\U"`, try replacing all `\` with `\\` or `/`.

14.9 Checking tasks and killing jobs

Windows	Mac / Linux	Description
`tasklist`	`ps -v`	List all processes on the command line
	`top -o [cpu/rsize]`	List all running processes, sorted by CPU or memory usage
`taskkill /F /PID pid_number`	`kill <PID_number>`	Kill a process by its process ID
`taskkill /IM "process name" /F`		Kill a process by its name
`start /b program.exe`		Runs jobs in the background (exclude `/b` if you want the program to run in a new console)
	`nohup`	Prevents jobs from stopping
	`disown`	Keeps jobs running in the background even if you close R
`taskkill /?`		Help, lists out other commands

To kill a task in Windows, you can also go to Task Manager > More details > Select your desired app > Click on End Task.

[29] https://www.howtogeek.com/118594/how-to-edit-your-system-path-for-easy-command-line-access/

14.10 Running big jobs

For big data workflows, the concept of "backgrounding" a bash script allows you to start a "job" (i.e. run the script) and leave it overnight to run. At the top level, a bash script (`0-run-project.sh`) that simply calls the directory-level bash scripts (i.e. `0-prep-data.sh`, `0-run-analysis.sh`, `0-run-figures.sh`, etc.) is a powerful tool to rerun every script in your project. See the included example bash scripts for more details.

- **Running Bash Scripts in Background**: Running a long bash script is not trivial. Normally you would run a bash script by opening a terminal and typing something like `./run-project.sh`. But what if you leave your computer, log out of your server, or close the terminal? Normally, the bash script will exit and fail to complete. To run it in background, type `./run-project.sh &; disown`. You can see the job running (and CPU utilization) with the command `top` or `ps -v` and check your memory with `free -h`.

Alternatively, to keep code running in the background even when an SSH connection is broken, you can use `tmux`. In terminal or gitbash follow the steps below. This site[30] has useful tips on using `tmux`.

```
# create a new tmux session called session_name
tmux new -ssession_name

# run your job of interest
R CMD BATCH myjob.R &

# check that it is running
ps -v

# to exit the tmux session (Mac)
ctrl + b
d

# to reopen the tmux session to kill the job or
# start another job
tmux attach -tsession_name
```

- **Deleting Previously Computed Results**: One helpful lesson we've learned is that your bash scripts should remove previous results (computed and saved by scripts run at a previous time) so that you never mix results from one run with a previous run. This can happen when an R script errors out before saving its result, and can be difficult to catch because your previously saved result exists (leading you to believe everything ran correctly).
- **Ensuring Things Ran Correctly**: You should check the `.Rout` files generated by the R scripts run by your bash scripts for errors once things are run. A utility file is include in this repository, called `runFileSaveLogs`, and is used by the example bash scripts to… run files and save the generated logs. It is an awesome utility and one I definitely recommend using. Before using `runFileSaveLogs`, it is necessary to

[30] https://medium.com/@jeongwhanchoi/install-tmux-on-osx-and-basics-commands-for-beginners-be22520fd95e

put the file in the home working directory. For help and documentation, you can use the command `./runFileSaveLogs -h`. See example code and example usage for `runFileSaveLogs` below.

14.10.1 Example code for `runfileSaveLogs`

```
#!/usr/bin/env python3
# Type "./runFileSaveLogs -h" for help

import os
import sys
import argparse
import getpass
import datetime
import shutil
import glob
import pathlib

# Setting working directory to this script's current directory
os.chdir(os.path.dirname(os.path.abspath(__file__)))

# Setting up argument parser
parser = argparse.ArgumentParser(description='Runs the argument R script(s) - in parallel i

# Function ensuring that the file is valid
def is_valid_file(parser, arg):
    if not os.path.exists(arg):
        parser.error("The file %s does not exist!" % arg)
    else:
        return arg

# Function ensuring that the directory is valid
def is_valid_directory(parser, arg):
    if not os.path.isdir(arg):
        parser.error("The specified path (%s) is not a directory!" % arg)
    else:
        return arg

# Additional arguments that can be added when running runFileSaveLogs
parser.add_argument('-p', '--parallel', action='store_true', help="Runs the argument R scri
parser.add_argument("-i", "--identifier", help="Adds an identifier to the directory name wh
parser.add_argument('filenames', nargs='+', type=lambda x: is_valid_file(parser, x))

args = parser.parse_args()
args_dict = vars(args)

print(args_dict)

# Run given R Scripts
```

```
for filename in args_dict["filenames"]:
  system_call = "R CMD BATCH" + " " + filename
  if args_dict["parallel"]:
    system_call = "nohup" + " " + system_call + " &"

  os.system(system_call)

# Create the directory (and any parents) of the log files
currentUser = getpass.getuser()
currentTime = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
logDirPrefix = "/home/kaiserData/logs/" # Change to the directory where the logs should be
logDir = logDirPrefix + currentTime + "-" + currentUser

# If specified, adds the identifier to the filename of the log
if args.identifier is not None:
  logDir += "-" + args.identifier

logDir += "/"

pathlib.Path(logDir).mkdir(parents=True, exist_ok=True)

# Find and move all logs to this new directory
currentLogPaths = glob.glob('./*.Rout')

for currentLogPath in currentLogPaths:
  filename = currentLogPath.split("/")[-1]
  shutil.move(currentLogPath, logDir + filename)
```

14.10.2 Example usage for `runfileSaveLogs`

This example bash script runs files and generates logs for five scripts in the `kaiserflu/3-figures` folder. Note that the `-i` flag is used as an identifier to add `figures` to the filename of each log.

```
#!/bin/bash

# Copy utility run script into this folder for concision in call
cp ~/kaiserflu/runFileSaveLogs ~/kaiserflu/3-figures/

# Run folder scripts and produce output
cd ~/kaiserflu/3-figures/
./runFileSaveLogs -i "figures" \
fig-mean-season-age.R \
fig-monthly-rate.R \
fig-point-estimates-combined.R \
fig-point-estimates.R \
fig-weekly-rate.R

# Remove copied utility run script
rm runFileSaveLogs
```

14.11 External Resources

For a comprehensive introduction to practical computing skills, shell tools, command-line workflows, and version control, see The Missing CS Quarter at UC Davis[31] (GitLab repository[32]).

[31] https://missing.cs.ucdavis.edu/
[32] https://gitlab.com/missing-cs-quarter-ucd/

15 Reproducible Environments

15.1 Package Version Control with renv

15.1.1 Introduction

Replicable code should produce the same results, regardless of when or where it's run. However, our analyses often leverage open-source R packages that are developed by other teams. These packages continue to be developed after research projects are completed, which may include changes to analysis functions that could impact how code runs for both other team members and external replicators.

For example, suppose we had used a function that took in one argument, such that our code contained `example_function(arg_a = "a")`. A few months after we publish our code, the package developers update the function to take in another mandatory argument `arg_b`. If someone runs our code, but has the most recent version of the package, they'll receive an error message that the argument `arg_b` is missing and will not be able to fully reproduce our results.

To ensure that the right functions are used in replication efforts, it is important for us to keep track of package versions used in each project.

`renv` can be to promote reproducible environments within R projects. `renv` creates individual package libraries for each project instead of having all projects, which may use different versions of the same package, share the same package library. However, for projects that use many packages, this process can be memory intensive and increase the time needed for a new users to start running code.

In this lab manual chapter, we provide a quick tutorial for integrating `renv` into research workflows. For more detailed instructions, please refer to the `renv` package vignette.

15.1.2 Implementing renv in projects

Ideally, `renv` should be initiated at the start of projects and updated continuously when new packages are introduced in the codebase. However, this process can be initiated at any point in a project

To add `renv` to your workflow, follow these steps:

1. Install the `renv` package by running `install.packages("renv")`
2. Create an RProject file and ensure that your working directory is set to the correct folder
3. In the R console, run `renv::init()` to initialize renv in your R Project
4. This will create the following files: `renv.lock`, `.Rprofile`, `renv/settings.json` and `renv/activate.R`. Commit and push these files to GitHub so that they're accessible to other users.

5. As you write code, update the project's R library by running `renv::snapshot()` in the R console
6. Add `renv::restore()` to the head of your config file, to make sure that all users that run your code are on the same package versions.

15.1.3 Configuring renv settings

The `renv/settings.json` file created during initialization allows you to customize how `renv` behaves in your project. One useful setting is `snapshot.dev`, which controls whether development dependencies are included by default when calling `renv::snapshot()` or `renv::status()`.

15.1.3.1 Reducing startup messages

When working on projects, you may encounter startup messages indicating that `renv` is out of sync with the lockfile. To reduce these messages in most projects, add the following setting to `renv/settings.json`:

```
"snapshot.dev": true
```

This setting (available in `renv` version 1.1.6 and later) includes development dependencies in snapshots by default, which helps keep the lockfile aligned with your actual usage and eliminates many synchronization warnings.

If startup messages about being out of sync persist after enabling this setting, use `renv::restore()` to sync your local library with the lockfile, or `renv::snapshot()` to update the lockfile with your current package versions.

For more details on `renv` configuration options, see the official renv documentation[1].

15.1.4 Using projects with renv

If you're starting to work on an ongoing project that already has `renv` set up, follow these steps to ensure that you're using the same project versions.

1. Install the `renv` package by running `install.packages("renv")`
2. Pull the most updated version of the project from GitHub
3. Open the project's RProject file
4. Run `renv::restore()`. In our lab's projects, this is often already found at the top of the config file, so you can just run scripts as is.
5. This will pull up a list of the project's packages that need to be updated for you to be consistent with the project. The console will ask if you want to proceed with updating these packages - type "Y" to continue.
6. Wait for the correct versions of each package to install/update. This may take some time, depending on how many packages the project uses.
7. Your R environment should now be using the same package versions as specified in the `renv` lock file. You should now be able to replicate the code.
8. If you make edits to the code and introduce new/updated packages, see the section above for instructions on how to make updates.

[1] https://rstudio.github.io/renv/

15.2 Fast Package Installation on Ubuntu with r2u

For users working on Ubuntu Linux systems, r2u[2] provides an alternative approach to package management that complements or replaces `renv` in certain workflows.

What is r2u?

r2u integrates R package installation with Ubuntu's `apt` package manager, providing pre-compiled binary packages for all CRAN packages. This integration offers several advantages over traditional R package installation:

- **Fast installation**: Binary packages install in seconds, not minutes
- **Automatic dependency resolution**: System dependencies are handled automatically
- **System-level integration**: Packages integrate with Ubuntu's package management
- **No compilation required**: Eliminates the need for build tools and compilers

Supported Ubuntu versions (as of early 2025):

- Ubuntu 24.04 "noble" (amd64 and arm64)
- Ubuntu 22.04 "jammy" (amd64)
- Ubuntu 20.04 "focal" (amd64, archived - no new updates)

When to use r2u vs. renv:

- **Use r2u when**: You work primarily on Ubuntu systems, want fast package installation, or need system-level package management
- **Use renv when**: You need cross-platform reproducibility, work on macOS/Windows, or require project-specific package libraries
- **Use both when**: You want fast installation on Ubuntu while maintaining reproducibility through renv lockfiles

Basic setup:

r2u requires a one-time system setup. See the r2u website[3] for detailed installation instructions.

Once configured, R's `install.packages()` automatically uses binary packages from r2u, making package installation much faster.

Docker containers:

The Rocker Project provides `rocker/r2u` Docker containers with r2u pre-configured, ideal for reproducible containerized workflows.

For complete documentation and setup instructions, see the r2u website[4].

[2] https://eddelbuettel.github.io/r2u/
[3] https://eddelbuettel.github.io/r2u/
[4] https://eddelbuettel.github.io/r2u/

15.3 Managing R Versions with rig

rig[5] is an R installation manager that makes it easy to install, manage, and switch between multiple versions of R on macOS, Windows, and Linux.

When working across projects that require different R versions, or when you need to match a specific R version for reproducibility, `rig` provides a simple command-line interface to handle R version management.

Key features:

- **Multi-version support**: Install and switch between multiple R versions
- **Cross-platform**: Works on macOS, Windows, and Linux
- **Simple CLI**: Install a version with `rig add release`, list installed versions with `rig list`
- **RStudio / Positron integration**: Registered R versions are automatically available in RStudio and Positron
- **Handles permissions for you**: switches to the root/administrator user as needed when installing R versions

When to use rig:

- You need a specific older R version to reproduce results from a published analysis
- You want to test code against multiple R versions
- You are setting up a new machine and want a streamlined R installation process

Basic usage:

```
# Install the latest release of R
rig add release

# Install a specific version
rig add 4.3.3

# List installed versions
rig list

# Switch the default R version
rig default 4.3.3
```

For full installation instructions and documentation, see the rig GitHub repository[6].

15.4 Clearing Docker Cache

When Docker images and containers accumulate on a server, they can consume significant disk space. To reclaim disk space by removing unused Docker data:

1. SSH into the Docker server.
2. Run `docker system prune -a`.

[5] https://github.com/r-lib/rig
[6] https://github.com/r-lib/rig

This command removes all stopped containers, all unused networks, all unused images (not just dangling ones), and all build cache.

> ⚠ Warning
>
> `docker system prune -a` removes **all** unused images, not just dangling ones. Confirm that no important containers are currently stopped before running it, as their associated images will also be removed.

16 Code Publication

16.1 Checklist overview

1. Fill out file headers
2. Clean up comments
3. Document functions
4. Remove deprecated filepaths
5. Ensure project runs via bash
6. Complete the README
7. Clean up feature branches
8. Create GitHub release

16.2 Fill out file headers

Every file in a project should have a header that allows it to be interpreted on its own. It should include the name of the project and a short description for what this file (among the many in your project) does specifically. See template here.[1]

16.3 Clean up comments

Make sure comments in the code are for code documentation purposes only. Do not leave comments to self in the final script files.

16.4 Document functions

Every function you write must include a header to document its purpose, inputs, and outputs. See template for the function documentation here.[2]

16.5 Remove deprecated filepaths

All file paths should be defined in 0-config.R, and should be set relative to the project working directory. All absolute file paths from your local computer should be removed, and replaced with a relative path. If a third party were to re-run this analysis, if they need to download data from a separate source and change a filepath in the 0-config.R to match, make sure to specify in the README which line of 0-config.R needs to be substituted.

[1] https://ucd-serg.github.io/lab-manual/coding-practices.html#file-headers
[2] https://ucd-serg.github.io/lab-manual/coding-practices.html#function-documentation

16.6 Ensure project runs via bash

The project should be configured to be entirely reproducible by running a master bash script, run-project.sh, which should live at the top directory. This bash script can call other bash scripts in subfolders, if necessary. Bash scripts should use the runFileSaveLogs utility script, which is a wrapper around the Rscript command, allowing you to specify where .Rout log files are moved after the R scripts are run.

See usage and documentation here.[3]

16.7 Complete the README

A `README.md` should live at the top directory of the project. Include a repository status badge from repostatus.org[4] near the top of the file to communicate whether the codebase is actively maintained, a work in progress, or an archived analysis (see Section 5.5). This file also includes a Project Overview and a Directory Structure, along with contributor names and the project license. See below for a template:

> **Overview**
>
> To date, coronavirus testing in the US has been extremely limited. Confirmed COVID-19 case counts underestimate the total number of infections in the population. We estimated the total COVID-19 infections – both symptomatic and asymptomatic – in the US in March 2020. We used a semi-Bayesian approach to correct for bias due to incomplete testing and imperfect test performance.
>
> **Directory structure**
>
> - 0-config.R: configuration file that sets data directories, sources base functions, and loads required libraries
> - 0-base-functions: folder containing scripts with functions used in the analysis
>   - 0-base-functions.R: R script containing general functions used across the analysis
>   - 0-bias-corr-functions.R: R script containing functions used in bias correction
>   - 0-bias-corr-functions-undertesting.R: R script containing functions used in bias correction to estimate the percentage of underestimation due to incomplete testing vs. imperfect test accuracy
>   - 0-prior-functions.R: R script containing functions to generate priors
> - 1-data: folder containing data processing scripts NOTE: some scripts are deprecated
> - 2-analysis: folder containing analysis scripts. To rerun all scripts in this subdirectory, run the bash script 0-run-analysis.sh.
>   - 1-obtain-priors-state.R: obtain priors for each state

[3] https://ucd-serg.github.io/lab-manual/unix.html#example-code-for-runfilesavelogs
[4] https://www.repostatus.org/

  - 2-est-expected-cases-state.R: estimate expected cases in each state
  - 3-est-expected-cases-state-perf-testing.R: estimate expected cases in each state, estimate the percentage of underestimation due to incomplete testing vs. imperfect test accuracy
  - 4-obtain-testing-protocols.R: find testing protocols for each state.
  - 5-summarize-results.R: summarize results; obtain results for in text numerical results.
- 3-figure-table-scripts: folder containing figure scripts. To rerun all scripts in this subdirectory, run the bash script 0-run-figs.sh.
  - 1-fig-testing.R: creates plot of testing patterns by state over time
  - 2-fig-cases-usa-state-bar.R: creates bar plot of confirmed vs. estimated infections by state
  - 3a-fig-map-usa-state.R: creates map of confirmed vs. estimated infections by state
  - 3b-fig-map-usa-state-shiny.R: creates map of confirmed vs. estimated infections by state with search functionality by state
  - 4-fig-priors.R: creates figure with priors for US as a whole
  - 5-fig-density-usa.R: creates figure of distribution of estimated cases in the US
  - 6-table-data-quality.R: creates table of data quality grading from COVID Tracking Project
  - 7-fig-testpos.R: creates figure of the probability of testing positive among those tested by state
  - 8-fig-percent-undertesting-state.R: creates figure of the percentage of under estimation due to incomplete testing
- 4-figures: folder containing figure files.
- 5-results: folder containing analysis results objects.
- 6-sensitivity: folder containing scripts to run the sensitivity analyses

Contributors: UCD-SeRG team (adapted from original contributors: Jade Benjamin-Chung, Sean L. Wu, Anna Nguyen, Stephanie Djajadi, Nolan N. Pokpongkiat, Anmol Seth, Andrew Mertens)

Wu SL, Mertens A, Crider YS, Nguyen A, Pokpongkiat NN, Djajadi S, et al. Substantial underestimation of SARS-CoV-2 infection in the United States due to incomplete testing and imperfect test accuracy. medRxiv. 2020; 2020.05.12.20091744. doi:10.1101/2020.05.12.20091744

When possible, also include a description of the RDS results that are generated, detailing what data sources were used, where the script lives that creates it, and what information the RDS results hold.

16.8 Clean up feature branches

In the remote repository on GitHub, all feature branches aside from master should be merged in and deleted. All outstanding PRs should be closed.

16.9 Create GitHub release

Once all of these items are verified, create a tag to make a GitHub release, which will tag the repository, creating a marker at this specific point in time.

Detailed instructions: Managing releases in a repository[5].

16.10 Releasing to CRAN

If your R package is intended for public distribution, you may want to release it on CRAN[6] (the Comprehensive R Archive Network). This section summarizes the release process and adds lab-specific guidance on versioning and tagging. For full details, see the R Packages book chapter on releasing to CRAN[7] by Hadley Wickham and Jennifer Bryan.

The CRAN submission pipeline goes through several stages: initial upload, automated checks, human review by CRAN maintainers, and final acceptance or rejection. For a visual overview of the full pipeline, see the CRAN stages diagram[8] by Edgar Ruiz.

16.10.1 Versioning

R package versions follow a `major.minor.patch` scheme (e.g., `1.2.3`). Use a fourth development component (e.g., `1.2.3.9000`) on the development version to distinguish it from the released version.

- Increment the **patch** version (`1.2.3` → `1.2.4`) for bug fixes
- Increment the **minor** version (`1.2.3` → `1.3.0`) for new features that are backward-compatible
- Increment the **major** version (`1.2.3` → `2.0.0`) for breaking changes

Update the version in `DESCRIPTION` before each release.

16.10.2 Pre-release Checklist

Before submitting to CRAN, work through the following steps:

1. **Update `NEWS.md`**: Document all changes since the last release in `NEWS.md`. Use `usethis::use_news_md()` to create this file if it doesn't exist. Each release should have a top-level heading (e.g., `# mypackage 1.2.0`).

[5] https://docs.github.com/en/repositories/releasing-projects-on-github/managing-releases-in-a-repository
[6] https://cran.r-project.org/
[7] https://r-pkgs.org/release.html
[8] https://github.com/edgararuiz-zz/cran-stages

2. **Run `devtools::check()`**: Ensure your package passes `R CMD CHECK` with no errors, warnings, or notes. Use `cran = TRUE` to enable the stricter checks that more closely mirror what CRAN runs. Address any issues before submitting.

```
devtools::check(cran = TRUE)
```

3. **Check spelling**: Run `devtools::spell_check()` to catch typos in documentation.

```
devtools::spell_check()
```

4. **Check URLs**: Run `urlchecker::url_check()` to catch broken or redirected URLs in your documentation, which CRAN flags.

```
urlchecker::url_check()
```

5. **Test on multiple platforms**: CRAN requires packages to work across operating systems and R versions. Use the following to test on additional platforms:

   - `devtools::check_win_devel()` - tests on Windows with the development version of R
   - R-hub[9] - tests on a wide range of platforms. In rhub v2[10], this runs via GitHub Actions: run `rhub::rhub_setup()` once to add the workflow to your repository, then `rhub::rhub_check()` to trigger a check run. (If you previously used older rhub APIs, see the rhub v2 documentation for migration details.)

6. **Review CRAN policies**: Ensure your package complies with the CRAN Repository Policy[11].

16.10.3 Submitting to CRAN

Once all checks pass, submit with:

```
devtools::submit_cran()
```

This bundles your package and uploads it to CRAN's submission portal. You will receive an email asking you to confirm the submission. Review time varies; watch for email from CRAN maintainers with acceptance or requested changes.

On the CRAN web submission form that `devtools::submit_cran()` opens, briefly describe any `R CMD CHECK` notes in the submission comments and explain why they are acceptable. Be concise and factual.

For first-time submissions, also check the new submission checklist[12] in the R Packages book for additional requirements.

[9] https://github.com/r-hub/rhub
[10] https://r-hub.github.io/rhub/articles/rhubv2.html
[11] https://cran.r-project.org/web/packages/policies.html
[12] https://r-pkgs.org/release.html#sec-release-initial

16.10.4 After CRAN Acceptance

Once CRAN accepts your package:

1. **Tag the release and create a GitHub release**: Follow the steps in Section 16.9, using a tag named `v` followed by the version number (e.g., `v1.2.0`). Copy the relevant section of `NEWS.md` as the release notes. Tags permanently mark the exact commit that was released, making it easy to return to or compare against any released version.

2. **Bump the development version**: After releasing, append the `.9000` development suffix to the current version in `DESCRIPTION`:

   ```
   usethis::use_version("dev")
   ```

   For example, after releasing `1.2.0`, the `DESCRIPTION` version becomes `1.2.0.9000`. This signals to contributors that the repository reflects development work toward the next release.

3. **Commit the version bump**: Stage and commit the updated `DESCRIPTION` (and `NEWS.md` if you added a `# mypackage (development version)` header):

   ```
   git add DESCRIPTION NEWS.md
   git commit -m "Start development toward next release (1.2.0.9000)"
   git push
   ```

4. **Track adoption over time**: the `{Visualize.CRAN.Downloads}`[13] package plots a CRAN package's download counts (retrieved via `{cranlogs}`[14]), producing static, interactive, and multi-package comparison plots, with moving averages and confidence bands to make trends visible. With no date range given, it defaults to the year of downloads ending today:

   ```
   Visualize.CRAN.Downloads::processPckg("yourpackage")
   ```

[13] https://github.com/mponce0/Visualize.CRAN.Downloads
[14] https://r-hub.github.io/cranlogs/

17 Data Publication

17.1 Overview

Warning! ***NEVER*** *push a dataset into the public domain (e.g., GitHub, OSF) without first checking with lab leadership to ensure that it is appropriately de-identified and we have approval from the sponsor and/or human subjects review board to do so. For example, we will need to re-code participant IDs (even if they contain no identifying information) before making data public to completely break the link between IDs and identifiable information stored on our servers.*

If you are releasing data into the public domain, then consider making available *at minimum* a `.csv` file and a codebook of the same name (note: you should have a codebook for internal data as well). We often also make available `.rds` files as well. For example, your `mystudy/data/public` directory could include three files for a single dataset, two with the actual data in `.rds` and `.csv` formats, and a third that describes their contents:

```
analysis_data_public.csv
analysis_data_public.rds
analysis_data_public_codebook.txt
```

In general, datasets are usually too big to save on GitHub, but occasionally they are small. Here is an example of where we actually pushed the data directly to GitHub: https://github.com/ben-arnold/enterics-seroepi/tree/master/data .

If the data are bigger, then maintaining them under version control in your git repository can be unwieldy. Instead, we recommend using another stable repository that has version control, such as the Open Science Framework ("Open Science Framework," n.d.). For example, all of the data from the WASH Benefits trials (led by investigators at Berkeley, icddr,b, IPA-Kenya and others) are all stored through data components nested within in OSF projects: https://osf.io/tprw2/. Another good option is Dryad Digital Repository ("Dryad Digital Repository," n.d.) or institutional digital repositories.

We recommend cross-linking public files in GitHub (scripts/notebooks only) and OSF/Dryad/institutional digital repositories.

Below are the main steps to making data public, after finalizing the analysis datasets and scripts:
1. Remove Protected Health Information (PHI)
2. Create public IDs or join already created public IDs to the data

3. Create an OSF repository and/or Dryad/institutional digital repository
4. Edit analysis scripts to run using the public datasets and test (optional)
5. Create a public github page for analysis scripts and link to OSF and/or Dryad/Zenodo
6. Go live

17.2 Removing PHI

Once the data is finalized for analysis, the first step is to strip it of Protected Health Information (PHI), or any other data that could be used to link back to specific participants, such as names, birth dates, or GPS coordinates at the village/neighborhood level or below. PHI includes, but is not limited to:

17.2.1 Personal information

These are identifiers that directly point to specific individuals, such as:
- Names, addresses, photographs, date of birth
- A combination of age, sex, and geographic location (below population 20,000) is considered identifiable

17.2.2 Dates

Any specific dates (e.g., study visit dates, birth dates, treatment dates) are usually problematic.
- If a dataset requires high resolution temporal information, coarsen visit or measurement dates to be two variables: year and week of the year (1-52).
- If a dataset requires age, provide that information without a birth date (typically month resolution is sufficient)

Caution! *If making changes to the format of dates or ages, make sure your analysis code runs on these modified versions of the data (step 3)!*

17.2.3 Geographic information

Do not include GPS coordinates (longitude, latitude) except in special circumstances where they have been obfuscated/shifted. Reach out to lab leadership before doing this because it can be complicated.

Do not include place names or codes (e.g., US Zip Codes) if the place contains <20,000 people. For villages or neighborhoods, code them with uninformative IDs. For sub-districts or districts, names are fine.

If an analysis requires GPS locations (e.g., to make a map), then typically we include a disclaimer in the article's data availability statement that explains we cannot make GPS locations public to protect participant confidentiality. As a middle ground, we typically

make our *code* public that runs on the geo-located data for transparency, even if independent researchers can't actually run that code (although please be careful to ensure the code itself does not in any way include geographic identifiers).

For more examples of what constitutes PHI, please refer to this link: https://cphs.berkeley.edu/hipaa/hipaa18.html

For learning about working with geospatial data, see the UC Davis DataLab GIS workshops[1], including resources on QGIS and spatial SQL.

17.3 Create public IDs

17.3.1 Rationale

The UC Davis IRB requires that public datasets not include the original study IDs to identify participants or other units in the study. The reason is that those IDs are linked in our private datasets to PHI. By creating a new set of public IDs, the public dataset is one step further removed from the potential to link to PHI.

17.3.2 A single set of public IDs for each study

For each study, it is ideal to create a single set of public IDs whenever possible. We could create a new set of public IDs for every public dataset, but the downside is that independent researchers could no longer link data that might be related. By creating a single set of public IDs associated with each internal study ID, public files retain the link.

Maintaining a single set of public IDs requires a shared "bridge" dataset, that includes a row for each study ID and has the associated public ID. For studies with multiple levels of ID, we would typically have separate bridge datasets for each type of ID (e.g,. cluster ID, participant ID, etc.)

Create a public ID that can be used to uniquely identify participants and that can internally be linked to the original study IDs. We recommend creating a subdirectory in the study's shared data directory to store the public IDs. The shared location enables multiple projects to use the same IDs. Create the IDs using a script that reads in the study IDs, creates a unique (uninformative) public ID for the study IDs, and then saves the bridge dataset. The script should be saved in the same directory as the public ID files.

Caution! *Note that small differences may arise if the new public IDs do not necessarily order participants in the same way as the internal IDs. The small differences are all in estimates that rely on resampling, such as Bootstrap CIs, permutation P-values, and TMLE, as the resampling process may lead tp slightly different re-samples. The key here, to ensure the results are consistent irrespective of the dataset used, is simply to not assign public IDs randomly. Use* `rank()` *on the internal ID instead of* `row_number()` *to ensure that the order is always the same.*

---

[1] https://github.com/ucdavisdatalab

17.3.3 Example scripts

Please see the reproducible example provided by the Proctor Lab at UCSF[2] that you can run and replicate when making data public for your projects. The example contains the following files and folders:

1. `data/final/`- folder containing the projects final data in both csv and rds formats

2. `code/DEMO_generate_public_IDs.R`- creates randomly generated public IDs that can be matched to the trial's assigned patient IDs.

3. `data/make_public/DEMO_internal_to_publicID.csv`- the output from step #2, a bridge dataset with two variables- the new public ID and the patient's assigned ID.

4. `code/DEMO_create_public_datasets.R`- joins the public IDs to the trial's full dataset, and strips it of the assigned patient ID.
5. `data/public/`- folder containing the output from step #3- de-identified public dataset, in csv and rds formats, with uniquely identifying public IDs that cannot be easily linked back to the patient's ID.

The example workflow is accessible via GitHub: https://github.com/proctor-ucsf/dcc-handbook/tree/master/templates/making-data-public

17.4 Create a data repository

First, ensure that you create a codebook and metadata file for each public dataset (see the DCC guide on Documenting datasets[3]). Use the same name as the datasets, but with `-codebook.txt`, `-codebook.html`, or `-codebook.csv` at the end (depending on the file format for the codebook). One option is the R `codebook` package, which also generates JSON output that is machine-readable.

For modern, standardized data dictionaries, consider the Tidyverse data-dict[4] project. The `data-dict.yaml` specification defines an open, lightweight, plain-text YAML format for describing tabular datasets—including table schemas, column data types, descriptions, relationships, and validation constraints. Because data dictionaries are defined in plain-text YAML files, they can be version-controlled directly in Git alongside your analysis code. The companion `data-dict` command-line tool can draft dictionaries from existing data files, validate dataset values against your dictionary rules, and render self-contained HTML documentation reports.

For additional guidance on data documentation best practices, see the UC Davis DataLab workshop on data documentation[5].

[2] https://proctor-ucsf.github.io/dcc-handbook/publicdata.html
[3] https://proctor-ucsf.github.io/dcc-handbook/datawrangling.html#documenting-datasets
[4] https://data-dict.tidyverse.org/
[5] https://github.com/ucdavisdatalab/workshop_how-to-data-documentation

17.4.1 Steps for creating an Open Science Framework (OSF) repository:

1. Create a new OSF project per these instructions: https://help.osf.io/article/252-create-a-project
2. Create a data component and upload the datasets in .csv and .rds format along with the codebooks. The primary format for public dissemination is .csv but we make the .rds files available too as auxiliary files for convenience.
3. Create a notebook component and upload the final .html files (which will not be on github… but see optional item below)
4. On the OSF landing Wiki, provide some context. Here is a recent example: https://osf.io/954bt/
5. Create a Digital Object Identifier (DOI) for the repository. A DOI is a unique identifier that provides a persistent link to content, such as a dataset in this case. Learn more about DOIs[6]
6. Optional: Complete the software checklist and system requirement guide for the analysis to guide others. Include it on the GitHub README for the project: https://github.com/proctor-ucsf/mordor-antibody

17.5 Edit and test analysis scripts

Make minor changes to the analysis scripts so that they run on public data. If using version control in GitHub, the most straight-forward way is to create a branch from the main git branch that reads in the public files, and then renames the new public ID variable, e.g., "id_public" to the internally recognized ID variable name, e.g. "recordID", when reading in the public data. Re-run all the analysis scripts to ensure that they still work with the public version of the dataset.

17.6 Create a public GitHub page for public scripts

At minimum, we should include all of the scripts required to run the analyses. **IMPORTANT**: ensure you have taken a snapshot and saved your computing environment using the `renv` package (`renv`).

See examples:
- ACTION - https://github.com/proctor-ucsf/ACTION-public
- NAITRE - https://github.com/proctor-ucsf/NAITRE-primary

> ***Caution!*** *Read through the scripts carefully to ensure there is no PHI in the code itself*

Once a public GitHub page exists, you can create a new component on an OSF project (step 3, above) and link it to the public version of the GitHub repo.

[6] https://researchdata.princeton.edu/research-lifecycle-guide/publishing-and-preservation/dois

17.7 Go live

On GitHub, it is useful to create an official "release" version to freeze the repository, where you can have "associated files" with each version. Include the .html notebook output as additional files — since they aren't tracked in GitHub, it does provide a way of freezing / saving the HTML output for us and others. OSF examples of studies from UCSF's Proctor Foundation:

- ACTION - https://osf.io/ca3pe/
- NAITRE - https://osf.io/ujeyb/
- MORDOR Niger antibody study - https://osf.io/dgsq3/

Further reading on end-to-end data management: How to Store and Manage Your Data - PLOS ("How to Store and Manage Your Data," n.d.)

18 High-performance computing (HPC)

When you need to run a script that requires a large amount of RAM, large files, or that uses parallelization, UC Davis provides several high-performance computing (HPC) resources.

18.1 UC Davis Computing Resources

18.1.1 Available Resources

UC Davis HPC Clusters: - **Farm Cluster** (hpc.ucdavis.edu[1]): UC Davis's primary HPC cluster providing shared computing resources for research

PHS Shared Compute Environments: For lab members affiliated with the School of Public Health Sciences (PHS), additional shared computing environments are available. These environments provide secure, HIPAA-compliant computing resources suitable for working with sensitive health data.

- **Shiva** (shiva.ucdavis.edu): SLURM-based cluster for computational work
- **Mercury** (mercury.ucdavis.edu): RStudio GUI computing environment

For detailed information about PHS shared compute environments, including access procedures, security guidelines, and usage policies, please refer to the PHS Shared Compute Environments Guide[2].

Contact lab leadership for assistance with: - Requesting access to computing resources - Choosing the appropriate computing environment for your project - Setting up your computing environment

18.2 Getting started with SLURM clusters

To access a UC Davis HPC cluster, in terminal, log in using SSH. For example, to access shiva:

```
ssh USERNAME@shiva.ucdavis.edu
```

You will be prompted to enter your UC Davis credentials and may need to complete two-factor authentication.

Once you log in, you can view the contents of your home directory in command line by entering `cd $HOME`. You can create subfolders within this directory using the `mkdir` command. For example, you could make a "code" subdirectory and clone a Github repository there using the following code:

[1] https://hpc.ucdavis.edu/
[2] assets/files/PHS_Shared_Compute_Environments.pdf

```
cd $HOME
mkdir code
git clone https://github.com/jadebc/covid19-infections.git
```

18.2.1 One-Time System Set-Up

To keep the install packages consistent across different nodes, you will need to explicitly set the pathway to your R library directory.

Open your `~/.Renviron` file (`vi ~/.Renviron`) and append the following line:

Note: Once you open the file using `vi [file_name]`, you must press `i` (on Mac OS) or `Insert` (on Windows) to make edits. After you finish, hit `Esc` to exit editing mode and type `:wq` to save and close the file.

```
R_LIBS=~/R/x86_64-pc-linux-gnu-library/4.0.2
```

Alternatively, run an R script with the following code on the cluster:

```
r_environ_file_path = file.path(Sys.getenv("HOME"), ".Renviron")
if (!file.exists(r_environ_file_path)) file.create(r_environ_file_path)

cat("\nR_LIBS=~/R/x86_64-pc-linux-gnu-library/4.0.2",
    file = r_environ_file_path, sep = "\n", append = TRUE)
```

To load packages that run off of C++, you'll need to set the correct compiler options in your R environment.

Open the Makevars file (`vi ~/.R/Makevars`) and append the following lines

```
CXX14FLAGS=-O3 -march=native -mtune=native -fPIC
CXX14=g++
```

Alternatively, create an R script with the following code, and run it on the cluster:

```
dotR = file.path(Sys.getenv("HOME"), ".R")
if (!file.exists(dotR)) dir.create(dotR)

M = file.path(dotR, "Makevars")
if (!file.exists(M)) file.create(M)

cat("\nCXX14FLAGS=-O3 -march=native -mtune=native -fPIC",
    "CXX14=g++",
    file = M, sep = "\n", append = TRUE)
```

18.3 Moving files to the cluster

The `$HOME` directory is a good place to store code and small test files. Save large files to the `$SCRATCH` directory or other designated storage areas. Check with the UC Davis HPC documentation[3] for specific quotas and retention policies. It's best to create a bash script that records the file transfer process for a given project. See example code below:

```
# note: the following steps should be done from your local
# (not after ssh-ing into the cluster)

# securely transfer folders from Box to cluster home directory
# note: the -r option is for folders and is not needed for files
scp -r "Box/project-folder/folder-1/" USERNAME@shiva.ucdavis.edu:/home/users/USERNAME/

# securely transfer folders from Box to your cluster scratch directory
scp -r "Box/project-folder/folder-2/" USERNAME@shiva.ucdavis.edu:/scratch/users/USERNAME/

# securely transfer folders from Box to shared scratch directory
scp -r "Box/project-folder/folder-3/" USERNAME@shiva.ucdavis.edu:/scratch/group/GROUPNAME/
```

18.4 Installing packages on the cluster

When you begin working on a cluster, you will most likely encounter problems with installing packages. To install packages, login to the cluster on the command line and open a development node. Do not attempt to do this in RStudio Server, as you will have to re-do it for every new session you open.

```
ssh USERNAME@shiva.ucdavis.edu

sdev
```

You should only have to install packages once. The cluster may require that you specify the repository where the package is downloaded from. You may also need to add an additional argument to install.packages to prevent the packages from locking after installation:

```
install.packages(<PACKAGE NAME>, repos="https://cran.r-project.org",
                 INSTALL_opts = "--no-lock")
```

In order for some R packages to work on clusters, it is necessary to load specific software modules before running R. These must be loaded each time you want to use the package in R. For example, for spatial and random effects analyses, you may need the modules/packages below. These modules must also be loaded on the command line prior to opening R in order for package installation to work.

[3] https://hpc.ucdavis.edu/

```
module --force purge # remove any previously loaded modules, including math and devel
module load math
module load math gmp/6.1.2
module load devel
module load gcc/10
module load system
module load json-glib/1.4.4
module load curl/7.81.0
module load physics
module load physics udunits geos
module load physics gdal/2.2.1 # for R/4.0.2
module load physics proj/4.9.3 # for R/4.0.2
module load pandoc/2.7.3

module load R/4.0.2

R # Open R in the Shell window to install individual packages or test code
Rscript install-packages.R # Alternatively, run a package installation script in the Shell
```

Figuring out the issues with some packages will require some trial and error. If you are still encountering problems installing a package, you may have to install other dependencies manually by reading through the error messages. If you try to install a dependency from CRAN and it isn't working, it may be a module. You can search for it using the `module spider` command:

```
module spider DEPENDENCY NAME
```

You can also reach out to UC Davis HPC support for help. Visit hpc.ucdavis.edu[4] for support information.

18.5 Testing your code

Both of the following ways to test code on a cluster are recommended for making small changes, such as editing file paths and making sure the packages and source files load. You should write and test the functionality of your script locally, only testing on the cluster once major bugs are out.

18.5.1 The command line

There are two main ways to explore and test code on computing clusters. The first way is best for users who are comfortable working on the command line and editing code in base R. Even if you are not comfortable yet, this is probably the better way because these commands will transfer between different cluster computers using Slurm.

Typically, you will want to initially test your scripts by initiating a development node using the command `sdev`. This will allocate a small amount of computing resources for 1 hour. You can access R via command line using the following code.

[4] https://hpc.ucdavis.edu/

```
# open development node
sdev

# Load all the modules required by the packages you are using
module load MODULE NAME

# Load R (default version)*
module load R

# initiate R in command line
R
```

*Note: for collaboration purposes, it's best for everyone to work with one version of R. Check what version is being used for the project you are working on. Some packages only work with some versions of R, so it's best to keep it consistent.

18.5.2 RStudio Server

For RStudio GUI computing, UC Davis provides mercury.ucdavis.edu. This is accessed through a web browser and provides an RStudio interface. You will be prompted to authenticate with your UC Davis credentials. This is the best way to work with R for people who are not comfortable accessing & editing in base R in a Shell application.

Note that mercury does not have SLURM, so it's best suited for interactive work and smaller computations. For large-scale computations requiring SLURM job scheduling, use shiva.ucdavis.edu instead.

When using RStudio Server, you can test your code interactively. However, do NOT use the RStudio Server's Terminal to install packages and configure your environment for SLURM-based clusters, as you will likely need to re-do it for every session/project. For SLURM clusters, use the command line approach described earlier.

18.5.3 Filepaths & configuration on the cluster

In most cases, you will want to test that the file paths work correctly on the cluster. You will likely need to add code to the configuration file in the project repository that specifies cluster-specific file paths. Here is an example:

```
# set cluster-specific file paths
if(Sys.getenv("LMOD_SYSHOST")!=""){

  cluster_path = paste0(Sys.getenv("HOME"), "/project-name/")

  data_path = paste0(cluster_path, "data/")
  results_path = paste0(cluster_path, "results/")
}
```

18.6 Running graphical programs with X11 forwarding

Shiva's SSH server permits X11 forwarding (checked August 2026), so a graphical program started on the cluster can draw its window on your own computer. This is worth setting up when you want a GUI text editor on the cluster, or a web browser that sees the network from Shiva's position rather than from yours.

X11 forwarding sends every window update across the network, so it suits a text editor much better than it suits a browser. When all you want is to view a web page that Shiva itself is serving, such as an RStudio Server[5] session or a rendered report, forward the port instead and use the browser already on your own computer:

```
# from your own computer, not from Shiva
ssh -L 8787:localhost:8787 USERNAME@shiva.ucdavis.edu
```

With that running, `http://localhost:8787` in your local browser reaches the service on Shiva. This is far faster than forwarding a browser window, because only the page data crosses the network rather than every repainted pixel.

18.6.1 Install an X server on your own computer

The cluster draws into an X server, and that server runs on your machine rather than on Shiva. macOS and Windows do not ship one, so install the software listed in Table 18.1 before your first attempt.

Table 18.1: Where to get an X server, by operating system

Your computer	What to install
macOS	XQuartz[6]. Log out and back in after installing, so that `DISPLAY` is set.
Windows	VcXsrv[7], or MobaXterm[8], which bundles an X server together with an SSH client.
Windows, using WSL2	Nothing. WSLg[9] supplies the X server.
Linux	Usually nothing. An X11 session is itself an X server; a Wayland session relies on Xwayland[10] for X11 compatibility. Confirm `DISPLAY` is set locally before you connect.

18.6.2 Connect with X11 forwarding enabled

Ask for trusted X11 forwarding with `-Y`:

[5] https://posit.co/products/open-source/rstudio-server/
[6] https://www.xquartz.org/
[7] https://sourceforge.net/projects/vcxsrv/
[8] https://mobaxterm.mobatek.net/
[9] https://learn.microsoft.com/en-us/windows/wsl/tutorials/gui-apps
[10] https://wiki.archlinux.org/title/Wayland

```
ssh -Y USERNAME@shiva.ucdavis.edu
```

Use `-Y` rather than `-X` here. `-X` requests *untrusted* forwarding, and by default OpenSSH refuses untrusted X11 connections made more than twenty minutes into the session ("Ssh_config(5) Manual Page," n.d.). Any window already on screen keeps working, so the symptom is that a program you launch later in the session simply never appears. Untrusted forwarding also restricts clients under the X11 SECURITY extension, to stop them reading or tampering with data belonging to trusted clients ("Ssh_config(5) Manual Page," n.d.), and some graphical toolkits fail outright under those restrictions.

The exception is a Linux client running Debian or Ubuntu, where `ForwardX11Trusted` defaults to `yes` and `-X` therefore already means the same thing as `-Y` ("Ssh_config(5) Manual Page, Debian Edition," n.d.). Passing `-Y` is correct on every platform, so prefer it and do not worry about which case you are in.

Trusted forwarding is a real tradeoff rather than a free upgrade. It gives programs running on Shiva full access to your local display, which includes reading input directed at your other windows ("Ssh_config(5) Manual Page," n.d.). That is an acceptable risk for a cluster our lab administers and trusts; do not extend the same setting to machines you do not trust.

To avoid retyping the flag, add an entry to `~/.ssh/config` on your own computer:

```
Host shiva
  HostName shiva.ucdavis.edu
  User USERNAME
  ForwardX11 yes
  ForwardX11Trusted yes
  Compression yes
```

`ssh shiva` then forwards X11 automatically. `Compression yes` is worth setting because X11 traffic compresses well over a slow link.

18.6.3 Check that the forward is working

After logging in, `DISPLAY` should be set:

```
echo $DISPLAY
# localhost:10.0
```

An empty result means the forward did not happen. The usual causes are omitting `-Y`, or not having an X server running on your own computer.

To confirm that the connection reaches your display, ask the display to describe itself:

```
xdpyinfo | head -3
```

Use `xdpyinfo` rather than `xeyes` for this test. Shiva has the `x11-utils` package, which provides `xdpyinfo`, but not `x11-apps`, which provides `xeyes` and `xclock` (checked August 2026).

18.6.4 Install graphical programs without `sudo`

Lab accounts on Shiva are not in the `sudo` group, which held only a handful of administrator accounts when this was checked in August 2026. Run `groups` to see which groups your own account belongs to. This means `apt install` is unavailable, and most installation instructions written for Ubuntu will not apply. Install into your home directory instead, which needs no special privileges.

Note that this is necessary even for programs that appear to be present already. The system Emacs at `/usr/bin/emacs` is the `emacs-nox` build, which links no X libraries at all and so cannot open a window under any circumstances. No web browser is installed on Shiva. `module avail` on the login node lists only MPI and numerical libraries, so it supplies neither one.

For Emacs, use conda-forge[11], whose build includes GTK and X11 support. If you do not already have conda, install Miniforge[12] into your home directory first.

```
conda create -y -n gui -c conda-forge emacs
conda activate gui
emacs &
```

For Firefox, use Mozilla's official Linux build, which is a self-contained directory that runs from anywhere:

```
mkdir -p ~/opt && cd ~/opt
curl -L -o firefox.tar.xz \
  'https://download.mozilla.org/?product=firefox-latest-ssl&os=linux64&lang=en-US'
tar xf firefox.tar.xz && rm firefox.tar.xz
~/opt/firefox/firefox &
```

Adding `~/bin` to your `PATH` and putting small wrapper scripts there saves typing the full path each time.

18.6.5 Limitations

R cannot open plot windows on Shiva. The installed R is built without X11 support, which you can confirm with `capabilities()["X11"]`. No amount of forwarding configuration changes this, because the R binary itself has no X11 device compiled in. Write plots to a file with `{ragg}`[13] or `ggplot2::ggsave()` and copy them back with `scp`, or forward a port and use a browser-based graphics device such as `{httpgd}`[14].

The faster remote-desktop tools are unavailable. Xpra[15], VNC, and X2Go[16] all compress far better than plain X11 forwarding over a slow link. All of them need an X server installed on the cluster itself, which requires root, so none can be set up from a lab account. Contact the cluster administrators if the performance of plain forwarding is blocking your work.

[11] https://conda-forge.org/
[12] https://github.com/conda-forge/miniforge
[13] https://ragg.r-lib.org/
[14] https://cran.r-project.org/package=httpgd
[15] https://github.com/Xpra-org/xpra
[16] https://wiki.x2go.org/

18.7 Recovering a disconnected interactive session

Interactive sessions on a SLURM cluster — a development node opened with `sdev`, or a session started with `srun --pty` ("Slurm Workload Manager," n.d.-e) or `salloc` ("Slurm Workload Manager," n.d.-a) — are tied to the terminal that launched them. If your SSH connection drops (a broken pipe, a network blip, or closing your laptop), SLURM usually sends the launching process a hang-up signal and **tears down the whole job allocation**, not just your shell. When that happens there is nothing left to reconnect to, and any unsaved work in that session is lost.

There is no way to reattach to the *identical* shell once the allocation has been released. The reliable strategy is to prevent the drop in the first place; if the allocation does happen to survive, you can open a *new* shell inside it.

18.7.1 Prevent drops with a terminal multiplexer (recommended)

Start a terminal multiplexer such as `tmux`[17] or `screen`[18] **on the login node**, then launch your interactive session from inside it:

```
ssh USERNAME@shiva.ucdavis.edu

# start a tmux session on the login node
tmux

# request your interactive allocation from inside tmux
sdev          # or: srun --pty bash
```

Because `tmux` keeps running on the login node even after your SSH connection drops, the launching process survives and the allocation stays alive. When you reconnect, reattach to your multiplexer and you are back where you left off:

```
ssh USERNAME@shiva.ucdavis.edu
tmux ls         # list existing sessions
tmux attach     # reattach to the most recent session
```

18.7.2 Reconnect to a surviving allocation

If you did not use a multiplexer, you can still open a new shell on the nodes of a job that is **already running** — most commonly a batch job submitted with `sbatch`, or an interactive allocation that happened to survive. This gives you a *new* shell (a new job step), not the original session.

First, find the job ID ("Slurm Workload Manager," n.d.-d):

```
squeue --me        # or: squeue -u $USER
```

[17] https://github.com/tmux/tmux/wiki
[18] https://www.gnu.org/software/screen/

Then launch a new interactive step inside that allocation, replacing `JOBID` with the ID reported by `squeue` ("Slurm Workload Manager," n.d.-e):

```
srun --jobid=JOBID --overlap --pty bash
```

> **!** The `--overlap` flag is required on recent SLURM
>
> Since SLURM 20.11, a new job step requests exclusive use of the job's resources by default, so without `--overlap` the `srun` above will simply hang waiting for resources the running step already holds. Always include `--overlap` when attaching to a running job.

If you only need to reattach to the input and output streams of an existing step (rather than start a fresh shell), `sattach JOBID.STEPID` does that instead ("Slurm Workload Manager," n.d.-b).

> **i** Note
>
> Exact behavior varies by cluster: some sites disable `srun` into running jobs, wrap it in a local helper, or require different flags. When in doubt, check the UC Davis HPC documentation[a] or contact HPC support.
>
> [a] https://hpc.ucdavis.edu/

18.8 Storage & group storage access

18.8.1 Individual storage

There are multiple places to store your files on computing clusters. Each user has their own `$HOME` directory as well as a `$SCRATCH` directory. These are directories that can be accessed via the command line once you've logged in to the cluster:

```
cd $HOME
cd /home/users/USERNAME # Alternatively, use the full path

cd $SCRATCH
cd /scratch/users/USERNAME # Full path
```

You can also navigate to these using the File Explorer if available through a web interface.

`$HOME` typically has a volume quota (e.g., 15 GB). `$SCRATCH` typically has a larger volume quota (e.g., 100 TB), but files here may get deleted after a certain period of inactivity. Thus, use `$SCRATCH` for test files, exploratory analyses, and temporary storage. Use `$HOME` for long-term storage of important files and more finalized analyses.

Check with the UC Davis HPC documentation[19] for specific storage options and quotas.

[19] https://hpc.ucdavis.edu/

18.8.2 Group storage

The lab may have shared `$GROUP_HOME` and `$GROUP_SCRATCH` directories to store files for collaborative use. These typically have larger quotas and may have different retention policies. You can access these via the command line or navigate to them using the File Explorer:

```
cd $GROUP_HOME
cd /home/groups/GROUPNAME

cd $GROUP_SCRATCH
cd /scratch/groups/GROUPNAME
```

However, saving files to group storage can be tricky. You can try using the scp command in the section "Moving files to the cluster" to see if you have permission to add files to group directories. Read the next section to ensure any directories you create have the right permissions.

18.8.3 Folder permissions

Generally, when we put folders in `$GROUP_HOME` or `$GROUP_SCRATCH`, it is so that we can collaborate on an analysis within the research group, so multiple people need to be able to access the folders. If you create a new folder in `$GROUP_HOME` or `$GROUP_SCRATCH`, please check the folder's permissions to ensure that other group members are able to access its contents. For example, you can inspect a folder's permissions by running `ls -l` from the level above it. For a quick refresher on interpreting the permission bits and updating them with `chmod`, see this guide[20].

18.9 Running big jobs

Once your test scripts run successfully, you can submit an sbatch script for larger jobs. These are text files with a `.sh` suffix. Use a text editor like Sublime to create such a script. Documentation on sbatch options is available from Slurm Workload Manager ("Slurm Workload Manager," n.d.-c). Here is an example of an sbatch script with the following options:

- `job-name=run_inc`: Job name that will show up in the SLURM system
- `begin=now`: Requests to start the job as soon as the requested resources are available
- `dependency=singleton`: Jobs can begin after all previously launched jobs with the same name and user have ended.
- `mail-type=ALL`: Receive all types of email notification (e.g., when job starts, fails, ends)
- `cpus-per-task=16`: Request 16 processors per task. The default is one processor per task.
- `mem=64G`: Request 64 GB memory per node.
- `output=00-run_inc_log.out`: Create a log file called `00-run_inc_log.out` that contains information about the Slurm session

[20] https://www.chriswrites.com/how-to-change-file-permissions-using-the-terminal/

- `time=47:59:00`: Set maximum run time to 47 hours and 59 minutes. If you don't include this option, the cluster will automatically exit scripts after 2 hours of run time (default may vary by cluster).

The file `analysis.out` will contain the log file for the R script `analysis.R`.

```
#!/bin/bash

#SBATCH --job-name=run_inc
#SBATCH --begin=now
#SBATCH --dependency=singleton
#SBATCH --mail-type=ALL
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --mem=64G
#SBATCH --output=00-run_inc_log.out
#SBATCH --time=47:59:00

cd $HOME/project-code-repo/2-analysis/

module purge

# load R version 4.0.2 (required for certain packages)
module load R/4.0.2

# load gcc, a C++ compiler (required for certain packages)
module load gcc/10

# load software required for spatial analyses in R
module load physics gdal
module load physics proj

R CMD BATCH --no-save analysis.R analysis.out
```

To submit this job, save the code in the chunk above in a script called `myjob.sh` and then enter the following command into terminal:

```
sbatch myjob.sh
```

To check on the status of your job, enter the following code into terminal:

```
squeue -u $USERNAME
```

19 Working with AI

Our notes on working responsibly and effectively with AI coding assistants have moved to a dedicated site: **Working with AI**[1] (source: https://github.com/Morrison-Lab/wai).

That site covers:

- **Policies for Using AI**[2]: responsibility for validation, disclosure, attribution, and using AI for journal articles
- **Coding Agents**[3]: what language models, coding agents, and harnesses are; how to work with them; benefits, hazards, and best practices; and configuring your environment
- **Pull-Request Workflow with Agents**[4]: filing issues, claiming work, and driving a pull request to a clean, mergeable state

Lab members who use AI tools must still adhere to the guidelines described there.

> **Looking for a specific section that used to be here?**
>
> This chapter previously had section anchors like `#sec-ai-best-practices` or `#sec-ai-disclosure` for each subtopic. Those anchors still resolve on this page (so old bookmarks and links won't 404), but the content itself now lives on the linked wai pages above — follow the matching bullet to find it.

19.1 Institutional AI account access

19.1.1 UC Davis Health OpenAI access

For lab members affiliated with UC Davis Health (UCDH) or conducting health sciences research, enterprise OpenAI accounts are provisioned through the UCDH IT Self-Service (ITSS) ServiceNow catalog.

To request an account:

1. Navigate to the UCDH ServiceNow OpenAI catalog item[5].
2. Log in with your UC Davis Health credentials and complete Duo two-factor authentication if prompted.
3. Fill out the request form with your department, contact details, and business or research justification.

[1] https://morrison-lab.github.io/wai/
[2] https://morrison-lab.github.io/wai/chapters/ai-use-policies.html
[3] https://morrison-lab.github.io/wai/chapters/coding-agents.html
[4] https://morrison-lab.github.io/wai/chapters/pr-workflow-with-agents.html
[5] https://ucdh.service-now.com/itss?id=sc_cat_item&sys_id=738dd2ac93eba650c4dbb1847aba100d

4. Submit the request for processing by UCDH IT.
5. Once approved, follow the instructions in the confirmation email to join the UC Davis Health OpenAI workspace.

19.1.2 UC Davis campus OpenAI access

For UC Davis campus affiliates, access to ChatGPT Edu is managed through Information and Educational Technology (IET) Aggie AI. See the IET Aggie AI ChatGPT Edu documentation[6] for setup instructions and eligibility details.

19.1.3 Data security and compliance

When using institutional AI accounts:

- Never submit Protected Health Information (PHI), personally identifiable information (PII), or restricted research data to AI tools unless authorized under an approved protocol.
- Follow all institutional data handling policies and the lab's guidelines on Working with AI[7].

[6] https://iet.ucdavis.edu/aggie-ai/ai-tools/chatgpt-edu
[7] https://morrison-lab.github.io/wai/chapters/ai-use-policies.html

20 Checklists

20.1 Onboarding checklist

Welcome to the lab! New members should complete the following checklist during their first week:

- ☐ **Account setup**: Request and confirm access to GitHub (join the `UCD-SERG` organization), Microsoft Teams, UC Davis email, and lab Box and SharePoint folders.
- ☐ **Required trainings**: Complete required CITI Human Subjects Biomedical Group 1 training and share your certificate with lab leadership (see Section 2.3).
- ☐ **Read the lab manual**: Review lab culture and conduct (Chapter 2), coding style (Chapter 7), and collaborative development (Chapter 12).
- ☐ **Environment setup**: Configure your local R development environment, clone a starter repository, and run `renv::restore()` to verify reproducible package installation (see Chapter 15).
- ☐ **HPC access**: Confirm High-Performance Computing cluster access (Farm or Shiva) and verify SSH keys if your project involves cluster computing (see Chapter 18).
- ☐ **Team introductions**: Introduce yourself on Microsoft Teams and at the next weekly lab meeting.
- ☐ **Initial 1:1 meeting**: Schedule a one-on-one meeting with lab leadership to align on project objectives, onboarding timeline, and initial milestones.

20.2 Pre-analysis plan checklist

- ☐ Brief background on the study (a condensed version of the introduction section of the paper)
- ☐ Hypotheses / objectives
- ☐ Study design
- ☐ Description of data
- ☐ Definition of outcomes
- ☐ Definition of interventions / exposures
- ☐ Definition of covariates
- ☐ Statistical power calculation
- ☐ Statistical model description
- ☐ Covariate selection / screening
- ☐ Standard error estimation method
- ☐ Missing data analysis
- ☐ Assessment of effect modification / subgroup analyses
- ☐ Sensitivity analyses
- ☐ Negative control analyses

20.3 Code checklist

- ☐ Does the script run without errors?
- ☐ Is code self-contained within repo and/or associated Box folder?
- ☐ Is all commented out code / remarks removed?
- ☐ Does the header accurately describe the process completed in the script?
- ☐ Is the script pushed to its github repository?
- ☐ Does the code adhere to the coding style guide[1]?
- ☐ Are all warnings ignorable? Should any warnings be intentionally suppressed or addressed?

20.4 Manuscript checklist

This is adapted in part from How to tackle the reproducibility crisis in ten steps (Baker 2019).

- ☐ Have you completed the relevant reporting checklist, if applicable? (See EQUATOR Network ("EQUATOR Network," n.d.) for a collection of checklists)
- ☐ Are the study results within the manuscript replicable (i.e., if you rerun the code in the study's repository, the tables and figures will be exactly replicated?)
- ☐ Is a target journal selected?
- ☐ Is the title declarative, in other words, does it state the object/findings rather than suggest them?
- ☐ Is the word count of the manuscript close to the target journal's allowance?
- ☐ Does the manuscript adhere to the formatting guide of the target journal?
- ☐ Does the manuscript use a consistent voice (passive or active – usually active is preferred … pun intended)?
- ☐ Is each figure and table (including supplementary material) referenced in the main text?
- ☐ Is there a caption for each figure and table (including supplementary material)?
- ☐ Are tables/figures and supplementary material numbered in accordance with their appearance in the main text?
- ☐ Does the text use past tense if it is reporting research findings or future tense if it is a study protocol?
- ☐ Does the text avoid subjective wording (e.g., "interesting", "dramatic")?
- ☐ Does the text use minimal abbreviations, and are all abbreviations defined at first use?
- ☐ Does the text avoid directionless words? (e.g., instead of writing, 'Precipitation influences disease risk', write, 'Precipitation was associated with increased disease risk').
- ☐ Does the text avoid making causal claims that are not supported by the study design? Be careful about the words "effect", "increase", and "decrease", which are often interpreted as causal.
- ☐ Does the text avoid describing results with the word "significant", which can easily be confused with statistical significance? (see references on this topic here[2])
- ☐ Have you drafted author contributions? Do they follow the CRediT: Contributor Roles Taxonomy ("CRediT," n.d.) for author contributions?

[1] https://ucd-serg.github.io/lab-manual/coding-style.html
[2] https://journals.lww.com/epidem/Fulltext/2001/05000/The_Value_of_P.2.aspx

20.5 Figure checklist

- ☐ Are the x-axis and y-axis labeled?
- ☐ If the figure includes panels, is each panel labeled?
- ☐ Are there sufficient numerical / text labels and breaks on the x-axis and y-axis?
- ☐ Is the font size appropriate (i.e., large enough to read, not so large that it distracts from the data presented in the figure?)
- ☐ Are the colors used colorblind friendly? See a colorblind-friendly palette here[3], a neat palette generator with colorblind options here[4], and an article on why this matters: The misuse of colour in science communication (Crameri et al. 2020)
- ☐ Are colors/shapes/line types defined in a legend?
- ☐ Are the legends and other labels easy to understand with minimal abbreviations?
- ☐ If there is overplotting, is transparency used to show overlapping data?
- ☐ Are 95% confidence intervals or other measures of precision shown, if applicable?

20.6 Offboarding checklist

When preparing to depart the lab, members should complete this offboarding checklist to ensure project continuity:

- ☐ **Code handoff**: Ensure all analysis code is committed, pushed to GitHub (`UCD-SERG` organization), and documented with a self-contained `README` and reproducible execution instructions (see Chapter 5).
- ☐ **Data archiving**: Archive or transfer large datasets, raw files, and simulation outputs per lab storage protocols (see Chapter 17 and Chapter 18). Ensure all shared Drive and Box datasets have clear data dictionaries and codebooks.
- ☐ **Repository ownership**: Transfer any remaining repository or branch administrative permissions to the `UCD-SERG` organization and lab leadership.
- ☐ **Publication plan**: Coordinate with lab leadership on outstanding manuscripts, co-authorship agreements, remaining revisions, and target submission timelines.
- ☐ **Account transition**: Coordinate with lab leadership to transition or revoke personal access to shared lab accounts, server credentials, and IRB protocols.
- ☐ **Equipment return**: Return any lab-issued equipment (keys, hardware, storage media) and report any known hardware or configuration issues.
- ☐ **Exit interview**: Complete a departure check-in with lab leadership to provide feedback and share forward contact details for future manuscript correspondence.

[3] http://www.cookbook-r.com/Graphs/Colors_(ggplot2)/#a-colorblind-friendly-palette
[4] https://medialab.github.io/iwanthue/?utm_source=Nature+Briefing&utm_campaign=2c68711076-briefing-dy-20211006&utm_medium=email&utm_term=0_c9dfd39373-2c68711076-44335685

21 Resources

21.1 Resources for R

21.1.1 Books and Comprehensive Guides

- R for Data Science (Wickham et al. 2023) - comprehensive introduction to doing data science with R
- R Packages (Wickham and Bryan 2023) - complete guide to R package development
- Awesome R Package Development Tools[1] - curated list of tools for R package development
- Advanced R (Wickham 2019) - deep dive into R programming and internals
- Mastering Shiny (Wickham 2021) - comprehensive guide to building web applications with Shiny
- Engineering Production-Grade Shiny Apps (Fay et al. 2021) - best practices for production Shiny applications
- Happy Git and GitHub for the useR (Bryan 2023) - guide to using Git and GitHub with R
- litedown: R Markdown Reimagined[2] - a book by Yihui Xie on `{litedown}`[3], a lightweight R Markdown implementation aimed at simplicity and speed
- DataCamp[4] - email Kristen for access

21.1.2 UC Davis DataLab Workshops and Tutorials

The UC Davis DataLab[5] provides extensive workshops and learning materials for data science:

- Workshop Index[6] - comprehensive catalog of all DataLab workshops
- R Basics Workshop[7] - foundational R programming for beginners
- Research Toolkits[8] - in-depth guides for research tools and methods
- Install Guides[9] - setup instructions for data science software

[1] https://github.com/IndrajeetPatil/awesome-r-pkgtools
[2] https://pkg.yihui.org/litedown/book/
[3] https://pkg.yihui.org/litedown/
[4] https://www.datacamp.com/
[5] https://github.com/ucdavisdatalab
[6] https://ucdavisdatalab.github.io/workshop_index/
[7] https://github.com/ucdavisdatalab/workshop_r_basics
[8] https://github.com/ucdavisdatalab/research-toolkits
[9] https://ucdavisdatalab.github.io/install_guides/

21.1.3 Cheat Sheets

- Data transformation with dplyr cheat sheet[10]
- Data tidying with tidyr cheat sheet[11]
- Data visualization with ggplot2 cheat sheet[12]
- data table cheat sheet[13]
- R Markdown cheat sheet[14]

21.1.4 Style and Best Practices

- Hadley Wickham's R Style Guide[15]

21.1.5 Data Documentation and Dictionaries

- Tidyverse data-dict[16] - open specification and CLI tool for creating, validating, and documenting data dictionaries in YAML format

21.1.6 Tidy Evaluation Resources

- Tidy Eval in 5 Minutes[17] (video)
- Tidy Evaluation[18] (e-book)
- Data Frame Columns as Arguments to Dplyr Functions[19] (blog)
- Standard Evaluation for *_join[20] (stackoverflow)
- Programming with dplyr[21] (package vignette)

21.1.7 Continuous Integration for R

- r-ci: Easy and portable CI for R packages[22] - a portable shell script by Dirk Eddelbuettel for testing R packages across GitHub Actions, Azure Pipelines, CircleCI, and local Docker environments
- r-codeberg[23] - community organization supporting R package development and hosting on Codeberg, including the r-codeberg migration guide[24] for porting packages from GitHub to Codeberg

[10] https://rstudio.github.io/cheatsheets/html/data-transformation.html
[11] https://rstudio.github.io/cheatsheets/html/tidyr.html
[12] https://rstudio.github.io/cheatsheets/html/data-visualization.html
[13] https://s3.amazonaws.com/assets.datacamp.com/blog_assets/datatable_Cheat_Sheet_R.pdf
[14] https://rstudio.github.io/cheatsheets/html/rmarkdown.html
[15] http://adv-r.had.co.nz/Style.html
[16] https://data-dict.tidyverse.org/
[17] https://www.youtube.com/watch?v=nERXS3ssntw
[18] https://dplyr.tidyverse.org/articles/programming.html
[19] https://www.brodrigues.co/blog/2016-07-18-data-frame-columns-as-arguments-to-dplyr-functions/
[20] https://stackoverflow.com/questions/28125816/r-standard-evaluation-for-join-dplyr
[21] https://dplyr.tidyverse.org/articles/programming.html
[22] https://eddelbuettel.github.io/r-ci/
[23] https://codeberg.org/r-codeberg
[24] https://codeberg.org/r-codeberg/migration-guide

- Woodpecker CI[25] - community-driven, container-based continuous integration system that integrates with Codeberg, Forgejo, and Gitea for running automated R package checks and pipeline workflows
- shinycoreci: CI Testing for shiny[26] - R package developed by the Posit Shiny team for testing the Shiny ecosystem across GitHub Actions runner environments

21.2 Resources for Git & Github

- Happy Git and GitHub for the useR (Bryan 2023) - comprehensive guide to using Git and GitHub with R
- GitHub Learn[27] - official interactive courses, Skills exercises, learning pathways, and credentials
- GitHub Skills: Introduction to GitHub[28]
- UC Davis DataLab Git Sandbox[29] - hands-on Git practice repository
- GitKraken[30] - cross-platform Git client and developer platform with visual commit graphs, interactive rebasing, and GitKraken Kepler[31] for agent-driven development workflows
- About stacked pull requests[32] - official GitHub guide to breaking large changes into stacked pull request chains
- Claude Code on the Web: Auto-Fix Pull Requests[33] - documentation on Claude Code's web interface for automatically investigating and resolving check failures and review comments on pull requests
- Claude Code Workflows[34] - documentation on Claude Code's dynamic subagent orchestration feature for parallel codebase audits, migrations, and evaluations
- Google Code Review Developer Guide[35] - comprehensive guidelines on code review standards and pull request authoring; the upstream source for the Tidyverse code review principles[36] (see acknowledgments[37])

21.3 Practical Computing and Command Line

- The Missing CS Quarter at UC Davis[38] - practical computing skills, shell tools, command-line workflows, and version control (GitLab repository[39])

[25] https://woodpecker-ci.org/
[26] https://rstudio.github.io/shinycoreci/
[27] https://learn.github.com/
[28] https://github.com/skills/introduction-to-github
[29] https://github.com/ucdavisdatalab/sandbox_git
[30] https://gitkraken.com/
[31] https://gitkraken.com/kepler
[32] https://docs.github.com/en/pull-requests/get-started/about-stacked-prs
[33] https://code.claude.com/docs/en/claude-code-on-the-web#auto-fix-pull-requests
[34] https://code.claude.com/docs/en/workflows
[35] https://google.github.io/eng-practices/review/
[36] https://code-review.tidyverse.org/
[37] https://code-review.tidyverse.org/#acknowledgements
[38] https://missing.cs.ucdavis.edu/
[39] https://gitlab.com/missing-cs-quarter-ucd/

21.4 Resources for Python

- UC Davis DataLab Python Basics Workshop[40] - foundational Python programming
- Natural Language Processing with Python[41] - text analysis and NLP techniques
- Jupyter Notebooks in a Git Repository[42] - guide by Matthias Geier on managing Jupyter notebooks in Git repositories, including workflows for stripping cell outputs before committing and strategies for sharing executed notebooks

21.5 Resources for Julia

- UC Davis Julia Users Group Julia Basics Workshop[43] - foundational Julia programming

21.6 Artificial Intelligence & Coding Agents

- Working with AI (wai)[44] - our lab's comprehensive guide to AI coding agents, harnesses, policies, and workflows
- When AI builds itself: Recursive self-improvement in practice[45] - Anthropic Institute analysis of recursive self-improvement mechanisms, AI capability trajectories, and safety considerations
- `pi`[46] - minimalist, extensible AI agent toolkit providing a unified LLM API, interactive TUI, and terminal coding agent CLI

21.7 Scientific figures

- Ten Simple Rules for Better Figures (Rougier et al. 2014)
- Tufte CSS[47] - style HTML articles like Edward Tufte's books and handouts, with sidenotes, typography, and tight integration of graphics with text

21.8 Writing

- Unpacking the Scientific Toolbox (Silbiger and Stubler 2019)
- ICMJE Definition of authorship (International Committee of Medical Journal Editors, n.d.)
- Computational science: …why scientific programming does not compute (Merali and Giles 2010)
- The Pathway to Publishing: A Guide to Quantitative Writing in the Health Sciences[48]

[40] https://github.com/ucdavisdatalab/workshop_python_basics
[41] https://github.com/ucdavisdatalab/workshop_nlp_with_python
[42] https://mgeier.github.io/git-jupyter.html
[43] https://ucjug.github.io/workshop_julia_basics/
[44] https://morrison-lab.github.io/wai/
[45] https://www.anthropic.com/institute/recursive-self-improvement
[46] https://github.com/earendil-works/pi
[47] https://edwardtufte.github.io/tufte-css/
[48] https://link.springer.com/book/10.1007/978-3-030-98175-4

- Principles of Scientific Writing[49] - a handbook covering scientific writing principles including citations and evidence, word choice, and conciseness
- Secret, actionable writing tips[50]

21.9 Presentations

- How to tell a compelling story in scientific presentations (Van Noorden 2021)
- How to give a killer narratively-driven scientific talk[51]
- How to make a better poster[52]
- How to make an even better poster[53]

21.10 Professional advice

- Professional advice, especially for your first job[54]
- Team Public Health Substack[55]

21.11 Funding

- Building Your Funding Train[56]
- NIH Grant Writing Resources[57]

21.12 Ethics and global health research

- Global Code of Conduct For Research in Resource-Poor Settings[58]
- Addressing power asymmetries in global health (Abimbola et al. 2022)
- Transforming Global Health Partnerships[59]

21.13 Open Science and Reproducibility Initiatives

- The Open Science Manual[60] - an open-source guide from the ARCA group[61] at the University of Padova covering programming good practices, version control, and reproducible workflow tools.

[49] https://github.com/d-morrison/psw
[50] https://x.com/acagamic/status/1680381737816424450
[51] https://www.sciencedirect.com/science/article/pii/S1471491423000928
[52] https://www.youtube.com/watch?v=1RwJbhkCA58&t=1171s
[53] https://mitcommlab.mit.edu/be/2023/09/27/toward-an-evenbetterposter-improving-the-betterposter-template/
[54] https://docs.google.com/document/d/1ckgRCcr7FFPyymyMA6Y_-cB_vftOyrrxT28c6gRg0PI/edit
[55] https://teampublichealth.substack.com/
[56] https://grantwriting.stanford.edu/resources/funding-train/
[57] https://www.niaid.nih.gov/grants-contracts/write-grant-application
[58] https://www.globalcodeofconduct.org/
[59] https://link.springer.com/book/9783031537929
[60] https://arca-dpss.github.io/manual-open-science/
[61] https://github.com/arca-dpss

- Openscapes[62] - an open science mentorship and community initiative founded by Dr. Julia Stewart Lowndes, focused on helping research groups transition to data-intensive, reproducible, and collaborative scientific practices through cohort-based Champions programs and shared open tooling.

21.14 Comparable Lab Manuals

Exploring lab manuals from peer research groups helps us identify best practices, adapt proven onboarding workflows, and align our open science standards with the broader computational biology and epidemiology communities:

- Benjamin-Chung Lab Manual[63] (GitHub repository[64]) — The computational epidemiology manual from which our manual originally adapted its structure, covering reproducible R workflows, collaborative development, and computational research practices.
- Fay Lab Manual[65] (GitHub repository[66]) — An Openscapes-origin Quarto manual for quantitative fisheries ecology (UMass Dartmouth), featuring detailed offboarding checklists, departure handoff protocols, and an open science communication philosophy.
- ContextLab Lab Manual[67] (GitHub repository[68]) — The Contextual Dynamics Lab manual at Dartmouth, notable for requiring incoming lab members to make an interactive pull request to learn contribution workflows hands-on.
- Proctor-UCSF DCC Handbook[69] (GitHub repository[70]) — A comprehensive handbook for the Data Coordinating Center at UCSF, focusing on clinical trial data management, REDCap workflows, quality assurance, and public data curation in global health settings.
- Quantitative Marine Ecology Lab Manual[71] (GitHub repository[72]) — A Quarto-based manual for Easton White's research group at UNH, featuring role-specific administrative checklists and curated reading lists organized by career stage.

21.15 Reading and Watching List

- *Good Luck, Have Fun, Don't Die*[73] — A science fiction film directed by Gore Verbinski following a traveler from the future who recruits patrons at a Los Angeles diner to prevent a rogue artificial intelligence from destroying the world.

[62] https://openscapes.org/
[63] https://jadebc.github.io/lab-manual/
[64] https://github.com/jadebc/lab-manual
[65] https://thefaylab.github.io/lab-manual/
[66] https://github.com/thefaylab/lab-manual
[67] https://contextlab.github.io/lab-manual/
[68] https://github.com/ContextLab/lab-manual
[69] https://proctor-ucsf.github.io/dcc-handbook/
[70] https://github.com/proctor-ucsf/dcc-handbook
[71] https://quantmarineecolab.github.io/lab-manual/
[72] https://github.com/quantmarineecolab/lab-manual
[73] https://en.wikipedia.org/wiki/Good_Luck,_Have_Fun,_Don%27t_Die

22 Professional Development

22.1 Mentoring Philosophy

We believe in individualized mentoring that supports each person's unique career goals. Effective mentoring requires:

- Regular, open communication between mentees and mentors
- Mutual respect and trust
- Clear expectations and goals
- Constructive feedback
- Support for both research and career development

22.2 Individual Development Plans

All graduate students and postdocs should maintain an Individual Development Plan (IDP) that outlines:

- Short-term and long-term career goals
- Skills to develop
- Training needs and opportunities
- Timeline and milestones
- Progress toward goals

Update your IDP at least annually and discuss it with your PI. Useful resources:

- myIDP - Individual Development Plan tool[1]
- NIH OITE Career Resources[2]

22.3 Navigating the Hidden Curriculum

The "hidden curriculum" refers to the unspoken norms, unwritten expectations, and cultural practices in graduate school and academic research that are rarely taught in formal coursework but are critical for long-term success and well-being. Demystifying these implicit rules ensures equitable access to academic opportunities for scholars from all backgrounds.

[1] https://myidp.sciencecareers.org/
[2] https://www.training.nih.gov/career-services/

22.3.1 Managing Up and Mentoring Relationships

Mentorship is a collaborative partnership. Taking active ownership of your mentoring relationships helps ensure you receive the guidance and feedback you need:

- **Set clear meeting agendas**: Send a brief bulleted agenda and updated materials at least 24 hours before 1-on-1 meetings.
- **Clarify priorities and timelines**: Agree on concrete milestones, deadlines, and expectations for feedback turnarounds.
- **Build a mentoring team**: Seek guidance from multiple mentors, including thesis committee members, peer mentors, senior graduate students, and external collaborators. No single mentor can meet all training, professional, and personal needs.

22.3.2 Normalizing Setbacks and Imposter Feelings

Graduate training involves tackling open-ended scientific problems where methods fail, experiments yield inconclusive results, and manuscripts face rejection:

- **Imposter feelings are common**: Experiencing self-doubt or feeling underqualified is a widely shared experience in high-rigor research environments, especially during early transitions. It reflects the challenging nature of scientific discovery, not personal inadequacy.
- **Analytical iterations are normal**: Bugs, refactored data pipelines, and revisions are standard components of the scientific process.
- **Establish healthy boundaries**: Sustainable productivity requires rest, structured time off, and hobbies outside academia. Seek out campus health and counseling resources when facing persistent stress.

22.3.3 Demystifying Publishing and Peer Review

Navigating scientific publication requires understanding the review lifecycle:

- **Rejection is a normal part of publishing**: In academic publishing, manuscripts frequently undergo multiple rounds of submission; rejection provides an opportunity to refine arguments and submit to a better-matched venue.
- **Approaching reviewer feedback**: Read peer reviews, step back for a day to process feedback, and draft systematic, point-by-point response letters using the guidance in Section 24.2.
- **Authorship conversations**: Initiate discussions regarding authorship order and contribution roles early in every project and revisit them regularly.

22.3.4 Strategic Networking and Visibility

Building professional connections expands research opportunities and career horizons:

- **Conferences as networking venues**: Prepare a clear 60-second summary of your research question, key methods, and scientific impact.
- **Engage with speakers**: Introduce yourself after departmental seminars or during poster sessions to ask focused questions about their work.

- **Peer networks**: Cultivate strong relationships with cohort members and lab peers; colleagues at your career stage will become future collaborators, reviewers, and lifelong allies.

22.4 Presentations and Conferences

We encourage lab members to present their work at conferences and seminars:

- Discuss conference opportunities with PIs early
- Submit abstracts with PI approval
- Practice presentations in lab meeting before the conference
- Prioritize clear visuals over text-dense slides following our Show, Don't Tell[3] principle
- Funding for conferences depends on availability and should be discussed in advance

22.4.1 Guidelines for Slide Decks

When preparing slide presentations for lab meetings, conferences, qualifying exams, or thesis defenses, follow these practical guidelines:

- **Always include slide numbers**: Slide numbers are essential for receiving constructive feedback. During question-and-answer periods or practice feedback sessions, collaborators and audience members need to refer back to specific slides quickly (e.g., "On slide 14, how was that variable defined?") without fumbling through visual descriptions or guessing where a topic occurred.
- **Complete, record, and review practice runs**: Before any significant presentation, rehearse the complete talk out loud, record the practice run (using screen recording software, a webcam, or your phone), and review the recording. Watching your own practice run provides invaluable, objective feedback on pacing, timing, verbal filler habits, transition clarity, and whether slide text and figures remain legible when viewed on screen. Both students and faculty benefit substantially from this practice.
- **One primary takeaway per slide**: Limit the cognitive load of each slide. If a slide presents multiple distinct findings or complex arguments, split them across sequential slides so the audience can focus on each point.
- **Prioritize visual evidence**: Audiences cannot read dense paragraphs while simultaneously listening to a speaker. Favor clean charts, diagrams, and annotated figures over text-heavy bulleted lists.
- **Ensure high contrast and readability**: Use large, legible fonts and high-contrast color palettes. Verify that axis labels, legends, and annotations remain easily readable from the back of a large room or on low-resolution projector displays.

22.4.2 Presentation and Poster Resources

- How to tell a compelling story in scientific presentations[4]
- How to give a killer narratively-driven scientific talk[5]
- How to make a better poster[6]

[3] @sec-show-dont-tell
[4] https://www.nature.com/articles/d41586-021-03603-2
[5] https://www.sciencedirect.com/science/article/pii/S1471491423000928
[6] https://www.youtube.com/watch?v=1RwJbhkCA58

- How to make an even better poster[7]

22.5 Scientific Figures

Creating clear, effective figures is essential for communicating research findings:

- Label x-axis and y-axis clearly
- Use panel labels when including multiple subplots
- Include sufficient tick marks and labels
- Use appropriate font sizes (readable but not distracting)
- Use colorblind-friendly palettes (see ColorBrewer[8] or iwanthue[9])
- Define colors, shapes, and line types in legends
- Minimize abbreviations in labels and legends
- Use transparency to show overlapping data
- Show measures of precision (e.g., 95% confidence intervals)

Resources:

- Ten Simple Rules for Better Figures[10]

22.6 Grant Writing

- Graduate students and postdocs are encouraged to apply for fellowships (e.g., NIH F31, NSF GRFP, K awards)
- PIs will support fellowship applications with feedback, letters of support, and mentoring
- Start planning fellowship applications well in advance of deadlines (typically 3-6 months)
- Attend grant writing workshops and seek feedback from multiple sources

Resources:

- Building Your Funding Train[11]
- NIH Funding Opportunities[12]
- NIH Grant Writing Resources[13]

22.7 PhD Dissertation Requirements

Understanding what constitutes a sufficient PhD dissertation is crucial for setting realistic expectations and timelines. The dissertation represents an important milestone, but it doesn't need to be your magnum opus.

[7] https://mitcommlab.mit.edu/be/2023/09/27/toward-an-evenbetterposter-improving-the-betterposter-template/
[8] http://colorbrewer2.org/
[9] https://medialab.github.io/iwanthue/
[10] https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1003833
[11] https://grantwriting.stanford.edu/resources/funding-train/
[12] https://grants.nih.gov
[13] https://www.niaid.nih.gov/grants-contracts/write-grant-application

22.7.1 Review Previous Dissertations

Before setting your dissertation goals, read previous dissertations from students in your program. This helps you:

- Understand the typical scope and depth expected
- See different approaches to structure and presentation
- Calibrate your expectations based on successful examples
- Identify common patterns and standards in your field

Most universities maintain electronic dissertation repositories, making it easy to access recent examples from your program.

22.7.2 Publication Requirements

Three first-author papers typically suffice for a dissertation in public health and biomedical sciences. If academic peers in reputable journals have approved your work through peer review, this demonstrates that your research meets professional standards. Your dissertation committee should recognize this external validation.

The specific publication requirements may vary by program and institution, so consult your program's guidelines and discuss expectations with your committee early. However, three substantial first-author publications generally demonstrate:

- Independent research capability
- Ability to communicate findings effectively
- Contribution to the scientific literature
- Readiness for an academic or research career

22.7.3 External Validation and Fast-Tracking

If you have a job offer waiting, you can usually get fast-tracked through the dissertation process. The reasoning is straightforward: your work has been externally validated as worthwhile by prospective employers.

Since most post-PhD positions offer better compensation than graduate stipends, it's difficult to justify prolonging your graduation when you've already demonstrated professional competence. This applies whether the job offer is in academia, industry, government, or nonprofit sectors.

22.7.4 Historical Context: The Masterpiece Tradition

The dissertation[14] (or thesis), a requirement for earning a PhD[15], is a spiritual successor to an apprentice's masterpiece[16] in craft guilds. Historically, a masterpiece was the piece of work that demonstrated an apprentice had achieved sufficient skill to join the guild as a master craftsperson. It was not meant to be the best work they would ever produce—it was meant to prove they were ready to work independently.

[14] https://en.wikipedia.org/wiki/Thesis
[15] https://en.wikipedia.org/wiki/Doctor_of_Philosophy#History
[16] https://en.wikipedia.org/wiki/Masterpiece#History

Similarly, your dissertation should demonstrate that you're ready to conduct independent research. It's your first professional-level work, not your career highlight. This perspective helps set appropriate expectations:

- The dissertation proves you can conduct rigorous research
- It doesn't need to solve every problem in your field
- It doesn't need to be flawless
- It doesn't need to be all-encompassing
- It just needs to constitute incremental progress in your field

22.7.5 Setting Realistic Expectations

Many PhD students struggle with perfectionism or "scope creep" in their dissertations. Remember:

- Done is better than perfect when it comes to dissertations
- You'll have your entire career to refine and expand on these ideas
- The goal is to finish and move forward, not to write the definitive work on your topic
- Your committee wants to see you succeed and graduate

Focus on making a solid, incremental contribution to knowledge in your field. That's what a dissertation is meant to be—no more, no less.

22.7.6 Resources

Dissertation writing tools:

- quarto-thesis[17] - Quarto extension for creating masters or PhD theses with professional LaTeX formatting

22.8 Teaching and Outreach

Teaching and outreach are valuable professional development opportunities:

- Graduate students are encouraged to gain teaching experience
- We support science communication and outreach activities
- Discuss opportunities with PIs

The UC Davis DataLab[18] offers various workshops and learning materials that can support your teaching and professional development. Their workshop index[19] provides a comprehensive catalog of available resources.

[17] https://github.com/nmfs-opensci/quarto-thesis
[18] https://github.com/ucdavisdatalab
[19] https://ucdavisdatalab.github.io/workshop_index/

22.9 Networking

Building a professional network is important for your career:

- Attend seminars and departmental events
- Connect with researchers in your field
- Join professional societies
- Use professional social media platforms to share research and engage with the scientific community

23 Writing

23.1 Writing to Clarify Your Thinking

Writing is a powerful tool for clarifying your thoughts, even before you start searching for answers. When questions or ideas come up, it's good practice to write them down immediately—this helps you:

- Organize your thoughts and identify what you actually need to know
- Articulate the problem more precisely
- Often realize the answer yourself through the process of writing

As Leslie Lamport, a Turing Award winner and computer scientist, states: "If you think you understand something, and don't write down your ideas, you only think you're thinking" (Lamport 2019).

23.2 Plain, Direct Prose

Write user-facing prose in a plain, direct style. This applies to everything a reader sees — pull request, issue, and commit text, docs, READMEs, code comments, release notes, and messages.

The guide of record is the lab's **Principles of Scientific Writing (PSW)**: https://morrison-lab.github.io/psw/. The rules below operationalize it. When PSW and this guidance disagree, PSW wins.

- **Limit dependent (subordinate) clauses.** One per sentence is plenty. When two or more stack up, split the sentence.
- **Cut low-content filler and jargon.** Delete words that add no information ("it is worth noting", "in order to" -> "to", "due to the fact that" -> "because").
- **Prefer plain (Anglish) words over Latin-derived ones** (PSW, "Word choice"). "before", not "prior to"; "needed", not "necessary"; "use", not "utilize". A heuristic, not a purity rule.
- **Prefer simple declarative sentences and active voice.** Subject, verb, point. Name the actor, then the action.
- **Join independent clauses with coordinating conjunctions** (and, but, so, or) over subordinate constructions. Prefer "X is fast, but Y is correct" over "While X is fast, Y is correct."

This is a default, not an absolute rule. Keep a clause or a technical term when removing it would lose meaning or precision. Never trade an honest hedge for false confidence.

23.3 Scanning for AI Tells

When AI tools draft or edit prose, the result often carries telltale signs of machine authorship.

Write plainly up front, then scan the draft for AI tells before sending. Before presenting non-trivial prose — pull request and issue descriptions, commit bodies, README, doc, and vignette text, or long answers — self-check it and cut the tells. Apply the same catalog when reviewing someone else's prose too:

- **Overused vocabulary:** actionable (-> "to act on"), delve, leverage, utilize, tapestry, testament, realm, robust, seamless, holistic, nuanced, multifaceted, pivotal, crucial, "in today's fast-paced world", "stands as a testament to".
- **Rhetorical reflexes:** the "it's not just X, it's Y" antithesis (the biggest tell), mechanical rule-of-three lists, signposting filler ("it is worth noting that", "importantly"), hedging stacks, hollow "in conclusion" restatements.
- **Structural and typographic:** em-dash overuse as a default connector, bold-leading `**Term:**` bullets applied mechanically, emoji section headers, conspicuously uniform paragraph rhythm.
- **Tonal:** promotional register, reflexive both-sidesing, vague universals with no concrete names or numbers.

De-slop — cut the filler and the reflexes — but do not ban words outright or sand the text into a flat, voiceless register. Any single tell is innocent; clustering and mechanical repetition are the signal. Code, terse status lines, and short conversational replies are exempt.

23.4 Show, Don't Tell

The adage "show, don't tell" (or "a picture is worth a thousand words") is a foundational communication principle across research manuscripts, technical documentation, and oral presentations.

Whenever conveying complex concepts, workflows, data patterns, or system architectures, favor concrete visual evidence and direct demonstration over dense verbal exposition.

23.4.1 In scientific manuscripts and reports

- **Lead with clear figures and tables:** Instead of describing multi-step analytical pipelines or multi-group comparative trends in lengthy paragraphs, present the data in a well-labeled plot or diagram.
- **Support visuals with interpretation, not repetition:** Use the narrative text to guide the reader through the key insights and takeaways of a figure rather than exhaustively listing every number already legible in a table or plot.
- **Use diagrams for conceptual models:** Directed acyclic graphs (DAGs), CONSORT flow diagrams, and workflow schematics clarify methodological structures much faster than textual descriptions alone.

23.4.2 In conference and lab presentations

- **Minimize text on slides:** Audiences cannot read dense bulleted paragraphs while simultaneously listening to a speaker. Replace text-heavy slides with clean visuals, diagrams, or summary charts.
- **Highlight key focal points:** Use callouts, annotations, or sequential visual reveals to focus attention on the specific relationship being discussed.
- **Demonstrate rather than assert:** When presenting software tools or analytical pipelines, show live output, reproducible code snippets, or screenshots rather than merely listing features.

24 Manuscript Preparation and Publication

24.1 Publication Process

The typical workflow for manuscript preparation and publication:

1. **Planning**: Discuss target journals, outline, and timeline with PIs
2. **Drafting**: Lead author prepares initial draft
3. **Internal review**: Co-authors review and provide feedback
4. **Revision**: Lead author incorporates feedback iteratively
5. **PI approval**: Obtain final approval from PIs before submission
6. **Submission**: Submit to journal
7. **Revisions**: Lead author coordinates response to peer reviewers
8. **Publication**: Celebrate and share!

24.2 Responding to Peer Review

When a journal asks for revisions, you will typically need to provide a point-by-point response to the reviewers' comments. The key strategy is to use the "yes-and" approach[1] from improvisational theater—building on what reviewers say rather than contradicting them. Try to avoid explicitly disagreeing with reviewers if at all possible.

Standard response format:

The typical response follows this pattern:

> "Thank you for raising this important point. We agree, and now state (page ##):
>
> [quote a section of the revised manuscript here]"

Balancing direct response and manuscript content:

There can be some overlap between the direct response to the reviewer (the sentences before "now state") and what goes into the revised manuscript (the part after "(page ##)"). This is acceptable, but we usually try to put as much of the content into the revised manuscript as possible, and minimize the direct response section. The goal is to make the manuscript self-contained while showing the reviewer that you've addressed their concern.

When content already exists:

Sometimes, a reviewer asks for content that was already in the manuscript. In these cases, first see if there's anything you can elaborate or clarify. Even if you don't see room for improvement, you can usually respond with:

[1] https://en.wikipedia.org/wiki/Yes,_and_...

"We agree, and now state (page ##):

[quote the content that was already there before]"

The reviewer doesn't need to know that they missed it the last time (see also: implicature[2].

Additional tips:

- Always thank reviewers for their time and feedback
- Quote page numbers from the revised manuscript
- Use direct quotes from your revised text when possible
- Maintain a respectful and collaborative tone throughout
- If you make changes beyond what the reviewer requested, mention them briefly

24.3 Preprints and Open Access

- We encourage posting preprints prior to or during peer review on platforms like medRxiv[3] or bioRxiv[4]
- Preprints allow rapid dissemination of findings and can be cited in grant applications
- We support publishing in open access journals when possible to maximize accessibility
- Many funders, including NIH, have public access policies that require making publications freely available

A preprint is a scientific manuscript that has not been peer reviewed. Preprint servers create digital object identifiers (DOIs) and can be cited in other articles and in grant applications. Because the peer review process can take many months, publishing preprints prior to or during peer review enables other scientists to immediately learn from and build on your work. Importantly, NIH allows applicants to include preprint citations in their biosketches. In most cases, we publish preprints on medRxiv[5].

24.4 Reporting Checklists

Using reporting checklists ensures that publications contain information needed for readers to assess validity and reproducibility. We use checklists appropriate to study design:

- CONSORT for randomized trials
- STROBE for observational studies
- PRISMA for systematic reviews
- Others as appropriate (see EQUATOR Network[6])

[2] https://en.wikipedia.org/wiki/Implicature
[3] https://www.medrxiv.org/
[4] https://www.biorxiv.org/
[5] https://www.medrxiv.org/
[6] https://www.equator-network.org/

24.5 Manuscript Checklist

Before submitting a manuscript:

- ☐ Completed relevant reporting checklist
- ☐ Results are reproducible (rerunning code replicates tables/figures exactly)
- ☐ Target journal selected
- ☐ Title is declarative and states findings clearly
- ☐ Word count meets journal requirements
- ☐ Manuscript follows journal formatting guidelines
- ☐ Consistent voice throughout (typically active voice)
- ☐ All figures and tables referenced in main text
- ☐ Captions for all figures and tables
- ☐ Tables/figures numbered by order of appearance
- ☐ Abbreviations defined at first use and used sparingly
- ☐ Avoid subjective wording (e.g., "interesting", "dramatic")
- ☐ Avoid directionless statements (specify direction of associations)
- ☐ Causal language only when supported by study design
- ☐ Avoid "significant" (easily confused with statistical significance)
- ☐ Author contributions drafted using CRediT Taxonomy

24.6 Formatting tables with `gtsummary` + `flextable` + `flexlsx`

One practical workflow for publication-ready tables is to use `{gtsummary}`[7] to create summary/regression tables, convert to `{flextable}`[8] for styling, and export to Excel with `{flexlsx}`[9].

````
```{r}
#| eval: false

tbl1 <-
  trial |>
  gtsummary::tbl_summary(by = trt) |>
  gtsummary::as_flex_table() |>
  flextable::autofit()

flexlsx::write_xlsx(tbl1, path = here::here("results", "table-1.xlsx"))
```
````

This gives you a reproducible table pipeline:

- Define table content with `gtsummary::tbl_summary()` (or `gtsummary::tbl_regression()` for model tables).
- Apply presentation formatting with `flextable` verbs.
- Export an `.xlsx` file for collaborator review while preserving your R workflow.

[7] https://www.danieldsjoberg.com/gtsummary/
[8] https://davidgohel.github.io/flextable/
[9] https://cran.r-project.org/package=flexlsx

24.7 Scientific Writing: Claims and Evidence

All factual claims in scientific writing should be supported by appropriate evidence.

Citation requirements:

- **Cite primary sources** for factual statements about established knowledge, methods, or findings
- **Cite official documentation** when describing how software, tools, or systems work
- **Link to authoritative sources** like peer-reviewed publications, official repositories, or technical specifications
- Avoid citing secondary sources when primary sources are available

When you can't find a citation:

- **Demonstrate directly**: Show the behavior through experiments, data, or explicit examples
- **Acknowledge uncertainty**: Use appropriate hedging language ("may", "appears to", "in our experience") when evidence is limited
- **Remove the claim**: If you cannot substantiate a claim with either citations or direct evidence, consider whether it needs to be included

Why this matters:

- Builds reader trust and credibility
- Enables readers to verify information independently
- Maintains scientific rigor in all communications
- Prevents propagation of misinformation

This principle applies to all lab writing, including: manuscripts, documentation, grant applications, and technical reports.

Using AI tools for writing:

When using AI tools to help develop manuscripts or other academic writing, follow the special guidance in the wai site's "Using AI for Journal Articles" section[10] to ensure transparency, maintain intellectual ownership, and avoid plagiarism.

[10] https://morrison-lab.github.io/wai/chapters/ai-use-policies.html#sec-ai-journal-articles

References

Abimbola, Seye, Sheena Asthana, Cristina Montenegro, et al. 2022. "Addressing Power Asymmetries in Global Health: Imperatives in the Wake of the COVID-19 Pandemic." *PLOS Global Public Health* 2 (10). https://doi.org/10.1371/journal.pgph.0002269.

"About Stacked Pull Requests." n.d. GitHub. https://docs.github.com/en/pull-requests/get-started/about-stacked-prs.

Baker, Monya. 2019. "How to Tackle the Reproducibility Crisis in Ten Steps." *Nature*, ahead of print. https://doi.org/10.1038/d41586-019-01431-z.

Benjamin-Chung, Jade, Kunal Mishra, Stephanie Djajadi, et al. 2024. *Benjamin-Chung Lab Manual*. https://jadebc.github.io/lab-manual/.

Bryan, Jennifer. 2023. *Happy Git and GitHub for the useR*. https://happygitwithr.com/.

Bryan, Jennifer, Jim Hester, David Robinson, Hadley Wickham, and Christophe Dervieux. 2024. *Reprex: Prepare Reproducible Example Code via the Clipboard*. https://reprex.tidyverse.org/.

Crameri, Fabio, Grace E. Shephard, and Philip J. Heron. 2020. "The Misuse of Colour in Science Communication." *Nature Communications* 11. https://doi.org/10.1038/s41467-020-19160-7.

"Creative Commons Attribution-NonCommercial 4.0 International License." n.d. Creative Commons. http://creativecommons.org/licenses/by-nc/4.0/.

"CRediT: Contributor Roles Taxonomy." n.d. PLOS ONE. https://journals.plos.org/plosone/s/authorship.

"Dryad Digital Repository." n.d. Dryad. https://datadryad.org/.

"EQUATOR Network: Enhancing the QUAlity and Transparency of Health Research." n.d. EQUATOR Network. https://www.equator-network.org/.

Fay, Colin, Sébastien Rochette, Vincent Guyader, and Cervan Girard. 2021. *Engineering Production-Grade Shiny Apps*. Chapman; Hall/CRC. https://doi.org/10.1201/9781003029878.

Gewerd-Strauss. 2023. "Mermaid Rendering to Docx Halts Indefinitely When Run Through Command Line." Quarto Development Team. https://github.com/orgs/quarto-dev/discussions/6085.

"GitHub Desktop." n.d. GitHub. https://desktop.github.com/.

"GitKraken." n.d. GitKraken. https://gitkraken.com/.

"GitKraken Kepler: Mission Control for Agent-Driven Development." n.d. GitKraken. https://gitkraken.com/kepler.

"How to Store and Manage Your Data." n.d. PLOS. https://plos.org/resource/how-to-store-and-manage-your-data/.

Hunt, Andrew, and David Thomas. 1999. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley.

International Committee of Medical Journal Editors. n.d. "Defining the Role of Authors and Contributors." ICMJE. http://www.icmje.org/recommendations/browse/roles-and-responsibilities/defining-the-role-of-authors-and-contributors.html.

Lamport, Leslie. 2019. *Think and Write with Leslie Lamport*. Podcast interview. https://mentors.fm/2019/08/13/think-and-write-with-leslie-lamport/.

Martin, Robert C. 2008. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.

"medRxiv: The Preprint Server for Health Sciences." n.d. Cold Spring Harbor Laboratory. https://www.medrxiv.org/.

Merali, Zeeya, and Jim Giles. 2010. "Computational Science: Error ... Why Scientific Programming Does Not Compute." *Nature* 467: 775–77. https://doi.org/10.1038/467775a.

Munafò, Marcus R., Brian A. Nosek, Dorothy V. M. Bishop, et al. 2017. "A Manifesto for Reproducible Science." *Nature Human Behaviour* 1. https://doi.org/10.1038/s41562-016-0021.

Nuzzo, Regina. 2015. "How Scientists Fool Themselves – and How They Can Stop." *Nature* 526: 182–85. https://doi.org/10.1038/526182a.

"Open Science Framework." n.d. Center for Open Science. https://osf.io/.

"Quarto Cannot Render Mermaid or Dot Images to Docx and Pdf Formats." 2023. Quarto Development Team. https://github.com/quarto-dev/quarto-cli/issues/3809.

"Requesting an Archive of Your Personal Account's Data." n.d. GitHub. https://docs.github.com/en/get-started/archiving-your-github-personal-account-and-public-repositories/requesting-an-archive-of-your-personal-accounts-data.

"REST API Endpoints for Organization Migrations." n.d. GitHub. https://docs.github.com/en/rest/migrations/orgs.

"REST API Endpoints for User Migrations." n.d. GitHub. https://docs.github.com/en/rest/migrations/users.

Robertson, Seth. n.d. "On Undoing, Fixing, or Removing Commits in Git." https:

//sethrobertson.github.io/GitFixUm/fixup.html.

Rougier, Nicolas P., Michael Droettboom, and Philip E. Bourne. 2014. "Ten Simple Rules for Better Figures." *PLOS Computational Biology* 10 (9). https://doi.org/10.1371/journal.pcbi.1003833.

Silbiger, Nyssa J., and Ariel D. Stubler. 2019. "Unpacking the Scientific Toolbox: Five Skills for the Modern Scientist." *Nature*, ahead of print. https://doi.org/10.1038/d41586-019-02918-5.

"Slurm Workload Manager: Salloc Documentation." n.d.-a. SchedMD. https://slurm.schedmd.com/salloc.html.

"Slurm Workload Manager: Sattach Documentation." n.d.-b. SchedMD. https://slurm.schedmd.com/sattach.html.

"Slurm Workload Manager: Sbatch Documentation." n.d.-c. SchedMD. https://slurm.schedmd.com/sbatch.html.

"Slurm Workload Manager: Squeue Documentation." n.d.-d. SchedMD. https://slurm.schedmd.com/squeue.html.

"Slurm Workload Manager: Srun Documentation." n.d.-e. SchedMD. https://slurm.schedmd.com/srun.html.

"Ssh_config(5) Manual Page." n.d. OpenBSD. https://man.openbsd.org/ssh_config.

"Ssh_config(5) Manual Page, Debian Edition." n.d. Debian Project. https://manpages.debian.org/stable/openssh-client/ssh_config.5.en.html.

Stoddart, Charlotte. 2019. "Is There a Reproducibility Crisis in Science?" *Nature*, ahead of print. https://doi.org/10.1038/d41586-019-00067-3.

Tidyverse Team. 2023. *Tidyverse Code Review Principles*. https://code-review.tidyverse.org/.

Van Noorden, Richard. 2021. "Scientists and Science Communicators Swap Tips on How to Tell Compelling Stories." *Nature*, ahead of print. https://doi.org/10.1038/d41586-021-03603-2.

Wickham, Hadley. 2019. *Advanced r*. 2nd ed. Chapman; Hall/CRC. https://doi.org/10.1201/9781351201315.

Wickham, Hadley. 2021. *Mastering Shiny*. O'Reilly Media. https://mastering-shiny.org/.

Wickham, Hadley. 2023a. *The Tidyverse Style Guide*. https://style.tidyverse.org/.

Wickham, Hadley. 2023b. *Tidyverse Design Guide*. https://design.tidyverse.org/.

Wickham, Hadley, and Jennifer Bryan. 2023. *R Packages*. 2nd ed. O'Reilly Media. https://r-pkgs.org/.

Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolemund. 2023. *R for Data Science*. 2nd ed. O'Reilly Media. https://r4ds.hadley.nz/.

Wickham, Hadley, Peter Danenberg, Gábor Csárdi, and Manuel Eugster. 2024. *Roxygen2: In-Line Documentation for r*. https://roxygen2.r-lib.org/.

Copilot Instructions File

For the complete `.github/copilot-instructions.md` file, please view the HTML version of this appendix at: https://ucd-serg.github.io/lab-manual/appendix-copilot-instructions.html

Copilot Setup Steps File

For the complete `.github/workflows/copilot-setup-steps.yml` file, please view the HTML version of this appendix at: https://ucd-serg.github.io/lab-manual/appendix-copilot-setup-steps.html

Document Generation Metadata

This document was generated from the following git commit:

- **Branch:** HEAD
- **Commit:** 337021e
- **Full commit hash:** 337021e19bde30d2c3e7dd731cf6058bfa434007
- **Commit date:** 2026-10-11 20:31:09 -0700

When transferring edits from this DOCX file back to the Quarto source files, use this commit information to set up the PR correctly and account for any commits that have been added since this document was generated.